

CALCOLATORI ELETTRONICI

Giuseppe Lettieri

A.A. 2024/2025

Indice

1. Introduzione e richiami	1
2. Indirizzi e oggetti	7
3. La memoria centrale	13
4. Memoria Cache	19
5. Protezione	25
6. Interruzioni	33
7. Memoria virtuale	41
8. Multiprogrammazione	49
9. Cache, DMA e bus PCI	57
10. Architettura interna del processore	65

Calcolatori Elettronici: introduzione

G. Lettieri

27 Febbraio 2023

1 Introduzione e richiami

La Figura 1 mostra l'architettura generale di un calcolatore moderno. Nel corso approfondiremo ogni componente. I principali argomenti di cui parleremo sono:

- protezione;
- interruzioni;
- memoria virtuale.

Questi tre argomenti ci permetteranno di parlare della *multiprogrammazione*: come eseguire “contemporaneamente” più programmi su un sistema che ha un solo processore. Studieremo tutti i dettagli che ci permetteranno di realizzare un (semplificato, ma funzionante) *nucleo di sistema operativo multiprogrammato*. Raffineremo inoltre l'architettura dell'elaboratore parlando di cache, DMA (Direct Memory Access), bus PCI. Infine, esamineremo la struttura interna di un processore moderno in grado di eseguire le istruzioni in parallelo, fuori ordine e speculativamente.

Esaminiamo di seguito i componenti principali (che comunque dovrebbero essere tutti già noti da corsi precedenti).

1.1 Il bus (senza DMA)

Il bus connette tra loro i vari dispositivi in modo che ciascuno possa comunicare con ciascun altro. La comunicazione deve avvenire sempre tramite operazioni di

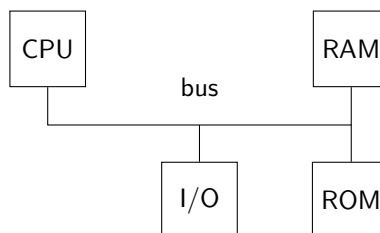


Figura 1: Architettura generale di un calcolatore moderno.

lettura o scrittura. Entrambi i tipi di operazione richiedono che venga specificato un “indirizzo”. Gli indirizzi sono normalmente dei numeri in sequenza. Il nome indirizzo fu scelto per ricordare gli indirizzi delle case. La metafora vuole che immaginiamo una lunghissima via in cui ci sono case in cui abitano numeri, ciascuna con il proprio indirizzo. L’operazione di lettura all’indirizzo x chiede chi abita all’indirizzo x . L’operazione di scrittura del numero y all’indirizzo x fa sì che ora y abiti all’indirizzo x (scacciando il precedente abitante). Si noti che sia gli indirizzi che gli abitanti sono numeri: è importante non confondere le due cose.

Ogni operazione è richiesta da un componente ed eseguita da un altro. La Fig. 1 mostra una architettura basata su un bus comune che permette a qualunque componente di richiedere una operazione a qualunque altro. Questo diventerà importante quando parleremo di DMA, ma per il momento le uniche combinazioni possibili sono le seguenti:

1. la CPU legge o scrive sulla memoria;
2. la CPU legge o scrive sull’I/O.

Le operazioni devono essere eseguite una per volta. In ogni operazione è l’indirizzo che permette di capire se è coinvolta la memoria o l’I/O. Il discriminare può essere operato riservando indirizzi diversi alla memoria e all’I/O, oppure definendo *spazi di indirizzamento* separati (vie diverse, nella metafora degli indirizzi delle case). Nel secondo caso, ogni operazione deve specificare anche lo spazio di indirizzamento coinvolto (memoria o I/O). Ogni operazione è vista da tutti i dispositivi collegati al bus e ciascuno di essi deve autonomamente capire se l’operazione è rivolta a lui oppure no. Per farlo deve confrontare l’indirizzo dell’operazione con gli indirizzi a lui riservati. Se l’operazione non è rivolta a lui, deve ignorarla. Se nessun dispositivo riconosce l’indirizzo possono succedere cose diverse in dipendenza dal tipo di bus: noi faremo l’ipotesi che le operazioni vengano comunque completate, con le scritture che non hanno effetto e le letture che restituiscono un valore casuale.

1.2 La memoria

Si tratta di un sistema organizzato in “celle”, ciascuna composta di un certo numero di bit che è uguale per tutte le celle. Una volta che la memoria è montata sul bus, ogni cella ha un indirizzo unico che la identifica. Nelle memorie moderne ogni cella ha la dimensione di un byte e la dimensione del byte è ormai universalmente fissata a 8 bit.

Per il resto, i seguenti punti continuano a essere veri:

- ogni cella contiene più precisamente una sequenza di bit; questa può essere sempre interpretata (da chi usa la memoria) come un numero naturale espresso in base due, ma il significato del contenuto di una cella dipende esclusivamente dall’uso che se ne fa;

- la memoria sa eseguire soltanto due tipi di operazioni: lettura e scrittura;
- sa eseguire una sola operazione per volta;
- per eseguire una lettura si deve specificare l'indirizzo della cella che si vuole leggere (la memoria risponde con il contenuto della cella);
- per eseguire una scrittura si deve specificare l'indirizzo della cella che si vuole scrivere, e il nuovo contenuto della cella (la memoria sostituisce il vecchio contenuto con il nuovo);
- la memoria è completamente passiva; non cambia il suo stato se non quando qualcuno ordina una operazione di scrittura;
- *tutte* le celle della memoria contengono *sempre* qualcosa, anche prima di eseguire qualunque scrittura: questo perché ogni bit è memorizzato da un dispositivo che può assumere solo uno tra due stati (0 o 1); all'accensione ci saranno zeri e uno a caso;
- ciascuna operazione richiede un tempo all'incirca costante, indipendentemente da quali altre operazioni sono state richieste precedentemente (memoria ad accesso casuale).
- nella memoria non c'è scritto cosa significhino i vari numeri, se rappresentino istruzioni, caratteri, o qualunque altra cosa.

1.3 I/O (senza interruzioni e DMA)

Anche se i dispositivi di ingresso/uscita sono tanti e vari, tutte le possibili interazioni sono trasformate in operazioni di lettura o scrittura. Queste operazioni coinvolgono particolari celle, dette registri o porte di I/O. Ogni dispositivo (tastiera, stampante, ...) sarà collegato al sistema tramite una *interfaccia*, all'interno della quale si trovano i registri. Indipendentemente dal dispositivo a cui appartiene, nel momento in cui l'interfaccia è montata sul bus ogni suo registro acquisisce un indirizzo che lo identifica univocamente, esattamente come le celle della memoria. Le operazioni di lettura e scrittura sui registri sono del tutto simili alle analoghe operazioni sulla memoria. La differenza è che ciascuna operazione può avere effetti collaterali, come causare la stampa di un carattere sulla carta di una stampante. Anche la sola *lettura* di un registro di I/O può avere effetti collaterali, come cambiare lo stato interno di una periferica. Per esempio, leggere il codice dell'ultimo tasto premuto dal registro di ingresso della tastiera fa sì che la tastiera smetta di memorizzare quel codice; la prossima volta che il registro verrà letto, la tastiera restituirà il codice del *nuovo* ultimo tasto premuto (se ve ne sono). Inoltre, a differenza della memoria, non è detto che le interfacce di I/O permettano accessi ai registri con dimensioni diverse da quelle del registro stesso: in genere un registro di n byte può essere letto (o scritto) solo con una operazione su n byte al suo indirizzo base.

Un'altra importante differenza è che i dispositivi di I/O possono cambiare il loro stato interno in seguito alla loro interazione con il mondo esterno al

calcolatore (per esempio, la tastiera memorizza il codice di un nuovo tasto se qualcuno lo preme). Ciò è completamente diverso dalla memoria, in cui il contenuto delle celle cambia solo se vi si scrive e due operazioni di lettura consecutive restituiranno sempre lo stesso valore. Dall'interno del calcolatore, il mutamento di stato di una periferica è visibile esclusivamente dal fatto che cambia il contenuto dei registri della sua interfaccia. Le interfacce, inoltre, (in assenza dei meccanismi di interruzione e DMA) aspettano passivamente che qualcuno legga il nuovo valore dei registri.

1.4 La CPU (senza interruzioni e protezione)

La CPU è un sistema che sa eseguire una sequenza di *istruzioni*. Le istruzioni operano sullo *stato* della CPU e/o interagiscono con l'esterno ordinando operazioni di lettura o scrittura sul bus. Lo stato della CPU è contenuto in un insieme di registri. I registri sono funzionalmente simili alle celle della memoria, con la differenza che si trovano all'interno della CPU. Le istruzioni sono codificate in numeri e contenute in memoria. Uno dei registri della CPU contiene l'indirizzo della prossima istruzione da eseguire (Instruction Pointer, IP). La CPU, da quando la accendiamo a quando la spegniamo, fa solo ed esclusivamente le seguenti cose:

1. preleva dalla memoria l'istruzione puntata dall'IP;
2. la decodifica, la esegue e aggiorna l'IP;
3. torna al punto 1.

Il modo in cui viene aggiornato l'IP al punto 2 dipende dall'istruzione eseguita. Per la maggior parte delle istruzioni, l'IP viene semplicemente incrementato in modo che punti all'istruzione che segue in memoria, secondo l'ordine degli indirizzi. Un programma, dunque, è per lo più una sequenza di azioni da eseguire una dopo l'altra (come la parola "programma" normalmente indica in altri contesti). Alcune istruzioni possono cambiare l'IP per eseguire dei salti. L'istruzione `hlt` ferma il processore (possiamo pensare che non modifichi l'IP).

Punti (tutti già noti) da tenere a mente:

1. tutto ciò che il processore fa, lo fa perché sta prelevando o eseguendo una istruzione;
2. tranne che quando esegue `hlt`, il processore non smette mai di eseguire istruzioni;
3. le istruzioni sono quelle del linguaggio macchina del processore; il processore non è in grado di eseguire nient'altro;
4. tutto quello che vede il processore è il suo stato corrente e l'istruzione da eseguire; il processore non ricorda le istruzioni passate e non si aspetta particolari istruzioni future:

- tutto ciò che resta di una istruzione alla fine della sua esecuzione è l'effetto che essa ha avuto sullo stato dei registri del processore, delle celle di memoria, o sui registri di I/O;
- dualmente, tutto ciò che una istruzione vede quando comincia la sua esecuzione è ciò che si trova nei registri del processore, nelle celle di memoria o nei registri di I/O.

1.5 La memoria ROM

Il fatto che, in un calcolatore moderno, l'ingresso/uscita non funziona direttamente ma deve essere comandato da un programma, crea un problema di *bootstrap*: come facciamo a caricare un programma in memoria se l'unica via di ingresso ha essa stessa bisogno che ci sia un programma in memoria per poter funzionare? La soluzione è di avere una memoria non volatile che contenga già un programma, fisso, che ha lo scopo di caricare il programma vero e proprio da qualche periferica di I/O (hard disk, rete, DVD, etc.).

1.6 Il flusso di controllo (in assenza di interruzioni)

Quanto esposto sopra è tutto ciò che l'hardware sa fare. Tutto ciò che un calcolatore fa, per quanto complicato possa apparire, deve ridursi a questo. È il software (il programma contenuto in memoria ed eseguito dalla CPU) a orchestrare questi comportamenti elementari in modo da ottenerne di più complessi.

Tutta l'architettura esiste solo per eseguire il software, ed è il software che comanda. La CPU è sotto il controllo del software: la CPU deve eseguire l'istruzione corrente (quella il cui indirizzo si trova nel suo instruction pointer), ed è inoltre l'istruzione stessa a dire quale deve essere l'istruzione successiva. Le istruzioni che non sono di salto lo dicono implicitamente (l'istruzione successiva è la prossima in memoria), mentre quelle di salto lo dicono esplicitamente. L'unico altro meccanismo che dice al processore qual è l'istruzione successiva è quello delle interruzioni, che per il momento ignoriamo.

Il controllo “fluisce” dunque da una istruzione all'altra come deciso dal software stesso. Se il software è composto di più parti, come per esempio più subroutine o diverse applicazioni o librerie, è sempre una sola di queste parti per volta che ha il controllo del processore. Il controllo può essere solo “palleggiato”: quando una subroutine S_1 ne invoca un'altra S_2 (per es. tramite una istruzione **call**) si dice che S_1 *trasferisce il controllo* a S_2 . A questo punto il processore obbedisce a S_2 e non più a S_1 , fino a quando S_2 non deciderà di *restituire il controllo* a S_1 (per es. eseguendo una istruzione **ret**).

1.7 Hardware e software

Alcuni trovano difficoltà nel capire se una certa cosa è fatta in hardware o in software, e in generale a capire “chi” svolge determinate azioni.

Per orientarsi può essere utile tenere sempre a mente cosa i vari componenti *sanno*. In particolare, si rifletta sul fatto che la CPU sa solo ciò che è contenuto nei suoi registri e, in particolare, vede del programma solo l'istruzione che di volta in volta sta eseguendo, senza ricordare le istruzioni passate e senza guardare quali siano le istruzioni future (che pure sono già scritte in memoria). Questo limita fortemente le cose che la CPU può fare. Nello scenario di S_1 che trasferisce il controllo a S_2 , per esempio, la CPU non può controllare che S_2 ritorni a S_1 : non sa cosa S_2 stia facendo e, con la visione limitata di una istruzione alla volta, non è in grado di capirlo. La CPU di cui ci occupiamo noi, per di più, non sa nemmeno che nel passato era stata eseguita una **call** e che dunque prima o poi deve essere eseguita una **ret**.¹

Non si deve però giungere alla conclusione che dunque tutto è fatto in software. Quando si ritiene che una certa operazione x sia fatta in software, si deve essere in grado di rispondere alle seguenti domande:

- se x è fatta in software, vuol dire che da qualche parte in memoria ci deve essere un programma p (anche di una sola istruzione) che fa x ; so scrivere questo programma?
- il software non ha effetto se la CPU non lo esegue; come fa il flusso di controllo a passare da p nel momento in cui serve fare x ?

Se anche una sola di queste domande risulta assurda, è probabile che x non sia fatta in software.

¹Verso la fine del corso vedremo una CPU più intelligente che cerca di ricordare e prevedere più cose, ma lo fa al solo scopo di andare più veloce, restando per il resto equivalente a quella stupida.

Indirizzi e oggetti

G. Lettieri

29 Febbraio 2024

Gli indirizzi sono relativi ad un bus: tutti i componenti collegati al bus, in grado di rispondere a richieste di lettura o scrittura, devono avere degli indirizzi assegnati in modo univoco. I possibili indirizzi vanno praticamente sempre da 0 fino ad un massimo della forma $2^n - 1$ per qualche n che dipende dal bus. Questo perché gli indirizzi viaggiano su un numero prefissato di piedini, fili o tracce, ciascuna delle quali può assumere solo i valori 0 o 1. Se queste tracce sono n , il numero totale di indirizzi possibili è dunque 2^n .

Gli indirizzi esistono sempre tutti, nel senso che è sempre possibile richiedere una lettura o una scrittura a qualunque indirizzo, anche se l'indirizzo non è assegnato a nessun componente (il risultato di una tale richiesta dipende dal bus; come abbiamo detto, assumiamo per ora che le scritture non abbiano effetto e le letture restituiscano un valore casuale).

Per questi motivi, il modo più naturale di rappresentare gli indirizzi di un bus è come la sequenza di tutti i numeri binari, senza segno, su n bit. Conviene acquisire familiarità con le rappresentazioni dei numeri in base 16 e 8, in quanto queste basi permettono di rappresentare gli indirizzi in forma molto più compatta e, allo stesso tempo, facilmente convertibile da e verso la base 2. Alcuni numeri molto frequenti devono subito richiamare alla mente la loro rappresentazione nelle varie basi: in base 2, 2^a è un 1 seguito da a zeri mentre $2^a - 1$ è composto da a 1. Se a è un multiplo di 4, allora 2^a in base 16 è 1 seguito da $a/4$ zeri, mentre $2^a - 1$ è rappresentato come $a/4$ cifre F (questo perché ogni cifra esadecimale corrisponde a 4 cifre binarie). Similmente per la base 8: se a è multiplo di 3, allora 2^a è 1 seguito a $a/3$ zeri e $2^a - 1$ è composto da $a/3$ cifre 7. In generale, conviene usare queste basi ogni volta che dobbiamo ragionare sulla struttura binaria di un qualche valore. È bene familiarizzarsi con i seguenti casi notevoli:

- un byte corrisponde a 2 cifre esadecimali;
- 512 corrisponde a 1000 in base 8;
- 4Ki corrisponde a 1000 in base 16 e 1.0000 in base 8;
- 1 Mi corrisponde a 10.0000 in base 16;
- 4 Gi corrisponde a 1.0000.0000 in base 16.

Le operazioni sugli indirizzi (come aggiungere una costante a un indirizzo, o sottrarre due indirizzi) vanno sempre pensate come operazioni modulo 2^n . Gli indirizzi hanno un ordinamento naturale, in cui l'indirizzo successivo di un indirizzo x è $x + 1$ e il precedente è $x - 1$. Si noti però che, dal momento che le operazioni vanno intese modulo 2^n , l'indirizzo che precede l'indirizzo 0 è l'indirizzo $2^n - 1$ e l'indirizzo successivo a $2^n - 1$ è l'indirizzo zero.

1 Scostamenti (*offset*)

Dati due indirizzi x e y su n bit, possiamo chiederci quanto y si discosta da x calcolando il valore $y - x$ modulo 2^n , detto *offset* di y rispetto a x .

In ogni caso, l'offset conta il numero di indirizzi che è necessario “saltare”, partendo da x , per raggiungere y nell'ordine naturale. In particolare, l'offset di x rispetto a se stesso è zero. Gli offset possono essere anche negativi: il loro valore assoluto rappresenta comunque il numero di indirizzi da saltare partendo da x per raggiungere y , ma andando nella direzione degli indirizzi decrescenti. Per esempio, l'offset -1 rappresenta l'indirizzo che precede x (nel caso $x = 0$ si ricordi che l'indirizzo precedente è $2^n - 1$).

1.1 Caso particolare: offset in complemento a 2 su n bit

Se rappresentiamo gli offset come numeri in complemento a 2 su n bit (cioè lo stesso numero di bit dello spazio di indirizzamento), diventa indifferente considerarli con o senza segno. Facciamo un esempio con $n = 4$. Gli indirizzi vanno dunque da 0 a 15. L'offset -1 si rappresenta come 1111 in complemento a 2 su 4 bit. Prendiamo $x = 3$. Se interpretiamo 1111 come numero con segno, l'offset è -1 e saltando un indirizzo all'indietro arriviamo a $y = 2$. Se ora interpretiamo l'offset 1111 come il numero senza segno 15, dobbiamo saltare 15 indirizzi in avanti partendo da $x = 3$. Dopo averne saltati 12 arriviamo all'indirizzo 15, saltandone un altro ripartiamo dall'indirizzo 0 e con gli ultimi due salti arriviamo a $y = 2$, come prima.

Se, però, l'offset è rappresentato su un numero di bit inferiore a n , diventa importante sapere se la rappresentazione è con segno o senza, in quanto l'offset va esteso a n bit prima di poterlo sommare all'indirizzo, e l'estensione va realizzata in modo diverso a seconda della rappresentazione. In particolare, gli offset di 4 byte (o un byte) che si trovano all'interno delle istruzioni AMD64 usano la rappresentazione in complemento a 2 *con* segno.

2 Intervalli (*range*)

Un *intervallo* è una sequenza di indirizzi. Ci sono vari modi di rappresentare un intervallo, e purtroppo nessuno è privo di difetti se siamo vincolati ad usare numeri rappresentati su n bit. Noi adotteremo la convenzione di rappresentare gli intervalli specificando il primo indirizzo che ne fa parte, sia x , e il primo,

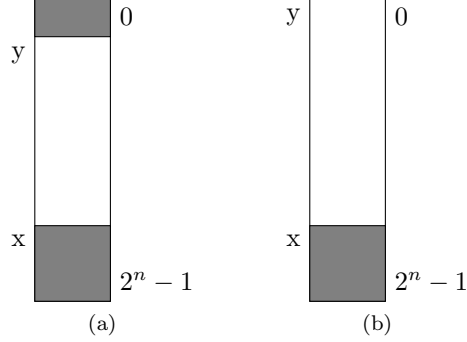


Figura 1: (a) Intervallo con wrap-around. (b) Intervallo che non fa wrap-around, ma comprende l'ultimo indirizzo.

successivo a x , che *non* ne fa parte, sia y . Diremo che x è la *base* dell'intervallo, mentre y ne è il *limite*.

Matematicamente, se x e y sono (per esempio) numeri naturali, un tale intervallo viene definito come

$$[x, y) = \{ i \mid x \leq i < y \}. \quad (1)$$

Escludendo il caso $y < x$ (privo di interesse), questa definizione ha il pregio che $l = y - x$ è il numero di elementi dell'intervallo (*lunghezza*). Viceversa, se di un intervallo conosciamo la base x e la lunghezza l , l'intervallo è semplicemente $[x, x+l)$. Questo è un vantaggio rispetto ad una rappresentazione come $[x, y]$ che include il limite nell'intervallo e richiede di aggiungere 1 alla prima espressione e di sottrarre uno nella seconda. La rappresentazione $[x, y)$, viceversa, ha come difetto che l'ultimo elemento dell'intervallo (assumendo che non sia vuoto) è $y - 1$ e non y .

I problemi principali, però, derivano dal fatto che nel nostro caso x e y non sono numeri naturali, ma numeri rappresentati su n bit, con wrap-around (matematicamente, sono naturali modulo 2^n). Per fortuna le espressioni $l = y - x$ e $[x, x+l)$ valgono ancora, purché si intendano la somma e la sottrazione modulo 2^n . Purtroppo, però, la semplice definizione (1) fallisce con intervalli come quello in Figura 1(a). In questo caso, infatti, abbiamo $y < x$ e secondo la definizione (1) l'intervallo dovrebbe essere vuoto, quando chiaramente non lo è. Per non complicare la definizione¹ eviteremo sempre questo tipo di intervalli richiedendo che sia sempre $x \leq y$, ma ci resta un caso particolare da considerare: un intervallo come quello di Figura 1(b). Pur non facendo wrap-around, un intervallo del genere ha comunque $y = 0 < x$ e dunque è vuoto stando alla

¹Una definizione che non ha problemi con gli intervalli di Figura 1 è

$$[x, y) = \{ i \mid o(x, i) < o(x, y) \} \quad (2)$$

Dove $o(a, b)$ è $b - a$ modulo 2^n , cioè l'offset tra a e b .

definizione (1). Per ammettere anche questi ultimi intervalli accettiamo anche la forma $[x, 0)$ con $x \neq 0$ come valida, con l'accordo che in questo caso il limite è in realtà il numero naturale 2^n che non possiamo rappresentare.

3 Allineamenti

Diamo un po' di definizioni:

- si dice che un *indirizzo* è *allineato a 2^b* se è multiplo di 2^b ;
- si dice che un *intervallo* è *allineato a 2^b* se la sua base è allineata a 2^b ;
- se i è un intervallo la cui lunghezza è una potenza di 2, si dice che i è *allineato naturalmente* se è allineato alla sua lunghezza.

Un indirizzo allineato a 2^b ha i b bit meno significativi della propria rappresentazione binaria tutti nulli. Notare che se un indirizzo è allineato a 2^b , allora è anche allineato a $2^{b'}$ con $b' < b$ (se ha b bit meno significativi a zero, a maggior ragione ne ha $b' < b$ a zero). L'indirizzo 0 è allineato a 2^b per qualunque b .

4 Confini (*boundaries*) e regioni naturali

Dato un qualunque $b \leq n$, tutti gli indirizzi allineati a 2^b sono detti *confini* di 2^b . Tutti i confini di 2^b si trovano partendo da 0 e procedendo ad offset successivi di 2^b . Questi confini delimitano degli intervalli di indirizzi, allineati naturalmente, che chiamiamo *regioni naturali* di 2^b . Ometteremo l'aggettivo "naturali" quando non si crea ambiguità. Useremo il termine regione (naturale) in modo generico, ma in molti casi introdurremo dei nomi particolari per regioni di particolare interesse (per esempio *riga*, *cacheline*, *pagina*, ...). Le regioni sono in totale 2^{n-b} , o 2^a se poniamo $a = n - b$ (Figura 2).

Possiamo assegnare ad ogni regione un numero progressivo r compreso tra 0 e $2^a - 1$. Il numero r , detto *numero di regione*, serve ad identificare univocamente ogni regione. Si noti che tutti (e soli) gli indirizzi che fanno parte della stessa regione contengono negli a bit più significativi proprio il numero della regione a cui appartengono. Invece, i b bit meno significativi contengono l'offset dell'indirizzo rispetto alla base della regione (il confine), detto *offset all'interno della regione*. Ogni indirizzo è dunque identificato dal numero di regione e dal suo offset (all'interno della regione). Ovviamente, lo stesso indirizzo può essere identificato in modi diversi in base alla dimensione della regione scelta.

Dato un numero b e un indirizzo x su n bit è dunque immediato, in hardware, trovare il suo numero di regione (di 2^b) e il suo offset: gli $a = n - b$ bit più significativi x danno il numero di regione, e i b bit meno significativi danno l'offset. Supponiamo ora di voler fare la stessa cosa in software, supponendo di avere una variabile **unsigned long** x che contiene un indirizzo, e di conoscere b . Prendiamo come n la dimensione in bit di un **unsigned long**, che nel nostro caso è 64. Per ottenere il numero di regione possiamo scrivere semplicemente

	$a = n - b$	b	
confine	00...00	00...00	} 2^b
	00...00	00...01	
regione $r = 0$	00...00	00...10	
	$\vdots$	$\vdots$	
	00...00	11...11	
confine	00...01	00...00	} 2^b
	00...01	00...01	
regione $r = 1$	00...01	00...10	
	$\vdots$	$\vdots$	
	00...01	11...11	
confine	00...10	00...00	} 2^b
	00...10	00...01	
regione $r = 2$	00...10	00...10	
	$\vdots$	$\vdots$	
	00...10	11...11	
	$\vdots$	$\vdots$	
confine	11...11	00...00	} 2^b
	11...11	00...01	
regione $r = 2^a - 1$	11...11	00...10	
	$\vdots$	$\vdots$	
	11...11	11...11	

Figura 2: Scomposizione degli indirizzi in regioni naturali di 2^b .

```
r = x >> b; // numero di regione
```

Per ottenere l'offset usiamo l'AND bit a bit con una maschera che ha b cifre meno significative ad 1 e tutte le altre a zero:

```
o = x & ((1UL << b) - 1); // offset
```

I caratteri UL che seguono la costante 1 servono a specificarne il tipo come Unsigned Long. Si ricordi che per default le costanti numeriche del C++ hanno tipo **int** se, come in questo caso, sono rappresentabili su un intero.

Vediamo anche come si ottiene la base della regione a cui x appartiene. Possiamo ottenere questo indirizzo con una maschera che ha a bit significativi a 1 e gli altri a 0. Questa maschera non è altro che il NOT della maschera che abbiamo usato prima:

```
c = x & ~(1UL << b) - 1; // confine
```

Dato un intervallo non vuoto $[x, y)$, vogliamo spesso sapere quali sono la prima regione toccata dall'intervallo e la prima regione *non toccata* dall'intervallo (con questo intendiamo la prima regione che si incontra dopo la prima toccata e che non contiene indirizzi dell'intervallo).

La prima regione toccata è semplicemente la regione in cui cade x , ma per la prima regione non toccata bisogna fare più attenzione. Si noti che questa non è sempre la regione successiva a quella in cui si trova y , perché y non fa parte dell'intervallo $[x, y)$: se y cade su un confine, la regione in cui si trova y è lei stessa la prima non toccata. Possiamo operare nel modo seguente:

```
nt = (y + (1UL << b) - 1) >> b.
```

In altre parole, sommiamo prima a y il valore $2^b - 1$: se y cade su un confine, $y + 2^b - 1$ è ancora nella stessa regione di y ; se y non cade su un confine, $y + 2^b - 1$ si trova nella regione successiva. Un altro metodo consiste nel sommare 1 alla regione in cui cade $y - 1$:

```
nt = ((y - 1) >> b) + 1,
```

ma bisogna stare attenti al wrap-around quando $y = 0$: il numero di regione deve stare su $n - b$ bit, quindi `nt` andrebbe poi mascherato con una maschera di $n - b$ bit a 1.

Calcolatori Elettronici: la memoria centrale

G. Lettieri

1 Marzo 2024

Nell'architettura moderna ogni singolo byte della memoria ha il proprio indirizzo e le istruzioni del processore permettono di accedere al singolo byte. Per esempio, nel processore Intel, `mov %al, (%rbp)` permette di sovrascrivere il singolo byte all'indirizzo contenuto in `rbp`, senza modificare nessun altro byte. Il sottosistema di memoria, quindi, deve essere in grado di supportare questo tipo di operazioni. Allo stesso tempo, però, il processore possiede istruzioni che leggono e scrivono in unità più grandi: l'istruzione `mov %rax, (%rbp)` scrive 8 byte all'indirizzo contenuto in `rbp`. Vorremmo che il sottosistema di memoria fosse in grado di supportare in maniera efficiente tutti i tipi di accessi che il processore supporta: al byte, alla parola (2 byte, nella terminologia Intel/AMD64), doppia parola (4 byte) o parola quadrupla (8 byte).

Il byte rappresenta l'unità di indirizzamento: non è possibile leggere o scrivere una unità più piccola di un byte in una singola operazione di lettura o scrittura. Se, per esempio, è necessario modificare un singolo bit all'interno di un byte, è necessario leggere l'intero byte, modificare il bit lasciando gli altri inalterati, quindi riscrivere il nuovo byte.

Quando si passa a oggetti di più byte, la prima cosa da chiedersi è che in che ordine i byte che compongono l'oggetto si trovino in memoria. Architetture diverse possono utilizzare ordini diversi. Nel nostro caso l'architettura usa l'ordinamento detto *little endian*: il byte meno significativo si trova all'indirizzo più piccolo, seguito dagli altri byte in ordine di significatività. Per esempio, supponiamo di avere una parola quadrupla in memoria, a partire da un indirizzo i , che memorizza il numero esadecimale 0x1122334455667788. Il byte di indirizzo i conterrà 0x88, il byte di indirizzo $i + 1$ conterrà 0x77, e così via fino al byte di indirizzo $i + 7$ che conterrà 0x11. Un altro ordinamento molto utilizzato (per esempio da noi stessi quando scriviamo i numeri su un foglio) è quello detto *big endian*¹, che consiste nello scrivere il numero partendo dalla parte più significativa. In Fig. 1 abbiamo ordinato i byte di ogni riga da destra verso sinistra proprio per ovviare alla differenza tra il modo in cui l'architettura AMD64 memorizza le parole e il modo in cui noi siamo abituati a leggerle.

Possiamo realizzare un modulo di memoria che possa anche leggere o scrivere più di un byte in una singola operazione e nello stesso tempo impiegato per un

¹I nomi *little endian* e *big endian* vengono indirettamente da *I viaggi di Gulliver* di Jonathan Swift, per il tramite di un famoso articolo del 1980 reperibile a questo indirizzo: <https://www.ietf.org/rfc/rfc137.txt>

numero di riga	+7	+6	+5	+4	+3	+2	+1	+0	indirizzo di riga
0									0
1									8
2									16
3									24
				...					
$2^{61} - 1$									$2^{64} - 8$

Figura 1: Organizzazione dello spazio di memoria su righe di 64 bit. Ogni casella rappresenta un byte. I numeri +0, +1, ... sono gli offset all'interno della riga.

singolo byte. Nel nostro caso il modulo può leggere o scrivere una intera parola di 64 bit (8 byte contigui) purché questa sia *allineata naturalmente*. Il nostro modulo di memoria può anche leggere o scrivere, in una sola operazione, una parola di 32 bit (4 byte) o una parola di 16 bit (2 byte), purché queste non attraversino i confini di 8 byte (cioè siano interamente contenute in una regione naturale di 8 byte).

Per visualizzare più facilmente queste limitazioni conviene rappresentare lo spazio di indirizzamento di memoria non come una sequenza di indirizzi di byte, ma come una sequenza di *righe* o *linee* di 8 indirizzi di byte (il numero massimo di byte che può essere trasferito in una singola operazione da un modulo di memoria). Si veda la Fig. 1, dove le righe sono disposte in orizzontale. In questo modo i dati accessibili in un'unica operazione saranno sempre contenuti in una singola riga. In pratica adottiamo una scomposizione degli indirizzi in regioni di 2^3 e disegniamo le regioni in orizzontale.

Si noti che per avere le parole da 2 byte e le parole da 4 byte sempre contenute in una riga è sufficiente che anch'esse siano allineate naturalmente, ma non è necessario. Per esempio, una parola da 4 byte che inizi in una riga alla colonna +3 sarebbe interamente contenuta nella riga, pur non essendo allineata naturalmente (per essere allineata naturalmente dovrebbe iniziare alla colonna +0 o alla colonna +4).

Nel seguito assumeremo che gli indirizzi della CPU e del BUS siano su n bit. Non assumiamo che n sia 64, in quanto questo è solo il valore massimo teorico dell'architettura Intel/AMD64. I processori di questa famiglia usciti fino ad ora hanno, di fatto, un numero inferiore di bit di indirizzo (per esempio, 36) e altre considerazioni, che vedremo, limitano il massimo numero di bit dei processori futuri della stessa famiglia a un valore comunque inferiore a 64.

Per specificare completamente una richiesta di operazione di lettura la CPU deve presentare alla memoria l'indirizzo del primo byte e il numero di byte (per una operazione di scrittura dovremo anche presentare il nuovo contenuto dei byte in questione). Queste due informazioni vengono specificate in modo indiretto nel seguente modo:

1. la CPU scompone l'indirizzo del primo byte in numero di riga e offset all'interno della riga;

2. la CPU e il bus prevedono $n-3$ fili, che chiamiamo $A\{n-1\}$ – $A3$, destinati a trasportare soltanto il numero di riga;
3. al posto dell'offset del primo byte (che sarebbe stato contenuto nei fili $A2$, $A1$ e $A0$) e del numero di byte sono previsti 8 fili di *byte enable*, uno per ogni byte della riga selezionata da $A\{n-1\}$ – $A3$.

Lo scopo delle linee di byte enable, che chiamiamo $/BE7$ – $/BE0$, è di selezionare singolarmente i byte della riga che la CPU intende leggere (o scrivere) nell'operazione.

Esempi:

- Supponiamo che la CPU stia eseguendo una istruzione **movq** 512, **%rax**. Deve dunque ordinare una operazione di lettura di una parola da 8 byte all'indirizzo 512. Si tratta della riga n. $512/8 = 64$, che va dagli indirizzi 512 a 519. Avremo $A\{n-1\} = A\{n-2\} = \dots = A10 = 0$, $A9 = 1$, $A8 = A7 = \dots = A3 = 0$, in quanto 512 è 1000000000 in binario, e $/BE7 = /BE6 = \dots = /BE0 = 0$ (tutti i byte abilitati);
- Supponiamo invece che l'istruzione sia **movl** 512, **%eax**. Si tratta ora di una lettura di una parola da 4 byte all'indirizzo 512: avremo $A\{n-1\}$ – $A3$ come prima, ma $/BE7 = /BE6 = /BE5 = /BE4 = 1$ (byte 7–4 disabilitati) e $/BE3 = /BE2 = /BE1 = /BE0 = 0$ (byte 3–0 abilitati);
- Consideriamo ora **movl** 516, **%eax**. Ora la lettura coinvolge una parola da 4 byte all'indirizzo 516: avremo $A\{n-1\}$ – $A3$ ancora come prima, in quanto siamo sempre all'interno della stessa riga. I byte enable saranno però diversi: $/BE7 = /BE6 = /BE5 = /BE4 = 0$ (byte 7–4 abilitati) e $/BE3 = /BE2 = /BE1 = /BE0 = 1$ (byte 3–0 disabilitati).

Si noti che questo tipo di codifica permetterebbe di specificare molti più casi di quelli che ci servono. Per esempio, con $/BE7 = /BE5 = /BE3 = /BE1 = 1$ e $/BE6 = /BE4 = /BE2 = /BE0 = 0$ potremmo leggere solo i byte di indirizzo pari all'interno della riga selezionata. Di fatto tali possibilità non sono sfruttate e sono utilizzate solo le combinazioni che corrispondono a byte contigui.

1 Collegamento al bus

Consideriamo ora un modulo di memoria da 2^k byte, per esempio $k = 30$ per una memoria di 1 GiB. Possiamo realizzare le funzionalità che ci servono se costruiamo il modulo usando 8 dispositivi di memoria, ciascuno capace di memorizzare $2^k/8 = 2^{k-3}$ byte (nell'esempio precedente, con $k = 30$, ci servono 8 moduli da 128 MiB). I dispositivi devono essere collegati come in Fig. 2. Ciascuna riga di Fig. 1 è memorizzata utilizzando tutti e 8 i dispositivi: il byte della colonna “+0” sarà memorizzato nel dispositivo contrassegnato con 0, il byte “+1” nel dispositivo 1, e così via. Ogni dispositivo memorizza una intera colonna di Fig. 1. Si noti come la presenza dei byte enable semplifichi l'implementazione:

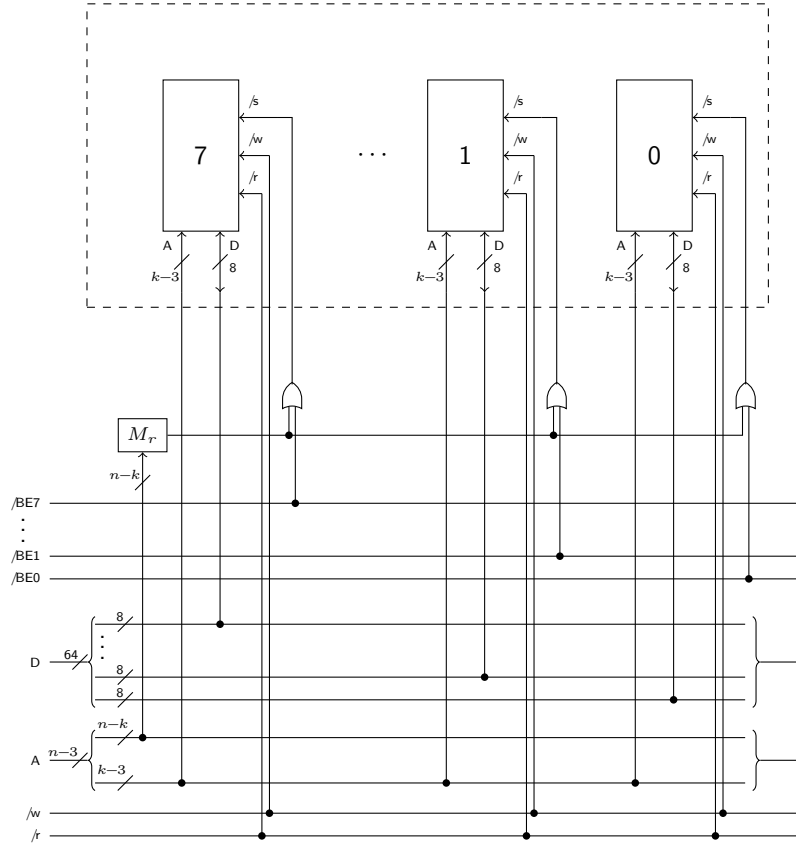


Figura 2: Realizzazione di una memoria organizzata a linee di 64 bit (parole quadruple), ma accessibile anche a byte, parole e doppie parole. Le parti dentro e fuori del rettangolo tratteggiato possono essere realizzate separatamente e poi collegate. La parte interna è una scheda di memoria, la parte esterna è il bus.

ogni byte enable contribuisce a generare il *chip select* del chip che contiene il corrispondente byte.

Il nostro modulo di memoria convive nello spazio di indirizzamento di memoria insieme ad altri dispositivi (altri moduli di memoria, oppure anche interfacce di I/O con registri mappati in memoria). Dobbiamo assegnargli un intervallo di indirizzi e fare in modo che risponda solo alle richieste di lettura e scrittura che ricadono in quell'intervallo. Per farlo conviene scegliere una regione naturale di 2^k byte e fare in modo che la nostra memoria la occupi interamente (facciamo in modo, cioè, che il nostro modulo sia allineato naturalmente all'interno dello spazio di indirizzamento). Sia r il numero della regione grande 2^k che abbiamo scelto. Data una operazione di lettura o scrittura ad un certo numero di riga i , la nostra memoria deve sapere se ignorarla o no in base al fatto che il numero di riga i appartenga o meno alla regione r . Abbiamo già visto come fare a vedere se un indirizzo su n bit appartiene o no ad una regione grande 2^k : è sufficiente guardare gli $n - k$ bit più significativi dell'indirizzo. In questo caso non abbiamo tutto l'indirizzo (di byte) ma solo il numero di riga. Questo non è un problema: ci mancano solo i 3 bit meno significativi dell'indirizzo di byte, e questi sicuramente rientrano nei k che già dovevamo ignorare. Un altro modo per visualizzare l'operazione è di riportare tutto alle righe: una regione di 2^k byte è anche una regione di 2^{k-3} righe. I numeri di regione restano gli stessi che per i byte e per sapere se una riga appartiene o no alla regione scelta è sufficiente ignorare $k - 3$ bit meno significativi del numero di riga e confrontare gli altri con il numero di regione r . I $k - 3$ bit meno significativi, invece, sono l'offset *della riga* all'interno della regione. Questi possono essere usati per selezionare il byte corretto in ciascun chip di RAM.

Il circuito risultante è quello di Figura 2. La maschera M_r serve ad abilitare o disabilitare complessivamente tutta la scheda di memoria.

La parte interna al rettangolo tratteggiato in Fig. 2 rappresenta la scheda di memoria vera e propria, mentre la parte esterna rappresenta i circuiti del bus a cui la scheda è collegata. Il bus può proseguire a destra e sinistra e avere altre maschere che riconoscono valori di r diversi. Tipicamente avremo una "scheda madre" che prevede un certo numero di slot in cui si possono inserire le schede di memoria. Ogni slot avrà una maschera diversa. Le schede di memoria possono invece essere tutte uguali tra loro ed essere montate in un qualunque slot; più schede possono essere montate contemporaneamente sul bus, ciascuna ovviamente inserita in uno slot diverso.

2 Accessi non allineati

I processori AMD/Intel a 64 bit permettono anche accessi non allineati. Per esempio, è possibile leggere con una sola istruzione una parola quadrupla che inizi ad un indirizzo che non è multiplo di 8:

```
mov 4097, %rax
```

numero di riga	+7	+6	+5	+4	+3	+2	+1	+0	indirizzo di riga
				...					
511									4088
512	22	33	44	55	66	77	88		4096
513								11	4104
514									4112
				...					

Figura 3: Accesso non allineato.

Supponiamo che all'indirizzo 4097 sia memorizzato il numero 1122334455667788 (base 16). I byte che compongono il numero si troveranno nelle posizioni indicate in Figura 3. In questo caso il processore eseguirà due accessi in memoria: uno alla riga numero 512 (4096/8) con tutti i byte enable attivi, tranne /BE0; un altro alla riga successiva (513), con tutti i byte enable *non* attivi, tranne /BE0. In questo modo riesce a recuperare tutti i byte che compongono il numero, ma non basta: la prima lettura porterà i byte 22–88 in un registro interno del processore, ma in una posizione che è traslata di 8 bit a sinistra rispetto a quella desiderata; la seconda lettura porterà il byte 11 nella posizione meno significativa del registro interno, invece che in quella più significativa. Il processore, automaticamente, provvederà ad eseguire i necessari shift in modo che **rax**, alla fine, contenga il valore corretto.

Si noti come il soggetto di tutte queste azioni è *il processore*: il software contiene solo l'istruzione “`mov 4097, %rax`” e non menziona in alcun modo le due letture e gli shift. Queste sono azioni svolte dal processore per eseguire correttamente l'istruzione richiesta.

Da quanto abbiamo descritto, possiamo capire che gli accessi non allineati comportano un costo in termini di tempo, e richiedono hardware aggiuntivo nel processore. Alcuni tipi di processori (per es., quelli di ARM) non contengono questo hardware e non ammettono accessi non allineati.

3 Allineamento dei dati in C++

Anche nei processori che possiedono l'hardware per l'accesso non allineato, come quelli Intel/AMD64, ha senso cercare di non portarsi mai nella situazione in cui occorrono accessi disallineati, per non pagarne il costo in termini di tempo. Per questo motivo, l'ABI (Application Binary Interface) di un sistema definisce delle regole di allineamento che il compilatore deve rispettare. Nel nostro caso valgono le regole dell'ABI System V, che è quella adottata da Linux². L'ABI stabilisce quale deve essere il risultato degli operatori **sizeof** e **alignof** per ogni tipo, sia base che derivato.

²https://calcolatori.iet.unipi.it/deep/psABI-x86_64.pdf

tipo	sizeof	alignof
char	1	1
short	2	2
int	4	4
long	8	8

Tabella 1: Allineamenti dei tipi base nell'ABI System V.

```

struct S1 {
    char c;
    int i;
    long l;
};
(a)

struct S2 {
    char c;
    int i;
    long l;
    char c2;
};
(b)

struct S3 {
    char c;
    char c2;
    int i;
    long l;
};
(c)

```

Figura 4: Tre esempi di strutture.

Per i tipi base che ci interessano vale la Tabella 1. Come si vede, tutti i tipi sono allineati naturalmente. La tabella non riporta i corrispondenti tipi **unsigned**, perché non c'è differenza con i tipi **signed**.

Più in generale, l'ABI garantisce che, per ogni tipo t , sia base che derivato, **sizeof**(t) sia un *multiplo* di **alignof**(t). L'utilità di ciò è semplice da vedere nel caso degli array. Se abbiamo un array tipo $a[\text{dim}]$, allora **alignof**(a) è uguale a **alignof**(tipo) (gli array hanno lo stesso allineamento dei loro elementi). Se **sizeof**(tipo) è un multiplo di **alignof**(tipo), possiamo disporre in memoria gli elementi $a[0]$, $a[1]$, ..., $a[\text{dim}-1]$ semplicemente uno dopo l'altro, perché ciascuno di loro rispetterà il proprio allineamento e sarà possibile calcolare l'indirizzo dell'elemento i -esimo come $a + i \times \text{sizeof}(\text{tipo})$. Inoltre, **sizeof**(a) sarà semplicemente $\text{dim} \times \text{sizeof}(a)$, rispettando dunque il vincolo anche per l'array nella sua interezza.

Passiamo ora alle strutture. Calcolare l'allineamento è semplice: una struttura ha per allineamento il *massimo* degli allineamenti dei suoi membri. Calcolare la dimensione, invece, è più complicato, perché bisogna garantire che *ogni* membro della struttura rispetti il proprio allineamento, e che i membri siano disposti in memoria nell'ordine in cui il programmatore li ha dichiarati. Questo comporta che ogni tanto si debbano saltare dei byte (*internal padding*).

Consideriamo, per esempio, la struttura S1 definita in Figura 4(a). Il suo allineamento è 8, per via del campo `l`. Per calcolare la sua dimensione dobbiamo disegnarne la *layout* in memoria, assumendo di partire da un indirizzo allineato a 8. Il risultato si può vedere in Figura 5. Il primo campo da allocare dentro la struttura è `c`, che ha allineamento uno e dunque occupa il primo byte. Il secondo campo da allocare è `i`, che ha allineamento 4. La prima posizione disponibile subito dopo `c`, però, non ha un indirizzo multiplo di 4, quindi bi-

i				-	-	-	c
1							

Figura 5: Layout di memoria della struttura S1 di Figura 4(a).

i				-	-	-	c
1							
-	-	-	-	-	-	-	c2

Figura 6: Layout di memoria della struttura S2 di Figura 4(b).

sogna saltare un numero sufficiente di byte, fino ad arrivare al primo indirizzo allineato a 4: in questo caso è necessario saltare 3 byte. Questi byte concorrono a definire la dimensione della struttura, ma non sono utilizzati in alcun modo. Dopo aver allocato `i` si procede ad allocare `1`, che ha allineamento 8. In questo caso la prossima posizione è allineata a 8, e quindi `1` può occuparla. A questo punto possiamo contare i byte che abbiamo usato (inclusi quelli saltati): sono 16. Siccome 16 è multiplo di `alignof(S1)`, che avevamo detto essere 8, `sizeof(S1)` è effettivamente 16.

Consideriamo ora la struttura S2 di Figura 4(b). Rispetto alla struttura S1 abbiamo aggiunto un unico **char** `c2` in fondo. L'allineamento di S2 è ovviamente sempre 8. Il layout della struttura è illustrato in Figura 6. Notare come `c2` deve essere allocato per ultimo, perché questo è l'ordine stabilito dal programmatore (colui che ha definito S2). Quindi `c2` deve finire su una terza riga. Inoltre, la dimensione di S2 non può essere 17, perché 17 non è multiplo di 8: dobbiamo sprecare ulteriori 7 byte in modo che `sizeof(S2)` diventi 24.

È importante ribadire che queste sono *regole* stabilite dall'ABI, non suggerimenti su come rappresentare le strutture: non seguire le regole comporta incompatibilità con il compilatore e con il sistema operativo. Se il programmatore è interessato a minimizzare lo spreco di byte, deve lui stesso pensare a definire le proprie strutture in maniera opportuna. Per esempio, la struttura S3 di Figura 4(c) contiene le stesse informazioni della struttura S2, ma il suo layout in memoria è quello illustrato in Figura 7, in cui si vede che solo 2 byte sono andati sprecati.

i				-	-	c2	c
1							

Figura 7: Layout di memoria della struttura S3 di Figura 4(c).

Memoria Cache

G. Lettieri

12 Marzo 2023

1 Introduzione

La memoria centrale è molto più lenta del processore. Possiamo rendercene conto scrivendo un programma che accede ripetutamente agli elementi di un array, misurando quanti cicli di clock sono in media necessari per ogni accesso. La Figura 1 mostra i risultati ottenuti su un processore Intel Core i7-12700 con clock massimo a 4.9 GHz e memoria DDR4 a 3200 Hz, per varie dimensioni dell'array. Si vede che quando l'array è più grande di 25 MiB, gli accessi in memoria possono richiedere circa 70 nanosecondi. Si tratta di un tempo enorme rispetto alla frequenza della CPU. Ricordiamo che il processore è in grado potenzialmente di completare una istruzione per ogni ciclo di clock, quindi circa una istruzione ogni quarto di nanosecondo. Ma le istruzioni si trovano in memoria: se per prelevare una istruzione sono necessari 70 nanosecondi, è del tutto inutile avere un processore velocissimo, in quanto per la stragrande maggioranza del tempo starebbe fermo ad aspettare la prossima istruzione.

In Figura 1, però, notiamo anche che le operazioni di lettura sembrano richiedere molto meno tempo se l'array è sufficientemente piccolo. Per esempio, per array più piccoli di 48 KiB sembra essere sufficiente poco più di un nanosecondo. Questo è l'effetto della memoria *cache*, che ora vogliamo studiare.

2 La memoria Cache

Esistono diverse tecnologie che permettono di costruire una memoria ad accesso casuale. In generale, è possibile costruire memorie grandi ed economiche, ma lente, oppure memorie piccole e veloci, ma costose. Noi vorremmo invece avere memorie grandi, economiche e veloci. Queste non possiamo ottenerle direttamente, ma possiamo avere qualcosa di equivalente usando contemporaneamente i due tipi di memoria e sfruttando alcune caratteristiche dei programmi. Queste caratteristiche prendono il nome di *principi di località*, e sono i seguenti:

- *località spaziale*: se un programma accede ad un certo indirizzo, è molto probabile che in breve tempo accederà ad un indirizzo vicino;
- *località temporale*: se un programma accede ad un certo indirizzo, è molto probabile che in breve tempo vi accederà di nuovo.

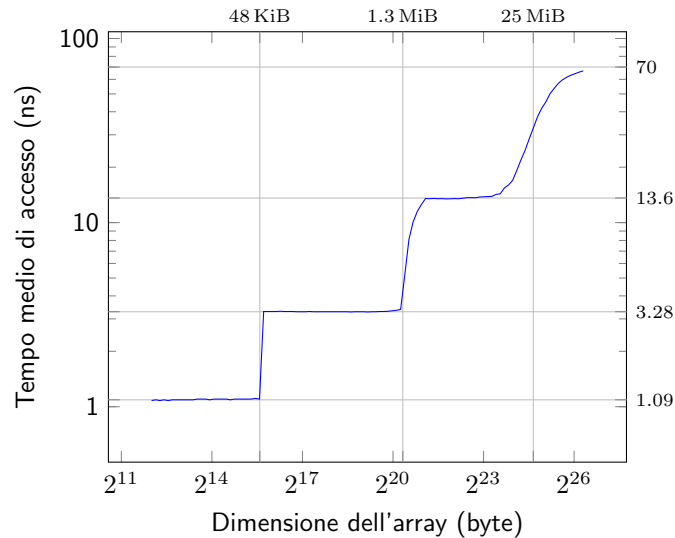


Figura 1: Numero medio di nanosecondi per ogni accesso in memoria per un programma che accede ciclicamente a tutti gli elementi di un array di dimensione variabile (Intel Core i7-12700, DRAM DDR4 3200).

Questi sono principi puramente statistici che i programmi tipicamente rispettano. Per esempio, le istruzioni sono eseguite per lo più in sequenza, quindi il prelievo di una istruzione ad un certo indirizzo è molto spesso seguito dal prelievo della successiva (quindi ad un indirizzo vicino). La presenza di cicli nel programma comporterà il prelievo ripetuto delle stesse istruzioni. Analoghe considerazioni valgono anche per i dati, in quanto i programmatori li organizzano spesso in strutture dati in cui le variabili usate insieme si trovano vicine, e vi accedono ripetutamente. Quindi, anche se un programma può avere complessivamente bisogno di molta memoria, se lo osserviamo per un intervallo di tempo sufficientemente breve vedremo che si concentra su una parte molto più piccola (magari diversa man mano che l'esecuzione procede). Se riuscissimo a scoprire qual è questa parte e copiarla nella memoria veloce, il nostro programma potrebbe essere eseguito molto più velocemente.

L'idea è di realizzare la memoria centrale con la tecnologia lenta, ma economica, in modo da poterla avere molto grande. Allo stesso tempo aggiungiamo al sistema una memoria cache come illustrato in Figura 2. La cache è composta da un *controllore cache* e dalla memoria cache vera e propria, realizzata con la tecnologia costosa, ma veloce. La memoria cache è dunque molto più piccola della memoria centrale.

Il controllore intercetta tutte le operazioni di lettura e scrittura nello spazio di memoria eseguite dal processore. Per ogni operazione controlla prima se il dato a cui il processore vuole accedere si trova in memoria cache, nel qual caso l'accesso può essere completato velocemente; altrimenti esegue l'operazione in

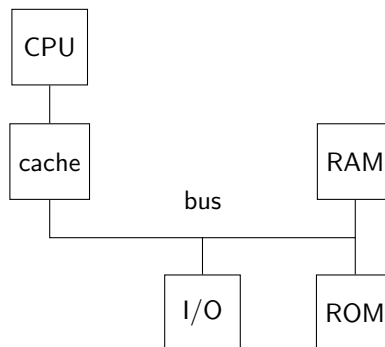


Figura 2: Architettura generale di un calcolatore con memoria cache.

memoria al posto del processore e mantiene una copia del dato in cache (in quanto si spera che il processore lo chiederà di nuovo, per il principio di località temporale). All'inizio la memoria cache non conterrà niente, ma a poco a poco si riempirà con le locazioni a cui il programma sta accedendo. Se la memoria cache si riempie, il controllore dovrà decidere quali locazioni tenere e quali eliminare (*rimpiazzare*), sempre cercando di mantenere quelle che è più probabile che vengano richieste in seguito. Si noti che né il controllore cache, né il processore conoscono le locazioni che verranno richieste in futuro: il controllore vede solo le operazioni che il processore genera e il processore vede una sola istruzione per volta. Le decisioni del controllore sono dunque euristiche.

Tipicamente il controllore legge dalla memoria più di quanto il processore ha chiesto, sia per sfruttare il principio di località spaziale, sia per altri motivi che vedremo tra breve. L'unità di trasferimento utilizzata dal controllore cache è detta *cacheline*. Un esempio tipico è di avere cacheline di 64 byte, allineate naturalmente. Per esempio, se il processore esegue una operazione di lettura all'indirizzo 8, il controllore trasferirà dalla memoria tutti i byte che vanno da 0 a 63 (cioè tutta la cacheline che contiene la locazione richiesta dal processore).

Si noti che tutto quanto abbiamo detto si svolge completamente in hardware, nel controllore cache, ed è trasparente al software: i programmi possono essere scritti ignorando che la cache esista e funzioneranno lo stesso. In presenza della cache, però, verranno in genere eseguiti più velocemente.

Anche il processore può ignorare completamente la presenza della cache, nel senso che non sono richieste modifiche al suo funzionamento, purché disponga di un modo per variare la lunghezza delle operazioni di lettura e scrittura in memoria. Questo perché tali operazioni richiederanno tempi molto diversi, a seconda che il dato richiesto si trovi in cache o debba essere prelevato dalla memoria. A tale scopo è sufficiente che il processore possieda un piedino di ingresso tramite il quale il controllore cache può segnalare quando l'operazione si è conclusa, oppure quando deve essere prolungata rispetto ad un tempo di default.

Si noti infine che il meccanismo della cache non ha alcun senso per le ope-

razioni di I/O, in quanto queste operazioni hanno effetti collaterali che non devono essere cancellati: se un programma vuole leggere il prossimo carattere battuto sulla tastiera è necessario interpellare la tastiera. Non avrebbe alcun senso re-inviare al processore sempre lo stesso carattere conservato in cache. Il controllore cache, dunque, farà passare inalterate tutte le operazioni di lettura e scrittura nello spazio di I/O. Questo però non basta: si ricordi che le interfacce di I/O possono anche avere i loro registri mappati nello spazio di memoria. Per disabilitare la cache anche durante gli accessi a queste interfacce è necessario dunque operare una discriminazione anche in base all'indirizzo usato dal processore, ma per vedere come fare dovremo aspettare di aver introdotto la paginazione.

3 Cache ad indirizzamento diretto

La memoria cache è, dal punto di vista funzionale, una normale memoria ad accesso casuale: le operazioni possibili sono sempre quelle di lettura e scrittura, eseguite una per volta, ognuna ad un certo *indirizzo di cache*.

Il controllore cache intercetta le operazioni di lettura e scrittura del processore. Ognuna di queste è relativa ad un certo indirizzo di memoria. Si ricordi che il processore genera l'indirizzo di una certa parola quadrupla allineata naturalmente, che abbiamo chiamato *numero di riga*, usando poi i fili di byte enable per selezionare i byte all'interno della riga selezionata. La memoria cache sarà organizzata nello stesso modo, così che le operazioni di lettura e scrittura del processore si mappino facilmente su quelle della cache.

Il controllore, però, lavora con unità più grandi, dette *cacheline*, che per esempio possono essere di 8 parole quadruple (64 byte), allineate naturalmente. Il controllore può dunque scomporre il numero di riga generato dal processore in una parte meno significativa detta *offset* (di 3 bit se le cacheline sono di 8 parole quadruple) e nel rimanente numero di cacheline. Poiché il controllore trasferisce sempre intere cacheline, il numero di cacheline è sufficiente per determinare se la locazione richiesta dal processore è in cache oppure no.

Dato il numero di cacheline, il controllore cache deve essere in grado di sapere se la corrispondente cacheline di memoria è stata precedentemente copiata in cache e, in caso affermativo, a quale indirizzo di cache. Un modo per realizzare questo meccanismo è di avere una funzione hash da numeri di cacheline a indirizzi di cache. Nelle cache a *indirizzamento diretto* questa funzione è estremamente semplice (e dunque veloce): l'indirizzo di cache (detto *indice*) è dato dai bit meno significativi del numero di cacheline. In particolare, se la cache è grande 2^a cacheline, l'indice sarà di a bit. Si noti che tutti i numeri di cacheline che distano tra loro 2^a cacheline avranno lo stesso indice, e dunque vorrebbero essere memorizzate nella stessa posizione della cache (si dice che c'è un conflitto tra le due cacheline). Il controllore deve sapere quale tra le tante cacheline che potrebbero trovarsi ad un certo indirizzo di cache è quella effettivamente caricata. Queste diverse cacheline si distinguono per la parte del numero di cacheline che non è utilizzata nell'indice e che è detta *etichetta*. Il controllore può

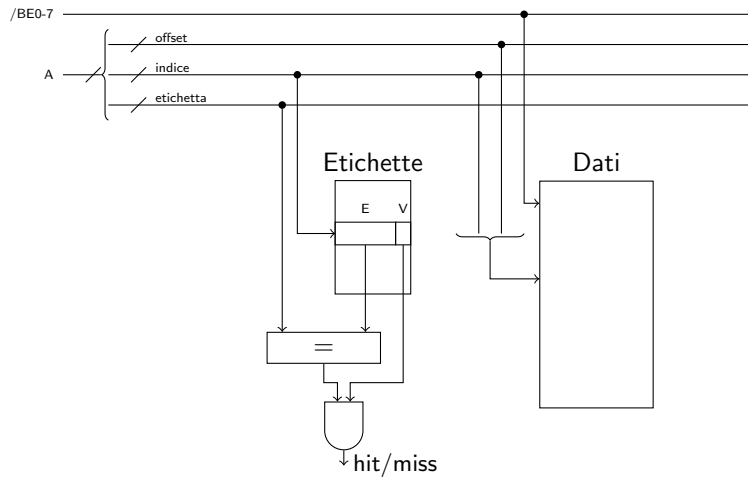


Figura 3: Cache a indirizzamento diretto.

utilizzare una memoria aggiuntiva, detta memoria delle etichette (o *tag*), in cui memorizzare, per ogni indice, l'etichetta della cacheline caricata a quell'indice. Il controllore deve anche sapere a quali indici non è stata caricata ancora alcuna cacheline. Poiché le memorie contengono sempre qualcosa e qualunque sequenza di bit potrebbe essere una valida etichetta, è necessario memorizzare un bit in più per ogni indice, detto *bit di validità*, che valga 1 se e solo se l'etichetta (e dunque la cacheline) contenuta a quell'indice è significativa. All'inizio tutti i bit di validità saranno a 0; passeranno a 1 man mano che il controllore cache caricherà cacheline in risposta agli accessi in memoria del processore. Lo schema è dunque quello di Figura 3.

Per ogni operazione di lettura in memoria da parte del processore, le operazioni svolte dal controllore cache saranno le seguenti:

- se la cacheline è presente (*read hit*), completa la lettura con i dati presenti in cache; per leggere dalla memoria cache dati la parola quadrupla richiesta dal processore è sufficiente usare come indirizzo l'indice e l'offset;
- se la cacheline non è presente (*read miss*), il controllore deve leggere la cacheline della memoria, scriverla nella cache dati a partire dalla posizione dettata dall'indice, e aggiornare l'etichetta; a questo punto i dati sono in cache e si procede come nel caso precedente.

Si noti che, in caso di miss, una eventuale cacheline che si trovava in cache allo stesso indice di quella appena caricata verrà rimpiazzata. Questo è il difetto principale delle cache ad indirizzamento diretto: due cacheline che hanno lo stesso indice non possono essere mantenute contemporaneamente in cache, anche se il resto della cache è vuota.

Per le operazioni di scrittura abbiamo diverse opzioni.

- se la cacheline è assente (*write miss*), il controllore può completare la scrittura in memoria senza caricare la cacheline in cache (*write no-allocate*) oppure caricare la cacheline in cache (*write allocate*) e proseguire come in una *write hit*;
- se la cacheline è presente (*write hit*), il processore può aggiornare solo la cache (*write back*) o sia la cache che la memoria (*write through*).

Si noti che nel caso di *write back* la cacheline dovrà essere riscritta in memoria prima di essere rimpiazzata da un'altra cacheline, perché altrimenti le scritture eseguite dal programma andrebbero perse. La tecnica è comunque conveniente se il programma riesce ad eseguire tante scritture sulla stessa cacheline prima che questa debba essere scritta, in quanto si riduce il numero di scritture in memoria. Per evitare scritture inutili, inoltre, il controllore cache può associare ad ogni cacheline un bit *dirty* e porlo a 1 quando il processore ha eseguito almeno una scrittura in quella linea. Le cacheline che hanno il bit dirty a zero non hanno bisogno di essere riscritte in memoria. Le cache moderne sono tipicamente write-allocate e write-back.

La memoria delle etichette è un costo aggiuntivo ed è bene che sia sufficientemente piccola. Notiamo che abbiamo bisogno di una etichetta per ogni cacheline della memoria dati. Se usiamo cacheline più grandi ne riduciamo il numero, e quindi riduciamo anche la dimensione della memoria delle etichette. Questo è il motivo principale per cui la cacheline contiene più byte di quanti ne può richiedere il processore in una singola operazione. D'altro canto, la cacheline non deve essere nemmeno troppo grande, altrimenti leggere o scrivere una cacheline dalla memoria richiederebbe troppo tempo. La dimensione di 64 byte è un buon compromesso tra queste due esigenze contrastanti.

In presenza di cache non è più il processore ad accedere alla memoria centrale, ma sempre il controllore cache. Dal momento che questo trasferisce sempre cacheline intere, si può ottimizzare questo tipo di trasferimento. In genere si introduce un nuovo tipo di operazione sul bus, specifico per la lettura/scrittura di cacheline. La memoria RAM può essere organizzata in modo tale che, mentre è in corso il trasferimento di una riga, inizi a la lettura della riga successiva all'interno della stessa cacheline. Inoltre, il numero di linee dati può essere reso indipendente dal numero di linee dati del processore¹. In questo modo il costo della lettura/scrittura di una cacheline è reso inferiore alla somma dei costi dei trasferimenti delle singole righe che compongono la cacheline.

4 Cache associative ad insiemi

Le cache associative ad insiemi sono più costose di quelle ad indirizzamento diretto, ma permettono di alleviare il problema dei conflitti. L'idea è di permettere la memorizzazione in cache di più di una cacheline per ogni indice. Una cache associativa ad insiemi che permette di memorizzare n cacheline per ogni

¹Per esempio, il Pentium era un processore a 32 bit, ma il bus dati era a 64 bit per velocizzare il trasferimento delle cacheline.

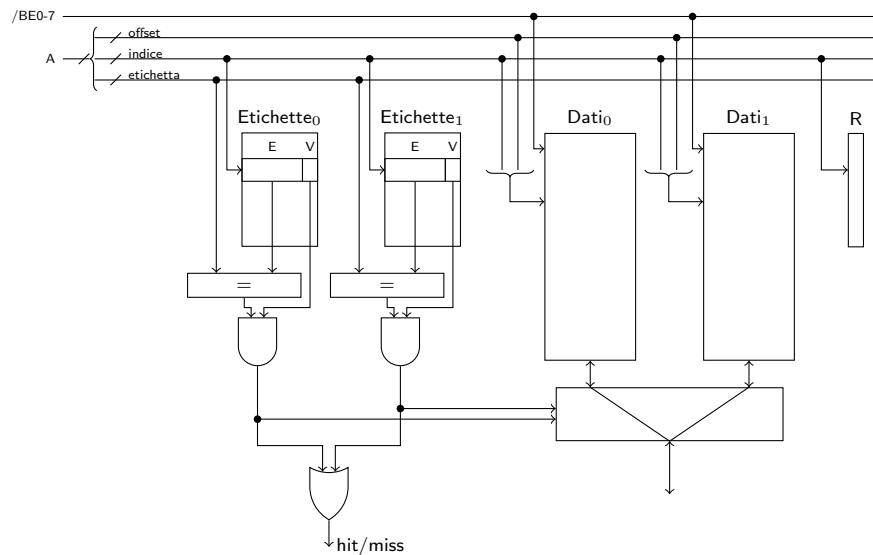


Figura 4: Cache associativa a insiemi (a 2 vie).

indice è detta a n vie. In Figura 4 è mostrato lo schema di una cache associativa a 2 vie. Al suo interno ci sono due repliche di una cache ad indirizzamento diretto, collegate in parallelo. Ogni cacheline ha sempre un indice, ma questo ora individua due possibili locazioni: una in *Dati₀* e una in *Dati₁*. Ogni cacheline può essere memorizzata indifferentemente nell'una o nell'altra. Ogni indice, dunque, individua un *insieme* di possibili locazioni (di cache) per la cacheline.

Si avrà un hit se una delle vie dell'insieme contiene la cacheline cercata, da cui la porta OR che riceve le uscite delle due porte AND. La cacheline a cui accedere dipenderà ovviamente da quale via ha prodotto l'hit. (Si noti che non è possibile che entrambe le vie producano hit, in quanto una cacheline viene caricata solo se non è già presente, e all'inizio la cache è vuota.) In Figura 4 questo è indicato sommariamente dal circuito che si trova sotto le due memorie *Dati*.

In caso di miss, il controllore può scegliere quale delle due cacheline rimpiazzare. La politica più usata in questo caso è la LRU (Least Recently Used): si rimpiazza la cacheline che non è acceduta da più tempo. Questa politica è quella che si comporta meglio nella maggior parte dei casi, anche se non è ottima in assoluto (in alcuni casi è anzi la peggiore). Purtroppo la politica matematicamente ottima richiede la conoscenza di tutti gli accessi futuri e non è realizzabile in pratica.

La memoria R in Figura 4 contiene le informazioni necessarie ad implementare la politica LRU. Con solo due vie è sufficiente ricordare, per ogni indice, la via riferita dall'ultimo accesso: la via da rimpiazzare in caso di miss sarà evidentemente l'altra. Con più di due vie sarebbe però necessario ricordare

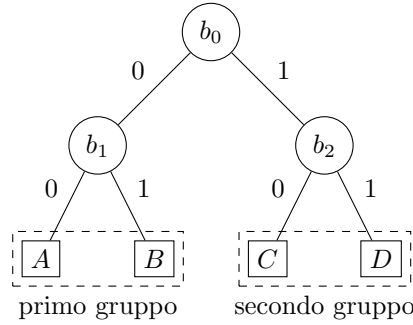


Figura 5: Scelta della via da rimpiazzare nell'algoritmo pseudo-LRU.

l'ordine degli ultimi accessi di ogni via, e aggiornarlo ad ogni accesso, operazione che potrebbe essere troppo costosa. In pratica conviene utilizzare politiche approssimate, se non addirittura effettuare un rimpiazzamento casuale.

La cache dell'80486 era a 4 vie e realizzava un interessante algoritmo che approssima LRU ed è molto semplice da realizzare. L'algoritmo si chiama Pseudo-LRU e richiede 3 bit per ogni insieme, chiamiamoli b_0 , b_1 e b_2 . Chiamiamo A , B , C e D le quattro vie. L'algoritmo raggruppa le vie a due due: A e B da una parte e C e D dall'altra. Il bit b_0 dirige la scelta sul primo gruppo o sul secondo gruppo:

- se $b_0 = 0$ la via da rimpiazzare è una tra A e B . La scelta tra le due è decisa dal valore di b_1 , ignorando b_2 :
 - se $b_1 = 0$ si sceglie A ;
 - se $b_1 = 1$ si sceglie B ;
- se $b_0 = 1$, invece, la scelta è tra C e D (secondo gruppo) ed è decisa dal valore di b_2 , ignorando b_1 :
 - se $b_2 = 0$ si sceglie C ;
 - se $b_2 = 1$ si sceglie D ;

Si veda anche la Figura 5. Ad ogni accesso vanno aggiornati due bit, lasciando inalterato il terzo, in base alla via interessata dall'accesso. L'idea è che la via appena acceduta deve diventare l'ultima ad essere rimpiazzata, così come accadrebbe nell'algoritmo LRU:

- accesso a A : $b_0 \rightarrow 1$ e $b_1 \rightarrow 1$;
- accesso a B : $b_0 \rightarrow 1$ e $b_1 \rightarrow 0$;
- accesso a C : $b_0 \rightarrow 0$ e $b_2 \rightarrow 1$;
- accesso a D : $b_0 \rightarrow 0$ e $b_2 \rightarrow 0$.

Il fatto di lasciare sempre inalterato uno tra b_1 o b_2 permette di ricordare l'ordine relativo tra le vie del gruppo non interessato dall'accesso. D'altro canto, la via che si trova nello stesso gruppo di quella interessata dall'accesso può ora trovarsi più in alto in coda (grazie al valore scritto in b_0) rispetto a dove sarebbe stata nel vero algoritmo LRU (da qui l'approssimazione).

Si noti inoltre che, se la memoria R è organizzata in tre banchi indipendenti da un bit ciascuno, ogni aggiornamento può essere implementato con una operazione di scrittura in due dei tre banchi, senza la necessità di dover prima eseguire una operazione di lettura.

L'algoritmo di Pseudo-LRU può essere esteso a cache con più di 4 vie ricorrendo ad alberi più profondi, ma può essere anche ulteriormente approssimato, per esempio eliminando i livelli più bassi dell'albero e sostituendoli con scelte casuali.

A parità di dimensione complessiva della cache, aumentando il numero di vie diminuisce il numero di insiemi. Nel caso estremo avremo una cache con un solo insieme, nel qual caso si parla di cache *completamente associativa*. Una cache completamente associativa lascia piena libertà sul piazzamento delle cacheline all'interno della cache, eliminando così i conflitti. Ovviamente si tratta di una soluzione costosa che è praticabile solo in cache molto piccole.

A LRU e gli accessi ciclici

Cerchiamo di capire meglio cosa succede in Fig. [1](#). La politica LRU ha questo difetto: nel caso in cui il programma acceda ciclicamente ad un numero di cacheline superiore, anche di una sola cacheline, a quelle che possono essere contenute in cache, LRU causa una miss ad *ogni* accesso. Possiamo facilmente convincerci di questo simulando a mano LRU su una piccola cache completamente associativa, diciamo di 4 cacheline, provando ad accedere ripetutamente alle cacheline numero 1, 2, 3, 4, e 5. Anche se è più complicato da vedere, la stessa cosa accade anche con Pseudo-LRU. Questo difetto può essere usato per scoprire la dimensione di una cache che usi questi algoritmi. Concentriamoci sul punto di Fig. [1](#) in cui la dimensione dell'array attraversa il valore 48 KiB: per valori immediatamente inferiori il tempo di accesso medio è 1.09 nanosecondi, mentre per valori immediatamente superiori è 3.28. Questa è una forte indicazione della presenza di una cache di 48 KiB con rimpiazzamento LRU o Pseudo-LRU: sotto i 48 KiB tutte le cacheline si trovano in cache (dopo il primo ciclo), mentre sopra i 48 KiB *nessuna* cacheline si trova mai in cache (LRU l'ha sempre rimpiazzata poco prima che il programma vi accedesse). La dimensione delle cache può essere confermata con vari strumenti—per es., in Linux su Intel/AMD, con il comando `lscpu --cache`. La CPU Intel Core i7-12700 contiene in effetti tre livelli di cache, con dimensioni 48 KiB, 1.3 MiB e 25 MiB, che corrispondono ai punti in cui il grafico di Fig. [1](#) ha dei salti (più o meno smussati).

Il comando `lscpu` ci permette anche di sapere che la prima cache è in realtà associativa a 12 vie, mentre la seconda e la terza sono associative a 10 vie. Per la prima cache avremmo potuto scoprire il numero di vie osservando più preci-

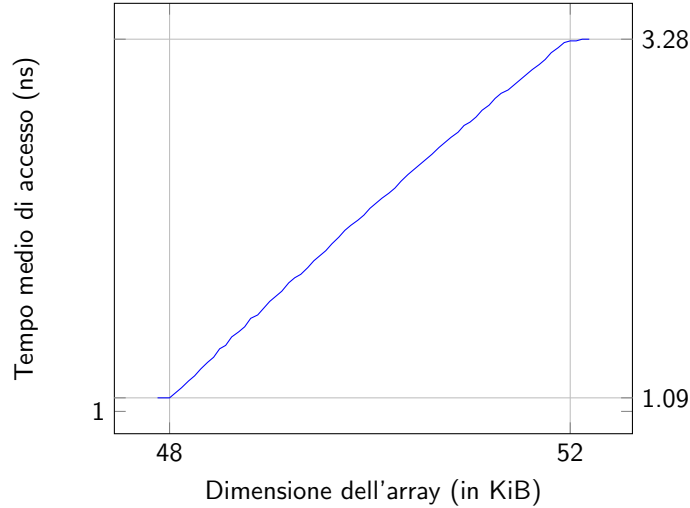


Figura 6: Ingrandimento di Fig. 1 nei pressi del primo salto.

samente cosa accade intorno al primo salto. La Fig. 6 mostra il grafico di Fig. 1 ingrandito intorno a quel punto, con gli assi non più in scala logaritmica: vediamo che l'aumento del tempo di accesso medio è in realtà lineare. Come mai questo accada è presto detto, se teniamo a mente che (Pseudo-)LRU si applica ad ogni *insieme* indipendentemente dagli altri. Quando l'array è grande 48 KiB può essere ancora contenuto completamente nella prima cache e tutti gli accessi si risolvono lì. Il tempo medio di accesso misurato è dunque interamente dovuto alla prima cache. Pensiamo ora a cosa accade se aumentiamo la dimensione dell'array di una cacheline. L'insieme in cui la cacheline dovrebbe essere caricata, chiamiamolo i , è sicuramente pieno (tutti gli insiemi sono pieni), quindi è necessario un rimpiazzamento. In pratica, 13 cacheline dell'array vorrebbero andare tutte nello stesso insieme i , che però ne può contenere solo 12—diciamo che i è “sovraffollato”. Il programma accede ciclicamente a tutto l'array, e dunque accede ciclicamente anche a queste 13 cacheline: (Pseudo-)LRU causerà dunque una miss (nell'insieme i) ogni volta che il programma accede ad una di queste 13 cacheline, ma non quando accede alle altre. Osserveremo allora che il tempo medio di accesso è aumentato leggermente. Quando la dimensione dell'array è $48 \text{ KiB} + 128 \text{ byte}$ entra in gioco una nuova cacheline, che andrà a causare un sovraffollamento su un altro insieme, e così via. Ogni nuova cacheline aumenta il tempo medio di accesso di un valore costante, fino a quando tutti gli insiemi saranno sovraffollati e quindi tutti gli accessi causeranno miss. A quel punto il tempo medio di accesso che misuriamo sarà interamente dovuto alla seconda cache, come se la prima non ci fosse.

Siccome la cache ha $v = 12$ vie, gli insiemi sono grandi ciascuno $v \times 64 \text{ byte}$

e la prima cache contiene

$$n = \frac{48 \times 1024}{v \times 64} = 64$$

insiemi. Tutti gli insiemi saranno sovraffollati quando l'array raggiunge la dimensione di

$$48 \text{ KiB} + n \times 64 B = 52 \text{ KiB},$$

come confermato in Fig. 6. Se non avessimo saputo il numero di vie avremmo potuto ricavarlo da queste relazioni, osservando i punti in cui il grafico si appiattisce.

I salti intorno a 1.3 MiB e 25 MiB in Fig. 1 non rispettano questo modello. Ciò può essere dovuto ad una implementazione approssimata di Pseudo-LRU, oppure (soprattutto per il secondo salto) all'effetto di altre cache che vedremo in seguito (TLB).

Periferiche

G. Lettieri

15 Marzo 2024

Studiamo ora le periferiche essenziali di un PC. Faremo riferimento alle vecchie periferiche del PC AT dell'IBM, che per molti anni hanno fatto parte della cosiddetta *Industry Standard Architecture*, uno standard di fatto seguito da tutti i produttori di PC IBM-compatibili e cloni. Queste periferiche hanno il vantaggio di essere più semplici da programmare dei loro equivalenti moderni. Inoltre, molti PC moderni continuano a supportarle, per mantenere la compatibilità software. In particolare, le periferiche che studiamo sono supportate dalla macchina virtuale QEMU che usiamo per sviluppare tutti gli esempi. In ogni caso i concetti base sono rimasti sostanzialmente gli stessi nel tempo.

Per ogni periferica daremo una breve descrizione del suo funzionamento interno, quindi descriveremo l'interfaccia visibile al programmatore, almeno quanto basta per poter programmare dei semplici esempi. Il codice degli esempi può essere scaricato da

<https://calcolatori.iet.unipi.it/resources/esempiIO-7.4.tar.gz>

Tutti gli esempi fanno uso della libreria `libce`, scaricabile dallo stesso sito.

1 Tastiera

Lo scopo della tastiera è di rilevare i tasti premuti e rilasciati e di comunicarli al PC. A (quasi) ogni tasto fisico sono associati due codici numerici: il *make code*, che indica che il tasto è stato premuto, e il *break code*, che indica che il tasto è stato rilasciato. Questi codici sono associati al tasto e non alla sua funzione: per esempio, i due tasti shift hanno codici diversi. È il software che assegna (liberamente) un significato ai tasti.

Per svolgere il proprio compito, le tastiere contengono tipicamente una matrice di collegamenti elettrici. Nelle tastiere più comuni (ed economiche) questa è realizzata con tre fogli di plastica sovrapposti, chiamiamoli 1, 2 e 3. Sul foglio 1 sono tracciati dei collegamenti verticali e sul foglio 3 dei collegamenti orizzontali. Le tracce si incrociano in corrispondenza di ciascun tasto, ma sono tenute separate dal foglio 2, che si trova in mezzo agli altri due. Il foglio 2, però, contiene un buco in corrispondenza di ogni tasto (e dunque di ogni incrocio). Ogni volta che si preme un tasto si mette in collegamento, attraverso il buco,

una traccia verticale del foglio 1 con una traccia orizzontale del foglio 3, semplicemente esercitando una pressione sul foglio superiore tramite un bastoncino di gomma che si trova sotto il tasto.

La matrice è monitorata da un *microcontrollore* che si trova a bordo della tastiera (originariamente un Intel 8042). Un microcontrollore è un piccolo computer, con una sua CPU, una sua RAM, una ROM e delle porte di I/O, interamente contenuto in un unico chip. Le porte di I/O del microcontrollore sono direttamente collegate a dei piedini del chip. Nella tastiera, i piedini di I/O del microcontrollore sono collegati alle tracce verticali e orizzontali della matrice di cui sopra. Il microcontrollore è programmato in modo da inviare un segnale su una colonna alla volta e leggere lo stato di tutte le righe della matrice. Se un tasto di quella colonna è premuto, il segnale ritornerà sulla corrispondente riga: in questo modo il controllore può rilevare la pressione di un tasto¹. Il microcontrollore passa poi alla colonna successiva, e così via. La scansione viene ripetuta migliaia di volte al secondo, cosa che è sufficiente a non perdere le battute anche del più veloce dattilografo. Per rilevare quando un tasto è stato rilasciato, il microcontrollore ricorda lo stato di ogni tasto nella propria RAM. Quando un tasto cambia stato, il microcontrollore trasmette un appropriato codice di scansione al PC.

L'informazione di "tasto rilasciato" (break code) serve al software per gestire le combinazioni di tasti (se ricevo il make code di shift seguito dal make code del tasto 'a', prima di aver visto il break code del tasto shift, vuol dire che l'utente sta cercando di scrivere 'A'). Le tastiere dei PC, invece, gestiscono autonomamente i tasti *typematic*: se il controllore della tastiera si accorge che uno dei tasti typematic è sempre premuto per tutto un certo intervallo di tempo (configurabile), inizia a inviare il make code di quel tasto ripetutamente, con un certo ritmo (anch'esso configurabile)².

La comunicazione tra tastiera e PC può avvenire in vari modi. Nel seguito facciamo riferimento alle tastiere di tipo PS/2, la cui interfaccia di programmazione rispecchia ancora quella del PC AT. In questo caso la comunicazione tra PC e tastiera è su un cavo seriale, e all'altro capo del cavo, sulla scheda madre del PC, si trova un altro microcontrollore (originariamente anch'esso un Intel 8042), che riceve i codici, li converte nei corrispondenti make code o break code (la conversione avviene, al solito, per motivi di compatibilità software), e li rende disponibili in un registro mappato nello spazio di I/O del bus, da cui il software può poi leggerli. I make code e break code sono su 8 bit (per quasi tutti i tasti). Inoltre, il break code differisce dal corrispondente make code solo per il bit più significativo (1 per i break code e 0 per i make code).

¹Per semplicità evitiamo di discutere i problemi che si verificano quando più di un tasto per colonna o riga sono contemporaneamente premuti (*ghosting* e *jamming*), o dei limiti al cosiddetto *roll-over*, cioè del massimo numero di tasti contemporaneamente premuti che la tastiera può correttamente rilevare e/o riportare.

²Non tutti i tasti sono typematic. Per esempio, i tasti shift, ctrl, alt non lo sono.

1.1 Interfaccia per il programmatore

Il programmatore interagisce esclusivamente con il microcontrollore a bordo del PC, tramite una interfaccia che espone quattro registri: un registro di lettura, RBR, un registro di stato STR, un registro di scrittura, TBR, e un registro di controllo, CMR, tutti da 8 bit. Questi registri occupano solo due indirizzi dello spazio di I/O, in modo da risparmiare piedini di indirizzo nell'interfaccia: RBR e TBR sono entrambi all'indirizzo 0x60, mentre STR e CMR sono entrambi all'indirizzo 0x64. Questo è possibile perché RBR e STR sono di sola lettura, mentre TBR e CMR sono di sola scrittura.

I bit 0 e 1 del registro STR fungono da flag “busy” per i registri RBR e TBR, rispettivamente. In particolare, il programmatore deve assicurarsi di leggere RBR solo se il bit 0 di STR è 1, per essere sicuro di star leggendo un nuovo codice.

Tramite il microcontrollore a bordo del PC è possibile anche *inviare* comandi alla tastiera, per esempio per accendere o spegnere i led, o configurare i parametri del typematic. Questo è uno degli scopi dei registri CMR e TBR. Un altro scopo, non legato alla tastiera, è quello di iniziare il reset del PC da software: uno dei piedini del microcontrollore è collegato alla circuiteria di reset del PC (e in particolare al piedino di reset del processore) e per attivarlo è sufficiente scrivere 0xFE in CMR (la funzione `reboot()` di `libce` fa esattamente questo). Vedremo poi un altro utilizzo di questi registri quando parleremo delle interruzioni.

La cartella `esempiIO` contiene due programmi che utilizzano l'interfaccia della tastiera: `tastiera-1` e `tastiera-2`.

L'esempio `tastiera-1` legge ciclicamente un nuovo codice e lo stampa sul video in binario. L'esempio fa uso di alcuni tipi e funzioni definite in `libce`:

1. `ioaddr` è un **typedef** per **unsigned short** (16 bit) ed è utilizzato per definire gli indirizzi nello spazio di I/O;
2. `inputb()` serve a leggere un byte dallo spazio di I/O, all'indirizzo passato come argomento;
3. `char_write()` serve a mostrare un carattere sullo schermo.

La funzione `inputb()` è definita in assembly (nel file `64/inputb.s` nei sorgenti di `libce`) in modo da poter utilizzare l'istruzione **inb**, che è sconosciuta al compilatore C++. La funzione `char_write()` la vedremo quando parleremo del video. La funzione `get_code()` legge ciclicamente STR fino a quando non trova il bit 0 settato a 1, quindi legge il nuovo codice da RBR. Si noti che una funzione sostanzialmente identica è definita anche in `libce`, e negli esempi successivi useremo direttamente quella offerta dalla libreria.

Il programma `tastiera-2` vuole mostrare un semplice esempio di operazioni tipicamente svolte dal software, come convertire i codici letti dalla tastiera in codici ASCII, tenendo conto dello stato di tasti modificatori come lo shift, e farne l'eco sul video. In particolare, la funzione `char_read()` legge un nuovo codice dalla tastiera e tiene traccia dello stato dello shift sinistro nella variabile

globale `shift`. Il **while** serve ad ignorare i codici dello `shift`, che abbiamo già gestito, e i `break code` degli altri tasti, che non ci interessano (tutti i `break code` sono maggiori di `0x80`). La funzione `conv()` provvede poi ad associare un codice ASCII al `make code` ricevuto, usando la tabella `tabmin` o `tabmai`, in base allo stato corrente del tasto `shift`. La libreria `libce` contiene versioni di `conv()` e `char_read()` identiche a quelle mostrate in questo esempio; contiene anche le tabelle `tab`, `tabmin` e `tabmai`, ma con qualche codice in più.

2 Video

L'interfaccia video ruota attorno all'idea della *memoria video*. Si tratta di una memoria destinata a contenere una descrizione di ciò che deve apparire sullo schermo. Alla memoria video accedono concorrentemente sia il software (normalmente in scrittura), sia un *controllore video*, che la legge completamente (per esempio 60 o 90 volte al secondo) e ne interpreta il contenuto per generare i segnali per il monitor (analogici nei vecchi monitor VGA, digitali nei monitor moderni). Tranne che nelle schede video più economiche, la memoria video si trova a bordo della stessa scheda video che contiene anche il controllore ed è mappata nello spazio di indirizzamento di memoria del bus. Questo vuol dire che, per il programmatore, accedere alla memoria video è sostanzialmente come accedere ad una struttura dati. A livello assembly è possibile usare tutte le istruzioni che ammettono un operando in memoria (tipicamente `mov`).

Oltre a scrivere e leggere dalla memoria video, il software può dialogare anche con il controllore stesso. Quest'ultimo ha una interfaccia con vari registri che, nella scheda che vedremo, sono accessibili nello spazio di I/O. Scrivendo opportunamente in questi registri è possibile, in particolare, selezionare la *modalità* video, che stabilisce come deve essere interpretato il contenuto della memoria video. Esistono due modalità principali, con varie sotto-modalità:

- *Modalità testo*: in questo caso la memoria video contiene i codici (per es. ASCII) dei caratteri che devono essere visualizzati sullo schermo, più eventualmente altri attributi come il colore dei singoli caratteri;
- *Modalità grafica*: in questo caso la memoria video serve a specificare il colore di ciascun pixel dello schermo.

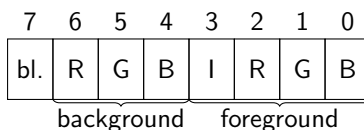
Nel caso della modalità testo, il controllore video ha anche bisogno di un *font* che gli dica come disegnare sullo schermo i vari caratteri. Normalmente uno o più font sono contenuti in una ROM a bordo della scheda video. Le varie sotto-modalità della modalità testo si distinguono per il numero delle righe e colonne in cui è suddiviso lo schermo. Le varie sotto-modalità della modalità grafica si distinguono invece per la risoluzione (normalmente espressa in pixel orizzontali per pixel verticali) e per il numero di colori possibili per ciascun pixel. Lo standard SVGA (Super Video Graphics Array) definisce i dettagli di varie modalità, sia testo che grafiche, che le schede che supportano lo standard

devono offrire. Nel seguito esaminiamo la modalità testo 80×25 (80 righe per 25 colonne), che è quella che tradizionalmente le schede video per PC offrono di default, e le modalità grafiche di tipo *framebuffer*, che sono le più semplici da usare.

Il controllore contiene un gran numero di registri interni, ma, per risparmiare piedini, espone al programmatore due soli registri: IND e DAT. Tramite questi registri il programmatore può accedere *indirettamente* a qualunque registro interno. Ogni registro interno è identificato da un indice e, per accedervi, il programmatore deve prima scrivere l'indice desiderato nel registro IND; a quel punto il registro DAT diventa una “finestra” sul registro interno selezionato: scrivendo e leggendo da DAT si legge e scrive nel registro interno.

2.1 Modalità testo

La scheda video emulata da QEMU si trova all'avvio in modalità testo 80×25 . In questa modalità lo schermo è idealmente diviso in una matrice di 80 per 25 posizioni, ciascuna delle quali può visualizzare un carattere. La memoria video è organizzata in una corrispondente matrice di 80×25 elementi, memorizzata per righe. Il primo elemento della matrice si trova all'indirizzo 0xb8000 e corrisponde alla posizione in alto a sinistra dello schermo. Ciascun elemento della matrice è grande due byte: il byte meno significativo contiene il codice del carattere da visualizzare, mentre il byte più significativo contiene l'*attributo colore*. Per quanto riguarda il codice del carattere, normalmente le schede video dei PC accettano un qualunque codice ASCII (7 bit) eventualmente esteso a 8 bit per includere altri caratteri come, in Italiano, le lettere accentate. L'attributo colore, invece, ha il seguente formato:



dove i bit contrassegnati con *foreground* selezionano il colore di primo piano su 4 bit (codificato come Intensity, Red, Green, Blue) e i bit contrassegnati con *background* selezionano il colore di sfondo (codificato come Red, Green, Blue). Il bit 7 (*blinking*), se attivo, rende il carattere lampeggiante, ma questa funzionalità non è emulata da QEMU. Quindi, per esempio, il byte 01001011 (4B in esadecimale) seleziona un colore azzurro chiaro (I+G+B) su sfondo rosso (R).

La cartella `esempiIO` contiene due esempi sul video in modalità testo: `video-testo-1` e `video-testo-2`.

L'esempio `video-testo-1` si limita a inizializzare tutta la memoria video con lo stesso carattere e lo stesso attributo colore. Il tipo `natw`, utilizzato nel codice, è definito in `libce` come un **typedef** per **unsigned short**, ed è dunque grande due byte, quanto ciascun elemento della memoria video. Per accedere alla memoria video direttamente dal C++ è sufficiente dichiarare un

puntatore a `natw` (chiamato `video` nel codice) e inizializzarlo con il valore numerico, noto, del primo indirizzo della memoria video (0xb8000, come detto). Per farlo accettare dal compilatore è però necessario un cast, visto che stiamo cercando di assegnare un intero ad un puntatore. Fatto questo, `video` può essere usato come un normale array di `natw`. Gli elementi da 0 a 79 corrispondono alla prima riga di caratteri, quelli da 80 a 159 alla seconda, e così via. Ogni `natw` deve contenere l'attributo colore nel byte più significativo e, nel byte meno significativo, il codice ASCII del carattere da visualizzare. Dopo aver inizializzato la memoria video, il programma attende che l'utente prema ESC, chiamando ripetutamente la funzione `char_read()` definita in `libce`, analoga a quella vista in `tastiera-2`.

L'esempio `video-testo-2` mostra come realizzare la funzione usata negli esempi precedenti per mostrare un carattere sul video, `char_write()`. La funzione tiene traccia, nelle variabili `x` e `y`, della posizione in cui dovrà apparire il prossimo carattere, in modo da emulare un "terminale a stampante", che stampa i caratteri uno a fianco all'altro. La funzione si preoccupa anche di andare automaticamente alla riga successiva quando si raggiunge l'ultima colonna, e di mostrare lo "scroll" verso l'altro del contenuto del monitor quando si raggiunge l'ultima riga. Lo scroll è realizzato copiando ogni riga della memoria video nella sua riga superiore, quindi riempiendo di spazi la riga in fondo. La funzione gestisce anche i caratteri di a-capo, che si limitano a spostare la posizione del prossimo carattere, senza stampare niente. Infine, la funzione sfrutta la possibilità, offerta dal controllore video, di mostrare un *cursore* nella posizione del prossimo carattere. Per farlo è sufficiente scrivere la posizione desiderata in due registri interni del controllore, cosa che è svolta dalla funzione `cursore()`.

2.2 Modalità grafica

Per attivare la modalità grafica, visto che la scheda che stiamo considerando parte in modalità testo, è necessario impostare un gran numero di registri interni. Questa operazione è molto complessa, tanto che molte schede contengono una ROM in cui sono presenti delle funzioni che svolgono questo compito. Noi "bareremo" un po', in quanto la scheda emulata da QEMU, a sua volta presa dall'emulatore Bochs, può essere configurata molto più facilmente, sostanzialmente scrivendo la risoluzione desiderata in alcuni registri. Queste operazioni sono svolte dalla funzione `bochsvga_config()`, definita in `libce`. La funzione riceve il numero di colonne e righe di pixel desiderato, mentre il numero di colori è fissato a 256. La funzione restituisce l'indirizzo del *framebuffer*, che è l'analogo della memoria video in modalità testo, solo che questa volta l'array contiene una posizione per ciascun pixel dello schermo (nella risoluzione selezionata). Ciascun elemento dell'array è grande un byte, in modo da poter contenere uno qualunque dei 256 colori possibili. Il tipo del valore restituito dalla funzione è direttamente un puntatore a `natb` (**unsigned char**), in modo che lo si possa usare senza dover fare un cast.

A questo punto, per disegnare qualcosa sullo schermo, è sufficiente colorare opportunamente i pixel desiderati, semplicemente scrivendo il codice del colore

nella posizione corrispondente del framebuffer. Si vedano i programmi `svga-1`, `svga-2` e `sgva-3` per tre semplici esempi: uno che colora tutto lo schermo di rosso, uno che disegna un bordo e degli assi, e uno che aggiunge una parabola e una retta.

3 Timer (conteggio)

Il PC AT conteneva una interfaccia di *conteggio* usata per vari scopi. Molti PC moderni continuano a emulare questa interfaccia, e la troviamo anche in QEMU. Queste interfacce decrementano un contatore interno ogni volta che si verifica un evento, segnalato da un impulso su un loro piedino di ingresso. In particolare, se questo piedino è collegato ad un generatore di clock, l'interfaccia permette di misurare il passaggio del tempo. Il valore iniziale del contatore può essere impostato via software, scrivendo in un apposito registro dell'interfaccia. La scrittura, normalmente, funge anche da *trigger* che avvia il conteggio. Quando il contatore arriva a zero, l'interfaccia genera un segnale su un piedino di uscita, con varie possibili forme d'onda. Queste interfacce possono funzionare a *ciclo singolo* o a *ciclo continuo*. Nel secondo caso l'interfaccia, a fine conteggio, ricarica automaticamente il contatore e fa ripartire il conteggio.

Il chip che si trovava a bordo del PC AT era l'Intel 8254, che conteneva tre interfacce di conteggio indipendenti, chiamate 0, 1 e 2, ciascuna con un contatore a 16 bit. Tutte e tre le interfacce erano collegate a un generatore di clock a 1,190 MHz. Ciascuna interfaccia era indipendentemente configurabile per selezionare, per esempio, il ciclo singolo o continuo, oppure il tipo di forma d'onda da generare (impulso, onda quadra, ...). L'interfaccia 0 era collegata, come vedremo, al controllore delle interruzioni. L'interfaccia 1 era utilizzata per il refresh della memoria dinamica (e non è più emulata nei PC moderni). Infine, l'interfaccia 2 era collegata all'unico dispositivo audio fornito, per default, dal PC AT: un "cicalino" in grado di produrre dei *beep*. Si tratta in pratica di un piccolo altoparlante, una membrana che può essere messa in vibrazione agendo su un elettromagnete.

3.1 Interfaccia per il programmatore

Il chip 8254 contiene 13 registri da 8 bit, 4 per ogni interfaccia di conteggio e uno a comune. Per ogni interfaccia, due registri sono di sola scrittura e permettono di impostare la parte alta e bassa del contatore (CTR_LSB e CTR_MSB); due registri sono di sola lettura e permettono di leggere lo stato corrente del contatore (STR_LSB e STR_MSB). Il registro comune (CWR) serve principalmente a configurare le interfacce. Il chip, però, possiede solo 2 piedini di indirizzo, e dunque occupa solo 4 indirizzi dello spazio di I/O, in particolare gli indirizzi 0x40-0x43. L'indirizzo 0x43 permette di accedere al registro comune, CWR. L'indirizzo 0x40 è utilizzato per tutti e 4 i registri dell'interfaccia 0, l'indirizzo 0x41 per i 4 dell'interfaccia 1 (non più utilizzato, come abbiamo detto), e l'indirizzo 0x42 per i 4 dell'interfaccia 2. Scrivendo all'indirizzo 0x40 si acce-

de alternativamente al registro `CTR_LSB` e `CTR_MSB` del contatore 0, mentre leggendo si accede alternativamente ai registri `STR_LSB` e `STR_MSB`, e così per gli altri due contatori. In pratica, per impostare il valore completo di un contatore, è necessario eseguire due scritture consecutive allo stesso indirizzo di I/O.

Dal momento che non conosciamo ancora le interruzioni, vediamo per il momento un esempio che usa il cicalino, e dunque il contatore 2. L'idea è che questo contatore deve essere programmato in modo da generare un'onda quadra a ciclo continuo, con un certo periodo. Questa onda quadra pilota (tramite un amplificatore e un filtro passa basso) direttamente l'elettromagnete dell'altoparlante, e dunque fa vibrare la membrana con la stessa frequenza, producendo una nota udibile. Il PC contiene anche un registro `SPR`, all'indirizzo di I/O `0x61`, che serve a controllare più finemente l'ingresso dell'altoparlante. In particolare, il bit 1 del registro `SPR` è in AND con l'uscita del contatore 2, e permette dunque di silenziare l'altoparlante se posto a 0; il bit 0 di `SPR`, invece, è in ingresso al contatore 2 e, se posto a 0, mette in pausa il conteggio.

La cartella `esempiIO` contiene il programma `timer`, che usa questo contatore per generare un sibilo a 1000 Hz. La funzione `avvia_timer()` configura e inizializza il contatore 2. La scrittura del valore `0xB6` in `CWR` imposta il contatore 2 per generare un'onda quadra a ciclo continuo. Le due successive scritture (si noti che `iCTR2_LSB` e `iCTR2_MSB` sono in realtà lo stesso indirizzo) impostano il valore del contatore a 1190, in modo che il contatore arrivi a zero 1000 volte al secondo. La funzione `main` provvede anche a scrivere 3 (11 in binario) nel registro `SPR`, in modo da *non* mettere in pausa il conteggio e permettere al segnale del contatore di raggiungere l'altoparlante. La funzione `pause()`, definita in `libce`, scrive un messaggio sul video e aspetta che l'utente prema ESC. Al ritorno dalla funzione, la funzione `main` scrive 0 in `SPR`, in modo da silenziare l'altoparlante, e quindi termina.

4 Hard disk

L'hard disk è un esempio di dispositivo *a blocchi*. Con questo termine ci si riferisce a dispositivi di memorizzazione in cui l'informazione è organizzata in unità relativamente grandi, dette appunto blocchi, che sono anche l'unità di trasferimento. Per esempio, negli hard disk comuni i blocchi sono tradizionalmente grandi 512 byte, e non è possibile leggere o scrivere meno di 512 byte per volta. Pur essendo dispositivi di memoria, dunque, non possono essere interfacciati direttamente con la CPU, che invece accede ad unità molto più piccole, e devono essere considerati dei dispositivi di ingresso/uscita: per poter accedere a informazioni contenute in un hard disk, il software deve tipicamente prima copiare i relativi blocchi in memoria centrale, ordinando il trasferimento tramite una interfaccia di I/O.

Internamente, gli hard disk si possono scomporre in un componente detto *drive*, che comprende i dischi, le testine di lettura/scrittura e i loro servomeccanismi, e un secondo componente detto *controllore*, che contiene la logica in

grado di pilotare il drive in base agli ordini impartiti dal software. Controllore e drive sono tipicamente venduti insieme, in un unico dispositivo, che poi deve essere collegato al resto del computer con un cavo che, nei PC moderni, è di tipo seriale.

L'informazione è memorizzata magneticamente sulle facce dei dischi, ciascuna delle quali è suddivisa in tracce concentriche. Ciascuna traccia è poi suddivisa in settori. Il numero di settori per traccia aumenta man mano che ci si sposta dal centro verso la periferia del disco, in modo che la dimensione fisica dei settori resti all'incirca costante. I dischi contenuti del drive (tipicamente 2 o poco più) sono imperniati ad uno stesso asse centrale e sono tenuti costantemente in rotazione (salvo essere messi a riposo in caso di inattività prolungata, per risparmiare energia), con velocità angolari che possono andare da qualche migliaio a qualche decina di migliaia di rpm (rotazioni per minuto), a seconda del modello. Le testine di lettura/scrittura, una per ciascuna faccia di ogni disco, sono anch'esse solidali ad uno stesso asse di rotazione, in modo che in ogni istante si trovino tutte in corrispondenza della stessa traccia su ogni faccia. Le tracce che si trovano alla stessa distanza dal centro su ogni faccia, e che dunque possono essere lette/scritte senza spostare le testine, formano un cosiddetto *cilindro*. Ogni settore è dunque identificato geometricamente da tre coordinate: il numero della faccia (o, equivalentemente, della testina), il numero della traccia sulla faccia (o, equivalentemente, il numero del cilindro), il numero del settore all'interno della traccia. Per poter riferire un settore, il drive ha bisogno di conoscere queste tre coordinate. A quel punto sposterà (se necessario) le testine per portarle sul cilindro richiesto (spendendo un tempo detto di *seek*), attiverà la testina corrispondente alla faccia richiesta e aspetterà che il settore richiesto, per effetto della rotazione costante dei dischi, vi passi di sotto (spendendo un ulteriore tempo detto *latency*). Solo a questo punto il settore può essere letto o scritto. In ogni caso il trasferimento dei dati avverrà da/verso un buffer interno, da cui poi dovrà essere trasferito in memoria centrale in qualche altro modo (come vedremo negli esempi più avanti).

Si noti che i tempi di seek e latency, essendo legati a fattori meccanici, sono migliorati nel tempo, ma molto più lentamente rispetto alla velocità delle componenti elettroniche, soprattutto del processore. In particolare, seek e latency sono entrambi dell'ordine di qualche *millisecondo*, contro le frazioni di *nanosecondo* del ciclo di un tipico processore. Questo comporta che, in un sistema moderno, è bene cercare di leggere/scrivere da settori contigui, in modo da minimizzare lo spostamento delle testine e ridurre i tempi di latenza. In pratica, oggi, conviene usarli come dispositivi sequenziali, anche se, quando furono introdotti dall'IBM negli anni '50, avevano proprio lo scopo di permettere l'accesso casuale e superare i limiti delle unità a nastro magnetico. Oggi, soprattutto nell'elettronica di consumo, sono stati praticamente sostituiti dagli hard disk a stato solido (SSD), che non hanno parti in movimento e hanno tempi di accesso nell'ordine dei microsecondi. Sono ancora comunque molto usati nei *data centre*, perché il loro prezzo per bit è ancora ordini di grandezza inferiore a quello degli SSD, e probabilmente sarà ancora così per molti anni.

4.1 Interfaccia per il programmatore

Il software interagisce con l'hard disk solo attraverso una interfaccia che, a sua volta, interagisce con il controllore a bordo dell'hard disk. Tramite i registri dell'interfaccia il software impartisce i comandi di lettura o scrittura, specificando quale settore deve essere coinvolto, e accede al buffer interno, sia per estrarne il contenuto dopo la fine di una operazione di lettura, sia per riempirlo con i dati da memorizzare prima dell'inizio di una operazione di scrittura.

Facciamo qui riferimento all'interfaccia ATA (AT Attachment), che standardizza l'interfaccia che si trovava nel PC AT. Questa interfaccia prevede diversi registri a 8 bit e uno a 16 bit, tutti mappati nello spazio di I/O. Ci interessano per il momento soltanto i seguenti:

- SNR (Sector Number), CNL (Cylinder Number Low), CNH (Cylinder Number High), HND (Head and Drive): tutti da 8 bit, permettono di specificare il (primo) settore coinvolto nell'operazione;
- SCR (Sector Counter, 8 bit): permette di specificare quanti settori (in sequenza a partire dal primo) sono coinvolti nell'operazione;
- CMD (CoMmanD, 8 bit): permette di specificare il tipo di operazione (per es., lettura o scrittura);
- BR (Buffer Register, 16 bit): permette di accedere al buffer interno, due byte alla volta;
- STS (STatuS, 8 bit): contiene due flag che permettono di sapere se la precedente operazione si è conclusa (e dunque è possibile accedere al registro BR);

Anche se l'interfaccia prevede che il programmatore identifichi un settore dandone le coordinate geometriche (testina, cilindro e settore), questo ha ormai soltanto un interesse storico, in quanto l'organizzazione interna dei dischi è oggi molto più complessa di un tempo. Quello che faremo è di usare il modo di indirizzamento detto LBA (Logical Block Address), in cui ogni settore è identificato da un numero sequenziale, a partire da 0 fino al numero di settori contenuti nell'hard disk. Questo numero va scomposto in quattro parti e scritto nei registri SNR, CNL, CNH e nei 4 bit meno significativi di HND, perché i 4 bit più significativi di questo registro hanno altre funzioni (tra cui, abilitare LBA). Si noti che in questo modo si dispone di 28 bit per identificare un settore; con settori di 512 byte questo limita la dimensione massima di un hard disk a $2^{28} \times 2^9 = 2^{37}$ byte, cioè 128 GiB, che è poco per un hard disk moderno. Per gli hard disk più grandi, lo standard prevede che il numero di settore sia spezzato in due parti di 28 e 20 bit (modalità LBA48) e scritto in due passaggi negli stessi registri, con un meccanismo simile a quello visto per il timer. L'interfaccia permette di ordinare il trasferimento di più settori consecutivi, specificandone il numero nel registro SCR. In quel caso il controllore provvederà ad incrementare automaticamente l'LBA dopo ogni trasferimento.

La macchina QEMU che usiamo per gli esempi fornisce un hard disk ATA, emulato dal file `CE/share/hd.img` nella nostra home directory. La cartella `esempiIO` contiene due programmi che mostrano come si può ordinare una scrittura (`hard-disk-1`) o una lettura (`hard-disk-2`) da questo hard disk. Entrambi i programmi sono strutturati in modo simile. La funzione `hd_set_lba()` accetta un LBA come parametro, lo spezza in quattro parti e lo scrive nei registri SNR, CNL, CNH e HND, abilitando anche il modo LBA. Il tipo `natl` è definito in `libce` come un **typedef** per **unsigned int** (32 bit). Per semplicità la funzione non abilita la doppia scrittura, quindi `lba` deve essere minore di 2^{28} . La funzione `hd_start_cmd(lba, quanti, cmd)` avvia un trasferimento di `quanti` settori a partire da quello di indirizzo `lba`. Il parametro `cmd` deve essere uno di quelli compresi dall'interfaccia. La libreria definisce le costanti `WRITE_SEC` e `READ_SECT`, che possono essere passate alla funzione per ordinare operazioni rispettivamente di scrittura e lettura. La funzione si limita a trasferire i parametri nei corrispondenti registri (usando `hd_set_lba()` per impostare l'LBA). Una volta avviata l'operazione dobbiamo aspettare che il controllore ci dica che è possibile accedere al buffer interno (funzione `hd_wait_for_br()`) e quindi eseguire 256 scritture (o letture) in BR. Per fare quest'ultima operazione possiamo comodamente usare le istruzioni **outs** e **ins** con prefisso **rep**, che trasferiscono da (verso) la memoria all'indirizzo contenuto in **rsi** il numero di byte/word/long (a seconda del suffisso) specificato in **rcx**. La libreria `libce` contiene le funzioni `outputb*` e `inputb*` (dove l'asterisco sta per `b`, `w` o `l`) che utilizzano queste istruzioni. Si noti che l'operazione di controllo di STS e successiva scrittura/lettura in BR va ripetuta per ogni settore coinvolto nell'operazione.

La funzione `main` usa queste funzioni per scrivere o leggere un paio di settori a partire da un certo LBA, trasferendoli da/verso l'array `buff`. Il programma `hard-disk-2` mostra poi sul video il contenuto dell'array `buff` a lettura ultimata. Il programma `hard-disk-1` mostra solo il messaggio OK per dire che l'operazione si è conclusa. Possiamo verificare che la scrittura è stata effettivamente eseguita correttamente esaminando il contenuto del file che emula l'hard disk "fuori" dalla macchina virtuale, per esempio con il comando

```
hexdump -C ~/CE/share/hd.img
```

che dovrebbe mostrare una serie di caratteri 'f' a partire dall'offset 512 (200 in esadecimale), corrispondente all'inizio del settore con LBA pari a 1.

Interruzioni

G. Lettieri

24 Marzo 2023

Il processore che abbiamo visto fino ad ora esegue un flusso di istruzioni dettato da un unico programma: ogni istruzione del programma ordina al processore sia quali operazioni deve compiere, sia qual è l'istruzione successiva. In particolare, ogni routine del programma va in esecuzione solo perché è stata esplicitamente invocata da una istruzione precedente nel flusso sequenziale, che le ha trasferito “volontariamente” il controllo. Vogliamo ora aggiungere un meccanismo nuovo: il programmatore può associare l'esecuzione di una routine al verificarsi di un *evento*. Più in generale, il sistema definisce un insieme (finito) di eventi $e_1, e_2, \dots, e_n$ e permette al programmatore di associarvi delle routine, chiamiamole $r_1, r_2, \dots, r_n$, con r_i associata all'evento e_i , per $1 \leq i \leq n$. Per programmare il sistema il programmatore deve ora scrivere un programma principale, chiamiamolo p , ma può anche scrivere le routine $r_1, \dots, r_n$ e associarle agli eventi $e_1, \dots, e_n$ (o un sottoinsieme di essi, in base alle proprie necessità). Il processore partirà eseguendo p e obbedendo alle istruzioni di p , in sequenza, esattamente come prima. Mentre esegue p , però, il processore controllerà se si è verificato uno degli eventi previsti e, in tal caso, salterà automaticamente alla routine associata, *interrompendo* il programma p . L'idea è che l'interruzione sia comunque momentanea e che, al termine della routine, l'esecuzione ritorni dal programma p , dal punto in cui era stato interrotto.

Per capire perché potrebbe servirci un meccanismo del genere, e a che tipo di eventi potremmo essere interessati, ispiriamoci ad un esempio che fece uno dei primi ricercatori che si occupò di studiare le interruzioni—Edsger Dijkstra¹. Immaginiamo di dover scrivere un programma che deve calcolare i valori di una funzione $f(x)$ per vari valori di x (il tipico problema per cui i computer sono stati inventati), in modo da stampare una tabella dei valori della funzione:

x	$\sin(x)$
0.1000	0.0998
0.1001	0.0999
0.1002	0.1000
...	

Per guadagnare tempo, ci piacerebbe che, mentre è in corso la stampa dell'ultimo valore calcolato, per esempio per $x = 0.1001$, il nostro programma

¹L'esempio originale si trova qui: <https://www.cs.utexas.edu/users/EWD/transcriptions/EWD13xx/EWD1303.html>.

potesse andare avanti a calcolare il successivo valore, per $x = 0.1002$. In questo modo stampa e calcoli procederebbero in parallelo. Come possiamo fare? Supponiamo di avere un dispositivo di uscita (una stampante) con i due classici registri TBR (Transmit Buffer Register) e un registro di stato STS. Per stampare un carattere ne dobbiamo scrivere il codice ASCII nel TBR. La stampante impiega un certo tempo per stampare il carattere, e per tutto quel tempo non può accettare un nuovo carattere in TBR. Un bit READY del registro di stato ci dice quando la stampante è pronta a ricevere un nuovo carattere. Il nostro programma dovrebbe essere organizzato così: quando ha finito di calcolare un valore di $f(x)$, prepara la riga da stampare in un buffer in memoria, aspetta che la stampante sia pronta (legge ripetutamente STS) e poi scrive il primo carattere. A questo punto, invece di aspettare che la stampante sia pronta a ricevere il prossimo carattere, prosegue con il calcolo del secondo valore. Qui viene la difficoltà: la routine che esegue i calcoli, ogni tanto, deve leggere STS e, se trova la stampante pronta, deve scrivere in TBR il prossimo carattere preso dal buffer, poi proseguire con i calcoli. Ogni quanto bisogna inserire le istruzioni che controllano STS? Se lo si fa troppo spesso si rallenta il calcolo del prossimo valore, se lo si fa troppo raramente si rallenta la stampa. Anche se troviamo una frequenza ideale, potrebbe non esserci una soluzione ottima: che succede se nella routine che fa i calcoli c'è un lungo ciclo con un corpo di poche istruzioni?

```
// controllo STS?
for (int i = 0; i < 1000000; i++) {
    // controllo STS?
    v[i]++;
}
// controllo STS?
```

Inserendo il controllo di STS solo fuori dal ciclo si rischia di far passare troppo tempo tra un controllo e il successivo, ma se lo si inserisce dentro il ciclo si rischia di farlo inutilmente troppo spesso. Che succede poi se il programma che fa i calcoli usa una funzione di libreria già scritta, magari da qualcun altro? Questa sicuramente non conterrà già i controlli di STS, e dovremmo crearne una versione modificata. A questi problemi aggiungiamo anche il fatto che il programma e le librerie diventano presto illegibili se dobbiamo inserire i controlli di STS in mezzo a funzioni che fanno tutt'altro.

1 Interruzioni (singola sorgente)

Per risolvere questi problemi modifichiamo leggermente l'hardware. L'idea di base (da perfezionare) è di portare il bit READY di STS direttamente in ingresso alla CPU (Figura 1), quindi di modificare il microprogramma della CPU in modo che, dopo ogni fase di esecuzione, controlli lo stato del nuovo ingresso e, se lo trova attivo, carichi un nuovo valore in **rip**. L'effetto è di avere inserito automaticamente un salto nel programma nel momento in cui la stampante diventa pronta. Visto che il controllo di READY viene eseguito dopo ogni

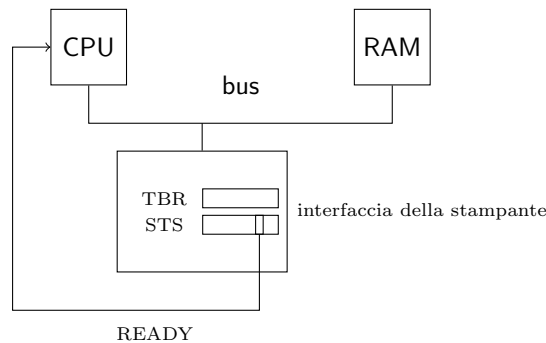


Figura 1: Idea di base del meccanismo delle interruzioni.

istruzione, non rischiamo di farlo troppo raramente (almeno rispetto ai tempi di una stampante); visto che abbiamo realizzato il controllo con un test in hardware di un piedino di ingresso, non paghiamo il costo di dover eseguire le istruzioni che leggono STS e controllano READY; visto che il controllo è fatto automaticamente dalla CPU, il programma principale non deve più contenere queste istruzioni “estrane” che ne compromettono la leggibilità. Il sistema è ora in grado di riconoscere autonomamente il verificarsi dell’evento e che corrisponde a “la stampante è pronta a ricevere il prossimo carattere”. Il programmatore deve solo scrivere una routine r che invia il prossimo carattere da stampare e associarla ad e . Il modo in cui avviene questa associazione, nel caso in cui ci sia un unico evento possibile, può essere molto semplice: il processore può caricare un indirizzo costante in **rip** quando riconosce l’evento, e il programmatore può semplicemente caricare la sua routine r a quell’indirizzo. Il programmatore poi scrive il programma principale p che ciclicamente calcola un nuovo valore, prepara la riga da stampare nel buffer e passa a calcolare il valore successivo. Periodicamente, il programma p verrà *interrotto* per eseguire la routine r che stampa il prossimo carattere preso dal buffer².

Un modo per concettualizzare ciò che avviene, dal punto di vista della stampante, è che questa ha bisogno di “attenzione” da parte del software quando ha finito di stampare un carattere, perché il software le deve dire cosa stampare dopo. Inoltre dunque una richiesta di attenzione, o di *servizio*, alla CPU, settando il bit READY. La CPU reagisce a questa richiesta interrompendo il programma principale (per questo la richiesta della stampante viene anche detta *richiesta di interruzione*) e cedendo forzatamente il controllo ad una *routine di servizio* per la stampante. Quando questa routine scrive il prossimo carattere in TBR, la stampante (la sua interfaccia) pone READY a zero, e questo “rimuove” la richiesta di servizio dalla CPU, a significare che la stampante ha ricevuto il

²Ovviamente il programma principale e la routine r si devono coordinare per capire quando la riga è stata completamente stampata; non affrontiamo questi dettagli che ci porterebbero troppo lontano.

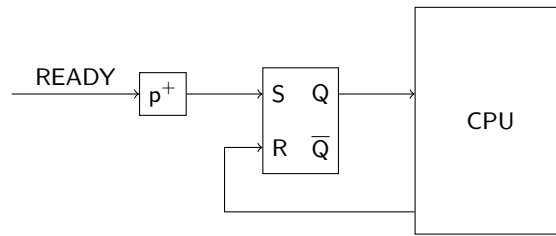


Figura 2: Rete per il riconoscimento di una nuova richiesta di interruzione.

servizio che aveva richiesto. La scrittura in TBR, dunque, funge da *risposta* alla richiesta di interruzione.

Questa è l'idea di base, che però richiede di essere perfezionata perché possa funzionare effettivamente.

- A livello hardware, dobbiamo intanto assicurarci che la CPU non veda due volte la stessa richiesta. Per come abbiamo descritto il meccanismo, infatti, quello che accade è che, non appena la CPU vede READY 1, salta alla prima istruzione della routine di servizio. Se questa istruzione non è esattamente quella che scrive in TBR, alla fine della sua esecuzione READY sarà ancora a 1 e, dunque, la CPU salterà nuovamente alla prima istruzione della routine di servizio, ciò quella appena eseguita, e così all'infinito. Questo problema può essere risolto in hardware con una rete come in Figura 2, in cui la richiesta viene trasformata in un impulso che setta un latch S/R. Il microprogramma della CPU provvederà a resettare il latch dopo aver visto la richiesta, in modo che il latch sia di nuovo attivo solo se arriva una *nuova* richiesta di interruzione.
- Se vogliamo che la routine di servizio possa tornare al programma principale nel punto in cui era stato interrotto, la CPU non può limitarsi a sovrascrivere **rip** quando accetta la richiesta, ma deve prima scrivere il precedente valore da qualche parte. Una soluzione è di scriverlo sullo stack, in modo che la routine di servizio possa terminare con una **ret** (o, come vedremo, con una istruzione simile); questa è la soluzione adottata nei procesori Intel.
- Anche a livello software vanno risolti molti problemi, per poter scrivere correttamente il programma principale e la routine di servizio. In realtà, i problemi sono così tanti che Dijkstra, non a torto, ha scritto che questo semplice meccanismo aprì un “vaso di Pandora”. Per il momento vi accenniamo soltanto—avremo modo di ritornarci.

Risolti questi problemi, possiamo pensare di raffinare ed estendere il meccanismo in vari modi. Il problema principale, dal punto di vista del software, è l'imprevedibilità del momento in cui può essere accettata una richiesta, con conseguente salto alla routine di servizio. Dal punto di vista del programmatore è

utile poter temporaneamente “disabilitare” le richieste di interruzione, fare cioè in modo che la CPU le ignori durante l’esecuzione di certe porzioni di codice. La soluzione tipicamente adottata, anche nei processori Intel, è che la CPU abbia un flag di *interrupt enabled* che possa essere settato e resettato via software, e che la CPU ascolti le richieste di interruzione solo se questo flag è settato. Nel processore intel questo è il flag IF (Interrupt Flag, bit 9) del registro dei flag e le istruzioni `cli` e `sti` possono essere usate per resettarlo e settarlo, rispettivamente. Le richieste che dovessero arrivare mentre IF è zero verranno servite non appena IF tornerà a 1.

Una porzione di codice che non vogliamo venga interrotta è, tipicamente, la routine di servizio stessa, cosa che invece può accadere non appena la routine esegue l’istruzione di risposta alla richiesta di servizio (la scrittura in TBR, nel nostro esempio). La situazione è talmente tipica che il processore Intel può (su richiesta del programmatore) resettare automaticamente il flag IF ogni volta che accetta una richiesta. Prima di farlo, il processore salva in pila il vecchio valore del registro dei flag, oltre al vecchio valore di `rip`. Il processore fornisce poi l’istruzione `iretq` che preleva dalla pila sia `rip` che i flag. Le routine di servizio devono terminare con `iretq`, invece che con `ret`, in modo da estrarre dalla pila entrambi i valori (e altri che vedremo) e riportare IF a 1.

2 Interruzioni (più sorgenti)

Un altro modo di estendere il meccanismo è quello di prevedere più sorgenti di interruzione, in modo che il sistema possa riconoscere un certo numero di eventi distinti e il programmatore possa associare una routine di servizio diversa a ciascuno di questi eventi, come avevamo detto all’inizio. Delle periferiche che abbiamo studiato, tre sono in grado di generare richieste di interruzione:

- la tastiera, quando RBR è pieno o TBR si svuota dopo una scrittura;
- il timer, quando il contatore 0, opportunamente programmato, termina il conteggio;
- l’hard disk, quando il buffer interno è pieno dopo una operazione di lettura o vuoto dopo una operazione di scrittura.

La tastiera e l’hard disk generano richieste di interruzione solo se queste sono state abilitate *nell’interfaccia*. Questa è un’altra soluzione molto tipica che permette al programmatore, per esempio, di non dover gestire richieste di interruzione da parte di interfacce che non sta usando. Le interfacce prevedono dunque uno o più registri di *controllo*, con varie funzioni tra cui abilitare o disabilitare l’interfaccia a generare richieste di interruzione (consultare gli esempi di I/O per i dettagli).

Per supportare più sorgenti di richieste di interruzione dobbiamo rispondere a tre domande:

- come inoltrare le richieste alla CPU;

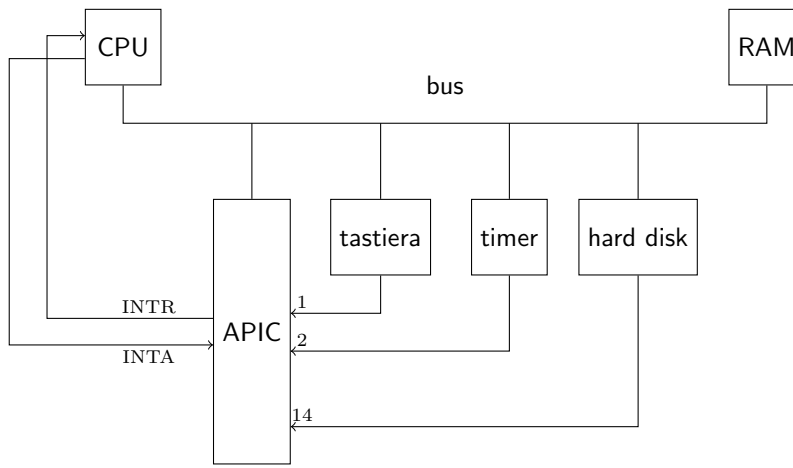


Figura 3: Controllore delle interruzioni.

- se e come associare una routine di servizio diversa ad ogni sorgente;
- cosa fare in caso di richieste che arrivano mentre ancora è in corso il servizio di una richiesta precedente da parte di un'altra sorgente.

2.1 Controllore delle interruzioni

Per rispondere alla prima domanda, teniamo presente che il meccanismo che abbiamo introdotto non ha cambiato la natura fondamentale sequenziale della CPU: la CPU, dal suo punto di vista, continua ad avere un unico flusso di controllo e ad ogni istante esegue un unico programma: il programma principale, oppure una routine di servizio³. Anche se abbiamo più sorgenti di richieste di interruzione e più routine di servizio, la nostra CPU potrà comunque eseguirne solo una alla volta. Ha allora senso che la CPU abbia un unico piedino su cui riceve richieste di interruzione, e un qualche componente esterno si preoccupi di raccogliere tutte le richieste provenienti dalle varie sorgenti, ordinarle in qualche modo e inoltrarle una alla volta alla CPU⁴. Questo componente è detto *controllore delle interruzioni*. Lo schema del sistema è ora quello di Figura 3. In particolare, facciamo riferimento al controllore APIC (Advanced Programmable Interrupt Controller) che si trova nei sistemi basati su chipset Intel e

³La CPU, in realtà, non è neanche consapevole della distinzione: per lei, la gestione della richiesta di interruzione termina subito dopo aver effettuato il salto, dopo di che riprende il normale ciclo di prelievo, decodifica, esecuzione, a partire dal nuovo **rip**. In particolare, la CPU non è “in attesa” che venga eseguita una **iretq** dopo l'accettazione di una richiesta di interruzione, come non è in attesa che venga eseguita una **ret** dopo una **call**.

⁴Anche se la CPU ha più di un piedino di ingresso per le richieste di interruzione, serve comunque una circuiteria, interna alla CPU stessa ma concettualmente distinta, che le raccolga e ne lasci passare una sola per volta. Vedremo che, in effetti, la CPU Intel, come molte altre, ha anche un altro piedino da cui riceve richieste di interruzione “non mascherabili”: queste hanno precedenza su tutte le altre.

anche, emulato, nella nostra macchina QEMU. L'APIC possiede 24 piedini di ingresso, numerati da 0 a 23, a cui possono essere collegate le linee di richiesta di interruzione di altrettante interfacce. In figura vediamo come sono collegate le periferiche della macchina QEMU. Il controllore permette la gestione *vettorizzata* delle richieste di interruzione. Con questo termine ci si riferisce al fatto che ad ogni sorgente di interruzione è associato un *tipo* numerico, che viene passato alla CPU insieme alla richiesta. La CPU utilizza questo tipo per consultare una tabella che associa i tipi agli indirizzi delle routine di servizio. Nei processori Intel la tabella si chiama Interrupt Descriptor Table (IDT) e il programmatore la può allocare ovunque in memoria, quindi caricarne l'indirizzo nel registro IDTR del processore. In questo modo è possibile associare una routine di servizio diversa ad ogni sorgente: basta scrivere l'indirizzo della routine nell'entrata della IDT corrispondente al tipo da associare. In assenza della vettorizzazione, ad ogni richiesta andrebbe in esecuzione sempre la stessa routine, che a quel punto dovrebbe capire via software quale interfaccia ha richiesto il servizio (per esempio, consultando i registri di stato di ogni interfaccia).

Nell'APIC l'associazione tra sorgente e tipo è configurabile dal programmatore. Il controllore possiede infatti un diverso registro interno, accessibile via software, per ognuno dei piedini di ingresso. Per ogni piedino è possibile specificare il tipo, su 8 bit, e altre informazioni legate a come l'APIC dovrà riconoscere le richieste in ingresso. In particolare, per ogni piedino è possibile decidere:

- se il riconoscimento di una nuova richiesta deve avvenire sul fronte (segnale in ingresso che cambia livello) o sul livello (vedere più avanti);
- se il segnale di ingresso deve essere considerato attivo quando è alto oppure quando è basso;
- se le richieste devono essere ignorate o meno (mascherate).

Per il dialogo con il processore supponiamo che ci siano due collegamenti: uno, INTR (INTerrupt Request), dall'APIC alla CPU, tramite il quale l'APIC inoltra una richiesta di interruzione; un altro, INTA (INTerrupt Acknowledge), dalla CPU all'APIC, usato per un protocollo di handshake:

1. quando l'APIC vuole inviare una richiesta di interruzione, attiva INTR e aspetta che sia attivo INTA;
2. la CPU controlla lo stato di INTR dopo l'esecuzione di ogni istruzione, se IF è 1; quando lo trova attivo, attiva a sua volta INTA;
3. quando l'APIC vede INTA attivo, passa il tipo dell'interruzione al processore sul bus e disattiva INTR⁵

⁵Restiamo sul vago su come il tipo viene passato, in quanto il vero APIC è in realtà scomposto in due parti: un I/O APIC sulla scheda madre e un Local APIC all'interno della CPU. Il dialogo, compreso lo scambio del tipo, avveniva su un bus dedicato fino al Pentium III, e poi usando il bus generale. Questa organizzazione è pensata per i sistemi con più CPU, in modo da avere, per esempio, un unico I/O APIC che dialoga con tanti Local APIC, uno per CPU. Nel nostro caso possiamo semplificare la discussione, perché ci limitiamo al caso con una sola CPU.

4. quando il processore vede INTR disattivo, legge il tipo e disattiva INTA, chiudendo l'handshake.

L'APIC possiede altri tre registri interni che ci servono per capire in dettaglio il suo funzionamento: IRR (Interrupt Request Register), ISR (In Service Register) e EOI (End Of Interrupt). I registri IRR e ISR sono entrambi da 256 bit, un bit per ogni possibile tipo. Supponiamo per il momento che una sola interfaccia sia abilitata a inviare richieste di interruzioni, e chiamiamo i il piedino di ingresso dell'APIC a cui tale interfaccia è collegata. Supponiamo anche che il piedino i sia impostato per il riconoscimento sul fronte, e il tipo associato sia t . L'APIC svolge continuamente le seguenti azioni:

1. ad ogni fronte su i , se il bit t di IRR non è già attivo, lo attiva;
2. se il bit t di IRR è attivo e il bit t di ISR non lo è, l'APIC inizia l'handshake con la CPU;
3. quando l'handshake arriva al punto 3 (la CPU ha accettato la richiesta), invia il tipo t e "sposta" il bit attivo da IRR in ISR (disattiva t in IRR e lo attiva in ISR);

Lo scopo di ISR è di tenere traccia di quali richieste sono attualmente in servizio sulla CPU. L'APIC assume che la CPU, nel momento in cui accetta la sua richiesta, passi ad eseguire la routine di servizio corrispondente, e per questo setta il bit t di ISR a 1. L'APIC non inoltrerà altre richieste dello stesso tipo fino a quando si troverà in questo stato (si veda il punto 2 qui sopra: l'handshake parte solo se il bit t di ISR è a zero). Un eventuale altro fronte sul piedino i verrà comunque registrato in IRR (punto 1). In pratica è come se i due bit t in IRR e ISR realizzassero una coda di due richieste: una in servizio e l'altra in attesa. Una terza richiesta che dovesse arrivare in questo stato, però, verrebbe persa. È compito del software far sapere all'APIC che la routine di servizio è terminata, scrivendo un valore qualsiasi nel registro EOI. In risposta a questa scrittura l'APIC azzerà il bit t in ISR e, se il bit t di IRR è 1 (c'è un'altra richiesta pendente arrivata nel frattempo) si riporta al punto 2, altrimenti al punto 1.

Nel caso di riconoscimento sul livello l'APIC si comporta diversamente solo per quanto riguarda il riconoscimento di una nuova richiesta sul piedino: se il bit t di ISR è a zero e c'è un fronte su i , l'APIC registra una nuova richiesta (se non era già registrata), come nel punto 1 qui sopra; se però ISR è a 1 (richiesta già in servizio), l'APIC smette di osservare il piedino i ; lo osserverà di nuovo solo all'arrivo dell'EOI che rimette il bit t di ISR a zero: se in quel momento ritrova il piedino i ancora (o di nuovo) attivo, registra una nuova richiesta. L'idea è che normalmente la periferica dovrebbe aver disattivato il piedino i , una volta che la routine di servizio ha effettuato la risposta alla richiesta (si pensi all'esempio della stampante: la richiesta resta attiva fino a quando la routine non scrive in TBR) e il software dovrebbe inviare l'EOI *dopo* che la routine ha effettuato l'azione di risposta. Dunque, se all'arrivo dell'EOI la richiesta è ancora attiva, deve trattarsi di una nuova richiesta.

Negli esempi vedremo quando conviene scegliere un riconoscimento sul fronte o uno sul livello. Si veda anche l'appendice per un altro utilizzo del riconoscimento sul livello.

2.2 Gestione annidata delle richieste di interruzione

Passiamo ora a considerare il caso generale, in cui le richieste possono arrivare da qualsiasi piedino, anche contemporaneamente. In generale si preferisce realizzare la *gestione annidata* delle richieste di interruzione, in cui una routine di servizio può essere a sua volta interrotta da un'altra routine di servizio. La cosa ha senso se la seconda routine ha una *priorità* maggiore rispetto alla prima (per esempio, le richieste che arrivano dal timer hanno in genere massima priorità, in quanto non è possibile ritardarle). Ovviamente l'annidamento può avvenire a qualsiasi livello, con la seconda routine che può essere interrotta da una terza, a maggiore priorità, e così via. Dal momento che tutti gli indirizzi a cui ritornare sono salvati sulla stessa pila, le routine di servizio devono poi terminare in ordine LIFO: l'ultima ritorna alla penultima e così via.

Affinché l'annidamento sia possibile, la CPU non deve disabilitare le interruzioni mentre è in esecuzione una routine di servizio. Nel processore Intel questo può essere ottenuto facilmente, in quanto la disabilitazione delle interruzioni è facoltativa (basta settare un flag nelle entrate della IDT corrispondenti alle routine che si vuole eseguire in questo modo).

L'APIC può essere usato per aiutare nella gestione annidata delle interruzioni con priorità. L'APIC, infatti, interpreta il tipo associato ad un piedino come la priorità delle richieste provenienti da quel piedino. Più precisamente, i 4 bit più significativi del tipo rappresentano, per l'APIC, la sua *classe di priorità*, in ordine numericamente crescente (15 è dunque la classe di priorità massima). Quando una nuova richiesta viene registrata in IRR, l'APIC confronta la classe di priorità della richiesta più a sinistra in IRR (la richiesta pendente a priorità maggiore), sia p_r , con la classe di priorità della richiesta più a sinistra in ISR (la richiesta in servizio a priorità maggiore), sia p_s , e inizia l'handshake con la CPU se, e solo se, $p_r > p_s$. Quando la CPU accetta questa richiesta l'APIC si comporta come al solito, spostando il bit da IRR in ISR. Si noti che ora più bit di ISR possono essere a 1, riflettendo il fatto che più routine di servizio hanno iniziato la propria esecuzione e non sono ancora concluse. Si noti, inoltre, che il fatto che l'APIC non inoltra le richieste con $p_r \leq p_s$ comprende il caso particolare, che abbiamo discusso in precedenza, in cui arrivi una nuova richiesta dallo stesso piedino di una richiesta già accettata, perché in quel caso le classi di priorità sono uguali.

Quando il software scrive in EOI, l'APIC resetta il bit più a sinistra in ISR. In pratica, l'APIC assume che il software stia rispettando l'ordine LIFO di esecuzione delle routine di servizio, e dunque che la scrittura sia stata effettuata perché è terminata la routine a più alta priorità tra quelle attualmente attive. Dopo aver resettato il bit, l'APIC ricontrolla le classi di priorità in IRR e in ISR per vedere se è necessario inoltrare una nuova richiesta.

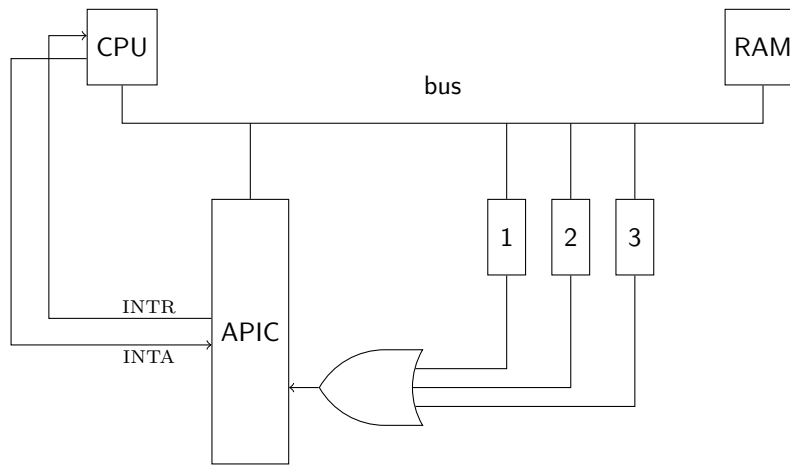


Figura 4: Tre interfacce collegate allo stesso piedino dell'APIC tramite una porta OR.

Si noti che l'APIC, per decidere se inoltrare o no una richiesta, guarda esclusivamente le classi di priorità, ignorando i 4 bit meno significativi dei tipi. Al momento di inoltrare una nuova richiesta alla CPU, però, usa l'intero tipo per scegliere quale richiesta inviare tra quelle registrate in IRR.

A Condivisione delle richieste di interruzione

È possibile collegare più di una interfaccia allo stesso piedino dell'APIC, e per molto tempo questa è stata una pratica comune, dal momento che i piedini disponibili erano spesso troppo pochi. La Figura 4 mostra un possibile modo in cui la condivisione può essere realizzata: all'ingresso dell'APIC abbiamo l'OR logico delle varie richieste di interruzione. Questo comporta che tutte le richieste in OR avranno tutte lo stesso tipo (in quanto il tipo è associato al piedino dell'APIC, che è lo stesso per tutte) e dunque che, in risposta alla richiesta di interruzione di una qualunque delle interfacce, partirà sempre la stessa routine di gestione (in quanto la routine è associata al tipo tramite la IDT). In altre parole, le interfacce collegate come in Figura 4 condividono la stessa richiesta di interruzione. Affinchè questa condivisione sia possibile, ciascuna interfaccia deve contenere almeno un registro da cui si possa capire se l'interfaccia ha inoltrato una richiesta oppure no, quindi la routine di servizio dovrà essere strutturata nel seguente modo:

```

void c_servizio()
{
    if (/* richiesta da interfaccia 1 */ {
        /* servo interfaccia 1 */
    } else if (/* richiesta da interfaccia 2 */) {

```

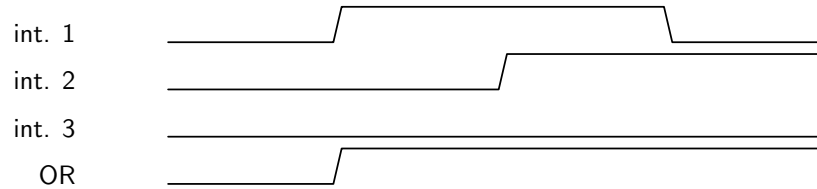


Figura 5: Mascheramento dei fronti con condivisione delle richieste di interruzione. L'interfaccia 2 inoltra la sua richiesta prima che l'interfaccia 1 abbia ritirato la propria.

```

    /* servo interfaccia 2 */
} else if (/* richiesta da interfaccia 3 */) {
    /* servo interfaccia 3 */
} // eventuali altre interfacce
apic_send_EOI();
}

```

Affinchè questa soluzione funzioni, però, è necessario che il piedino dell'APIC così condiviso sia configurato per il riconoscimento sul *livello*, e non sul *fronte*. Supponiamo, infatti, che mentre è in corso la gestione di una richiesta, per esempio, da parte dell'interfaccia 1, e prima che `c_servizio()` abbia fornito la risposta attesa dall'interfaccia 1, arrivi anche una richiesta da un'altra interfaccia, per esempio la 2 (si veda la Figura 5): il fronte in salita della richiesta proveniente dall'interfaccia 2 non sarà visibile in uscita dall'OR, in quanto la richiesta 1 sarà ancora attiva. Dunque, se l'APIC riconoscesse nuove richieste solo sul fronte, la richiesta proveniente dall'interfaccia 2 andrebbe persa. Invece, con il riconoscimento su livello, l'APIC si accorgerebbe della nuova richiesta quando `c_servizio()` invia l'EOI.

Si noti che il problema non potrebbe risolversi soltanto modificando la routine `c_servizio()`, per esempio facendole ricontrollare tutte le interfacce dopo averne servita una: la nuova richiesta potrebbe sempre arrivare subito dopo che `c_servizio()` ha interrogato l'interfaccia richiedente. Nè possiamo pensare di far ciclare `c_servizio()` all'infinito, perchè a quel punto non si tornerebbe più al programma principale e non verrebbero neanche più servite eventuali richieste provenienti da altri piedini con privilegio inferiore o uguale.

B Porte open-collector e wired-or

La porta OR che si vede in Figura 4 non è tipicamente presente nei sistemi reali. Invece, la condivisione di una linea di richiesta di interruzione è realizzata come in Figura 6. Le interfacce, in questo caso, devono avere uscite *open collector*. In questo tipo di uscite il livello logico zero è realizzato collegando l'uscita a massa, mentre il livello logico 1 è realizzato ponendo l'uscita in alta impedenza.

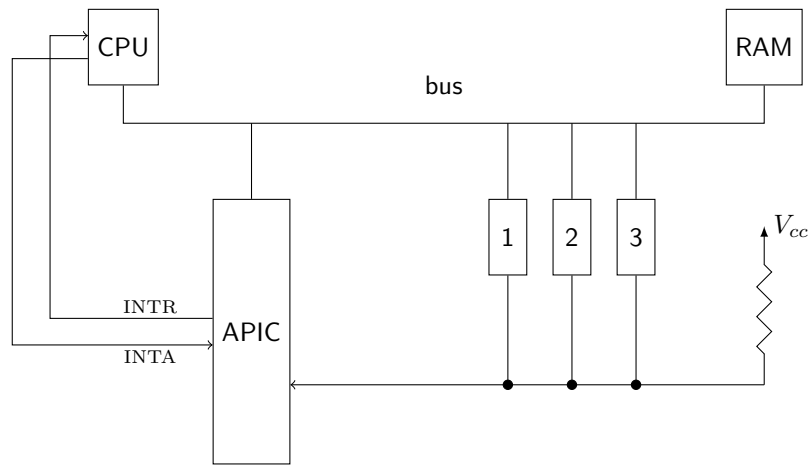


Figura 6: Tre interfacce con uscita open-collector collegate allo stesso piedino dell'APIC in configurazione *wired-or*.

In questo modo, se una o più interfacce vogliono produrre l'uscita zero, l'uscita complessiva è zero, mentre se tutte vogliono produrre l'uscita uno, la linea condivisa è portata al livello alto dalla resistenza (detta di *pull up*) collegata alla tensione di alimentazione. Questo corrisponde a calcolare l'OR delle uscite se le interpretiamo come attive basse (cioè se le interfacce pongono la loro uscita a zero quando vogliono inoltrare una richiesta di interruzione). Ovviamente bisogna anche configurare il piedino dell'APIC per il riconoscimento sul livello basso invece che alto.

Eccezioni

G. Lettieri

29 Marzo 2023

Le prime 32 entrate della Interrupt Descriptor Table sono riservate per le *eccezioni*. Con questo termine ci si riferisce a condizioni di errore o speciali che il processore rileva mentre sta eseguendo le normali istruzioni. Per esempio, una operazione di divisione (`div` o `idiv`) in cui il divisore è zero causa una eccezione di “divisione per zero”. Il processore tratta le eccezioni in modo molto simile alle interruzioni esterne: vi associa un tipo numerico (compreso tra 0 e 31) e lo usa per accedere ad un gate della IDT, dove trova l’indirizzo di una routine a cui saltare. I tipi delle eccezioni sono fissi e consultabili sul manuale del processore. Per esempio, l’eccezione di divisione per zero ha tipo 0. Il meccanismo di salto è del tutto simile a quello delle interruzioni esterne, con il salvataggio in pila di informazioni analoghe. In particolare, anche le routine di gestione delle eccezioni, se vogliono tornare al programma principale, devono farlo con una istruzione **`iretq`** e non con una semplice **`ret`**.

Le eccezioni sono classificabili ulteriormente come segue:

trap vengono sollevate solo tra l’esecuzione di una istruzione e la successiva;

fault vengono sollevate *durante* l’esecuzione di una istruzione;

abort possono verificarsi in qualsiasi momento e indicano errori particolarmente gravi.

Per le eccezioni di tipo fault il processore salva in pila l’indirizzo dell’istruzione che stava eseguendo mentre ha rilevato il fault, e non l’indirizzo dell’istruzione successiva come per i trap e le interruzioni esterne. L’idea è che a volte la routine di gestione dell’eccezione può correggere la condizione che ha causato il fault e poi ritornare (con la **`iretq`** finale) a *rieseguire* l’istruzione, che questa volta non dovrebbe più generare il fault o al massimo dovrebbe generarne uno diverso, per qualche altra condizione di errore.

Si noti che, per rendere possibile la riesecuzione di una istruzione che era stata precedentemente eseguita fino ad un certo punto (e dunque poteva aver già modificato qualche registro), il processore deve essere in grado di ritornare allo stato precedente l’inizio della prima esecuzione. Per quanto riguarda lo stato dei registri, il processore può apportare tutte le modifiche in delle copie di lavoro, trasferendo il contenuto dalle copie ai veri registri solo alla fine dell’esecuzione di ogni istruzione. Se si verifica un fault durante l’esecuzione, è sufficiente ignorare

```

1 int main()
2 {
3     int x = 3, y = 0;
4     return x / y;
5 }

```

Figura 1: Un programma che causa una eccezione di “divisione per zero”.

le copie di lavoro per ritornare allo stato precedente. Il caso delle istruzioni che scrivono in memoria va invece esaminato istruzione per istruzione, ma in generale non ci sono problemi se si esegue una sola scrittura alla fine dell’esecuzione (che è il caso più comune).

La libreria `libce`, che abbiamo usato fino ad ora per tutti gli esempi, inizializza la IDT in modo che ogni eccezione causi un salto ad una funzione (della libreria stessa) che stampa un messaggio di errore e blocca il processore eseguendo l’istruzione `hlt` (si veda la funzione `init_idt` nel file `64/init_idt.s`, seguendo la catena di chiamate fino a `gestore_eccezioni()`).

In Figura 1 troviamo un semplice programma che causa una eccezione. Si noti che il programma deve essere compilato senza ottimizzazioni, altrimenti il compilatore si accorge facilmente del tentativo di dividere per 0 e, dal momento che lo standard dichiara tale operazione come “indefinita”, può tradurre l’operazione in qualunque modo. In questo caso, molto probabilmente, si limiterebbe ad eliminare l’operazione di divisione. Scriviamo il codice di Figura 1 in un file di nome `prova.cpp` e compiliamolo con lo script `compile`¹. Possiamo controllare che il compilatore abbia effettivamente generato l’istruzione di divisione osservando l’output del comando

```
objdump -d | grep idivl
```

Dovremmo ottenere qualcosa di simile:

```
20015d: f7 7d f8          idivl  -0x8(%rbp)
```

Il primo numero è l’indirizzo a cui si troverà l’istruzione quando il programma verrà caricato.

Se lanciamo ora l’eseguibile con lo script `boot` dovremmo vedere che la macchina virtuale parte e si ferma, scrivendo sulla console un messaggio di errore. Il messaggio mostra il tipo dell’eccezione (0) e l’instruction pointer salvato in pila. In questo caso dovrebbe coincidere con `0x20015d`, perché l’eccezione 0 è di tipo `fault`.

Per vedere la differenza con le eccezioni di tipo `trap` proviamo a generare l’eccezione di *break-point*, che ha tipo 3. Questa eccezione è sollevata dall’istruzione `int3` alla fine della sua esecuzione. Essendo di tipo `trap`, il processore deve salvare in pila l’indirizzo dell’istruzione successiva. Dopo aver copiato il codice di

¹Come per tutti gli esempi che usano gli script `compile` e `boot`, dobbiamo trovarci in una directory che contiene inizialmente solo i file `*.cpp` e `*.s` dell’esempio.

```

1 int main()
2 {
3     int x = 0;
4     x++;
5     asm("int3");
6     x++;
7     return x;
8 }

```

Figura 2: Un programma che causa una eccezione di “break-point”.

Figura 2 in un file `prova.cpp` e averlo compilato con lo script `compile`, prendiamo nota dell’indirizzo dell’istruzione `int3` e delle istruzioni che le stanno attorno, con il comando:

```
objdump -d | grep -C 1 int3
```

Dovremmo ottenere un output del genere:

```

200152: 83 45 fc 01          addl    $0x1,-0x4(%rbp)
200156: cc                  int3
200157: 83 45 fc 01          addl    $0x1,-0x4(%rbp)

```

Se ora lanciamo il programma sulla macchina virtuale questa si ferma nuovamente con un messaggio di errore, questa volta per l’eccezione di tipo 3. L’instruction pointer salvato deve essere quello dell’istruzione *successiva* alla `int3` (0x200157 nel nostro caso).

Per eseguire una routine di nostra scelta ogni volta che si verifica una eccezione, operiamo in modo del tutto analogo a come abbiamo fatto per le interruzioni esterne: prepariamo un gate della IDT in modo che punti alla nostra funzione. Questa volta, però, non possiamo scegliere il gate che preferiamo, ma dobbiamo usare quello relativo all’eccezione che vogliamo intercettare. Per esempio, se vogliamo intercettare le eccezioni di divisione per zero dobbiamo usare il gate 0, e se vogliamo intercettare i break-point dobbiamo usare il gate 3.

Come esempio intercettiamo le eccezioni di tipo 1. Queste sono eccezioni di *debug* generate, tra le altre cose, dal meccanismo del *Single Step* (esecuzione passo passo). Tale meccanismo viene attivato settando il flag TF del registro dei flag. Quando tale flag è attivo il processore genera una eccezione di debug dopo aver eseguito *ogni istruzione*. Si tratta di una eccezione di trap, quindi l’instruction pointer salvato è quello dell’istruzione successiva (ancora da eseguire).

Il programma in Figura 3 associa la funzione `a_debug` al gate numero 1 (linea 14). La funzione `a_debug` è definita in Figura 4 e si limita a chiamare la funzione `c_debug` passandole il valore dell’instruction pointer salvato in pila dal processore (linea 17). Si noti che la funzione salva e ripristina tutti i registri, perché verrà chiamata dopo ogni istruzione del programma principale (se TF

```

1  #include <libce.h>
2
3  extern "C" void enable_single_step();
4  extern "C" void disable_single_step();
5  extern "C" void a_debug();
6  extern "C" void c_debug(void *rip)
7  {
8      flog(LOG_DEBUG, "rip=%p", rip);
9  }
10
11 int main()
12 {
13     int x = 0;
14     gate_init(1, a_debug);
15     enable_single_step();
16     x++;
17     x++;
18     x++;
19     x++;
20     disable_single_step();
21     pause();
22     return x;
23 }

```

Figura 3: Un programma che intercetta l'eccezione di debug, parte C++.

```

1 #include "libce.s"
2 .global enable_single_step
3 enable_single_step:
4     pushf
5     orw $0x0100, (%rsp)
6     popf
7     ret
8 .global disable_single_step
9 disable_single_step:
10    pushf
11    andw $0xFEFF, (%rsp)
12    popf
13    ret
14 .global a_debug
15 a_debug:
16     salva_registri
17     movq 120(%rsp), %rdi
18     call c_debug
19     carica_registri
20     iretq

```

Figura 4: Un programma che intercetta l’eccezione di debug, parte Assembler.

è settato). La macro `salva_registri` è definita nel file `libce.s` (nella libreria) e salva in pila tutti i registri generali tranne `rsp` (perché questo verrà ripristinato dal normale utilizzo della pila). Dunque, al momento di eseguire l’istruzione alla linea [17](#), l’instruction pointer si troverà 120 byte più in basso rispetto alla cima corrente della pila ($120 = 8 \times 15$).

La funzione `c_debug` ([Figura 3](#)) stampa il valore dell’instruction pointer utilizzando la funzione di libreria `flog()`, che invia il messaggio sulla porta seriale. Nel nostro caso abbiamo impostato la macchina virtuale affinché tutto ciò che viene inviato alla porta seriale venga mostrato sul terminale da cui abbiamo eseguito `boot`, quindi è lì che vedremo questi messaggi. La funzione `flog()` accetta un primo valore che indica il tipo di messaggio che si sta inviando (i possibili valori sono `LOG_DEBUG`, `LOG_INFO`, `LOG_WARN` e `LOG_ERR`). L’effetto è solo quello di cambiare le prime tre lettere della linea che viene inviata alla porta seriale, ma l’informazione potrebbe essere usata per filtrare i tipi di messaggi che si vuole o non si vuole vedere. Il secondo argomento è una stringa che verrà interpretata in modo simile a come opera la funzione `printf()` della libreria standard del C (la funzione si può usare anche in C++, includendo `<cstdio>`). La stringa rappresenta un modello per il messaggio che deve essere generato. Tutti i caratteri diversi da “%” verranno riprodotti così come sono, mentre i caratteri “%” rappresentano dei segnaposto per dei valori che devono essere inseriti in quel punto. Per ogni segnaposto deve essere passato

un ulteriore parametro (in ordine, dopo la stringa) e deve essere specificato, dopo il carattere “%”, in che modo il valore del parametro deve essere mostrato. Nel nostro caso usiamo il carattere “p” per specificare che vogliamo mostrare un puntatore (verrà stampato in esadecimale) e passiamo `rip` come parametro corrispondente. Altri caratteri possibili sono “d” per numeri da mostrare in base 10, “x” per numeri da mostrare in esadecimale, e “s” per stringhe di caratteri. I caratteri “d” e “x” richiedono parametri di tipo **int**, ma possono essere preceduti dal carattere “l” per usare parametri di tipo **long**.

La funzione `main()` abilita il Single Step (linea 15), esegue un po’ di istruzioni, e poi lo disabilita (linea 20). Le funzioni di abilitazione e disabilitazione sono scritte in Assembler (Figura 4). Si noti che non ci sono istruzioni apposite per modificare il flag TF, quindi utilizziamo le istruzioni **pushf** per salvare il contenuto del registro dei flag in pila, manipoliamo il valore in cima alla pila, e infine lo ricarichiamo nel registro dei flag con l’istruzione **popf**.

Compilando e avviando il programma dovremmo vedere un output del genere sulla console (non sul monitor della macchina emulata):

```
DBG 0 rip=0000000000200190
DBG 0 rip=0000000000200194
DBG 0 rip=0000000000200198
DBG 0 rip=000000000020019c
DBG 0 rip=00000000002001a0
DBG 0 rip=00000000002001b8
DBG 0 rip=00000000002001b9
DBG 0 rip=00000000002001bf
DBG 0 rip=00000000002001c0
```

Se si va a guardare a cosa corrispondono i vari indirizzi (`objdump -dS`) si noteranno alcune cose.

- Il processore genera (o non genera) l’eccezione di debug alla fine di una istruzione, ma lo fa in base al valore che TF aveva subito prima di eseguirla. L’istruzione alla linea 6 di Figura 4 porta TF a 1, ma prima che iniziasse TF valeva 0. Quindi il processore non genera l’eccezione alla sua fine, ma solo alla fine dell’istruzione successiva, che parte quando TF è già 1. L’eccezione è di tipo trap, quindi l’instruction pointer salvato è quello dell’istruzione ancora successiva. Il primo **rip** stampato, dunque, dovrebbe essere quello della prima istruzione `x++` di `main()`. L’ultimo **rip** stampato dovrebbe essere invece quello dell’istruzione **ret** alla linea 13 di Figura 4: l’istruzione **popf** porta TF a 0, ma quando era partita TF era ancora 1, e dunque verrà generata ancora una eccezione di debug alla sua fine, salvando l’instruction pointer dell’istruzione successiva.
- Il processore non genera ulteriori eccezioni di Single Step mentre sono in esecuzione le funzioni `a_debug` e `c_debug`. Questo perché il flag TF viene automaticamente resettato ogni volta che si attraversa un gate della IDT. L’idea è che vogliamo eseguire passo passo un certo programma da

```

1 #include <libce.h>
2 void foo() {
3     printf("foo()\n");
4 }
5
6 int main()
7 {
8     foo();
9     pause();
10    return 0;
11 }

```

Figura 5: Esercizio 1

debuggare, ma non vogliamo eseguire passo passo anche il debugger stesso. Il vecchio valore di TF viene salvato in pila, insieme a tutti gli altri flag, e ripristinato dalla **iretq** che termina la routine di eccezione. Si noti come, in base alla regola precedente, non ci sarà una nuova eccezione subito dopo la **iretq**, ma solo dopo l'istruzione successiva, che è quello che vogliamo.

Esercizio 1

L'istruzione **int3** può essere usata da un debugger per inserire un break-point in un punto del programma da debuggare. La codifica dell'istruzione **int3** in linguaggio macchina è 0xCC, su un solo byte. Il debugger può dunque sostituire il primo byte dell'istruzione a cui ci si vuole fermare con 0xCC, e poi restituire il controllo al programma. Il programma ora esegue liberamente, ma il processore genererà una eccezione di break-point se l'esecuzione arriva alla **int3** così inserita. L'eccezione sarà intercetta dal debugger, che così riacquisirà il controllo del processore e potrà chiedere ulteriori istruzioni all'utente.

Vogliamo modificare il main di Figura 5 in modo da inserire un break-point all'inizio della funzione `foo()` che faccia saltare ad una funzione che invii il messaggio "breakpoint all'indirizzo x " (dove x è l'indirizzo della prima istruzione di `foo`) su log. Dopo l'invio del messaggio l'esecuzione del programma deve riprendere normalmente.

Soluzione

La soluzione è in Figura 6. Alla riga 18 del file C++ associamo la funzione `a_debug` all'eccezione 3 (breakpoint), quindi sovrascriviamo il primo byte di `foo` con l'istruzione **int3** (riga 21). Per poter poi eseguire correttamente il programma, ci salviamo il byte che stiamo sostituendo (riga 20).

La chiamata a `foo()` (riga 23) causerà ora un salto alla `a_debug` e poi alla `c_debug()`. Questa invia il messaggio al log (riga 12) e poi ripristina il primo byte della `foo()` (riga 13). Si noti che **int3** è di tipo trap e dunque il

```

1 #include <libce.h>
2 void foo() {
3     printf("foo()\n");
4 }
5
6 natb old;
7 extern "C" void a_debug();
8 extern "C" void c_debug(void *rip)
9 {
10     natb *p = static_cast<natb*>(rip);
11     p--;
12     flog(LOG_DEBUG, "breakpoint all'indirizzo %p", p);
13     *p = old;
14 }
15
16 int main()
17 {
18     gate_init(3, a_debug);
19     natb *p = reinterpret_cast<natb*>(foo);
20     old = *p;
21     *p = 0xCC;
22
23     foo();
24     pause();
25     return 0;
26 }

```

```

1 #include "libce.s"
2 .global a_debug
3 a_debug:
4     salva_registri
5     movq 120(%rsp), %rdi
6     call c_debug
7     carica_registri
8     decq (%rsp)
9     iretq

```

Figura 6: Soluzione esercizio 1

processore salva in pila l'indirizzo dell'istruzione successiva. Per questo motivo dobbiamo sottrarre 1 a `rip` prima di usarlo (riga 11).

Sempre per questo motivo, la `a_debug` deve decrementare di 1 l'indirizzo in cima alla pila prima di eseguire la **`iretq`** che ritornerà al programma interrotto (riga 8 del file assembler). In questo modo possiamo eseguire l'istruzione che avevamo sostituito con **`int3`**.

Esercizio 2

Quando gdb raggiunge un breakpoint ridà il controllo all'utente. Se questo decide di continuare l'esecuzione (istruzione `c`), come fa gdb garantire che l'istruzione su cui era stato inserito il break-point venga ora eseguita, e allo stesso tempo che ci sia una nuova eccezione di break-point se il programma dovesse ripassare in seguito da quella stessa istruzione?

Protezione

G. Lettieri

24 Marzo 2024

Per capire perché abbiamo bisogno del meccanismo della protezione, partiamo da un esempio. Supponiamo di trovarci negli anni '50 o '60 del secolo scorso, quando un computer occupava l'area di una palestra ed una Università poteva averne uno o forse due¹. In questo periodo i computer venivano usati in modalità *batch*. Gli utenti—ricercatori o studenti—preparavano i programmi a casa, su fogli di carta, scrivendoli in linguaggio macchina o in FORTRAN. Portavano poi i loro fogli al centro di calcolo, dove alcuni impiegati potevano trascriverli su nastro o su schede perforate. Ogni pacco di schede, contenente il programma di un utente, rappresentava un *job*. L'utente consegnava poi il suo job agli operatori del computer; in un secondo momento sarebbe dovuto ritornare a ritirare i risultati, tipicamente sotto forma di un tabulato stampato su carta.

Gli operatori, gli unici ad avere accesso alla sala del computer, aspettavano di avere un mazzo (*batch*) di job e poi lo caricavano sul lettore di schede del computer. Questo eseguiva i job uno alla volta, caricando automaticamente il prossimo job dopo aver terminato il precedente.

In questi sistemi si voleva massimizzare il numero di job completati ogni ora, sfruttando il costosissimo processore nel modo più efficiente possibile.

Supponiamo ora che il job dell'utente 1 debba caricare una serie di dati da un nastro magnetico e decida di svolgere questa operazione a controllo di programma. Il nastro va però riavvolto, operazione che richiede diversi secondi. Il costosissimo processore verrà così sprecato in un banale ciclo di istruzioni che legge ripetutamente i registri del controllore del nastro, per attendere che il nastro raggiunga la posizione desiderata. La “vera” elaborazione del job 1, quella che ha davvero bisogno delle piene capacità della CPU, comincerà solo quando tutti i dati saranno stati letti. Per gli operatori del computer sarebbe molto meglio se il controllore del nastro fosse programmato per inviare una richiesta di interruzione dopo il riavvolgimento. In questo modo, durante il tempo di riavvolgimento, si potrebbe utilizzare la CPU per cominciare ad eseguire il prossimo job del batch. All'arrivo della richiesta di interruzione si potrebbe poi ritornare al job 1.

¹L'Università di Pisa era una delle poche ad averne due: la CEP (Calcolatrice Elettronica Pisana), il primo calcolatore scientifico interamente progettato e costruito in Italia, ora custodita al Museo degli Strumenti per il Calcolo, e l'IBM 7090, frutto di una donazione dell'IBM stessa.

Si noti come l'esempio sia per certi versi simile a quello che abbiamo usato per introdurre il meccanismo delle interruzioni, in quanto in entrambi i casi vorremmo usare l'unica CPU per portare avanti concorrentemente due compiti: la stampa e il calcolo nell'esempio di Dijkstra, i due job in questo esempio. C'è però una differenza cruciale: questa volta i due compiti arrivano da due utenti diversi, che non si conoscono fra loro. Questo vuol dire che non possiamo attenderci collaborazione: l'utente del job 1 non ha interesse a cedere la CPU ad un altro job, e l'utente del job 2 non ha interesse a ridarla all'utente 1 quando arriva l'interruzione. Purtroppo, però, mentre è in esecuzione un certo programma, la CPU obbedisce a *quel* programma e dunque, nel nostro caso, al volere degli utenti 1 e 2 e non al volere degli operatori.

Per focalizzare le idee, supponiamo che gli operatori abbiamo scritto due routine e le abbiamo caricate in memoria, insieme ai job degli utenti:

1. Routine 1: avvia il riavvolgimento del nastro e cede il controllo ad un altro job;
2. Routine 2: (da associare alle richieste di interruzione provenienti dal controllore del nastro) restituisce il controllo al job che aveva invocato la Routine 1.

Il problema che gli operatori hanno è: come costringere il primo utente a chiamare la Routine 1, invece di dialogare direttamente con il controllore del nastro; come impedire al secondo utente di disabilitare le richieste di interruzione. Ma non basta: ora in memoria possiamo avere più programmi: quello scritto dagli operatori (contenente almeno le Routine 1 e 2 qui sopra) e quelli scritti dagli utenti. Nella CPU che abbiamo visto fino ad ora un programma può accedere liberamente a tutte le locazioni di memoria. Gli operatori devono anche impedire agli utenti di modificare le Routine 1 e 2, o le loro strutture dati.

Abbiamo bisogno di un meccanismo aggiuntivo nel processore, in modo che questo non obbedisca *sempre* ciecamente alle istruzioni del programma corrente: deve farlo se il programma che sta eseguendo è stato scritto dagli operatori, ma se il programma è stato scritto dagli utenti deve quantomeno “rifiutarsi” di eseguire le istruzioni di I/O e le istruzioni che abilitano o disabilitano le interruzioni, o accedono agli indirizzi contenenti codice o dati degli operatori. Le istruzioni in sè, però, non hanno “colore”: come può il processore distinguere, per esempio, l'istruzione **in** di un operatore dall'identica istruzione **in** di un utente? Inoltre, il processore dovrebbe operare questa distinzione senza stravolgere completamente il suo funzionamento che, ricordiamo, consiste nell'eseguire le istruzioni una alla volta ricordando del passato solo ciò che è scritto nei suoi registri. Dovremo anche decidere cosa vuol dire che il processore si “rifiuta” di eseguire una istruzione, visto che comunque non sa fare altro.

Se paragoniamo un programma ad un *testo* che il processore legge (ed esegue), il problema che abbiamo è quello di stabilire quale sia il *contesto*. Con questa parola ci si riferisce, nel linguaggio comune come in quello tecnico che ci riguarda, a tutto ciò che è necessario sapere per interpretare correttamente un dato testo, ma che non è scritto esplicitamente nel testo stesso. Nel nostro

caso il testo è l'istruzione che il processore sta eseguendo. Ad ogni istante ci deve essere anche un *contesto corrente*, che determini come l'istruzione debba essere interpretata. Potremo avere un *contesto privilegiato*, nel quale verranno eseguiti i programmi degli operatori, e un contesto non privilegiato, o *utente*, in cui verranno eseguiti i programmi degli utenti. Il processore deve solo sapere qual è il contesto corrente: se è quello privilegiato deve obbedire a tutte le istruzioni, come siamo abituati, ma se il contesto corrente è quello non privilegiato, si deve rifiutare di eseguire le istruzioni che abilitano o disabilitano le interruzioni e tutte le istruzioni che leggono o scrivono nello spazio di I/O. Il vincolo sulle istruzioni che accedono alle routine o alle strutture dati degli operatori è più complicato perché si applica a qualunque istruzione che abbia operandi in memoria, ma può essere semplificato nel modo seguente: si stabilisce *a-priori* che certi indirizzi, per esempio, tutti quelli inferiori ad un valore fissato, siano riservati agli operatori. Nel contesto non privilegiato il processore confronterà l'indirizzo di ogni operando con il valore fissato, rifiutandosi di eseguire l'istruzione se lo trova inferiore.

Per fare in modo che il processore sappia quale è il contesto corrente è sufficiente che abbia un nuovo registro in cui questa informazione è memorizzata (nel nostro caso basta un bit). Oltre ad aggiungere questo registro e i controlli durante la decodifica ed esecuzione di ogni istruzione, modifichiamo il processore in modo che all'avvio si trovi nel contesto privilegiato, e prevediamo una istruzione che lo porti nel contesto non privilegiato. Gli operatori sono gli unici ad essere presenti, la mattina, quando l'elaboratore viene avviato, e dunque possono fare in modo che il loro programma venga caricato agli indirizzi riservati e sia il primo a partire. Questo programma inizializzerà tutto il necessario, caricherà il primo job utente, eseguirà l'istruzione che porta il processore nel contesto non privilegiato e poi cederà il controllo al job utente. Da questo momento in poi l'esecuzione del programma utente sarà soggetta a tutti i vincoli che abbiamo visto.

Ci rendiamo conto, però, che ci serve anche un meccanismo con cui sia possibile riportare occasionalmente il processore nel contesto privilegiato. Per capire perché, ricordiamoci che il computer è stato acquistato per servire gli utenti, non gli operatori: gli operatori devono fornire all'utente 1 un modo per riavvolgere il nastro e leggere i suoi dati. L'idea è che l'utente 1 debba farlo esclusivamente invocando la Routine 1 scritta dagli operatori. Ma, ricordiamo, le istruzioni (e dunque le routine) non hanno colore. Dobbiamo aggiungere una istruzione speciale che, oltre a saltare alla Routine 1, porti anche il processore nel contesto privilegiato, altrimenti nemmeno la Routine 1 potrebbe interagire con il controllore del nastro. Questo meccanismo dovrà essere a sua volta protetto, in modo che gli utenti non possano indurre la CPU a portarsi nel contesto privilegiato e poi eseguire codice scritto da loro stessi invece che dagli operatori: l'innalzamento di privilegio deve essere associato ad un trasferimento di controllo ad un indirizzo scelto dagli operatori e non dagli utenti. Come fare? Nel nostro esempio, questo ritorno al contesto privilegiato deve avvenire in almeno due occasioni:

1. Nel job 1, quando l'utente 1 invoca la Routine 1;
2. Nel job 2, quando il nastro ha finito di riavvolgersi e invia la richiesta di interruzione che manda in esecuzione la Routine 2.

Il secondo caso ci fornisce l'idea decisiva: legare strettamente l'innalzamento del livello di privilegio con la risposta alle richieste di interruzione e alle eccezioni. Se gli operatori caricano la tabella delle interruzioni nella parte di memoria inaccessibile agli utenti, le destinazioni dei salti in risposta alle richieste di interruzione e alle eccezioni non è sotto il controllo degli utenti. Il secondo caso è così automaticamente risolto. Per il primo caso, possiamo ricorrere al seguente trucco: gli operatori associano la Routine 1 ad una entrata della tabella delle interruzioni e dicono all'utente 1 che, per invocarla, deve volontariamente causare la corrispondente eccezione. Infine, abbiamo una risposta naturale a cosa dovrebbe fare il processore quando si "rifiuta" di eseguire una istruzione: solleva una eccezione. Gli operatori assoceranno a questa eccezione un programma che stampa un messaggio di errore e carica un altro job dal batch.

Si noti che l'esempio dei sistemi batch è stato scelto perché particolarmente semplice. La necessità di poter portare avanti più programmi contemporaneamente nasce però in diversi ambiti. Ormai è data per scontata in tutti i sistemi che vanno dai supercalcolatori, ai server, ai personal computer, agli smartphone/tablet fino anche a molti sistemi embedded, con la sola eccezione dei più semplici tra questi ultimi.

1 La protezione nei processori Intel/AMD a 64 bit

Il meccanismo della protezione è stato aggiunto dall'Intel nel processore 80286 (1982), che era a 16 bit. Oltre ad essere legato al meccanismo delle interruzioni, il meccanismo era strettamente intrecciato con un altro meccanismo, la segmentazione, che però non trovò molti utilizzi. Il meccanismo di protezione nell'80286 era molto sofisticato: i "livelli di privilegio" non erano soltanto due (privilegiato e non privilegiato), ma quattro.

Con il processore 80386 (1985) l'Intel passò a 32 bit e supportò anche la paginazione, ma sempre in aggiunta alla segmentazione, estesa a 32 bit. Il meccanismo di paginazione dell'80386, però, supportava (e continua a supportare nei processori moderni) solo due livelli di privilegio: *sistema* (privilegiato) e *utente* (non privilegiato). I processori successivi (80486, Pentium, Pentium Pro, ...) non hanno cambiato questa architettura di base, ma né la segmentazione, né il meccanismo di cambio di processo via hardware sono mai stati usati in modo significativo.

Quando l'AMD ha introdotto l'architettura a 64 bit ha *quasi* completamente disattivato il supporto alla segmentazione. Purtroppo, sempre per motivi di compatibilità, le cose sono rimaste molto più complicate di come avrebbero potuto essere. Nel seguito cercheremo di semplificare la descrizione nascondendo

il più possibile i riferimenti alla segmentazione; i dettagli si potranno trovare nel codice eseguibile del sistema che realizzeremo.

1.1 Livelli di privilegio

Il processore si trova ad ogni istante o a livello (di privilegio) sistema, o a livello utente. Il livello corrente è deciso da un campo nel registro **cs** (Code Selector).

Per quanto riguarda la protezione della memoria, facciamo subito una semplificazione (che poi rimuoveremo quando parleremo della paginazione). Immaginiamo che il processore abbia un registro, in cui si può scrivere solo da livello sistema, che contenga un indirizzo *limite* che faccia da spartiacque tra la memoria usata dal sistema e quella utilizzabile dagli utenti. Chiamiamo M1 la parte di memoria ad indirizzi inferiori al limite e M2 la rimanente. Quando il processore si trova a livello utente sono vietati tutti gli accessi alle locazioni di M1, mentre a livello sistema il limite viene ignorato (tutti gli indirizzi sono permessi). All'accensione il processore si trova a livello sistema e carica ed esegue il bootstrap loader (da una ROM). Il bootstrap loader carica le routine e le strutture dati del sistema all'inizio della memoria, quindi scrive nel registro limite l'indirizzo della prima locazione non utilizzata, definendo così la separazione tra M1 e M2.

Il cambio di livello è strettamente connesso con il meccanismo delle interruzioni. Infatti, il livello di privilegio (e dunque il contenuto di **cs**) può essere cambiato solo in due modi:

- innalzato (o lasciato inalterato) passando attraverso un gate della IDT;
- abbassato (o lasciato inalterato) tramite una istruzione **iretq**.

Il termine *gate* (cancello) ci deve proprio far pensare ad un cancello che permette di entrare nel territorio privilegiato del sistema. Ci sono tre modi per attraversare un cancello. Due li conosciamo già: interruzioni esterne ed eccezioni. Il terzo lo introdurremo a breve.

Come abbiamo visto nell'esempio, ha senso che la ricezione di una interruzione o il sollevarsi di una eccezione causino un salto ad una routine di sistema. Nell'esempio, la conclusione del riavvolgimento del nastro era segnalata da una interruzione. In risposta a questa vogliamo che vada in esecuzione una routine di sistema che faccia ripartire il job 1. Anche per le eccezioni dovrebbe essere chiaro cosa vogliamo: se un utente prova a disabilitare le interruzioni, a leggere o scrivere nei registri del controllore del nastro, o a leggere o scrivere nelle locazioni di M1, vogliamo che venga fermato. Un modo per fermarlo è che il processore, sapendo che queste operazioni sono vietate a livello utente, sollevi una "eccezione di protezione". In risposta all'eccezione vogliamo che vada in esecuzione un'altra routine del sistema, che termini il job corrente e ne metta in esecuzione un altro. Si capisce anche perché deve essere la **iretq**, che si trova in fondo a tutte le routine di risposta alle interruzioni ed eccezioni, l'istruzione che si preoccupa di riportare il processore a livello utente.

Affinché questo meccanismo sia efficace gli utenti non devono essere in grado di modificare il contenuto della IDT. Questo si ottiene rendendo privilegiata

l'istruzione **lidtr** (che carica l'indirizzo della IDT nel registro **idtr** che il processore usa per accedere alla tabella) e allocando la IDT nella memoria M1. Questo può essere fatto in fase di inizializzazione del sistema: la IDT è semplicemente una delle strutture dati del sistema, caricate in M1 dal bootstrap loader. Dopo il caricamento il bootstrap loader salta ad una ben precisa routine (**start**) del sistema appena caricato, la quale si preoccupa di inizializzare tutte le strutture dati di sistema (compresa la IDT) e il processore, in particolare caricando il registro **idtr** con l'indirizzo della IDT.

Sempre nell'esempio ci siamo resi conto che l'utente 1 deve avere un modo per invocare la routine che avvia l'operazione sul nastro. Riusare il meccanismo delle eccezioni, come suggerito nell'esempio, è possibile (ed è anche la soluzione usata in qualche vecchio processore, ma il processore Intel offre un modo più esplicito, sotto forma di una nuova istruzione:

int <i>\$tipo</i>

Questa istruzione ha un unico parametro immediato, *tipo*, che deve essere un numero tra 0 e 255. Il tipo ha lo stesso significato del tipo delle interruzioni esterne e delle eccezioni: è utilizzato per accedere ad un gate della IDT, dove il processore trova l'indirizzo della routine a cui saltare e l'informazione che gli dice se il livello di privilegio deve essere innalzato. Si dice che l'istruzione genera una "interruzione software"—il terzo, e ultimo, modo in cui si può attraversare un gate della IDT. Le interruzioni software sono del tutto analoghe a quelle esterne ed alle eccezioni di tipo trap, con salvataggio in pila di informazioni analoghe. In particolare, l'istruzione pointer salvato in pila è quello dell'istruzione successiva alla **int**. Anche le routine di risposta alle interruzioni software devono terminare con **iretq** e non **ret**.

Le routine che vanno in esecuzione tramite una **int** prendono comunemente il nome di *primitive di sistema*. Chi scrive il codice di sistema deve fornire ai suoi utenti la necessaria documentazione su: quali sono le primitive disponibili; quale tipo è necessario usare per invocare ciascuna primitiva; quali parametri ciascuna primitiva si aspetta e come le vanno passati (tipicamente tramite i registri). L'istruzione **int** svolge un compito simile a quello di una **call**. Ci si può chiedere come mai serva una nuova istruzione per invocare una primitiva, invece di aggiungere la possibilità di innalzare il livello di privilegio alla normale **call**. Ma l'istruzione **call** specifica direttamente l'indirizzo a cui saltare, e questo causa due problemi:

1. l'utente potrebbe saltare in mezzo al codice delle primitive (per esempio, per scavalcare dei controlli);
2. se gli indirizzi vengono specificati in forma simbolica, i programmi utente dovrebbero essere collegati con il codice di sistema per poter risolvere i simboli.

Il primo problema è di gran lunga più grave, mentre il secondo è solo una scomodità. L'istruzione **int** non ha questi problemi, in quanto l'utente specifica solo il tipo della primitiva che intende invocare, mentre l'indirizzo a cui saltare

è scritto nella IDT, a cui lui non ha accesso. C'è un ultimo vantaggio, che per il momento non possiamo apprezzare pienamente: è molto comodo, per chi scrive il sistema, che i modi con cui si può accedere al sistema siano il più possibile uniformi.

1.2 Interruzioni e protezione

Vediamo più in dettaglio come funziona il meccanismo delle interruzioni. Ogni gate della IDT occupa 16 byte e contiene le seguenti informazioni:

- un bit P (Presenza), che indica se il gate contiene informazioni significative (il sistema non deve necessariamente utilizzare tutti i gate);
- il puntatore alla routine a cui saltare (8 byte);
- un bit I/T che indica se il gate è di Interrupt o Trap;
- un campo L che indica il livello di privilegio a cui il processore si deve portare dopo aver attraversato il gate;
- un campo DPL (Descriptor Privilege Level, chiamato anche GL per Gate Level) che indica il livello di privilegio minimo che il processore deve avere per poter attraversare il gate nel caso di interruzione software.

Quando il processore accetta una interruzione esterna, genera una eccezione o esegue una interruzione software,

1. si procura il *tipo* dell'interruzione:
 - in caso di eccezione, il tipo è implicito;
 - in caso di interruzione esterna, riceve il tipo dall'APIC;
 - in caso di interruzione software, il tipo è il parametro dell'istruzione **int**.

Usa quindi il tipo come indice nella tabella IDT, per accedere al corrispondente gate.

2. Se il bit P del gate è zero, il processore smette di gestire l'interruzione e genera una eccezione per “gate non presente” (tipo 11).
3. Altrimenti, se sta gestendo una interruzione software (o eseguendo una **int3**), confronta il livello corrente (registro **cs**) con il campo DPL del gate. Se il livello corrente è meno privilegiato di DPL, genera una eccezione di protezione (tipo 13).
4. Altrimenti, confronta il **cs** con il campo L. Se L è inferiore, genera una eccezione di protezione (tipo 13).
5. Altrimenti, salva in un registro di appoggio (chiamiamolo SRSP) il contenuto corrente di **rsp**.

6. Se **cs** è diverso da L, esegue un *cambio di pila*, caricando un nuovo valore in **rsp** (si veda la Sezione 1.3 più avanti per sapere da dove viene prelevato il nuovo valore).
7. Salva in pila (la nuova o la vecchia, a seconda che al punto precedente abbia cambiato pila o meno) 5 parole lunghe (8 byte ciascuna), in ordine:
 - una parola lunga non significativa (segmentazione ...);
 - il contenuto di SRSP (questo punta alla vecchia pila, nel caso di cambio pila);
 - il contenuto di **rflags**;
 - il contenuto di **cs**;
 - il contenuto di **rip**.
 (in cima alla pila c'è dunque **rip**).
8. Azzerà in ogni caso TF (disabilita una eventuale modalità Single Step) e azzerà IF (Interrupt Flag) solo se il gate è di tipo Interrupt (campo I/T del gate).
9. Salta infine all'indirizzo della routine puntata dal gate.

Il controllo eseguito al punto 3 può essere usato per imporre che l'utente usi l'istruzione **int** solo con i tipi associati a primitive di sistema, e non con i tipi associati alle interruzioni esterne o alle eccezioni. I programmatori di sistema possono impostare DPL=sistema in tutti i gate delle interruzioni esterne e delle eccezioni, e DPL=utente in quelli che puntano alle primitive. In questo modo le interruzioni esterne e le eccezioni vengono gestite normalmente (perché DPL non viene controllato in questi casi), mentre ogni tentativo dell'utente di usare un gate che non è assegnato ad una primitiva genera una eccezione di protezione.

Il controllo effettuato al punto 4 corrisponde a quanto si era detto: le interruzioni devono solo poter innalzare il livello di privilegio (o lasciarlo inalterato) e mai abbassarlo. Il punto è che una richiesta di interruzione manda in esecuzione una routine e, normalmente, ci aspettiamo che questa routine faccia ciò che deve fare e poi ritorni al programma principale. Ma se la routine è di livello utente non possiamo fare nessuna assunzione, nemmeno sul fatto che prima o poi termini. Quindi, se il processore si trova a livello sistema (e dunque sta svolgendo operazioni importanti), è pericoloso che venga interrotto da una routine di livello utente. È comunque abbastanza insolito che un gate di interruzione contenga L=utente e noi assumeremo che non avvenga, rendendo dunque ridondante questo controllo.

Si noti che il meccanismo contempla anche i casi **cs**=utente/L=utente e **cs**=sistema/L=sistema. Possiamo ignorare quello con L=utente, per quando appena detto. Il caso **cs**=sistema/L=sistema si può invece facilmente verificare: se il sistema invoca esso stesso una primitiva, o se una routine di sistema di tipo trap viene interrotta da una eccezione o da una interruzione esterna. In questi casi non ci sarà cambio pila.

Il cambio pila eseguito al punto 6 in caso di innalzamento del livello di privilegio (da utente a sistema) ha due motivazioni:

- il processore deve garantire di poter scrivere le 5 parole lunghe senza sovrascrivere altre cose, e non può dunque fidarsi del contenuto di **rsp** controllato dall'utente;
- è bene che queste informazioni siano salvate nella memoria di sistema (M1), in modo che l'utente non le possa corrompere. In particolare, è bene che l'utente non possa modificare il valore salvato di **cs**, che dice a che livello si trovava il processore prima dell'interruzione.

Si noti però che la seconda motivazione è importante solo quando il sistema dispone di più di un processore (cosa che noi non prenderemo in considerazione). In presenza di un unico processore, infatti, il codice di sistema stesso potrebbe copiare le informazioni dalla pila in memoria M1, senza il timore che il codice utente possa interferire nella copia. La prima motivazione, invece, resta vera in ogni caso.

L'istruzione **iretq** svolge le operazioni opposte a quelle del meccanismo di interruzione.

- (a) Confronta il valore corrente di **cs** con quello salvato in pila; se quello salvato è più privilegiato di quello corrente genera una eccezione di protezione (tipo 13).
- (b) Ripristina i valori di **rip**, **cs**, **rflags** e **rsp** leggendo i corrispondenti valori dalla pila.

Si noti che con il ripristino di **rsp** processore ritorna ad usare la pila originaria, nel caso questa fosse stata cambiata all'avvio dell'interruzione. Il ripristino di **rflags** ripristina in particolare i vecchi valori di TF e IF, eventualmente riabilitando il Single Step e le interruzioni esterne. Il ripristino di **cs** riporta il processore al livello di privilegio precedente.

Il controllo eseguito al punto (a) corrisponde a quanto detto in precedenza: l'istruzione **iretq** può solo abbassare (o lasciare inalterato) il livello di privilegio, mai innalzarlo. Diversamente dal complementare controllo nel caso delle interruzioni (punto 4), questo controllo è molto importante. Se non venisse effettuato sarebbe facile per l'utente eseguire codice di suo piacimento a livello sistema (come?).

Si noti che la routine di inizializzazione del sistema deve avere un modo per poter passare a livello utente una volta che tutte le strutture dati sono state inizializzate. Per farlo è sufficiente che prepari una pila nello stesso stato in cui l'avrebbe lasciata una interruzione proveniente dal livello utente, ed esegua una **iretq**. Da quel momento in poi il controllo passa agli utenti. Il sistema interviene solo in risposta alle richieste di interruzione esterne, le eccezioni, e le interruzioni software.

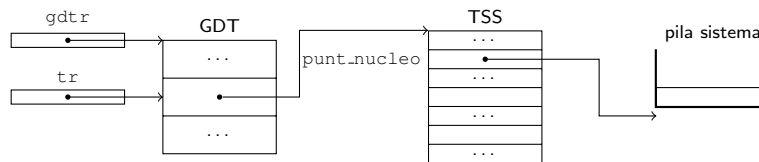


Figura 1: Strutture dati del sistema associate ad ogni processo.

1.3 Il registro **tr** e i Task State Segment

Al punto 6 della Sezione precedente, abbiamo visto che il processore cambia pila quando deve innalzare il livello di privilegio. In puntatore alla nuova pila è prelevato dal *Task State Segment* (TSS) corrente. I TSS sono delle strutture in memoria contenenti spazio per diversi campi, tra cui il puntatore alla pila che ci interessa. Erano usati dal meccanismo di cambio di processo in hardware, che, come abbiamo visto, non è più disponibile. È necessario, però, continuare a definirne almeno uno, per compatibilità con il fondamentale meccanismo delle interruzioni. Chiameremo `punt_nucleo` il campo del TSS che punta alla pila di livello sistema.

Nell'idea originaria ci doveva essere un TSS per ogni processo ma, ad ogni istante, uno solo è quello “attivo”. Il registro **tr** (Task Register) serve a identificare il TSS attivo e può essere caricato usando l'istruzione **ltr**. Tutti i TSS esistenti sono descritti da una tabella GDT (Global Descriptor Table), puntata dal registro `gdt_r` che può essere caricato con l'istruzione **lgdt_r**. La GDT contiene alcune entrate che non ci interessano (segmentazione...) e una entrata per ogni TSS. Le entrate relative ai TSS contengono l'indirizzo di partenza (base) del TSS e la sua grandezza in byte. Il registro **tr** contiene l'offset (rispetto all'inizio della GDT) dell'entrata della GDT che punta al TSS corrente. Si veda la Figura 1.

Ovviamente, le istruzioni **lgdt_r** e **ltr** sono privilegiate e la GDT e tutti i TSS devono trovarsi in M1.

Nell'idea originaria, il software di sistema deve aggiornare **tr** ogni volta che cambia processo. Noi useremo un unico TSS, caricando **tr** una sola volta in fase di inizializzazione. Avremo però una pila sistema diversa per ogni processo e, ad ogni cambio di processo, dovremo aggiornare il campo `punt_nucleo` di quell'unico TSS.

1.4 Livelli di privilegio e operazioni di I/O

Siamo partiti da un esempio in cui volevamo vietare agli utenti di abilitare/-disabilitare le interruzioni e accedere allo spazio di I/O. Nell'architettura Intel queste operazioni non sono automaticamente vietate quando il processore si trova a livello utente. La possibilità o meno di eseguire queste operazioni è determinata, invece, dal campo IOPL (I/O Privilege Level) che si trova nel registro dei flag. Questo campo specifica il livello di privilegio minimo che il

processore deve avere per poter abilitare o disabilitare le interruzioni ed eseguire le istruzioni **in** e **out** (in tutte le loro varianti). Il campo IOPL può essere modificato solo a livello sistema. Il tentativo di modificarli da livello utente (per esempio tramite **popf**) non genera, come ci potremmo aspettare, una eccezione di protezione, ma viene semplicemente ignorato: IOPL continua a mantenere il suo valore senza che venga segnalato alcun errore. Lo stesso accade se si tenta di modificare il flag IF tramite **popf**.

Introduzione al sistema multiprogrammato

G. Lettieri

3 Aprile 2024

1 Introduzione generale

Cominciamo a studiare come utilizzare i meccanismi hardware introdotti finora per realizzare un sistema in grado di eseguire più istanze di programmi concorrentemente. Nell'esempio dell'ultima lezione abbiamo parlato di job, ma il termine che si usa oggi è *processo*.

1.1 I processi

L'astrazione più importante introdotta dal sistema è quella dei *processi*. Un processo è un programma in esecuzione.

Proviamo ad illustrare la differenza tra programma e processo con una semplice metafora: in una pizzeria, il pizzaiolo riceve una ordinazione. Il pizzaiolo è inesperto e ha bisogno di avere la ricetta della pizza davanti a sé. Stende la pasta, la condisce, la inforna, aspetta che sia cotta, la farcisce e serve la pizza.

Il programma è la ricetta. Il pizzaiolo è il processore, cioè l'entità che interpreta la ricetta e la esegue. La pizza sono i dati da elaborare.

Il processo è un concetto un po' più astratto. È la *sequenza di stati* che il sistema “pizza + pizzaiolo” attraversa, passando dalla pasta alla pizza finale, secondo le istruzioni dettate dalla ricetta, eseguite pedissequamente dal pizzaiolo inesperto. Possiamo immaginarlo come il “filmato” del pizzaiolo che fa la pizza.

Uscendo dalla metafora, un processo è un programma in esecuzione su dei dati di ingresso. Questa esecuzione la possiamo modellare come la sequenza degli stati attraverso cui il sistema processore + memoria passa eseguendo il programma, su quei dati, dall'inizio fino alla conclusione. L'esecuzione di ogni istruzione del programma fa passare il processo da uno stato al successivo.

Notate che questa definizione si applica bene ai programmi di tipo *batch*, in cui gli ingressi vengono specificati tutti all'inizio, e il processo (generato dal programma in esecuzione su quei dati) prosegue indisturbato fino ad ottenere le uscite (ad esempio, pensate ad un programma per ordinare alfabeticamente un file). Una volta afferrato il concetto, però, in questo caso semplice, credo che non vi sarà difficile estenderlo ai programmi “interattivi” a cui ormai siamo più abituati (praticamente tutti i programmi con interfaccia grafica, ma non solo quelli).

A prima vista, il processo potrebbe sembrare molto simile al programma: la ricetta dice “stendere la pasta” e nel processo vediamo il pizzaiolo stendere la pasta; nel rigo successivo la ricetta ricetta dice “versare il condimento” e nel processo vediamo, subito dopo, il pizzaiolo versare il condimento. È molto semplice confondere i due concetti, soprattutto se il programma è molto semplice, ma si tratta di due cose completamente distinte, per i motivi illustrati di seguito.

- Uno stesso programma può essere associato a più processi: tanti clienti, in genere, chiedono lo stesso tipo di pizza. In questo caso, il programma è sempre lo stesso (la ricetta per quel tipo di pizza), ma ad ogni pizza corrisponde un processo distinto, che si svolge autonomamente nel tempo.
- Uno stesso processo può eseguire, in sequenza, più programmi. Si pensi, per esempio, ad una pizza *a metro*, composta da vari tipi di pizza uno dietro l'altro¹.
- In generale, non è esclusivamente il programma (la ricetta) a decidere attraverso quali stati il processo dovrà passare, ma anche la richiesta del cliente (l'input): la ricetta potrebbe, infatti, prevedere delle varianti (un **if**), che il cliente dovrà specificare. La ricetta conterrà istruzioni per entrambe le varianti (con o senza carciofi), ma un particolare processo seguirà necessariamente *una sola* variante.
- Il programma potrebbe contenere dei cicli (aggiungere pomodoro fino a coprire la parte centrale della pasta); nel programma vediamo le azioni da ripetere scritte una volta sola, mentre nel processo vediamo le azioni effettivamente ripetute tante volte.

Ma c'è dell'altro, nella metafora del pizzaiolo, che ci può aiutare a capire altri punti fondamentali. Il processo, se lo guardiamo nella sua interezza, si svolge necessariamente nel tempo (“procede” nel tempo). Possiamo anche, però, guardarlo ad un certo istante, facendone una fotografia, o osservando un singolo fotogramma del filmato di cui sopra. La fotografia che ne facciamo deve contenere tutte le informazioni necessarie a capire come il processo si svolgerà nel seguito. Nel nostro esempio, la foto dovrà contenere:

- la pizza nel suo stato semilavorato (la memoria dati del processo);
- il punto, sulla ricetta, a cui il pizzaiolo è arrivato (il contatore di programma);
- tutte le altre informazioni che permettono al pizzaiolo di proseguire (da quel punto in poi) con la corretta esecuzione della ricetta, come, ad esempio, il tempo trascorso da quando ha infornato la pizza (i registri del processore).

¹Nel sistema Unix, per esempio, uno processo può interrompere il programma che sta eseguendo e passare ad un altro, invocando la primitiva `execve()`.

Se la fotografia è fatta bene, contiene tutto il necessario per sospendere il processo e riprenderlo in un secondo tempo. Il pizzaiolo fa questa operazione continuamente, quando condisce, a turno, più pizze, o quando lascia una serie di pizze nel forno e, nel frattempo, comincia a prepararne un'altra (multiprogrammazione).

I processi sono rappresentati all'interno del sistema tramite una serie di strutture dati, la più importante delle quali è il *descrittore di processo*, una struttura che viene istanziata per ogni nuovo processo e che contiene, in particolare, lo spazio per memorizzare una istantanea dello stato del processo. Quando il sistema decide di portare avanti un processo P_1 , per prima cosa carica questa istantanea nei veri registri del processore e nella vera memoria del sistema. A questo punto la normale esecuzione delle istruzioni farà avanzare lo stato del processo P_1 . Quando il sistema decide di passare ad un altro processo P_2 , per prima cosa scatta una nuova istantanea dello stato di P_1 , che andrà a sostituire la precedente, e poi caricherà l'istantanea dello stato del processo P_2 .

1.2 Contesto

Per realizzare i processi riutilizziamo il concetto già introdotto di *contesto*. Ricordiamo che quando diciamo “testo” stiamo pensando al testo di un programma da eseguire. In un sistema multiprocesso il significato di una istruzione dipende dal processo che la sta eseguendo. Per esempio, se un processo P_1 esegue una istruzione

<code>mov %rax, 1000</code>

si sta riferendo al “suo” registro `%rax`, il cui aveva presumibilmente scritto qualcosa in un passo precedente, e sta tentando di copiarla al “suo” indirizzo 1000, dove avrà presumibilmente allocato una qualche sua variabile. La stessa identica istruzione, eseguita però da un altro processo P_2 , parlerà di un diverso `%rax` e di una diversa variabile. La corretta interpretazione dell'istruzione dipende dunque da qualcosa che non è scritto nell'istruzione: il processo che la esegue. Possiamo dunque pensare che ogni processo abbia un suo contesto. Il sistema deve essere organizzato in modo tale che, ogni volta che si esegue una qualunque istruzione, si tenga correttamente conto del contesto del processo a cui quella istruzione appartiene. Il contesto di un processo comprenderà, sicuramente, tutta la memoria usata dal processo e una copia privata di tutti i registri del processore. L'operazione di caricamento dello stato di un processo (l'istantanea di cui abbiamo parlato nella sezione precedente) non fa altro che rendere *corrente*, o attivo, il contesto di quel processo. Da quel momento in poi, e fino al prossimo cambio di contesto, le istruzioni eseguite dal processore opereranno implicitamente nel contesto di quel processo.

Oltre ai contesti dei singoli processi avremo anche il “contesto privilegiato” di cui avevamo già parlato quando abbiamo introdotto la protezione. I contesti dei processi possono essere manipolati (per esempio per eseguire un cambio di contesto) solo quando il processore si trova nel contesto privilegiato. In generale, a tutte le risorse che sono condivise tra i processi, come le periferiche, si potrà accedere solo dal contesto privilegiato. In questo contesto girerà il software

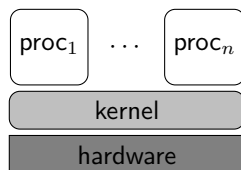


Figura 1: Rappresentazione dei livelli di privilegio e contesti.

detto *nucleo* (*kernel* in Inglese), il cui compito principale è quello di realizzare l'astrazione dei processi, e anche di fornire primitive che permettano ai processi utente di accedere alle risorse condivise in modo controllato.

La situazione a cui vogliamo arrivare è spesso rappresentata come in Figura 1. Figure di questo tipo vogliono mostrare l'esistenza di diversi contesti e i loro rapporti. I processi utente (in alto) sono meno privilegiati del “kernel” (nucleo del sistema operativo), ma non hanno in genere diverso privilegio tra essi stessi. La presenza dell'hardware in basso serve a mostrare che i processi utente devono per forza passare dal nucleo per potervi accedere, come nel nostro esempio del lettore di nastro.

Attenzione però a non farsi confondere e a non vedere in queste figure cose che non ci sono (e non possono esserci). In particolare, si potrebbe essere portati a immaginare la CPU, in quanto hardware, come posizionata “sotto” il nucleo. Questo è fuorviante, perché il software di Figura 1, sia quello utente che quello sistema, è eseguito direttamente dalla CPU e, mentre è in esecuzione, ne ha il controllo senza l'intermediazione di altro software. Per lo stesso motivo non bisogna pensare che la Figura 1 rappresenti lo stato del sistema in un qualche istante temporale: non è vero che mentre è in esecuzione $proc_1$, per esempio, abbiamo contemporaneamente, “di sotto”, il nucleo che fa qualcosa (lo controlla?). La CPU può eseguire un solo software alla volta e tutto ciò che può fare e passare da una istruzione all'altra, usando i meccanismi che conosciamo (normale flusso di controllo oppure interruzioni e eccezioni). Conviene immaginare la CPU come un puntino che si muove all'interno dei rettangoli arrotondati di Figura 1, guidata dal software che sta eseguendo. Occasionalmente la CPU salta in un altro contesto (tipicamente il nucleo) per via di una interruzione o perché è caduta in una botola (*trap*). Interruzioni e trap sono i meccanismi che permettono al nucleo di riacquisire il controllo della CPU indipendentemente dal volere degli utenti.

Il fatto che i processi debbano appartenere a diversi utenti non è fondamentale: in molte applicazioni uno stesso utente può voler eseguire più di un processo contemporaneamente e, anche in quel caso, vogliamo che ogni processo abbia un suo contesto indipendente (per fare in modo, per esempio, che i bug di un processo non si propaghino in altri processi). Se però uno stesso utente esegue più processi, può aver senso che questi lavorino su qualche struttura dati in comune, quindi i loro contesti potrebbero anche condividere in tutto o in parte la memoria.

2 Un semplice sistema multiprocesso

Il sistema che realizzeremo è organizzato in tre moduli:

- *sistema*;
- *io*;
- *utente*.

Ogni modulo è un programma a sé stante, non collegato con gli altri due. Il modulo *sistema* contiene la realizzazione dei processi, inclusa la gestione della memoria (che, come vedremo, usa la tecnica della memoria virtuale); il modulo *io* contiene le routine di ingresso/uscita (I/O) che permettono di utilizzare le periferiche collegate al sistema (tastiera, video, hard disk, ...). Sia il modulo *sistema* che il modulo *io* verranno eseguiti con il processore a livello sistema, in un contesto privilegiato. Solo il modulo *utente* verrà eseguito al livello utente.

I moduli *sistema* e *io* forniscono un supporto al modulo *utente*, sotto forma di primitive che il modulo *utente* può invocare. In particolare, il modulo *utente* può creare più processi, che verranno eseguiti concorrentemente. I processi avranno sia una parte della memoria condivisa tra tutti, sia una parte privata per ciascuno.

2.1 Sviluppo di programmi

Il sistema che sviluppiamo non è autosufficiente e, per motivi di semplicità, non lo diventerà. Quindi per sviluppare i moduli useremo un altro sistema come appoggio. In particolare, il sistema di appoggio sarà Linux. Come compilatore utilizziamo lo stesso compilatore C++ di Linux (g++), opportunamente configurato in modo che produca degli eseguibili per il nostro sistema, invece che per il sistema di appoggio (come farebbe per default). Come abbiamo già visto per gli esempi di I/O, questo comporta la disattivazione di alcune opzioni, l'ordine di non utilizzare la libreria standard (in quanto userebbe quella fornita con Linux, che non funziona sul nostro sistema) e la specifica di indirizzi di collegamento opportuni. Gli indirizzi di collegamento vanno cambiati in quanto quelli di default sono pensati per i programmi utente che devono girare su Linux, rispettando quindi l'organizzazione della memoria di Linux, che è diversa da quella che utilizzeremo nel nostro sistema. Per specificare un diverso indirizzo di collegamento è sufficiente, nel nostro caso, passare al collegatore l'opzione `-Ttext` seguita da un indirizzo. Il collegatore userà quell'indirizzo come base di partenza della sezione **.text**. La sezione **.data** sarà allocata agli indirizzi che seguono la sezione **.text**. Per il modulo *sistema* useremo l'indirizzo di partenza 0x200000 (secondo MiB, per motivi spiegati in seguito).

Il modulo *sistema* deve essere caricato da un bootstrap loader che è in grado di interpretare i file ELF. L'output del collegatore del sistema, dunque, è direttamente utilizzabile. Sarà il modulo *sistema*, durante la fase di inizializzazione, a caricare gli altri due moduli (*io* e *utente*). Si veda più avanti (Sezione [2.2](#)) per i dettagli.

Una volta scompattato il file `nucleo.tar.gz` si ottiene la directory `nucleo-x.y` (dove `x.y` è il numero di versione). All'interno troviamo:

- le sottodirectory `sistema`, `io` e `utente`, che contengono i file sorgenti dei rispettivi moduli;
- la sottodirectory `util`, che contiene alcuni script di utilità;
- la sottodirectory `include`, che contiene dei file `.h` inclusi dai vari sorgenti;
- la sottodirectory `build`, inizialmente vuota, destinata a contenere i moduli finiti;
- la sottodirectory `debug`, che contiene alcune estensioni per il debugger `gdb`;
- la sottodirectory `doc`, destinata a contenere la documentazione dei moduli.

La directory `nucleo-x.y` contiene anche due file: il file `Makefile`, contenente le istruzioni per il programma `make` del sistema di appoggio; uno script `run`, che permette di avviare il sistema su una macchina virtuale. Il `Makefile` può essere usato per generare la documentazione lanciando il comando

```
make doc
```

Attenzione: per generare la documentazione è necessario aver installato `Doxygen` e `pandoc`. Se il comando ha successo, la documentazione si troverà in formato HTML, con l'indice in `doc/html/index.html`. Per il resto, sia il `Makefile` che lo script `run` possono essere ignorati, in quanto gli script `compile` e `boot` funzionano anche per il nucleo.²

2.1.1 Scrittura di programmi utente

Si suppone che i moduli `sistema` e `io` cambino raramente e costituiscano il sistema vero e proprio, mentre il modulo `utente` rappresenta il programma, di volta in volta diverso, che l'utente del nostro sistema vuole eseguire. Per questo motivo la sottodirectory `utente` contiene solo alcuni file di supporto (`lib.cpp` e `lib.h`, contenenti alcune funzioni di utilità, e `utente.s`, contenente la parte assembler delle chiamate di primitiva, come vedremo), e una sottodirectory `examples` contenente alcuni esempi di possibili programmi utente.

²Nel caso del nucleo, lo script `compile` usa internamente `make`, che può anche essere usato direttamente. Il comando `make` legge a sua volta il file `Makefile` e vi trova i comandi da eseguire per costruire quanto richiesto. Si noti che il programma `make` cerca di eseguire solo le operazioni strettamente necessarie. Per esempio, se lo si lancia due volte di seguito si vedrà che la prima volta verranno eseguiti tutti i diversi comandi di compilazione e collegamento, ma la seconda volta, dal momento che i moduli esistono già e i file sorgenti non sono cambiati, non verrà eseguito alcun comando. Se si vuole forzare la ricompilazione di tutto si può prima lanciare il comando `make reset`, che cancella tutti i file `.o` e tutto il contenuto della directory `build`. In questo modo un successivo `make` sarà costretto a rifare tutto daccapo. Lo script `compile` esegue un `make reset` seguito da `make`.

```

1  #include <all.h>
2
3  void main()
4  {
5      writeconsole("Hello, world!\n", 14);
6      pause();
7      terminate_p();
8  }

```

Figura 2: Un esempio di programma utente (file `utente/utente.cpp`).

In Figura 2 vediamo un esempio minimo, che può essere scritto direttamente nel file `utente/utente.cpp`. Alla riga 1 si include un file che contiene le dichiarazioni delle funzioni di libreria e delle primitive di sistema (tra cui la dichiarazione delle primitive invocate alle righe 5 e 7. Il file, in realtà, si limita ad includere vari altri file, tra cui quelli contenuti nella directory `include`, il file `lib.h` e il file di intestazione di `libce`. La funzione `pause()`, invocata alla linea 6, è implementata nel file `lib.cpp`. La primitiva `writeconsole()`, implementata nel modulo `io` e dichiarata in `include/io.h`, permette di scrivere una stringa sul monitor. Si noti la necessità di chiamare la primitiva `terminate_p()`: la funzione `main` verrà eseguita da un processo utente, che deve chiedere al sistema di poter terminare. La funzione `pause()` alla riga 6 serve solo a impedire che il sistema esegua troppo velocemente lo shutdown impedendoci di vedere la stringa stampata alla riga 5. Questo perché il sistema esegue lo shutdown non appena tutti i processi utente sono terminati.

Per compilare i moduli e i programmi di utilità lanciare il comando `compile`, già usato per gli esempi di I/O.

2.2 Avvio del sistema

Una volta costruiti tutti i moduli, possiamo avviare il sistema. La procedura di *bootstrap* è la stessa già usata per gli esempi di I/O e può essere avviata lanciando lo script `boot`.

All'avvio il processore parte in modalità a 16 bit non protetta (il cosiddetto “modo reale”) e deve essere prima portato, via software, in modalità protetta a 32 bit. Questo compito è normalmente svolto da un programma di bootstrap caricato dal BIOS. Nel nostro caso, visto che caricheremo il sistema esclusivamente in un una macchina virtuale, questo compito sarà svolto dall'emulatore stesso. Tocca però a noi portare il processore nella modalità a 64 bit, e questo compito lo facciamo svolgere dal programma `boot.bin` fornito da `libce`³. Una volta fatto questo, il programma `boot.bin` può cedere il controllo al modulo sistema. Lo spazio da `0x100000` a `0x200000` può essere ora riutilizzato (vedremo che verrà utilizzato dallo heap di sistema). Lo spazio di memoria da `0` a `0x100000-1`, invece, contiene varie cose che hanno usi specifici (per esempio,

³I sorgenti sono in `boot64/boot.S` e `boot64/boot.cpp`.

la memoria video in modalità testo). Soli i primi 640 KiB sono liberamente utilizzabili.

Più in dettaglio, `boot.bin` viene caricato da QEMU a partire dall'indirizzo fisico 0x100000, subito seguito da una copia dei file `sistema`, `io` e `utente`. Il modulo `sistema` è collegato a partire dall'indirizzo 0x200000. Il programma `boot.bin` si preoccupa di copiare le sezioni `.text`, `.data`, etc. dalla copia del file `sistema` al loro indirizzo di collegamento, abilitare la modalità a 64 bit, quindi saltare all'entry point del modulo `sistema`.

Una volta avviato vediamo una nuova finestra che rappresenta il video della macchina virtuale. Notiamo anche dei messaggi sul terminale da cui abbiamo lanciato `boot`, qui riportati in Figura 3. Questi sono messaggi inviati sulla porta seriale della macchina virtuale. I messaggi nelle righe 1–15 arrivano dal programma `boot.bin`. Nelle righe 4–7 il programma `boot.bin` ci informa del fatto che il bootloader precedente (QEMU stesso, nel nostro caso) ha caricato in memoria tre file, e in particolare il file `build/sistema.strip`⁴ all'indirizzo 0x114000. Nelle righe 8–11 ci dice come sta copiando le sezioni di questo file nella loro destinazione finale. La riga 15 ci avverte che `boot.bin` ha finito e sta per saltare all'indirizzo mostrato nella riga 12 (0x200178), dove si trova l'entry point del modulo `sistema`. I messaggi successivi arrivano dal modulo `sistema` (alcuni, come quelli alle righe 45–46, arrivano dal modulo `io`). Alla riga 38 vediamo che viene inizializzato l'APIC. Le righe 19–24 contengono informazioni relative alla memoria virtuale, che per il momento ignoriamo. Di seguito viene inizializzato lo heap di sistema (riga 40, riutilizzando lo spazio occupato da `boot.bin`). Vengono poi creati i primi processi di sistema (righe 36, 37). Da questo punto in poi l'inizializzazione prosegue nel processo `main_sistema` (id 1) e `main I/O` (id 2). Il processo `main_sistema` attiva il timer alla riga 41 e crea il processo `main_io` (righe 42–43). Le righe 44–53 sono relative all'inizializzazione del modulo `io`, eseguita da questo processo. Quando il processo `main_io` termina, processo `main_sistema` crea il primo processo utente (righe 54–55) e gli cede il controllo (riga 56), semplicemente terminando (riga 57). In questo caso il processo utente esegue il codice di Figura 2 che stampa un messaggio sul video e poi termina (riga 59 di Figura 3). Il controllo passa quindi a `dummy` (id 0), che si accorge che non ci sono più processi utente e quindi ordina lo shutdown della macchina QEMU (riga 60).

In Figura 4 mostriamo un altro esempio di programma utente, che questa volta tenta di eseguire un'azione non permessa: leggere direttamente da una porta di I/O (linea 5). Il tentativo causa il sollevamento di una eccezione che restituisce il controllo al modulo `sistema`, il quale termina forzatamente il processo e invia alcuni messaggi sul log (Figura 5). È utile imparare a leggere questi messaggi, perché contengono informazioni che aiutano a trovare più velocemente eventuali errori. La colonna centrale (che in questo caso contiene il valore 5) è l'id del processo a cui fanno riferimento questi messaggi. Il messaggio alla riga 1 contiene informazioni sull'eccezione che ha causato la terminazione forzata del

⁴Si tratta del file `build/sistema` con alcune informazioni non necessarie rimosse, in modo da occupare meno spazio in memoria.

```

1 INF - Boot loader di Calcolatori Elettronici, v1.0
2 INF - Memoria totale: 32 MiB, heap: 636 KiB
3 INF - Argomenti: /home/giuseppe/CE/lib/ce/boot.bin
4 INF - Il boot loader precedente ha caricato 3 moduli:
5 INF - - mod[0]: start=114000 end=12f580 file=build/sistema.strip
6 INF - - mod[1]: start=130000 end=1414e0 file=build/io.strip
7 INF - - mod[2]: start=142000 end=147400 file=build/utente.strip
8 INF - Copio mod[0] agli indirizzi specificati nel file ELF:
9 INF - - copiati 108560 byte da 114000 a 200000
10 INF - - copiati 970 byte da 12edb8 a 21bdb8
11 INF - - azzerati ulteriori 79030 byte
12 INF - - entry point 200178
13 INF - Crea finestra sulla memoria centrale: [ 1000, 2000000)
14 INF - Crea finestra per memory-mapped-IO: [ 2000000, 100000000)
15 INF - Attivo la modalita' a 64 bit e cedo il controllo a mod[0]...
16 INF - Nucleo di Calcolatori Elettronici, v7.1.1
17 INF - Heap del modulo sistema: [1000, a0000)
18 INF - Numero di frame: 560 (M1) 7632 (M2)
19 INF - Suddivisione della memoria virtuale:
20 INF - - sis/cond [ 0, 8000000000)
21 INF - - sis/priv [ 8000000000, 10000000000)
22 INF - - io /cond [ 10000000000, 18000000000)
23 INF - - usr/cond [ffff800000000000, ffffc00000000000)
24 INF - - usr/priv [ffffc00000000000, 0)
25 INF - mappo il modulo I/O:
26 INF - - segmento sistema read-only mappato a [ 10000000000, 1000000f000)
27 INF - - segmento sistema read/write mappato a [ 10000010000, 10000031000)
28 INF - - heap: [ 10000031000, 10000131000)
29 INF - - entry point: start [io.s:11]
30 INF - mappo il modulo utente:
31 INF - - segmento utente read-only mappato a [ffff800000000000, ffff8000000005000)
32 INF - - segmento utente read/write mappato a [ffff8000000005000, ffff8000000007000)
33 INF - - heap: [ffff8000000007000, ffff8000000107000)
34 INF - - entry point: start [utente.s:10]
35 INF - Frame liberi: 7059 (M2)
36 INF - Creato il processo dummy (id = 0)
37 INF - Creato il processo main_sistema (id = 1)
38 INF - Inizializzo l'APIC
39 INF - Cedo il controllo al processo main sistema...
40 INF 1 Heap del modulo sistema: aggiunto [100000, 200000)
41 INF 1 Attivo il timer (DELAY=59659)
42 INF 1 Creo il processo main I/O
43 INF 1 proc=2 entry=start [io.s:11](1024) prio=1278 liv=0
44 INF 1 Attendo inizializzazione modulo I/O...
45 INF 2 Heap del modulo I/O: 100000B [0x10000031000, 0x10000131000)
46 INF 2 Inizializzo la console (kbd + vid)
47 INF 2 estern=3 entry=estern_kbd(int) [io.cpp:168] (0) prio=1104 (tipo=50) liv=0 irq=1
48 INF 2 kbd: tastiera inizializzata
49 INF 2 vid: video inizializzato
50 INF 2 Inizializzo la gestione dell'hard disk
51 INF 2 bm: 00:01.1
52 INF 2 estern=4 entry=estern_hd(int) [io.cpp:509] (0) prio=1120 (tipo=60) liv=0 irq=14
53 INF 2 Processo 2 terminato
54 INF 1 Creo il processo main utente
55 INF 1 proc=5 entry=start [utente.s:10] (0) prio=1023 liv=3
56 INF 1 Cedo il controllo al processo main utente...
57 INF 1 Processo 1 terminato
58 INF 5 Heap del modulo utente: 100000B [0xffff8000000006068, 0xffff8000000106068)
59 INF 5 Processo 5 terminato
60 INF 0 Shutdown

```

Figura 3: Esempio di messaggi di log inviati sulla porta seriale.

```

1 #include <all.h>
2
3 void main()
4 {
5     inputb(0x60)
6     pause();
7     terminate_p();
8 }

```

Figura 4: Un esempio di programma utente che tenta di eseguire un'azione illecita.

```

1 WRN 5 Eccezione 13 (errore di protezione), errore 0, RIP inputb [inputb.s:6]
2 WRN 5 proc 5: corpo start [utente.s:10](0), livello UTENTE, precedenza 1023
3 WRN 5 RIP=inputb [inputb.s:6] CPL=LIV_UTENTE
4 WRN 5 RFLAGS=246 [-- -- -- IF -- -- ZF -- PF --, IOPL=SISTEMA]
5 WRN 5 RAX= fee000b0 RBX= 0 RCX=fffffffffe68 RDX= 60
6 WRN 5 RDI= 60 RSI=fffffffffe68 RBP=fffffffff0 RSP=fffffffffe8
7 WRN 5 R8 =ffff80000106068 R9 = 0 R10= 0 R11= 0
8 WRN 5 R12= 0 R13= 0 R14= 0 R15= 0
9 WRN 5 backtrace:
10 WRN 5 > main [utente.cpp:5]
11 WRN 5 Processo 5 abortito

```

Figura 5: Esempio di messaggi di log relativi alla terminazione forzata di un processo in seguito al sollevamento di una eccezione.

processo: si tratta di una eccezione di protezione (tipo 13) e l'istruzione che l'ha generata si trova all'interno della funzione `inputb()`, alla riga 6 del file `inputb.s`. Questo è un file della `libce` (`as64/inputb.s` nella directory della libreria) e alla riga 6 troviamo appunto l'istruzione `inb %dx, %al`, che è vietata. La riga 2 contiene informazioni sul processo, tra cui il suo livello (UTENTE in questo caso) e la precedenza (1023 in questo caso). L'informazione più utile di questa riga è, in genere, quella che segue `corpo`: si tratta della funzione e del parametro che sono stati passati alla `activate_p()` quando questo processo è stato creato: in questo caso si tratta della funzione `start` (definita alla riga 10 di `utente.s`) con parametro 0 (mostrato tra le parentesi tonde). Le righe 3–8 mostrano anche il contenuto di tutti i registri, e in genere sono utili solo quando l'errore si è verificato in un file scritto in Assembler (come in questo caso, in cui possiamo vedere che il registro `rdx` conteneva 0x60 quando l'eccezione è stata sollevata). Molto più utili sono le informazioni che partono dalla riga 9: queste contengono il cosiddetto “backtrace”, ovvero la pila delle chiamate di funzione ancora attive al momento dell'errore: in questo caso possiamo vedere che `inputb()` era stata chiamata dalla funzione `main`, e più precisamente alla riga 5 di `utente.cpp`, come possiamo confermare dalla Figura 4.

2.3 Uso del debugger

Anche in questo caso, come per gli esempi di I/O, possiamo sfruttare la possibilità di collegare il debugger dalla macchina host e osservare tutto quello che accade nel sistema.

La procedura è quella già vista: avviamo la macchina virtuale passando l'opzione `-g` allo script `boot`; quindi, da un altro terminale, ci portiamo nella stessa directory e lanciamo lo script `debug`. Lo script, oltre alle estensioni già viste, carica altre estensioni dal file `debug/nucleo.py`, in modo che il debugger mostri informazioni specifiche sullo stato del nucleo. In particolare, ogni volta che il debugger riacquisisce il controllo, viene mostrato:

⁵Il sistema invia sul log soltanto degli indirizzi numerici, che sono poi trasformati in funzione, file e numero di riga dallo script `util/show_log.pl`, usando le informazioni di debug contenute nei file della directory `build`. Per vedere il contenuto del log non processato si può usare il comando `CERAW=1 boot`.

- lo stack delle chiamate (*backtrace*);
- il file sorgente nell'intorno del punto in cui si trova **rip**;
- se il sorgente è C++, i parametri della funzione in cui ci troviamo e tutte le sue variabili locali; altrimenti (assembler) i registri e la parte superiore della pila;
- il numero di processi (utente) esistenti e le liste esecuzione e pronti (e altre liste di processi);
- alcuni dettagli sul processo attualmente in esecuzione;
- lo stato di protezione della CPU.

Oltre ai normali comandi di gdb, sono disponibili i seguenti:

`process list`

mostra una lista di tutti i processi attivi (utente o sistema);

`process dump id`

mostra il contenuto (della parte superiore) della pila sistema del processo *id* e il contenuto dell'array `contexto` del suo descrittore di processo.

Altri comandi servono ad esaminare altre strutture dati che per il momento non abbiamo introdotto.

Si noti che il debugger è preimpostato per caricare i simboli di tutti e tre i moduli, quindi è possibile inserire breakpoint liberamente sia nel codice del modulo sistema, sia nel codice del modulo utente.

Realizzazione dei processi

G. Lettieri

5 Aprile 2022

1 I processi

Vediamo ora come i processi sono realizzati in generale, e poi come sono stati implementati nel sistema didattico.

1.1 Stati di esecuzione dei processi

Durante la sua vita un processo si trova in uno degli *stati di esecuzione* illustrati in Figura 1.1

I processi devono essere prima di tutto “attivati”, in modo che possano cominciare ad essere eseguiti. L’attivazione comporta la creazione di tutte le strutture dati necessarie (descrittore di processo, pile, etc.). In alcuni sistemi i processi da attivare sono decisi staticamente all’avvio del sistema. Noi realizzeremo il caso in cui i processi possono essere creati dinamicamente da altri processi (tranne ovviamente il primo processo, che sarà creato dal sistema stesso all’avvio).

Se un processo si trova in “esecuzione”, il processore sta eseguendo le sue istruzioni: il processo ha il controllo del processore e lo stato del processo (contenuto dei registri e della memoria) cambia nel tempo. Se abbiamo un solo processore (come stiamo supponendo in tutto il corso), un solo processo per volta può trovarsi in esecuzione. In tutti gli altri stati di esecuzione lo stato del processo resta costante nel tempo.

Mentre si trova in esecuzione un processo può chiedere di terminare, oppure di sospendersi in attesa di un evento. Esempi di evento sono il completamento di una operazione di I/O, o il passaggio di un determinato intervallo di tempo o anche, come vedremo, l’arrivo di un generico “segnale” da parte di un altro processo. Quando l’evento atteso si verifica il processo diventa “pronto” (può anche accadere che vada direttamente in esecuzione, anche se ciò non è mostrato esplicitamente in figura).

I processi che si trovano nello stato (di esecuzione) “pronto” sono processi che potrebbero proseguire se avessimo a disposizione sufficienti processori: sono

¹Attenzione a non confondere lo stato *di esecuzione* di un processo (che è uno tra nuovo, pronto, esecuzione, bloccato e terminato) con lo *stato del processo*, di cui abbiamo parlato precedentemente, e che consiste nel contenuto dei registri e della memoria in un qualche punto della vita del processo.

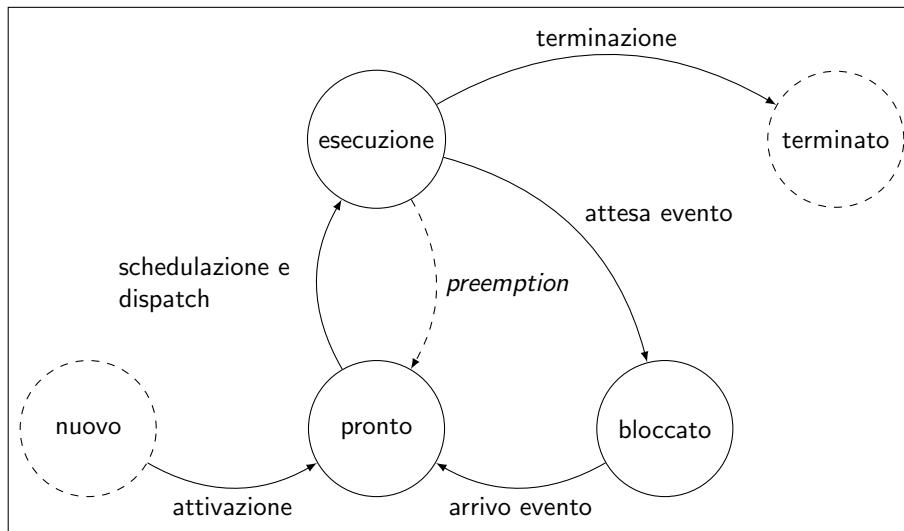


Figura 1: Stati di esecuzione un processo.

fermi solo perché il processore è già impegnato a portare avanti un altro processo. Un processo pronto può essere scelto per andare in esecuzione tramite una operazione che è detta di “schedulazione”. Il processo andrà effettivamente in esecuzione (prendendo il controllo del processore) tramite una successiva operazione che è detta di *dispatch*. In alcuni sistemi (ma non in quello che realizzeremo) schedulazione e dispatch sono combinate in un’unica azione. In ogni caso l’azione di dispatch segue dopo poco quella di schedulazione, e il fatto che siano distinte è solo un dettaglio implementativo.

Ovviamente, al momento del dispatch, il processo che era precedentemente in esecuzione deve aver prima liberato il processore, per esempio chiedendo di bloccarsi e passando nello stato “bloccato”. Un processo in esecuzione può anche essere costretto a liberare forzatamente il processore, con una azione che è detta di “*preemption*” (prelazione). La *preemption* può avvenire sia in seguito ad una interruzione o eccezione, sia come effetto collaterale di una azione richiesta dal processo in esecuzione stesso (vedremo che nel nostro caso ciò può accadere quando un processo invia un segnale ad un altro che lo stava aspettando). Il caso di *preemption* differisce dal caso di blocco, in quanto il processo passa nello stato “pronto” e non nello stato “bloccato”: il processo non sta attendendo un evento e non è più in esecuzione soltanto perché un altro processo sta occupando il processore.

Si noti che nei sistemi senza *preemption* un processo può occupare il processore indefinitamente (per esempio, eseguendo un ciclo infinito) senza lasciare mai il processore agli altri processi.

Sono possibili tante strategie di schedulazione. Nella strategia *time-sharing* (a divisione di tempo), per esempio, il processore viene assegnato ad ogni pro-

cesso pronto per un tempo massimo, passato il quale un timer interrompe il processore causando una preemption con schedulazione e dispatch del prossimo processo pronto. Noi realizzeremo un sistema a *priorità fissa*: ad ogni processo è assegnata un priorità numerica al momento della creazione e il sistema si impegna a garantire che, ad ogni istante, si trovi in esecuzione il processo che ha la massima priorità tra tutti quelli pronti. Questo ci permette di dover eseguire una azione di schedulazione solo quando un processo passa da “esecuzione” a “bloccato” oppure da “bloccato” a “pronto”. Nel primo caso il processore si libera, e dunque dobbiamo mettere in esecuzione il processo a maggiore priorità tra i pronti. Nel secondo caso c’è un novo processo pronto, che potrebbe avere priorità maggiore di quello attualmente in esecuzione: per rispettare la regola che abbiamo promesso di garantire potremmo fare preemption sul processo in esecuzione. Si noti che anche quando un processo P_1 ne attiva un altro P_2 ci troviamo in una situazione simile: abbiamo un nuovo processo pronto (P_2) mentre un altro (P_1) è in esecuzione. Noi però garantiremo che i processi non possano attivarne altri a priorità maggiore della propria, quindi non sarà mai necessaria una preemption in questo caso.

1.2 Realizzazione dei processi (nel sistema didattico)

Il nostro sistema, per semplicità, sarà mono-utente e mono-programma (perché prevediamo un singolo modulo utente), ma sarà comunque multiprocesso, nel senso che il programma potrà creare tanti processi a cui far eseguire funzioni definite nel programma stesso. I sistemi multi-processo si distinguono comunemente in “sistemi a memoria comune”, in cui tutti i processi hanno accesso alla stessa memoria, e “sistemi a scambio di messaggi”, in cui ogni processo ha una memoria completamente privata (e può comunicare con gli altri processi solo tramite messaggi). Noi realizzeremo un modello ibrido, perché questo ci permetterà di esplorare più dettagliatamente il meccanismo che rende possibile condividere o non condividere la memoria tra i processi (cioè la paginazione, come vedremo in seguito). Nel nostro sistema ogni processo utente ha una sua pila utente distinta da quella di tutti gli altri processi, a cui ha accesso esclusivo, mentre le sezioni `.text`, `.data` e `.bss`, e anche lo heap, sono condivise tra tutti. In pratica sono condivise tutte le entità globali definite nel modulo utente, più lo heap.

Dal punto di vista del sistema ogni processo ha inoltre un proprio descrittore di processo e una propria pila sistema. Lo scopo del descrittore di processo è di contenere tutte le informazioni che il sistema deve ricordare relativamente al processo. In particolare, il descrittore contiene un campo `contesto` destinato a contenere l’ultima “fotografia” scattata sullo stato del processore (vale a dire, il contenuto di tutti i registri del processore). Si ricordi che una foto completa dello stato del processo deve contenere anche lo stato di tutta la memoria. Per il momento supponiamo che lo stato di tutta la memoria del processo sia conservato su un hard disk e che nel descrittore di processo ci saranno le informazioni necessarie a recuperare tale stato dall’hard disk (almeno per la parte che non è a comune con gli altri processi).

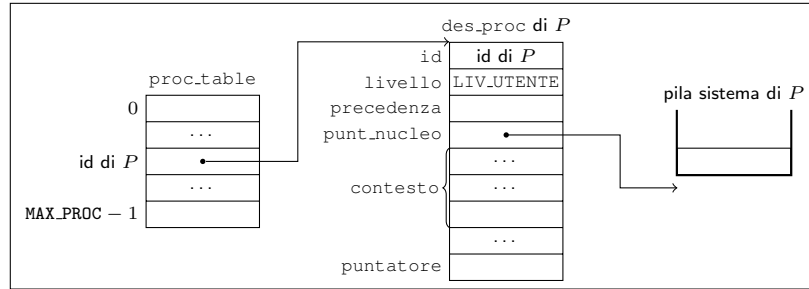


Figura 2: Strutture dati del sistema associate ad ogni processo utente.

Per gestire gli stati di Figura 1 faremo ricorso a *code di processi* realizzate come liste di descrittori di processo. La Figura 2 mostra le strutture dati che il modulo sistema deve associare ad ogni processo (con l'esclusione delle strutture dati relative alla memoria del processo). Per il descrittore di processo definiamo una struttura `des_proc` che memorizza l'identificatore numerico del processo (campo `id`), il suo livello di privilegio (utente o sistema) e la sua precedenza, come decisi al momento della sua creazione (campi `livello` e `precedenza`). Il campo `contesto` memorizza la copia privata dei registri del processore, che fanno parte appunto del contesto del processo. Questo campo è un array di `natq` (interi senza segno a 64 bit). Ogni registro ha un posto assegnato all'interno di questo array: definiremo delle costanti (`I_RAX`, `I_RBX`, etc.) per evitare di ricordare gli indici numerici. Osserviamo anche il campo `punt_nucleo`, che punta alla base della pila sistema del processo, e il campo `puntatore`, che serve a realizzare le code di processi a cui abbiamo accennato poco sopra.

Ricordiamo che il meccanismo di interruzione, nel caso in cui debba cambiare la pila corrente, prende l'indirizzo della nuova pila dal segmento TSS identificato dal registro TR. Noi allocheremo un unico segmento TSS e inizializzeremo il registro TR una sola volta all'avvio del sistema. Poi, per fare in modo che il meccanismo delle interruzioni utilizzi la pila sistema del processo corrente, dovremo sovrascrivere opportunamente il segmento TSS ogni volta che cambiamo processo.

Il processo attualmente in esecuzione sarà identificato dalla variabile globale `esecuzione`, di tipo `puntatore a des_proc`. I processi pronti si troveranno in una coda globale, la cui testa sarà puntata dalla variabile `pronti`, anch'essa di tipo `puntatore a des_proc`. Predisporemo inoltre una coda diversa per ogni tipo di evento atteso dai processi bloccati. Manterremo tutte queste code ordinate in base al campo `priorità`. In questo modo, in particolare, la schedulazione di un processo pronto può essere eseguita semplicemente estraendo il `des_proc` in testa alla coda `pronti`.

Per evitare di dover gestire in modo speciale il caso in cui tutti i processi sono bloccati è sufficiente che il sistema crei un processo a priorità minima che sia sempre pronto, detto processo dummy. Tale processo può eseguire un ciclo infinito. Nel nostro caso il processo dummy controllerà ciclicamente il numero

di processi attualmente esistenti nel sistema. Quando scopre che tutti gli altri processi sono terminati, il processo dummy può eseguire lo *shutdown* del sistema.

1.3 Cambio di processo

Il cambio di processo può avvenire solo quando il processo in esecuzione si porta a livello sistema, cosa che può avvenire solo nei seguenti modi:

- se il processo stesso esegue una istruzione **int**;
- se il processore genera una eccezione (per es., page fault);
- se il processore accetta una interruzione esterna.

In tutti e tre i casi il processore esegue azioni simili: consulta una entrata (“cancello”) della tabella IDT per prelevare l’indirizzo a cui saltare, salvando in pila l’indirizzo a cui ritornare. In base ai flag contenuti nel cancello, il processore può eseguire anche altre azioni: innalzare il livello di privilegio del processore (in base al valore del campo L); disabilitare le interruzioni (in base al valore del campo I/T). Se deve innalzare il livello di privilegio, provvede anche a cambiare pila (prelevando il puntatore alla nuova pila campo `punt_nucleo` del descrittore di processo puntato dal registro TR). Tutti i cancelli del nostro sistema prevedono l’innalzamento del livello di privilegio, quindi in tutti e tre i casi il processore passerà ad usare la pila sistema del processo corrente. In questa pila salverà il puntatore alla vecchia pila, il contenuto del registro **rflags**, il livello di privilegio precedente l’innalzamento e l’indirizzo della prossima istruzione da eseguire. Vedremo anche che tutti i cancelli che portano al modulo sistema prevedono la disabilitazione delle interruzioni (il campo I/T deve dunque specificare il tipo Interrupt e non Trap).

Inoltre, predisporremo le cose in modo che il codice a cui si salta in tutti e tre i casi segua il seguente schema:

```
call salva_stato
...
call carica_stato
iretq
```

L’ingresso nel modulo sistema, subito dopo le operazioni svolte dal meccanismo di interruzione, passa quindi per la funzione `salva_stato`. L’uscita dal modulo sistema (con successivo ritorno al modo utente tramite **iretq**) passa invece dalla funzione `carica_stato`. La funzione `salva_stato` salva lo stato del processore nel descrittore del processo identificato dalla variabile `esecuzione`, mentre la seconda carica nel processore lo stato contenuto nel descrittore del processo identificato da `esecuzione`.

In Figura 3 mostriamo un esempio con due processi, P_1 , P_2 (oltre al processo dummy, sempre presente) nel momento in cui P_1 , che è in esecuzione, si porta a livello sistema per uno qualunque dei tre motivi di cui sopra, mentre P_2 era

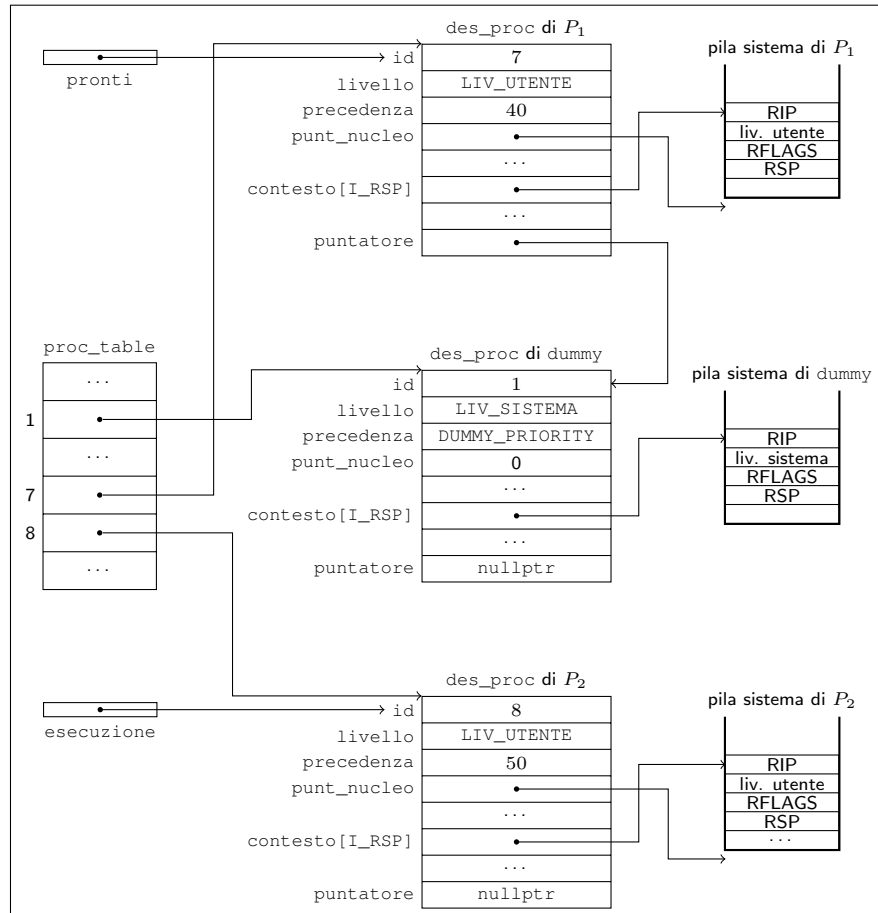


Figura 3: Esempio di istanziazione delle strutture dati per la gestione dei processi.

pronto. La Figura mostra lo stato delle strutture dati dopo il ritorno dalla `call salva_stato`.

La “fotografia” dello stato di P_1 e P_2 è costituita in parte dalle informazioni salvate in pila sistema dal meccanismo di interruzione (in particolare, il contenuto dei registri **rip** e **rsp**) e in parte dal contenuto del descrittore di processo come aggiornato dalla `salva_stato`.

Attenzione ai due valori salvati di **rsp**: quello che fa parte dello stato del processo è quello che punta alla pila utente e che si trova salvato in pila sistema. La `salva_stato` salva anche (nel campo `contesto[I_RSP]` del descrittore di processo) il contenuto del registro **rsp** come lasciato dal processore dopo il cambio di pila e il successivo salvataggio in questa delle cinque parole lunghe. La funzione `carica_stato` ripristinerà anche questo registro in modo che l’istruzione **iretq** finale possa estrarre dalla pila sistema queste informazioni, facendo dunque proseguire il processo dal punto in cui era stato interrotto.

Si noti che, quando il processore sta eseguendo codice del modulo sistema, *tutti* i processi si trovano in uno stato simile: non solo quello in esecuzione, ma anche tutti i pronti e tutti quelli bloccati. Infatti, se questi erano stati precedentemente in esecuzione, l’unico modo in cui possono esserne usciti è passando dal meccanismo delle interruzioni, in uno dei tre modi possibili. Tutti saranno dunque passati anche dalla `salva_stato`, e dunque i loro descrittori di processo e pile sistema saranno in una configurazione adatta ad essere interpretati da una `carica_stato` seguita da una **iretq**². In Figura 3 questo è illustrato dal fatto che tutte le pile sistema contengono lo stesso tipo di informazioni. Per passare da un processo ad un altro è dunque sufficiente cambiare il valore della variabile `esecuzione` in un momento qualunque tra la `call salva_stato` e la `call carica_stato`. La `carica_stato` finale caricherà (insieme agli altri registri) il valore di **rsp** che punta alla pila sistema del nuovo processo, e la successiva **iretq** farà ripartire quest’ultimo. In questo modo possiamo entrare nel sistema eseguendo un processo e, al ritorno, saltare ad un altro.

1.4 Attivazione di un processo

Nel nostro sistema un processo ne può attivare un altro invocando la primitiva `activate_p()`. La Figura 4 mostra un esempio di programma utente che usa la primitiva per attivare un nuovo processo. La primitiva accetta i seguenti parametri:

- un puntatore f alla funzione che il nuovo processo dovrà eseguire; la funzione deve essere di tipo **void (int)**, cioè accettare un unico parametro, di tipo **int**, e non restituire niente (`mioproc` in Figura);
- un’espressione di tipo `natq`, che sarà il valore ricevuto come argomento dalla funzione eseguita dal processo (10 in Figura);
- un’espressione di tipo `natl` che indica la priorità del processo (20 in Figura);

²Per i processi che non erano ancora mai andati in esecuzione si veda la sezione successiva.

```

1  #include <all.h>
2
3  void mioproc(natq i)
4  {
5      printf("mioproc: i = %d", i);
6      pause();
7      terminate_p();
8  }
9
10 int main()
11 {
12     natl id = activate_p(mioproc, 10, 20, LIV_UTENTE);
13     printf("Ho creato il processo %d", id);
14     terminate_p();
15 }

```

Figura 4: Un esempio di programma utente (file `utente/utente.cpp`).

- uno tra `LIV_UTENTE` o `LIV_SISTEMA`, per indicare se il nuovo processo deve essere di tipo utente o sistema.

Si noti che un processo utente non può creare un processo di livello sistema e non può creare un processo a priorità maggiore della propria. La primitiva restituisce l'identificatore numerico del nuovo processo, o `0xFFFFFFFF` in caso di errore.

Nell'esempio di Figura 4 il nuovo processo eseguirà `mioproc(10)`. Si noti che nulla vieta di invocare `activate_p()` più volte, anche usando lo stesso puntatore a funzione. Ogni invocazione attiverà un nuovo processo (si ricordi la distinzione tra processo e programma: `mioproc` è solo il programma).

1.5 Realizzazione della `activate_p()`

La primitiva `activate_p()` dovrà fare in modo che il nuovo processo, quando verrà messo in esecuzione la prima volta, parta dalla prima istruzione della funzione `f`, usando una nuova pila utente.

Per attivare un processo è necessario allocare e inizializzare tutte le sue strutture dati: descrittore di processo (`des_proc`) e pila sistema. Per capire come inizializzarle, pensiamo a cosa accadrà al processo dopo averlo attivato: sarà inserito in coda pronti (si veda la Figura 1), dove prima o poi verrà schedato e portato in esecuzione. Sappiamo che ciò avverrà facendo puntare esecuzione al nuovo `des_proc`, quindi eseguendo **call** `carica_stato` e infine **iretq**. Per i processi che erano arrivati in coda pronti essendo stati precedentemente in esecuzione, il descrittore letto dalla `carica_stato` e la pila sistema letta dalla **iretq** erano stati opportunamente inizializzati (dalla `salva_stato` e dal meccanismo di interruzione) l'ultima volta che il processo era uscito dallo stato esecuzione. Se vogliamo che il nuovo processo si comporti come tutti gli altri che sono già in coda pronti, possiamo inizializzare il descrittore di processo e pila sistema *come se* anch'esso si fosse precedentemente portato da livello utente a livello sistema, subito prima di eseguire la sua prima istruzione (cioè quella all'indirizzo `mioproc`, nell'esempio di Figura 4). La Figura 5 mostra il risultato

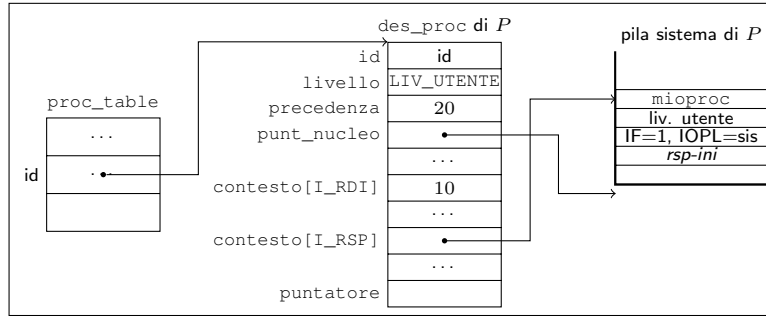


Figura 5: Attivazione del processo utente di Figura 4. Tutte le strutture dati mostrate si trovano nella parte M1 della memoria.

finale. In pila sistema scriviamo l'indirizzo di partenza (*mioproc* nell'esempio), il codice del livello utente, un campo **rflags** con flag *IF*=1 e *IOPL*=sistema, e *rsp-ini*, che punta alla pila utente opportunamente allocata. Inizializziamo il campo *contesto[I_RSP]* in modo che, in seguito alla *carica_stato*, il registro **rsp** del processore punti alle informazioni che abbiamo scritto in pila sistema. In questo modo la *iretq* che segue sempre la *call carica_stato* farà saltare il processore all'istruzione di indirizzo *mioproc*, a livello utente, con le interruzioni abilitate (e l'impossibilità di disattivarle) e pronto a usare la pila utente puntata da *rsp-ini*.

Si noti che il campo *punt_nucleo* deve puntare comunque alla base della pila sistema come se questa fosse vuota. Questo campo, infatti, verrà utilizzato dal meccanismo delle interruzioni quando il processo sarà ormai in esecuzione a livello utente. Quando un processo si trova a livello utente la sua pila sistema è sempre vuota: si riempie passando da utente a sistema e si svuota al ritorno.

Si noti che per fare in modo che la funzione che il processo eseguirà riceva il parametro che l'utente ha chiesto (secondo argomento della *activate_p()*) dobbiamo fare in modo che questo si trovi nel registro **rdi**. Per ottenere questo è sufficiente che all'attivazione del processo scriviamo il parametro attuale nel campo *contesto[I_RDI]*. La prima *carica_stato* lo porterà poi nel registro **rdi**.

Per completare l'attivazione dobbiamo anche occupare una nuova entrata della *proc_table*, scrivendovi dentro il puntatore al nuovo *des_proc*. L'indice di questa entrata nella *proc_table* funge anche da identificatore del processo. Scriviamo l'identificatore nel campo *id* del nuovo *des_proc*. Il campo *precedenza* del *des_proc* deve contenere la priorità (*prio*) del nuovo processo, che è semplicemente quella ricevuta come terzo parametro dalla *activate_p()*.

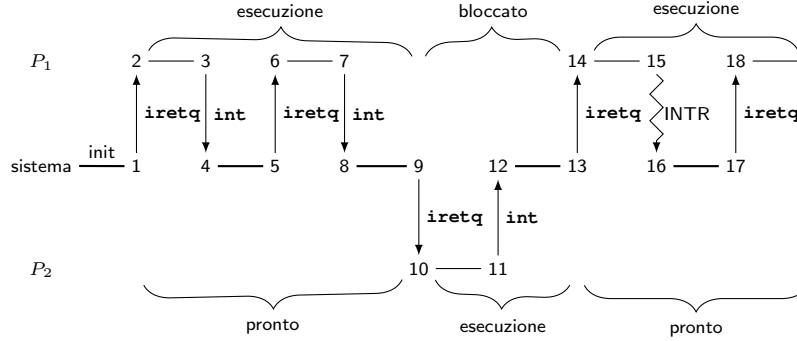


Figura 6: Un esempio di evoluzione del sistema con due processi.

1.6 Esempio

La Figura 6 mostra un esempio di una possibile evoluzione del sistema con due processi, P_1 e P_2 , con P_1 che ha una priorità maggiore di P_2 . Durante l'inizializzazione supponiamo che vengano attivati sia P_1 che P_2 e inseriti in coda pronti. All'istante 1 il sistema esegue la **iretq** che salta alla prima istruzione di P_1 . All'istante 3 P_1 invoca una primitiva del sistema tramite una istruzione **int**. Subito dopo, all'istante 4, il sistema salva lo stato di P_1 nel suo descrittore di processo. Una volta terminata, la primitiva torna al processo P_1 senza cambiare processo, all'istante 5. Si noti che il descrittore di processo viene aggiornato solo quando un processo si porta da livello utente a sistema. Nel caso di P_1 ciò avviene agli istanti 4, 8, e 16. Per tutto il resto del tempo il descrittore di processo continua a memorizzare l'ultimo stato salvato. Inoltre, si faccia attenzione a *quale* stato viene salvato: all'istante 4 viene salvato lo stato che permette a P_1 di ripartire dal punto 6, dove ci sarà la prima istruzione successiva alla **int** che P_1 aveva eseguito all'istante 3. Ciò è semplice ed intuitivo da ricordare quando il sistema *non* cambia processo tra la *salva_stato* e la *carica_stato*. Ma si ricordi che ciò avviene sempre: all'istante 7 vediamo P_1 invocare di nuovo una primitiva, portandosi a livello sistema (istante 8). La situazione delle strutture dati del sistema è ora quella descritta in Figura 3. Il **rip** salvato nella pila sistema di P_1 è quello che punta all'istruzione successiva alla **int** appena eseguita. Questa volta supponiamo che la primitiva blocchi P_1 e schedi P_2 : la pila sistema attiva (cioè, puntata dal registro **rsp** del processore) diventa quella di P_2 e la **iretq** eseguita all'istante 9 ritorna a P_2 (alla sua prima istruzione, dal momento che P_2 non era ancora mai andato in esecuzione). Successivamente, P_2 invoca anch'esso una primitiva, all'istante 11. All'istante 12 il sistema salva il nuovo stato di P_2 e decide di rimettere in esecuzione P_1 (istante 13). Da dove riparte P_1 ? Dall'ultimo stato salvato, che è sempre quello salvato all'istante 8. Questo stato fa ripartire P_1 dall'istruzione successiva alla **int** che aveva eseguito all'istante 7. In particolare, P_1 riparte in stato utente (istante 14), come se fosse tornato adesso dalla primitiva che aveva

invocato in 7.

All'istante 15 il processore accetta una interruzione esterna e si porta in modo sistema. Anche questa volta viene salvato il nuovo stato di P_1 . In questo caso, il **rip** salvato in pila sistema punta all'istruzione successiva all'ultima eseguita prima dell'accettazione dell'interrupt (si ricordi che il processore ascolta gli interrupt solo dopo aver terminato una istruzione). In 17 il sistema termina la gestione dell' interrupt e torna in modo utente senza cambiare processo, quindi la **iretq** torna in P_1 in 18.

Si noti che anche la routine di interruzione usa la pila sistema del processo che si trovava in esecuzione al momento dell'accettazione dell'interruzione.

Realizzazione delle primitive

G. Lettieri

12 Aprile 2023

Vediamo ora come le primitive sono realizzate nel sistema didattico.

1 Atomicità

Le primitive del nostro sistema devono lavorare su un insieme di strutture dati globali, come i descrittori di processo e le code dei processi. Che succederebbe se, mentre una primitiva sta lavorando su una di queste strutture, sia S , una interruzione (sia essa una interruzione esterna, una eccezione o una INT) causasse un salto ad un'altra primitiva, la quale cercasse di accedere alla stessa struttura S ? Pensiamo, per esempio, ad una primitiva che sta cercando di inserire un nuovo `des_proc*`, sia $d1$, in lista `pronti`, supponiamo in testa. Per farlo deve modificare due puntatori: copiare `pronti` in $d1 \rightarrow \text{puntatore}$ e scrivere $d1$ in `pronti`. Supponiamo che, tra la prima e la seconda scrittura, il processore salti, per effetto di una interruzione, ad un'altra routine di sistema, e che questa cerchi di inserire un altro `des_proc*`, sia $d2$, in testa alla coda `pronti`. Questa seconda primitiva copierà `pronti` in $d2 \rightarrow \text{puntatore}$ e scriverà $d2$ in `pronti`. Al termine della seconda primitiva si ritornerà alla prima, che proseguirà dal punto in cui si era interrotta, e in particolare scriverà $d1$ in `pronti`, cancellando quanto vi aveva scritto la seconda. L'effetto è che il `des_proc* d2` non è più puntato da niente.

Chiaramente non vogliamo che quanto appena descritto possa accadere, ma questo è solo uno degli infiniti problemi che si potrebbero presentare (si pensi, per esempio, ad una `salva_stato` interrotta da un'altra `salva_stato`). Più in generale, quello che abbiamo descritto è un problema di “interferenza” tra due flussi di esecuzione che lavorano su una stessa struttura dati. In generale, noi vogliamo che ogni struttura dati si trovi in uno stato “consistente”. Per esempio, una lista è in uno stato consistente se tutti e soli i suoi elementi sono raggiungibili dalla testa. In un sistema che non prevede interruzioni, le operazioni che manipolano una struttura dati vengono scritte assumendo che la struttura dati si trovi sempre in uno stato consistente quando l'operazione inizia, e assicurandosi di portarla in un nuovo stato consistente *alla fine* dell'operazione. Nel mezzo dell'operazione, però, la struttura dati può passare temporaneamente attraverso stati non consistenti. Ripensiamo all'operazione di inserimento in testa alla coda `pronti` eseguita dalla prima primitiva: subi-

to dopo la copia di `pronti` in `d1->puntatore`, la lista non è in uno stato consistente, in quanto `d1` ne fa concettualmente parte, ma non è ancora puntato da `pronti`. Questo stato inconsistente non è un problema in un sistema senza interruzioni, in quanto non è osservabile da nessun'altra operazione sulla coda: la routine di inserimento proseguirà scrivendo `d1` in `pronti`, riportando così la lista in uno stato consistente. In presenza di interruzioni, però, lo stato inconsistente diventa improvvisamente visibile da un'altra operazione, che era stata scritta assumendo che ciò non potesse mai accadere, e che dunque non è preparata per affrontare la situazione.

Abbiamo due modi principali per evitare i malfunzionamenti causati dall'interferenza:

- scrivere tutte le routine in modo da tener conto di tutti i modi in cui queste si possono mescolare, in modo che funzionino in ogni caso (possibile e anzi desiderabile, ma molto complesso);
- prevenire *a priori* l'interferenza, eliminando tutte le sorgenti di interruzione durante l'esecuzione delle primitive (o almeno delle parti critiche di esse).

Noi adotteremo la seconda soluzione per tutte le primitive del modulo `sistema`, e per la loro intera durata. In particolare, i gate che portano a tali primitive saranno di tipo “interrupt”, con disabilitazione automatica delle interruzioni esterne mascherabili. Durante la scrittura delle primitive, poi, staremo attenti a non causare eccezioni e a non chiamare altre primitive tramite `int`. Con questi accorgimenti, le nostre primitive gireranno in un contesto “atomico”: una volta iniziate saranno portate a compimento, senza che niente le possa interrompere¹. Diventeranno in questo modo molto simili alle singole istruzioni di linguaggio macchina, la cui atomicità è garantita dal processore (che gestisce interruzioni esterne e eccezioni solo tra una istruzione e la successiva).

Si noti che in molti sistemi reali l'atomicità viene considerata un prezzo troppo alto da pagare e viene rilasciata in vari modi. Noi la rilasceremo solo nel modulo `io`, ma alcuni testi d'esame considerano il caso di rilassamento anche nel modulo `sistema`.

2 Meccanismo di chiamata

A livello Assembler, invocare una primitiva non è come invocare una semplice funzione in quanto, come abbiamo detto, è necessario passare attraverso un gate della IDT con una istruzione `int`.

Al livello del C++, però, possiamo fare in modo che la primitiva si usi come una qualunque funzione, per maggior comodità dell'utente. Il meccanismo che usiamo è illustrato in Figura 1. L'utente, nel file `utente.cpp`, dichiara e chiama la funzione `primitiva_i()`, con l'obiettivo di eseguire la funzione

¹Salvo bug e interruzioni esterne non mascherabili, che comunque causeranno il blocco dell'intero sistema.

`c_primitiva_i()` che è contenuta nel modulo `sistema`. La `primitiva_i()` è in realtà solo un piccolo programma di interfaccia, scritto in assembler e contenuto nel file `utente.s`, che si limita ad eseguire l'istruzione **int** con l'indice del gate della primitiva nella IDT (*tipo_i*).

L'istruzione **int** si preoccuperà di innalzare il livello di privilegio (con le varie operazioni connesse, tra cui il cambio di pila) e saltare al codice della primitiva nel modulo `sistema`, salvando in pila l'indirizzo dell'istruzione successiva (una **ret**, in questo caso). Se non ci sarà un cambio di processo, questa è l'istruzione a cui il processore salterà al termine dell'esecuzione della primitiva.

Si noti che, a livello Assembler, anche *ritornare* da una primitiva non è come ritornare da una normale funzione, in quanto è necessario eseguire l'istruzione **iretq**, che è l'unica che permette di riportare il processore a livello utente. Anche nel modulo `sistema` dovremo quindi aggiungere una funzione di interfaccia (`a_primitiva_i` in Figura), che chiami la `c_primitiva_i()` e poi esegua la **iretq**. Con questo accorgimento, la `c_primitiva_i()` può essere scritta in C++ e compilata normalmente.

Affinchè l'istruzione "**int** \$*tipo_i*" salti alla funzione `a_primitiva_i`, con innalzamento di privilegio, il programmatore di sistema deve predisporre l'entrata della IDT di offset *tipo_i* come illustrato in figura:

- il campo IND (indirizzo a cui saltare) contiene `a_primitiva_i`;
- il campo P (gate Present) contiene 1, ad indicare che il gate è implementato;
- il campo DPL (Descriptor Privilege Level) indica che il gate può essere utilizzato da livello utente tramite una istruzione **int**;
- il campo L (Level) indica che, dopo il salto, il processore si deve trovare a livello sistema;
- per i motivi che abbiamo visto nella sezione precedente, il campo I/T indica che il gate è di tipo "interrupt", in modo che le interruzioni esterne mascherabili siano disabilitate.

La `a_primitiva_i` dovrà anche chiamare `salva_stato` e `carica_stato`, per realizzare il meccanismo del cambio di processo. In questo modo la funzione `c_primitiva_i()` può sospendere il processo corrente e schedarne un altro, semplicemente cambiando il valore della variabile `esecuzione`.

I parametri formali della `c_primitiva_i()` sono gli stessi della corrispondente `primitiva_i()`. In Figura 1 si vede che, nel tradurre la chiamata a `primitiva_i()`, il compilatore C++ copierà i parametri attuali nei registri **rdi**, **rsi**, etc. Questi registri non vengono modificati mentre si passa da `primitiva_i` e `a_primitiva_i`, quindi la funzione `c_primitiva_i()` li troverà ancora lì, dove il compilatore C++ se li aspetta.

Sia `primitiva_i()` che `c_primitiva_i()` sono dichiarate **extern "C"**. In questo modo il compilatore C++ assume che le due funzioni seguano lo standard di aggancio del linguaggio C, che non prevede l'overloading delle funzioni e

dunque non richiede che i nomi delle funzioni vengano trasformati come abbiamo visto nel caso del C++. Facciamo questo per comodità, dal momento che non prevediamo di sfruttare l'overloading per le primitive.

Normalmente il programmatore di sistema fornisce all'utente anche il file `utente.s` e un header file che contenga le dichiarazioni delle primitive (`primitiva_i()`, nell'esempio). L'utente non deve fare altro che includere tale file, con la direttiva **#include**, nel suo `utente.cpp`. Nel nostro caso le dichiarazioni si trovano nel file `sys.h` (nella directory `utente/include`).

2.1 Primitive che restituiscono un risultato

Si noti che la primitiva in Figura 1 è di tipo **void**. La presenza della chiamata di `carica_stato` rende un po' complicato restituire un valore dalla `c_primitiva_i()` alla `primitiva_i()` tramite il registro **rax**. Una istruzione di **"return ris;"** nella `c_primitiva_i()` verrebbe tradotta dal compilatore C++ lasciando il valore *ris* nel registro **rax**, ma la `carica_stato` sovrascriverebbe tale valore prima che la `primitiva_i()` possa vederlo.

Se una primitiva deve restituire un valore, occorre operare come in Figura 2. La funzione `primitiva_j()`, che è quella direttamente invocata dall'utente, è dichiarata di tipo *tipo_r*, ma la corrispondente `c_primitiva_j()` è **void**. Il valore *ris* deve essere restituito modificando il campo `contesto[I_RAX]` del descrittore del processo, in modo che la successiva `carica_stato` ricopi *ris* nel registro **rax**, dove se lo aspetta il compilatore C++ dopo la chiamata a `primitiva_j()`.

2.2 Primitive che non causano cambi di processo

Se una primitiva non causa mai un cambio di processo, alcune cose possono essere semplificate come in Figura 3. Il programmatore di sistema può eliminare le chiamate a `salva_stato` e `carica_stato` in `c_primitiva_k`. Inoltre, se la primitiva deve restituire un valore, può tranquillamente lasciarlo in **rax**, in quanto ora non verrà sovrascritto. In Figura 3 vediamo che `c_primitiva_k()` è ora dichiarata di tipo *tipo_r*, come la corrispondente `primitiva_k()` e il risultato *ris* può essere restituito con una normale **return**.

Si noti però che, con questo schema, eventuali registri *scratch* sporcati dalla primitiva non verrebbero ripristinati al ritorno a livello utente. Questa può essere considerata una violazione della riservatezza dei dati del sistema.

3 Scrivere nuove primitive

Per aggiungere una nuova primitiva si deve seguire lo schema appropriato di Figura 1 o 2 e predisporre una entrata della IDT.

Supponiamo di voler aggiungere una primitiva `getid()`, senza parametri, che restituisce l'identificatore del processo che la invoca.

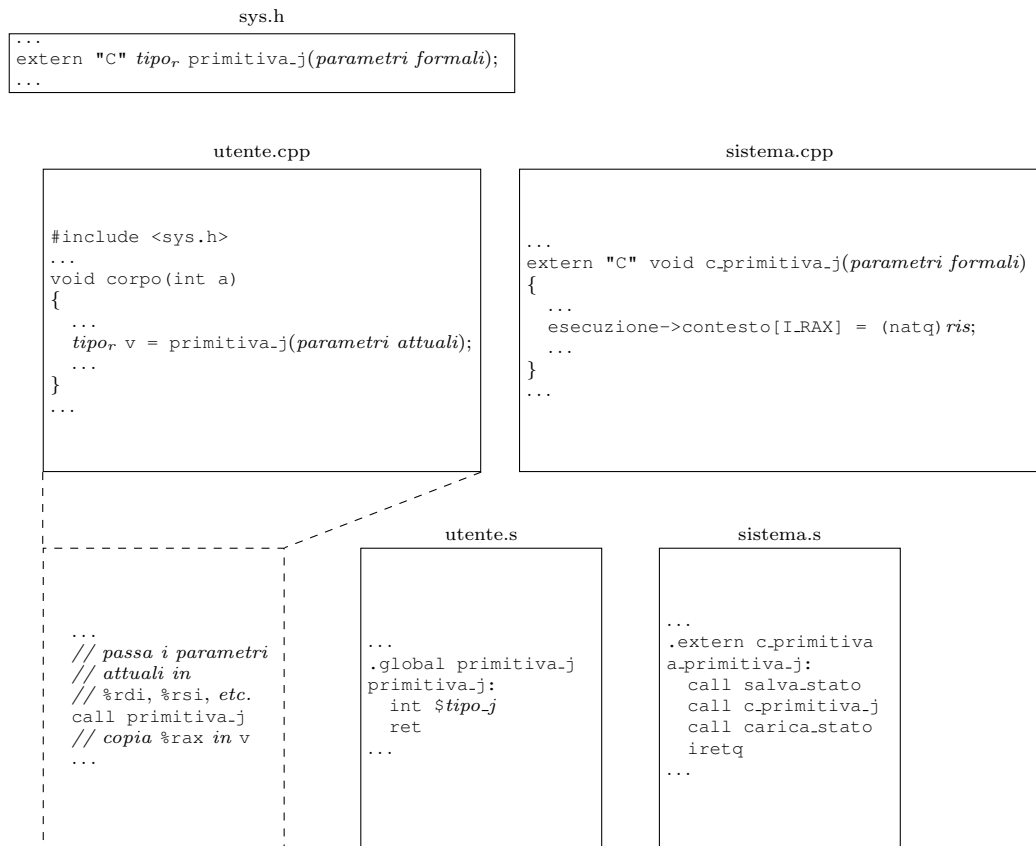


Figura 2: Primitive che devono restituire un risultato.

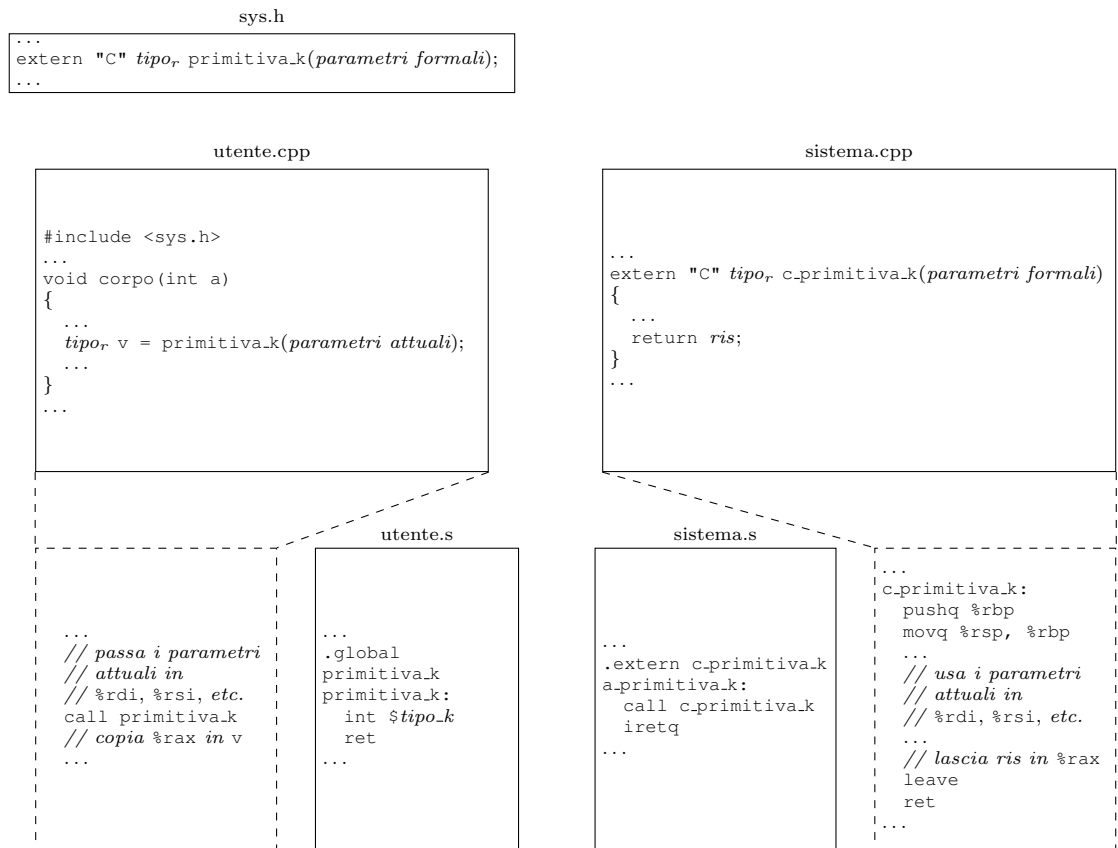


Figura 3: Primitive che non causano mai un cambio di processo.

Per prima cosa occorre assegnare un tipo di interruzione alla primitiva. I tipi di tutte le primitive già realizzate sono definiti nel file `costanti.h` nella cartella `include`, con nomi che iniziano con `TIPO_`. Si noti che i tipi sono definiti come macro, in modo che il file possa essere utilizzato sia dal C++ che dall'assembler. Scegliamo un tipo non utilizzato (per esempio, `0x2a`) e definiamo una nuova macro:

```
#define TIPO_GETID    0x2a
```

Per caricare il corrispondente gate della IDT possiamo aggiungere una riga alla funzione `init_idt` che si trova nel file `sistema.s`. Tale funzione è chiamata all'avvio del sistema e si occupa di inizializzare la tabella IDT. Possiamo usare la macro `carica_gate` che richiede tre parametri:

- il tipo della primitiva (da cui si deduce il gate della IDT da inizializzare);
- l'indirizzo a cui saltare quando qualcuno usa il gate;
- il livello di privilegio minimo richiesto per utilizzare il gate tramite una **int** (campo DPL del gate).

La macro inizializza sempre il campo P con 1 (gate implementato), il campo I/T con I (gate di tipo interrupt) e il campo L con S (la primitiva andrà in esecuzione a livello sistema). Nel nostro caso aggiungeremo la riga

```
carica_gate    TIPO_GETID    a_getid    LIV_UTENTE
```

dove `LIV_UTENTE` è una costante che specifica il livello utente (esiste anche la costante `LIV_SISTEMA` per indicare che il gate può essere usato solo da livello sistema).

Sempre nel file `sistema.s` dobbiamo scrivere la funzione `a_getid`.

```
.extern c_getid
a_getid:
    call salva_stato
    call c_getid
    call carica_stato
    iretq
```

Infine, scriviamo la primitiva vera e propria (in `sistema.cpp`):

```
extern "C" void c_getid()
{
    esecuzione->contesto[I_RAX] = esecuzione->id;
}
```

Dal punto di vista del modulo `sistema` non dobbiamo fare altro. Possiamo ricompilare il modulo `sistema` con il comando “make” e correggere eventuali errori di sintassi.

Il programmatore di sistema, come detto, dovrebbe anche fornire la dichiarazione e il programma assembler di interfaccia per l'utente. In `sys.h` aggiungiamo

```
extern "C" natw getid();
```

e nel file `utente.s`

```
.global getid
getid:
    int $TIPO_GETID
    ret
```

Il compito del programmatore di sistema finisce qui. A questo punto gli utenti possono scrivere i loro programmi e usare la nuova primitiva. Per esempio, scriviamo il seguente programma utente nel file `utente.cpp` nella directory `utente`:

```
#include <all.h>

void corpo(int a)
{
    natw id;

    id = getid();
    printf("Il mio id: %hu", id);
    terminate_p();
}

int main()
{
    activate_p(corpo, 0, 20, LIV_UTENTE);
    terminate_p();
}
```

Per provare il programma utente lanciamo il comando “make” (correggendo eventuali errori di sintassi) e poi avviamo il sistema con “boot”.

3.1 Funzioni di supporto

Le seguenti funzioni sono già definite in `sistema.cpp` e possono essere utilizzate nel definire nuove primitive.

`des_proc* des_p(natl id)`

Restituisce un puntatore al descrittore del processo di identificatore `id` (`nullptr` se tale processo non esiste).

`void schedulatore()`

Sceglie il prossimo processo da mettere in esecuzione (cambia il valore della variabile esecuzione).

```

void inserimento_lista(des_proc* p_lista, des_proc* p_elem)
    Inserisce p_elem nella lista p_lista, mantenendo l'ordinamento basato
    sul campo precedenza. Se la lista contiene altri elementi che hanno la
    stessa precedenza del nuovo, il nuovo viene inserito come ultimo tra questi.

des_proc* rimozione_lista(des_proc* p_lista)
    Estrae l'elemento in testa alla p_lista e ne restituisce un puntatore
    (nullptr se la lista è vuota).

void inspronti()
    Inserisce il des_proc puntato da esecuzione in testa alla coda pronti.

void c_abort_p()
    Distrugge il processo puntato da esecuzione e chiama schedulatore().

```

3.2 Chi esegue le primitive atomiche

È naturale pensare che le azioni svolte da una primitiva siano eseguite dal processo che l'ha invocata. Ci sono molte cose che ci inducono a concettualizzare l'esecuzione in questo modo: principalmente il fatto che il flusso di controllo è sostanzialmente continuo, almeno fino a quando non si “salta” ad un altro processo, ma anche il fatto che il contesto corrente è in buona parte quello del processo invocante: lo stato iniziale dei registri è quello lasciato dal processo, la primitiva usa la pila sistema di quel processo e, come vedremo, anche la sua memoria (virtuale). A ciò si aggiunga che, per alcuni tipi di primitive, questo è effettivamente il modo migliore di ragionare (nel nostro caso è così per le primitive del modulo di I/O, che non abbiamo ancora trattato).

Nel caso delle primitive atomiche, però, questo modo di pensare può portare a diverse confusioni. Ripensiamo alla definizione di processo come la sequenza di stati attraverso cui processore e memoria (privata del processo) passano durante l'esecuzione di un programma su dei dati; ricordiamo che possiamo visualizzare la sequenza di stati come il filmato della storia del processo, in cui ogni fotogramma ritrae il processo mentre ha appena finito di eseguire una istruzione e sta per eseguire la prossima. Tutta l'esecuzione di una primitiva atomica si svolge tra *due* di questi fotogrammi: uno in cui il processo è ritratto mentre sta per eseguire l'istruzione **int** e uno che lo ritrae subito dopo. Tutte le azioni svolte dalla primitiva non aggiungono nuovi fotogrammi tra questi due, e dunque non fanno parte della storia del processo. Tra questi due fotogrammi possono anche andare in esecuzione altri processi, che possono invocare altre primitive: tutto ciò è invisibile nella storia del processo.

Per rendere le cose meno astratte, pensiamo a cosa accade all'ingresso di una nostra primitiva: il meccanismo di interruzione e la funzione `salva_stato` scattano una foto dello stato del processo invocante. La foto coincide sostanzialmente con il primo fotogramma di cui abbiamo parlato, contenente lo stato di processore e memoria privata subito prima dell'esecuzione della **int**, con l'eccezione di **rip** che, per come funziona il meccanismo, è già quello dell'istruzione successiva e dunque fa già parte, a rigore, del secondo fotogramma. Tutto ciò

che accade fino a quando il processo non ritorna in esecuzione può servire a definire il secondo fotogramma, ma per tutto questo tempo dobbiamo immaginare che il processo sia fermo al fotogramma precedente e non stia compiendo azioni. Per esempio, a meno che il codice di sistema non abbia volutamente scritto nel campo `contesto` del descrittore del processo, i registri generali del secondo fotogramma conterranno gli stessi valori che avevano nel primo, anche se tra i due fotogrammi la CPU ha eseguito molteplici processi e primitive.

Queste considerazioni valgono in particolare per tutto il tempo di esecuzione della primitiva che il processo aveva invocato: mentre il processore sta eseguendo il codice della primitiva, il processo invocante va pensato “congelato”, in uno stato temporaneamente simile a quello di tutti gli altri processi che non erano in esecuzione. Questo è vero in generale per tutto il codice del modulo sistema eseguito atomicamente (che comprende anche le routine di risposta alle eccezioni e alle interruzioni esterne). Dobbiamo pensare quindi che la primitiva è eseguita da un’entità diversa dai processi e che, anche se usa le risorse del processo che l’ha invocata, le abbia soltanto prese “in prestito”.

Passiamo ora ad una delle cose che causano maggior confusione nella scrittura di primitive atomiche. La variabile `esecuzione` punta inizialmente al processo che *era* in esecuzione quando la primitiva è stata invocata. Quando cambiamo il valore della variabile `esecuzione` o invochiamo `schedulatore()` (che non fa altro che scrivere un nuovo valore in `esecuzione`) non significa che stiamo cedendo il controllo ad un altro processo *in quel momento*. Ci sono diversi campanelli di allarme che dovrebbero suonare e impedire di cadere in questo errore:

- come può la semplice scrittura in una variabile in memoria deviare immediatamente il flusso di controllo?
- le primitive atomiche non possono sospendersi, per definizione; come è possibile che `schedulatore()` contravvenga a questa regola?

Se questi non dovessero bastare, aggiungiamo la conclusione del ragionamento che abbiamo fatto fino ad ora: le primitive non sono eseguite da un processo, quindi non ha senso pensare che `schedulatore()` causi un cambio del processo corrente, perché non c’è alcun processo corrente. Il nuovo processo andrà in esecuzione alla prossima invocazione della coppia **call** `carica_stato;` **iretq**.

Semafori

G. Lettieri

13 Aprile 2023

Nel sistema didattico tutti i processi utente condividono le sezioni `.text`, `.data`, `.bss` del modulo utente, e lo heap. Ciascun processo ha invece una sua pila utente privata, inaccessibile agli altri processi. Questo tipo di condivisione può essere ottenuta evitando di rimpiazzare la parte di memoria condivisa quando si salta da un processo ad un altro.

Un sistema del genere ha senso se i processi appartengono tutti allo stesso utente e fanno parte di un'unica applicazione, che l'utente ha deciso di strutturare in più attività concorrenti. Da ora in poi ci limiteremo a considerare solo questo caso.

L'utente che scrive una applicazione strutturata su più processi concorrenti deve affrontare dei problemi, che in parte abbiamo già visto, perché sono molto simili a quelli già affrontati a livello sistema.

In particolare, anche l'utente deve affrontare il problema dell'*interferenza*. Mentre un processo sta eseguendo delle modifiche su una struttura dati comune, un altro processo potrebbe inserirsi e cominciare anche lui a modificare la *stessa* struttura dati. Se l'utente non scrive il codice con attenzione, questo può causare malfunzionamenti, come abbiamo visto.

Si noti che nel codice di sistema abbiamo risolto il problema ricorrendo all'atomicità, realizzata disabilitando le interruzioni mentre è in esecuzione codice di sistema. Qui non possiamo fare la stessa cosa, in quanto le interruzioni devono restare abilitate mentre è in esecuzione il codice utente (altrimenti non riusciamo a realizzare un sistema multiprogrammato), e non possiamo dare all'utente la possibilità di disabilitare e riabilitare le interruzioni a suo piacimento, perché potrebbe disabilitarle e non riabilitarle più.

L'atomicità è un caso estremo di *mutua esclusione*. Con questa locuzione ci si riferisce alla proprietà secondo la quale alcune operazioni non possono mescolarsi tra loro (si escludono a vicenda). Quello che vogliamo fare è fornire al nostro utente delle primitive tramite le quali possa realizzare la mutua esclusione delle procedure che accedono alle sue strutture dati, senza però compromettere il funzionamento di tutto il sistema.

Esaminiamo ora il problema, diverso, della *sincronizzazione* tra i processi. Nello strutturare la sua applicazione, l'utente può aver bisogno di fare in modo che una certa azione *B* avvenga sempre dopo una certa azione *A*. Se però le due azioni sono svolte da processi diversi, l'utente ha il problema di non poter prevedere quando questi due processi svolgeranno le loro azioni. Un caso comune

è quello in cui un processo produce dei dati e li scrive in un buffer intermedio, da cui un altro processo li preleva per svolgere ulteriori elaborazioni. Normalmente questi due processi sono ciclici, con il produttore che produce continuamente nuovi dati e il consumatore che li preleva continuamente. Il buffer è una zona di memoria che può contenere un numero limitato di dati—supponiamo uno solo, per semplicità. La scrittura di un nuovo dato sovrascrive dunque il vecchio. L'utente vorrebbe poter garantire che il produttore non possa sovrascrivere un dato che il consumatore non ha ancora letto, e che il consumatore non legga mai due volte lo stesso dato.

Si noti come i problemi di sincronizzazione siano diversi da quelli di mutua esclusione: nella sincronizzazione vogliamo garantire un ordinamento tra alcune azioni, che deve essere sempre rispettato. Non è così nella mutua esclusione. Si pensi a due processi, siano P_1 e P_2 , che vogliono inserire un elemento nella stessa lista. L'unica cosa che non vogliamo è che i due inserimenti vengano tentati contemporaneamente. Se, per esempio, P_1 ha iniziato un inserimento, P_2 deve aspettare che P_1 abbia finito prima di iniziare il suo. Questa *sembra* la stessa cosa della sincronizzazione, ma dobbiamo tenere presente che ci va bene anche il contrario: se P_2 arriva prima e comincia il suo inserimento, è P_1 che deve aspettare che P_2 abbia finito. Inoltre, nessuno dei due deve aspettare niente, se quando vuole fare l'inserimento l'altro non sta usando la lista. Non stiamo dunque stabilendo un ordinamento tra le operazioni di P_1 e P_2 .

1 Scatole e gettoni

Per risolvere i problemi di mutua esclusione e sincronizzazione, supponiamo di avere delle scatole che possono contenere degli oggetti, tutti uguali, che chiamiamo gettoni. Su queste scatole possono essere eseguite solo due operazioni:

- inserire un gettone—non è necessario che il gettone sia stato precedentemente preso da una scatola;
- prendere un gettone—se non ce ne sono, bisogna aspettare che qualcuno ne inserisca uno, *senza poter fare nient'altro*.

Supponiamo di dover risolvere un problema di mutua esclusione: abbiamo più persone, $P_1, P_2, \dots, P_n$, e un corrispondente numero di azioni $A_1, A_2, \dots, A_n$ che queste persone devono compiere. Vogliamo che le azioni non possano mai essere eseguite contemporaneamente. È sufficiente avere una scatola che inizialmente contiene un solo gettone e imporre la regola che solo chi ha il gettone può compiere una delle azioni: chiunque voglia agire, dunque, deve prima prendere un gettone e poi rimetterlo nella scatola quando ha finito. Dal momento che c'è un solo gettone, non è possibile che ci siano due azioni contemporaneamente in corso. Se qualcuno vuole iniziare una azione mentre ne è in corso un'altra, troverà la scatola vuota e dovrà aspettare che l'altro abbia finito.

Si noti che, mentre è in corso una azione, supponiamo da parte di P_1 , tante persone potrebbero volerne iniziare un'altra. Tutte queste dovranno aspettare.

Quando P_1 avrà finito poserà il suo gettone, che verrà preso da una delle persone in attesa, sia P_2 . Quando P_2 avrà a sua volta finito, il gettone passerà ad un'altra persona ancora, e così via. Si noti anche che la soluzione richiede una completa collaborazione da parte di tutti i partecipanti: se qualcuno compie una azione senza prendere il gettone, o si dimentica di rimetterlo a posto dopo aver finito, il sistema non funziona.

Vediamo ora come risolvere un problema di sincronizzazione. Abbiamo due persone, P_a che deve compiere l'azione A e P_b che deve compiere l'azione B . Vogliamo che l'azione B sia eseguita sempre dopo l'azione A . È sufficiente prevedere una scatola inizialmente vuota. Vogliamo che la presenza di un gettone nella scatola rappresenti il fatto che A è stata eseguita e B ancora no. Operativamente, *dopo* aver eseguito l'azione A , P_a deve lasciare un gettone nella scatola; *prima* di eseguire l'azione B , P_b deve prendere un gettone dalla scatola. Se P_a arriva per primo alla scatola, lascia il gettone e poi P_b potrà prenderlo e proseguire. Se invece arriva per primo P_b , trova la scatola vuota e deve aspettare che P_a inserisca un gettone. In entrambi i casi l'azione B non potrà partire prima che sia finita l'azione A .

In generale, per utilizzare le scatole di gettoni per risolvere un problema particolare, bisogna specificare quante scatole servono e quanti gettoni contengono inizialmente, quindi stabilire delle regole su chi deve inserire/estrarre gettoni, e quando. Il problema si può considerare risolto solo se le regole sono scelte bene e tutti le rispettano.

1.1 Esercizio

Come risolvere il problema del produttore e consumatore? Il produttore P invia dei messaggi al consumatore C tramite una lavagna. Per scrivere un nuovo messaggio cancella il precedente. Inoltre, non c'è modo di distinguere un messaggio nuovo da uno vecchio semplicemente guardandoli. Come fare per garantire che C legga tutti i messaggi di P e non legga mai due volte lo stesso messaggio?

2 Semafori

Nel nostro sistema, forniremo agli utenti l'astrazione delle scatole di gettoni, chiamate però, per motivi storici, *semafori*. Ogni semaforo è identificato da un numero. Forniamo le seguenti primitive di sistema:

- `natl sem_ini(natl v)`: crea un nuovo semaforo, che inizialmente contiene v gettoni, e ne restituisce l'identificatore (0xFFFFFFFF se non è stato possibile crearlo);
- `void sem_wait(natl sem)`: prende un gettone dal semaforo numero `sem`; blocca il processo se il semaforo è vuoto;
- `void sem_signal(natl sem)`: inserisce un gettone nel semaforo `sem`; risveglia uno dei processi bloccati in attesa di un gettone, se ve ne sono.

Si noti che, nonostante il nome, la primitiva `sem_wait()` non causa necessariamente una attesa con blocco del processo in esecuzione: se la scatola contiene già un gettone, il processo lo prende e va avanti senza bloccarsi.

Per semplicità, non diamo la possibilità di distruggere semafori. Dal momento che abbiamo deciso di supportare solo il caso di un singolo utente che esegue una applicazione multi-processo, assumiamo che tutti i semafori creati servano fino al termine dell'applicazione.

2.1 Mutua esclusione

Il problema della mutua esclusione tra le azioni $A_1, A_2, \dots, A_n$ dei processi $P_1, P_2, \dots, P_n$ può essere risolto come abbiamo visto nella Sezione 1. Prima che i processi siano stati attivati, l'utente deve creare un semaforo inizializzato ad 1:

```
nat1 mutex = sem_ini(1);
```

Quindi, tutte le azioni A_i , $1 \leq i \leq n$, devono essere protette dal semaforo:

```
Pi:    sem_wait(mutex);
        Ai;
        sem_signal(mutex);
```

Se, per esempio, le azioni A_i sono tutte invocazioni di funzioni membro di una classe su uno stesso oggetto, è sufficiente aggiungere l'identificatore del semaforo ai membri della classe, creare il semaforo nel costruttore, quindi aggiungere le chiamate `sem_wait()` e `sem_signal()` ad ogni funzione membro. In questo modo tutte le operazioni sullo stesso oggetto verranno eseguite in mutua esclusione. Se chiamiamo “libero” il semaforo, possiamo leggere l'operazione `sem_wait(libero)` come “attendi che l'oggetto sia libero (nessun altro lo sta usando)” e `sem_signal(libero)` come “segnala che ora l'oggetto è libero”.

2.2 Sincronizzazione

Il problema della sincronizzazione tra due processi— P_a che deve compiere l'azione A e P_b che deve compiere l'azione B —può essere risolto creando, prima che i processi siano attivati, un semaforo inizializzato a zero:

```
nat1 sync = sem_ini(0);
```

Quindi, i due processi devono essere codificati nel seguente modo:

```
Pa:    A;
        sem_signal(sync);

Pb:    sem_wait(sync);
        B;
```

Se P_b sta aspettando il verificarsi di una condizione, inizialmente falsa e resa vera dall'azione A , è utile dare al semaforo un nome x che ricordi questa condizione

(per esempio, `nuovo_messaggio`). Allora l'azione di P_b si legge come “aspetto che la condizione x sia vera” e l'azione di P_a si legge come “segnalo che la condizione x ora è vera”. Anche in questo caso non è detto che la `sem_wait()` debba sempre aspettare (bloccare il processo): se P_a completa l'azione A ed esegue la `sem_signal()` prima che P_b arrivi alla sua `sem_wait()`, P_b troverà il gettone già nella scatola e potrà prenderlo senza dover aspettare niente.

2.3 Realizzazione dei semafori

Per realizzare i semafori prevediamo la seguente struttura dati, definita nel codice di sistema:

```
struct des_sem {
    int counter;
    des_proc *pointer;
};
```

Il campo `counter` conta i gettoni contenuti nel semaforo, se maggiore o uguale a zero. Permettiamo al campo `counter` di scendere anche sotto zero. In questo modo la `sem_wait()` può decrementarlo in ogni caso. Se è negativo, il suo valore assoluto indica quanti processi sono in attesa di un gettone. I processi in attesa sono inseriti nella lista `pointer`. Queste liste realizzano le code dei processi bloccati in attesa di un “evento”. L'evento non è altro che l'inserimento di un gettone nel semaforo. I processi bloccati su un semaforo verranno rimessi in coda pronti quando riceveranno un gettone, per effetto di un altro processo che invoca `sem_signal()` sullo stesso semaforo. La primitiva `sem_signal()` deve incrementare il numero di gettoni e, se ci sono processi nella lista `pointer`, estrarne uno e inserirlo in coda pronti. Si noti che abbiamo deciso di realizzare un sistema in cui ogni processo ha una priorità, e ora dobbiamo garantire che in esecuzione ci sia sempre il processo che ha maggiore priorità tra tutti quelli pronti. Se la `sem_signal()` inserisce un processo in coda pronti, deve anche controllare che questo processo non abbia priorità maggiore di quello attualmente in esecuzione (quello che ha invocato la `sem_signal()`). In tal caso, il processo corrente deve andare in coda pronti (preemption) e quello appena risvegliato deve andare direttamente in esecuzione.

Come tutte le primitive, anche `sem_ini()`, `sem_wait()` e `sem_signal()` sono invocate tramite una istruzione **int** che, se eseguita da livello utente, causa un innalzamento del livello di privilegio del processore e un salto alla parte assembler della primitiva, che salva lo stato del processo corrente e chiama la parte C++, poi carica lo stato del processo puntato da esecuzione (che potrebbe essere cambiato) ed esegue una **iretq**.

La Figura 1 mostra la parte C++ della primitiva `sem_ini`. Per l'allocazione dei semafori riserviamo un array di strutture `des_sem` e ci limitiamo ad usarne una nuova, presa dall'array, ogni volta che l'utente ci chiede un nuovo semaforo. L'array ci permetterà di risalire facilmente alla struttura `des_sem` corretta, quando l'utente passerà l'identificatore del semaforo alle primitive `sem_wait()` e `sem_signal()`. Come vedremo, anche il modulo `io` utilizza dei semafori.

```

1 des_sem array_dess[MAX_SEM * 2];
2 extern "C" void c_sem_ini(int v)
3 {
4     natl i = alloca_sem();
5     if (i != 0xFFFFFFFF)
6         array_dess[i].counter = val;
7     esecuzione->contesto[I_RAX] = i;
8 }

```

Figura 1: Parte C++ della primitiva `sem_ini()`.

```

1 extern "C" void c_sem_wait(natl sem)
2 {
3     des_sem *s = &array_dessem[sem];
4     s->counter--;
5     if (s->counter < 0) {
6         inserimento_lista(s->pointer, esecuzione);
7         schedulatore();
8     }
9 }

```

Figura 2: La parte C++ della primitiva `sem_wait()`.

Per evitare che l'utente possa erroneamente usare i semafori allocati dal modulo `io`, i semafori vengono idealmente distinti in “utente” e “sistema”. I semafori utente sono quelli che si trovano nelle prime `MAX_SEM` posizioni dell'array, e i rimanenti sono di livello sistema. La funzione `alloca_sem()` allocherà un indice appartenente ad una delle due parti dell'array, in base al livello di privilegio che il processo aveva quando ha invocato la primitiva (come salvato dal processore nella pila sistema). La parte C++ della primitiva `sem_wait()` è mostrata in Figura 2. Nel caso di semaforo senza gettoni (linea 5), il processo attualmente in esecuzione viene inserito nella coda del semaforo (linea 6) e ne viene scelto un altro invocando la funzione `schedulatore()`. Questa estrae dalla coda pronti il processo a più alta priorità e lo fa puntare dalla variabile `esecuzione`, in modo che la routine `carica_stato` (che verrà eseguita subito dopo) faccia saltare al nuovo processo, di fatto bloccando il precedente.

La Figura 3, infine, mostra la parte C++ della primitiva `sem_signal()`. Se ci sono processi in coda sul semaforo (linea 5), la primitiva ne estrae uno (linea 6). Si noti che la funzione `rimozione_lista`, nel caso vi fosse più di un processo, estrae quello a maggiore priorità. A questo punto la primitiva deve scegliere chi deve proseguire, tra il processo in esecuzione e quello appena estratto. La cosa più semplice è di inserire entrambi i processi in coda pronti e lasciar scegliere alla funzione `schedulatore()` (linee 7-9).

Sia la `sem_wait()` che la `sem_signal()`, prima di usare `sem`, devono

```

1 extern "C" void c_sem_signal(natl sem)
2 {
3     des_sem *s = &array_dessem[sem];
4     s->counter++;
5     if (s->counter <= 0) {
6         des_proc* lavoro = rimozione_lista(s->pointer);
7         inspronti();
8         inserimento_lista(pronti, lavoro);
9         schedulatore();
10    }
11 }

```

Figura 3: La parte C++ della primitiva `sem_signal()`.

controllare che questo sia un valido identificatore di semaforo, cioè un numero precedentemente restituito da una invocazione di `sem_ini()` per il livello (utente o sistema) corretto, e terminare forzatamente il processo in caso contrario. Questi controlli sono stati omessi dalle Figure 2 e 3 per semplicità, ma sono presenti nel codice del nucleo.

2.4 Utilizzo del debugger

Le estensioni del debugger contengono anche alcuni comandi relativi ai semafori:

`sem` mostra lo stato di tutti i semafori allocati.

`sem waiting`

mostra lo stato di tutti i semafori la cui coda non è vuota.

Il comando `sem waiting` viene eseguito automaticamente ogni volta che il debugger riacquisisce il controllo. Lo stato dei semafori è mostrato nella forma

$\{counter, lista\ processi\}$.

Sospensione dei processi

G. Lettieri

8 Aprile 2024

Una funzionalità comune dei sistemi multiprogrammati è quella per cui un processo può chiedere di essere “sospeso” per un certo intervallo di tempo. Per esempio, in Unix, la funzione `sleep(x)` permette al processo che la invoca di “andare a dormire” per x secondi. Una funzione del genere viene tipicamente usata in processi che devono compiere azioni periodicamente, per esempio ogni minuto: il programma eseguito dal processo conterrà un ciclo che compie l’azione desiderata e poi si sospende per 60 secondi. Nel tempo in cui un processo è sospeso il sistema può eseguire altri processi.

Un processo sospeso è a tutti gli effetti bloccato e, come tutti i processi bloccati, è in attesa di un evento che lo sblocchi e lo rimetta in coda pronti (o in esecuzione): in questo caso l’evento è il passaggio del tempo richiesto. Per realizzare questa funzionalità il sistema deve dunque tenere traccia del tempo che passa, e il modo più semplice per farlo è di programmare un timer in modo che invii una richiesta di interruzione con un periodo fisso. Questa è la soluzione che adottiamo nel nostro sistema: programmiamo il timer 0 del PC AT (quello che ha il piedino di uscita collegato al controllore delle interruzioni) in modo che invii una richiesta ogni 50 ms¹. Forniamo una primitiva `void delay(nat1 n)` tramite la quale un processo può chiedere di essere sospeso per n cicli del timer. La primitiva inserirà il processo in una opportuna coda di processi sospesi, ricordando il valore iniziale di n . Chiamiamo *driver* (del timer) la routine che va in esecuzione ogni volta che il timer invia una richiesta di interruzione. Il driver dovrà decrementare n e rimettere il processo in coda pronti (o direttamente in esecuzione) quando n arriva a zero.

Supponiamo di avere tanti processi sospesi, siano $P_1, \dots, P_n$, con P_i che deve ancora attendere c_i cicli, per $1 \leq i \leq n$. Se n è grande, può essere molto costoso richiedere che il driver debba decrementare tutti gli n contatori ogni volta che va in esecuzione. Possiamo però operare nel seguente modo: manteniamo i processi ordinati in una lista in modo che $c_1 \leq c_2 \leq \dots \leq c_n$ e in ogni elemento della lista ricordiamo soltanto quanti cicli in più il processo corrispondente deve

¹Attenzione: programmiamo il timer una volta sola all’avvio del sistema e, da quel momento in poi, riceveremo la richiesta di interruzione ogni 50 ms, anche quando nessun processo ha chiesto di sospendersi e dunque non servirebbe tenere traccia del tempo che passa. Quella di avere una interruzione periodica comandata da un timer era una soluzione un tempo comune, ma è caduta in disuso con la diffusione dei PC portatili, in cui si cerca risparmiare la batteria evitando operazioni inutili. Nel nostro sistema continuiamo ad operare come un tempo, per semplicità.



Figura 1: Esempio di lista di processi sospesi.

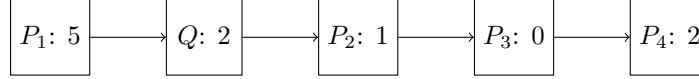


Figura 2: La lista di Figura 1 dopo l'inserimento di un nuovo processo.

attendere rispetto a quello che lo precede. In altre parole, l'elemento in cima alla lista, chiamiamolo r_1 , memorizza c_1 , mentre l'elemento r_i per $1 < i \leq n$ memorizza $c_i - c_{i-1}$. Per esempio, supponiamo di avere quattro processi con $c_1 = 5$, $c_2 = 8$, $c_3 = 8$ e $c_4 = 10$. La lista sarà nello stato mostrato in Figura 1. Il vantaggio di questa struttura dati è che il driver deve decrementare solo il primo elemento della lista: se il risultato è diverso da zero, sicuramente nessun processo deve essere risvegliato; altrimenti va risvegliato il processo in cima alla lista, più eventuali processi immediatamente successivi che abbiano già il contatore a zero. Per esempio, partendo dalla situazione di Figura 1 avremo che il processo 1 verrà risvegliato dopo $r_1 = 5$ esecuzioni del driver; a quel punto il processo 2 diventerà quello in cima alla lista e verrà risvegliato dopo altri $r_2 = 3$ cicli; insieme al processo 2 verrà risvegliato anche il processo 3 (avendo $r_3 = 0$); a quel punto il processo 4 diventerà quello in cima alla lista e verrà risvegliato dopo due ulteriori cicli ($r_4 = 2$). In generale, il processo che si trova in posizione i verrà svegliato dopo

$$\sum_{j=1}^i r_j = c_1 + (c_2 - c_1) + \dots + (c_{i-1} - c_{i-2}) + (c_i - c_{i-1}) = c_i$$

cicli, come desiderato.

Supponiamo che, partendo dalla situazione in Figura 1, un nuovo processo Q chieda di essere sospeso per 7 cicli. Il processo dovrà essere inserito in lista in modo che alla fine si ottenga la situazione in Figura 2. Si noti che è necessario decrementare il contatore di P_2 , che si trova in lista subito dopo Q , in modo che non cambi il suo tempo di attesa, ma non è necessario modificare i contatori dei processi successivi (P_3 e P_4) e precedenti (P_1).

1 Implementazione nel nucleo

La lista dei processi sospesi è definita in Figura 3. Ogni elemento della lista contiene, oltre al puntatore `p_rich` che serve a realizzare la lista, un puntatore `pp` al (descrittore del) processo sospeso e un contatore `d_attesa`, usato come spiegato sopra. La variabile `sospesi` punta alla testa della lista.

```

1 struct richiesta {
2     natl d_attesa;
3     richiesta *p_rich;
4     des_proc *pp;
5 };
6 richiesta *sospesi;

```

Figura 3: La lista dei processi sospesi.

```

1 void inserimento_lista_attesa(richiesta *p)
2 {
3     richiesta *r, *precedente;
4     r = sospesi;
5     precedente = nullptr;
6     while (r && p->d_attesa > r->d_attesa) {
7         p->d_attesa -= r->d_attesa;
8         precedente = r;
9         r = r->p_rich;
10    }
11    p->p_rich = r;
12    if (precedente)
13        precedente->p_rich = p;
14    else
15        sospesi = p;
16    if (r)
17        r->d_attesa -= p->d_attesa;
18 }

```

Figura 4: Inserimento di una nuova richiesta nella lista del timer.

```

1 extern "C" void c_delay(natl n)
2 {
3     if (!n)
4         return;
5     richiesta* p = new richiesta;
6     p->d_attesa = n;
7     p->pp = esecuzione;
8     inserimento_lista_attesa(p);
9     schedulatore();
10 }

```

Figura 5: La parte C++ della primitiva `delay()`.

La Figura 4 mostra l'implementazione della funzione che inserisce un nuovo elemento nella lista dei processi sospesi. La funzione usa la tecnica dei due puntatori, `r` e `precedente`, con `precedente` che resta sempre un passo indietro rispetto a `r`. Il ciclo alle linee 6-10 avanza i due puntatori fino a quando non puntano ai due elementi tra i quali va inserito il nuovo. Man mano che si avanza nella lista, al campo `d_attesa` del nuovo elemento vengono sottratti i campi `d_attesa` degli elementi attraversati (linea 5), in modo che alla fine contenga solo il numero di cicli in più da attendere rispetto agli elementi che lo precedono. All'uscita del ciclo, le linee 11-15 aggiustano i puntatori in modo da inserire l'elemento in lista nella posizione individuata. La linea 17 serve ad aggiustare il contatore dell'eventuale elemento successivo, in modo da non modificare il suo tempo di attesa nonostante l'inserimento del nuovo elemento.

Alla lista accedono due routine di sistema: la primitiva `delay()` e il driver del timer. Il driver, come la primitiva, è eseguito atomicamente, e questo garantisce la mutua esclusione nell'accesso alla lista. La primitiva `delay()`, con argomento `natl n`, è una normale primitiva di sistema, con un gate nella IDT e una parte scritta in Assembly analoga a quella delle altre primitive. In Figura 5 mostriamo solo la parte C++. Il caso `n` pari a zero non ha senso, ma dobbiamo gestirlo lo stesso perché non possiamo fare alcuna ipotesi sugli argomenti che l'utente decide di passare alle primitive. Decidiamo di trattarlo come una "nop" (linee 3-4). Nei casi normali allochiamo una nuova struttura `richiesta` e la inizializziamo con i valori opportuni: il numero di cicli da attendere e il processo che ha fatto la richiesta (linee 5-7). Si noti che usiamo **new**: l'operatore **new** è definito nel file `sistema.cpp` e usa una funzione di `libce` che gestisce una zona della memoria di sistema usata come heap. Alla linea 8 usiamo la funzione di Figura 4 per inserire il nuovo elemento nella lista dei processi sospesi. La sospensione vera e propria avviene perché chiamiamo la funzione `schedulatore()` che cambia il valore di `esecuzione`: al ritorno dalla funzione `driver_td()` verrà eseguita la coppia **call** `carica_stato;` **iretq** che cederà il controllo al nuovo processo, mentre quello che era precedentemente in esecuzione resterà bloccato in attesa che il driver del timer lo risvegli.


```

1      .extern c_driver_td
2 driver_td:
3      call salva_stato
4      call c_driver_td
5      call apic_send_EOI
6      call carica_stato
7      iretq

```

Figura 6: La parte assembler del driver del timer.

```

1 extern "C" void c_driver_td(void)
2 {
3     inspronti();
4     if (sospesi)
5         sospesi->d_attesa--;
6     while (sospesi && sospesi->d_attesa == 0) {
7         inserimento_lista(pronti, sospesi->pp);
8         richiesta *p = sospesi;
9         sospesi = sospesi->p_rich;
10        delete p;
11    }
12    schedulatore();
13 }

```

Figura 7: La parte C++ del driver del timer.

Il driver stesso segue sostanzialmente lo stesso schema di una primitiva, ma va in esecuzione per effetto di una richiesta di interruzione e non di una istruzione **int**. La parte in Assembly è mostrata in Figura 6. L'indirizzo `driver_td` sarà puntato da un gate della IDT. In fase di inizializzazione l'indice di questo gate deve essere scritto nel registro dell'APIC associato al piedino di interruzione collegato al timer. Rispetto ai gate di una primitiva, il gate del driver avrà DPL pari a sistema, perché vogliamo che il driver vada in esecuzione *solo* per effetto di una richiesta di interruzione da parte del timer e non vogliamo che l'utente lo faccia partire tramite una **int**.

Il codice in Assembly in Figura 6 è del tutto analogo a quello di una primitiva, con una sola differenza: prima di terminare inviamo l'EOI all'APIC (linea 5), che altrimenti non farebbe più passare richieste di interruzione con priorità minore o uguale di quella del timer. Nel nostro caso non passerebbe più nessun tipo di richiesta, in quanto abbiamo assegnato al timer la massima priorità (si veda la definizione di `TIPO_DRIVER` in `include/costanti.h`).

La parte C++ del driver è mostrata in Figura 7. Il driver potrebbe risvegliare processi che hanno precedenza maggiore di quello che era in esecuzione all'arrivo dell'interruzione, quindi dobbiamo prevedere una possibile preemp-

tion. Per gestire in modo uniforme tutti i casi, inseriamo preventivamente il processo in esecuzione in testa alla coda pronti (linea [3](#)) e chiamiamo `schedulatore()` alla fine (linea [12](#)): se il codice nel mezzo non inserisce altri processi in coda pronti le due operazioni si annullano a vicenda e il contenuto di esecuzione non cambia, altrimenti alla fine punterà al processo con priorità maggiore.

Ogni volta che va in esecuzione, il driver decrementa il contatore `d_attesa` dell'eventuale processo in testa alla lista `sospesi` (linea [5](#)), quindi estrae dalla testa tutti gli eventuali elementi con `d_attesa` pari a zero (ciclo alle linee [6](#) [11](#)) inserendo i corrispondenti processi in coda pronti (linea [7](#)) e facendo attenzione a deallocare (linea [10](#)) la struttura `richiesta` che era stata precedentemente allocata con **new** durante la `c_delay()` (linea [5](#) di Figura [5](#)).

Come sempre, il processo interrotto o uno di quelli risvegliati (in caso di preemption) andrà in esecuzione al termine di `c_driver_td()`, che ritornerà a `driver_td`, che alla fine eseguirà la coppia **call** `carica_stato; iretq`.

2 Chi esegue il driver

Come ce lo siamo chiesti per le primitive, chiedamoci ora chi esegue il driver. Probabilmente, rispetto al caso di una primitiva, è più semplice convincersi che c'è *nessun processo* che esegue il driver. Questo perché l'unico candidato (il processo che era in esecuzione all'arrivo dell'interruzione) non ha volontariamente chiamato il driver e, molto probabilmente, non è neanche interessato a ciò che il driver deve fare. Il driver usa la pila sistema del processo interrotto (e anche il resto della memoria privata) e questa volta, rispetto al caso della primitiva, è naturale pensare che l'abbia presa in prestito, proprio perché il processo interrotto non l'ha invocato volontariamente.

Eppure il contesto di esecuzione del driver è identico a quelle delle primitive: si tratta di un contesto atomico in cui si usano temporaneamente le risorse di un processo che era in esecuzione prima dell'attraversamento di un gate, e in tutte e due i casi eseguiamo una `salva_stato` all'ingresso e una `carica_stato` (seguita da **iretq**) alla fine. Conviene dunque pensare all'esecuzione delle primitive nello stesso modo in cui si pensa all'esecuzione del driver: in entrambi i casi l'esecuzione non va associata ad alcun processo.

Paginazione

G. Lettieri

14 Aprile 2024

Ricordiamo che lo stato di un processo contiene sia lo stato della CPU, sia lo stato della memoria (privata del processo). Per quanto visto finora, ogni volta che si esegue un cambio di processo tutta la memoria privata del processo uscente, contenuta in M2, deve essere ricopiata sull'hard disk e sostituita con la memoria privata del processo entrante, letta dall'hard disk. L'hard disk (o una sua partizione) dedicato a contenere le immagini delle memorie private dei processi è detto (dispositivo o partizione di) *swap* (scambio).

Nel caso generale in cui i processi potrebbero anche non avere parti condivise, può essere necessario trasferire da e verso lo swap l'intero contenuto di M2. Si noti che questo è vero anche per i processi che non usano tutta la memoria M2, in quanto il sistema non può sapere a quali parti della memoria i processi accedono mentre hanno il controllo della CPU.

Quello che vogliamo fare ora è di eliminare, o quantomeno ridurre, queste copie da e verso lo swap. Teniamo però presente che la soluzione che trasferisce tutta M2, per quanto costosa, ha dei vantaggi architetturali che vorremmo comunque preservare:

1. *Isolamento tra i processi.* Ciascun processo può accedere solo alla propria memoria privata (la memoria privata degli altri processi è al sicuro nell'area di swap, a cui i processi utente non hanno accesso).
2. *Semplicità nel collegamento dei programmi.* Il collegatore può sempre assumere che ogni programma abbia a disposizione l'intera memoria M2.
3. *Semplicità nel caricamento/scaricamento dei processi.* La memoria privata di ogni processo viene ricaricata esattamente nella stessa posizione ogni volta che il processo torna in esecuzione, quindi tutti gli indirizzi che il processo usa sono sempre validi.
4. *Possibilità di condivisione della memoria.* Se più processi hanno bisogno di condividere memoria tra loro, il sistema può concederlo evitando di sostituire le parti di memoria condivise ogni volta che cambia processo.

Un modo per riassumere la situazione è di dire che ogni processo “pensa” di avere una CPU e una memoria M2 tutte per sé, ma in realtà ha una CPU virtuale e una M2 virtuale. La vera CPU e la vera memoria M2 incarnano ad ogni istante la CPU virtuale e la memoria M2 virtuale del processo attualmente

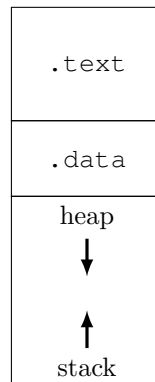


Figura 1: Organizzazione della memoria di un processo. Lo heap e lo stack si trovano ai capi opposti di un'unica regione di memoria, con lo heap che si espande verso il basso e lo stack verso l'altro.

in esecuzione, mentre l'ultimo stato delle CPU virtuali e delle memorie M2 virtuali degli altri processi sono al sicuro, rispettivamente, in M1 (nei campi contesto dei descrittori di processo) e nell'area di swap.

1 Ridurre i trasferimenti da/verso lo swap

Per ridurre i trasferimenti introdurremo una serie di meccanismi via via più sofisticati, in modo da capire le motivazioni dietro il meccanismo che adotteremo alla fine.

1.1 Registri limite inferiore e superiore

Supponiamo di sapere di quanta memoria ha bisogno ogni processo. Possiamo supporre che sia il programmatore stesso a dirlo al sistema (magari aiutato da compilatore e collegatore). Un processo avrà bisogno di un po' di memoria per contenere la sezione **.text**, contenente il codice del programma da eseguire, e di altra memoria per la sezione **.data**, contenente (in C++) le variabili globali. La dimensione di queste sezioni è nota al collegatore. Ci sono però altre due sezioni, inizialmente vuote, che si possono espandere durante l'esecuzione: la pila e lo heap (usato per la memoria dinamica). Per queste il programmatore deve stabilire un massimo. La memoria di un processo viene tipicamente organizzata come in Figura 1, con tutte le sezioni contigue e con lo heap e la pila che condividono una stessa zona di memoria. È sufficiente che il programmatore dica al sistema quanto deve essere grande la zona utilizzata dallo heap e dalla pila. Il compilatore, il collegatore, o anche il sistema stesso, possono assumere un valore di default, per non costringere il programmatore a specificare questa dimensione nei casi più comuni.

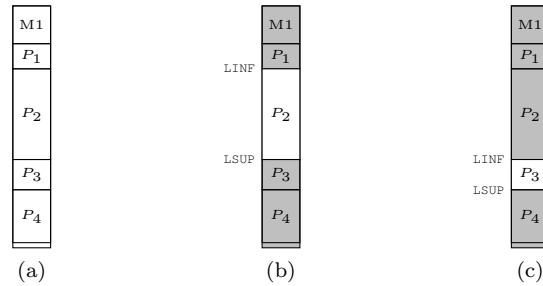


Figura 2: Un esempio di evoluzione del sistema. (a) vengono caricati P_1 , P_2 , P_3 e P_4 ; (b) quando è in esecuzione P_2 non si può accedere alla memoria degli altri processi; (c) cambiando il contenuto di LINF e LSUP cambia la zona di memoria accessibile.

Una volta che questa informazione è nota al sistema, questo può sfruttarla per copiare da e verso lo swap solo la memoria effettivamente usata dai processi entranti e uscenti, invece che l'intera M2. Ricordiamo, però, che il sistema non può fidarsi dei processi utente, quindi non può essere sicuro che, mentre un processo utente ha il controllo della CPU, acceda realmente soltanto alla parte di M2 che aveva dichiarato: per eseguire questo controllo c'è bisogno dell'aiuto dell'hardware.

Per il momento abbiamo previsto nella CPU un meccanismo che protegge la memoria M1 mentre la CPU è sotto il controllo dei processi utente. Ricordiamo in cosa consiste questo meccanismo:

- abbiamo introdotto un registro, chiamiamolo LINF, scrivibile solo da livello sistema, che contiene un indirizzo limite inferiore;
- abbiamo modificato la CPU in modo che, quando si trova a livello utente, controlli che ogni operazione in memoria (prelievo istruzioni e lettura/-scrittura operandi) avvenga a indirizzi maggiori o uguali di quello contenuto in LINF, sollevando una eccezione in caso contrario;
- all'avvio del sistema, il registro LINF viene inizializzato con il primo indirizzo di M2.

Per garantire che ciascun processo non acceda al di fuori della memoria massima dichiarata, possiamo fare un'altra piccola modifica all'hardware della CPU, aggiungendo anche un registro LSUP (limite superiore), anch'esso scrivibile solo da livello di privilegio sistema. Quando la CPU lavora in modalità utente, deve controllare che ogni accesso in memoria (sia esso per prelevare dati o istruzioni) avvenga ad indirizzi *compresi* nell'intervallo [LINF, LSUP). In caso contrario la CPU deve generare una eccezione che restituisca il controllo al sistema, che potrà decidere di terminare il processo con un errore. Durante i cambi di processo, il sistema dovrà anche provvedere a modificare opportunamente il contenuto di LSUP, mentre LINF resta costante.

Con questo meccanismo manteniamo tutti i vantaggi 1–4 e possiamo ridurre la quantità di byte da trasferire da/verso lo swap ad ogni cambio di processo, ma non il numero dei trasferimenti.

1.2 Caricamento di più processi contemporaneamente

Una volta che abbiamo i registri `LINF` e `LSUP`, possiamo pensare di usarli in un altro modo. L'idea di partenza è che la maggior parte dei processi non avrà bisogno di tutta la memoria `M2`, quindi potremmo pensare di caricare più di uno stato alla volta e di tenere in memoria anche lo stato dei processi che non sono in esecuzione, in modo da averli già pronti quando verranno schedulati. Si veda l'esempio in Figura 2(a). In pratica la memoria `M2` si comporterà come una cache dello swap, con gli stessi problemi da risolvere: quando tutta `M2` è piena e dobbiamo mettere in esecuzione un processo il cui stato non è in `M2`, dobbiamo fare spazio togliendo lo stato di qualche altro processo; quale scegliamo? Non ci addentreremo però in questi argomenti, che appartengono al corso di Sistemi Operativi. Ci limitiamo invece a studiare i meccanismi che permettono di realizzare questa cache.

Ovviamente, nella situazione di Figura 2(a), dobbiamo preoccuparci di mantenere l'isolamento tra i processi, che altrimenti potrebbero accedere liberamente alla memoria privata degli altri processi. Per garantire l'isolamento è sufficiente che il sistema scriva in `LINF` non il primo indirizzo di `M2`, ma l'indirizzo della prima locazione appartenente al processo in esecuzione, come mostrato in Figura 2(b)–(c). Entrambi i registri devono avere una posizione nel vettore `contesto` dei descrittori di processo, siano `I_LINF` e `I_LSUP`.

- Ogni volta che un processo viene caricato dallo swap (la prima volta, o dopo che era stato rimosso per fare spazio) il sistema deve inizializzare i campi `contesto[I_LINF]` e `contesto[I_LSUP]` con l'indirizzo iniziale e finale della parte di `M2` occupata dal processo.
- Ogni volta che si cambia processo, il sistema aggiorna anche il contenuto di `LINF` e `LSUP` con i valori presi dal descrittore del processo entrante.

In questo modo la CPU controllerà che ciascun processo non possa accedere al di fuori della zona di memoria che gli è stata assegnata.

Questa soluzione ci permette di ridurre enormemente il numero di trasferimenti da/verso lo swap. In compenso, però, perdiamo tutti i vantaggi architetturali che avevamo, tranne il primo:

- Non è più così semplice collegare i programmi. Prima tutti i processi venivano sempre caricati a partire dallo stesso indirizzo (l'indirizzo di partenza di `M2`), mentre ora possono essere caricati ovunque, in base a quali altri processi si trovano già in memoria al momento del caricamento. L'indirizzo di caricamento non è dunque noto durante la compilazione e il collegamento: come scegliere gli indirizzi a cui collegare i programmi? Ci sono due modi per risolvere questo problema: fare in modo che sia il caricatore a relocare il programma (con una tecnica del tutto simile a quella

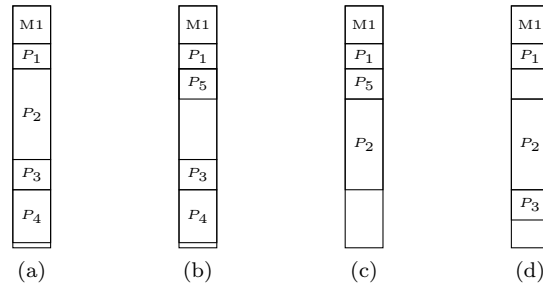


Figura 3: (a) ultimo stato dalla Figura 2; (b) P_2 viene temporaneamente rimosso per far posto a P_5 ; (c) P_3 e P_4 vengono temporaneamente rimossi per far posto a P_2 ; (d) viene ricaricato P_3 e P_5 termina: ora non c'è più posto per caricare P_4 .

usata dal collegatore) in modo da adattarlo all'indirizzo di caricamento; oppure, compilare tutti i programmi in modo che siano indipendenti dalla posizione.

- C'è però un secondo svantaggio, molto più grave: che succede se un processo viene rimosso dalla memoria e poi caricato in una posizione diversa, come il processo P_2 in Figura 3(c)? In generale non funzionerà, in quanto il suo stato potrebbe ora contenere indirizzi che erano validi solo nella posizione originaria: si pensi agli indirizzi di ritorno salvati in pila, o ai puntatori al prossimo elemento scritti in qualche lista. Questi indirizzi andrebbero corretti in base alla nuova posizione, ma in questa architettura è praticamente impossibile trovarli: lo stato è una sequenza di byte e non c'è niente che permetta di distinguere un indirizzo da qualunque altra cosa. In pratica perdiamo il vantaggio numero 3.
- Perdiamo anche il vantaggio numero 4: una eventuale parte di memoria condivisa tra i processi esisterebbe ora in più copie e non sarebbe dunque condivisa, a meno che il sistema non la copi da memoria a memoria ogni volta che cambia processo.

1.3 Registri base e limite

Possiamo recuperare il vantaggio numero 2 e parte del 3 modificando il comportamento del registro LINF. Facciamo in modo che, ad ogni accesso in memoria eseguito da livello utente, la CPU *sommi* il contenuto di LINF all'indirizzo generato dal programma, controllando che il risultato non superi LSUP. Il registro LINF contiene dunque una “base” per tutti gli indirizzi generati dal processo. I programmi utente possono ora essere collegati a partire dall'indirizzo zero e i processi che li eseguono possono continuare ad usare gli stessi indirizzi per tutta la loro esecuzione, anche se il loro stato viene tolto dalla memoria e ricaricato

in seguito ad un indirizzo diverso. Infatti, siccome LINF conterrà ora il nuovo indirizzo, tutti gli accessi in memoria verranno automaticamente aggiustati in modo da puntare alle locazioni corrette.

Si presti attenzione al fatto che questa correzione è operata a tempo di esecuzione dall'hardware ed è controllata dal software di sistema, l'unico che può scrivere nei registri LINF e LSUP. Gli utenti non possono vedere il meccanismo in opera, né tantomeno modificarlo. Quello che gli utenti vedono è che ora ogni processo sembra avere una memoria tutta per sé, come nel sistema iniziale. Questo perché il significato di un indirizzo dipende ora dal contesto: l'indirizzo x di un processo P_1 non è lo stesso indirizzo x di un diverso processo P_2 , in quanto ogni volta che P_1 usa x leggerà o scriverà una locazione di memoria diversa da quella letta o scritta da P_2 , per via dei diversi valori di LINF automaticamente sommati a x . Possiamo dunque di nuovo dire che ogni processo ha acquisito una sua memoria virtuale, che è l'unica a cui ha accesso. Tutti gli indirizzi generati durante l'esecuzione di un processo (sia per prelevare le istruzioni che per leggere o scrivere gli operandi in memoria) sono relativi alla memoria virtuale del processo, e sono detti “indirizzi virtuali”. L'azione di sommare LINF agli indirizzi generati dal processo corrisponde ad una *traduzione* da indirizzi virtuali a indirizzi *fisici*, che possono essere usati per accedere alla memoria del sistema, che da ora in poi chiameremo “memoria fisica”. I processi utente non hanno alcun controllo sulla traduzione, e dunque nessun accesso diretto agli indirizzi fisici: questo ci garantisce che la loro esecuzione non possa dipendere dagli indirizzi fisici, dando la libertà al sistema di allocarli come meglio crede.

Questo meccanismo non ci permette di recuperare il vantaggio numero 4, ma nemmeno completamente il vantaggio numero 3 (semplicità nel caricamento e scaricamento dei processi). Che succede nella situazione di Figura 2(d) in cui dobbiamo caricare un processo, ma lo spazio in memoria è frammentato in porzioni troppo piccole? Potremmo risolvere questo problema ricompattando lo spazio, ma per farlo dobbiamo copiare la memoria dei processi da una zona all'altra, operazione molto costosa.

2 Paginazione

I problemi che abbiamo visto potrebbero essere risolti se potessimo “spezzare” la memoria di un processo in più porzioni e gestire ogni porzione separatamente: potremmo dire che alcune porzioni sono condivise e altre no e caricare ogni porzione in uno spazio diverso. In presenza di memoria virtuale, questo “spezzettamento” sarebbe possibile se potessimo tradurre in modi diversi parti diverse dello spazio di indirizzamento di un processo. Questo è ciò che ci permette di fare il meccanismo della *paginazione*.

L'idea è di considerare tutti i possibili indirizzi generabili da un processo e di raggrupparli in regioni naturali dette “pagine” e di applicare una traduzione di indirizzi diversa per ogni pagina. In questo modo il sistema è libero di caricare la memoria di un processo ovunque vi sia spazio libero, anche se non contiguo. Inoltre, due o più processi possono condividere parte della loro memoria: è

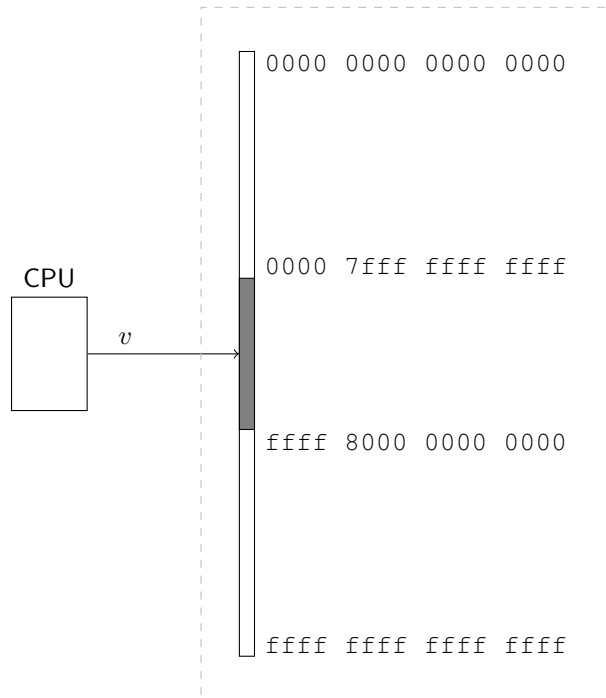


Figura 4: Spazio di indirizzamento nei processi Intel/AMD a 64 bit. Si ricordi che, per queste macchine, solo gli x bit meno significativi di un indirizzo di 64 bit possono assumere un valore qualsiasi, dove x vale 48 o 57 a seconda del modello della CPU. I $64 - x$ bit più significativi devono essere tutti uguali al bit n. $x - 1$ (contando da 0). Per fissare le idee consideriamo solo il caso $x = 48$. La parte in grigio non è dunque utilizzata. Il processore genera una eccezione se si tenta di usare un indirizzo che rientra in questa parte.

sufficiente applicare le stesse traduzioni per le pagine che contengono la zona da condividere.

Nel caso dei processori Intel/AMD a 64 bit i possibili indirizzi sono mostrati in Fig. 4, dove si mostra solo il collegamento tra la CPU e la memoria tramite il bus degli indirizzi. L'insieme di questi indirizzi forma lo *spazio di indirizzamento virtuale* di un processo. Gli indirizzi che abbiamo usato fino ad ora (quelli a cui risponde la memoria centrale, la memoria video, l'APIC e eventuali altre periferiche con registri mappati in memoria) fanno invece parte dello *spazio di indirizzamento fisico*. Gli indirizzi che fanno riferimento allo spazio di indirizzamento virtuale sono detti *indirizzi virtuali*, e quelli che fanno riferimento allo spazio di indirizzamento fisico sono detti *indirizzi fisici*.

Suddividiamo idealmente lo spazio di indirizzamento virtuale di Figura 4 in regioni naturali¹, che chiamiamo *pagine*. Per realizzare la traduzione suddivi-

¹Si ricordi che abbiamo chiamato “regioni naturali” intervalli di indirizzi allineati

diamo anche lo spazio di indirizzamento fisico in regioni naturali, dette *frame* (cornici), della stessa dimensione delle pagine. Assumiamo che le pagine e i frame siano grandi 4 KiB (0x1000 in esadecimale). Ogni indirizzo virtuale può essere scomposto in (p, o) , dove p è il *numero di pagina* e o l'offset all'interno della pagina. Similmente, ogni indirizzo fisico può essere scomposto in (n, q) , dove n è un *numero di frame* e q un offset all'interno del frame.

Il meccanismo della paginazione ci permette di mappare qualunque pagina su qualunque frame. Per farlo si introduce una struttura dati che, dato un numero di pagina p , ci permette di conoscere il numero di frame n su cui p è mappata. La più semplice struttura dati possibile è un array indicizzato dal numero di pagina: se chiamiamo $\mathbf{a}$ questo array, il numero di frame che cerchiamo è $n = \mathbf{a}[p]$. Chiamiamo *tabella di corrispondenza* l'array $\mathbf{a}$. In Fig. 5 si mostra come la tabella di corrispondenza sia un vettore in cui ogni elemento, in ordine, si occupa della traduzione della corrispondente pagina, in ordine, dello spazio di indirizzamento virtuale. Supponiamo ora che la CPU generi un indirizzo v , per esempio per accedere ad una variabile di un programma caricato in memoria centrale. La cella a cui la CPU vuole accedere non si trova in memoria all'indirizzo v : come nel caso dei registri LINF e LSUP, l'indirizzo v deve essere *tradotto* prima di poter essere usato per accedere alla memoria. La traduzione tramite LINF consisteva semplicemente in una somma, mentre ora è più complessa. L'indirizzo v cadrà all'interno di una certa pagina, sia la numero p , ad un certo offset o all'interno della pagina. Se p è caricata in $n = \mathbf{a}[p]$, la cella che corrisponde all'indirizzo v sarà quella che si trova all'offset o dentro il frame numero n . In altre parole, l'indirizzo (p, o) viene tradotto in $(\mathbf{a}[p], o)$ (si noti che l'offset resta lo stesso).

Per eseguire questa traduzione di indirizzi introduciamo un nuovo dispositivo tra la CPU e la memoria: la Memory Management Unit (MMU). La MMU intercetta tutti gli indirizzi generati dalla CPU e li traduce in base alla tabella di corrispondenza attiva, seguendo il procedimento che abbiamo descritto sopra: sostituire il numero di pagina con il numero di frame letto dalla tabella di corrispondenza, lasciando l'offset inalterato. L'operazione di traduzione è riassunta in Figura 6.

La paginazione ci permette di realizzare la cache dello swap senza perdere nessuno dei vantaggi architetturali che avevamo all'inizio:

- *Isolamento tra i processi.* Si ottiene utilizzando una diversa tabella di corrispondenza per ogni processo e impedendo che le tabelle di corrispondenza possano essere manipolate da livello utente. In questo modo ogni processo può accedere solo a quelle parti della memoria fisica che si trovano nel codominio della funzione di traduzione realizzata dalla MMU. Per impedire a un processo P di accedere al contenuto di un qualunque frame n è sufficiente che n non compaia mai nella tabella di corrispondenza di P .

naturalmente, con una dimensione che sia una potenza di 2.

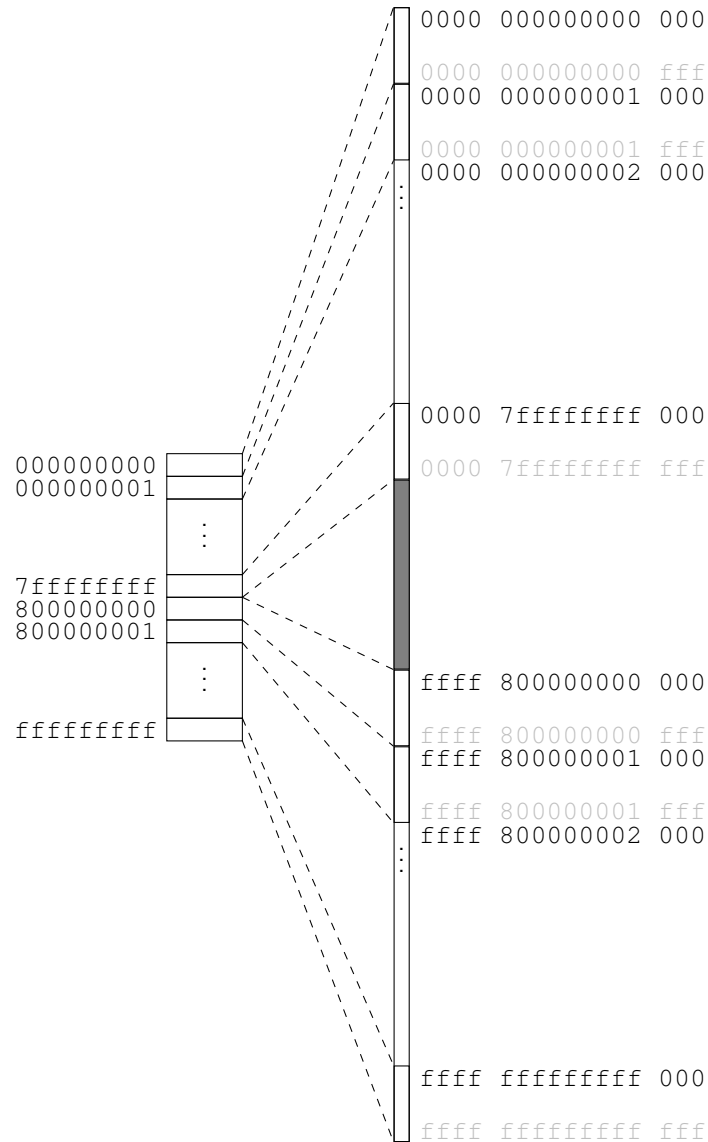


Figura 5: Tabella di corrispondenza e memoria virtuale. La tabella di corrispondenza è sulla sinistra, con a fianco gli indici (in esadecimale) delle sue entrate. Sulla destra è rappresentato lo spazio di indirizzamento virtuale, con gli indirizzi iniziali e finali delle pagine. Tutti gli indirizzi che cadono dentro una pagina sono tradotti usando l'entrata collegata tramite le linee tratteggiate. Si noti che, per il modo in cui sono definiti gli indirizzi possibili nell'architettura Intel/AMD a 64 bit, quando si passa dall'elemento di indice (in base 16) 7fffffff a quello di indice 800000000 si salta dalla pagina virtuale di indirizzo 0000 7fffffff 000 a quella di indirizzo ffff 800000000 000.

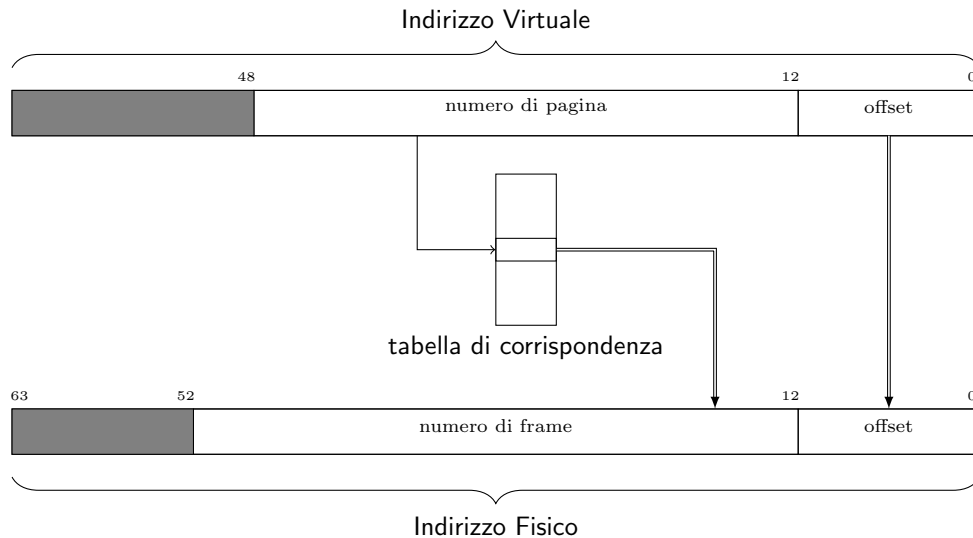


Figura 6: Traduzione da indirizzo virtuale a fisico. Si noti come in questa architettura il numero di pagina è dato solo dai bit 12–47 dell’indirizzo virtuale, mentre il numero di frame è più grande e occupa i bit 12–51 dell’indirizzo fisico.

- *Semplicità nel collegamento dei programmi.* Utilizzando una diversa tabella di traduzione per ogni processo, possiamo creare per ogni processo l’illusione che tutti gli indirizzi siano disponibili, quindi il collegatore li può assegnare liberamente.
- *Semplicità nel caricamento/scaricamento dei processi.* I processi usano solo ed esclusivamente gli indirizzi virtuali, che non cambiano anche se i processi vengono rimossi dalla memoria e caricati in un altro punto. Inoltre, ogni pagina può essere caricata ovunque ci sia un frame libero e non è più necessario caricare un intero processo in una porzione contigua della memoria.
- *Possibilità di condivisione della memoria.* Se si vuole che due o più processi condividano della memoria, è sufficiente che il sistema inserisca gli stessi numeri di frame nelle entrate opportune delle varie tabelle.

Il programmatore deve comunque dire al sistema quanto spazio occupa ogni processo, con l’unica differenza che ora lo spazio si misura in pagine.

2.1 Funzioni aggiuntive

La presenza della MMU permette al sistema di realizzare anche altre funzioni. La MMU, infatti, si trova in un punto in cui può osservare tutti gli indirizzi generati dal software e, per ognuno di essi, consulta la tabella di corrispondenza.

La tabella può essere usata per associare ulteriori informazioni ad ogni pagina (oltre al numero di frame) che dicano alla MMU come comportarsi quando intercetta un indirizzo che cade in quella pagina. Le entrate delle tabelle che si trovano nell'architettura AMD/Intel includono i seguenti campi:

- un flag P che dice se la traduzione è valida;
- un flag R/W che dice se sono ammesse scritture nella pagina;
- un flag U/S che dice se sono ammessi accessi alla pagina da livello utente.
- due flag, PWT e PCD, per agire sulla cache²;
- due flag, A e D, che danno indicazioni sugli accessi che la MMU ha osservato sulla pagina.

Il flag P serve a marcare le pagine che il processo non usa. Si ricordi che la tabella di corrispondenza contiene una entrata per ogni possibile pagina. Il caricatore di sistema (nel nostro caso, la `activate_p()`), provvederà a porre P=0 in tutte le pagine di cui il processo non ha bisogno. Se la MMU riceve dalla CPU un indirizzo che porta ad una entrata con P=0, fa in modo che la CPU sollevi una eccezione. Il sistema, tipicamente, terminerà il processo con un errore³. Il bit P può essere anche utilizzato per fermare i processi quando cercano di dereferenziare un puntatore nullo: è sufficiente porre P=0 nell'entrata di indice 0 (relativa alla pagina numero 0).

Il flag R/W può essere usato per proteggere dalla scrittura la sezione `.text`. Infatti, anche se l'architettura detta di “von Neumann” era stata introdotta proprio per permettere ai programmi di modificare il proprio stesso codice, questa pratica è da tempo fortemente limitata, in quanto crea molti più problemi di quanti ne risolve⁴. Tipicamente viene completamente vietata per i processi utente, settando opportunamente i flag R/W. La MMU fa sollevare una eccezione anche quando incontra R/W=0 nel tradurre l'indirizzo durante una operazione di scrittura.

Per capire la necessità del flag U/S, consideriamo che la MMU presente nei sistemi Intel/AMD è *sempre* attiva, anche quando il processore si trova a livello sistema. Se vogliamo che sia possibile saltare al codice di sistema quando un processo invoca una primitiva, genera una eccezione o riceve una interruzione esterna, dobbiamo fare in modo che tutta la memoria M1 si trovi nel codominio delle funzioni di traduzione di tutti i processi. Dobbiamo dunque riservare delle pagine nella memoria virtuale di ogni processo, in modo che vengano tradotte nei frame che coprono M1. Nel processore AMD64 sfruttiamo il fatto che la memoria virtuale è naturalmente divisa in due (Figura 4) e riserviamo la parte

²Questi flag sono stati introdotti nel processore 80486.

³In Unix il processo riceve una signal SIGSEV, che normalmente ne causa la terminazione con ritorno alla shell, che poi stampa “Segmentation fault”.

⁴Si noti che l'articolo originale di von Neumann già limitava i modi in cui i programmi dovevano poter modificare il proprio codice, ma i primi calcolatori poi realizzati non applicavano queste limitazioni. In ogni caso, anche il meccanismo più limitato suggerito da von Neumann non è realmente necessario.

superiore al sistema e la parte inferiore all'utente⁵. Allo stesso tempo non vogliamo che i processi utente possano accedere alla memoria M1. Potremmo ricorrere nuovamente al registro limite, ma ora che abbiamo la MMU possiamo sfruttarla per farle fare anche questo controllo: basta porre U/S=sistema in tutte le entrate relative alla parte alta dello spazio di indirizzamento (le entrate con indici da 000000000 a 7fffffff in Figura 5). Anche in questo caso la MMU fa generare una eccezione se rileva un accesso da livello utente ad una pagina con U/S=sistema.

I bit PWT e PCD sono un mezzo tramite il quale il sistema può indirettamente inviare ordini alla cache, sfruttando il fatto che la MMU si trova sul percorso che porta dal processore alla cache (la cache si trova tra la MMU e la memoria fisica). La MMU si preoccuperà di inoltrare l'ordine alla cache ogni volta che traduce un indirizzo.

1. Il bit PCD (Page Cache Disable) ordina alla cache di non intercettare l'operazione (sia essa di lettura o di scrittura) e di lasciarla passare inalterata sul bus, similmente a come si comporta per gli accessi nello spazio di I/O. La routine di inizializzazione porrà PCD=1 per tutte le pagine che contengono indirizzi di registri di I/O mappati in memoria, invece che locazioni di memoria. Un esempio è l'APIC, i cui indirizzi sono appunto mappati nello spazio di memoria.
2. Il bit PWT (Page Write Through) ordina alla cache di usare la politica *write-through* per questo particolare accesso (ovviamente, solo se si tratta di una scrittura). Se PCD è 1 il bit PWT non ha effetto. Se PCD è 0, porre PWT=1 può essere utile per la parte di indirizzi relativa alla memoria video: quando il programma scrive vogliamo che la scrittura arrivi nella vera memoria video e non si fermi in cache, in modo che il controllore video possa visualizzare l'informazione sul display; ma se il programma vuole leggere dalla memoria video possiamo tranquillamente farlo leggere dalla cache.

Incidentalmente, si noti che l'aver posizionato la cache dopo la MMU comporta che la cache non deve subire modifiche rispetto a quella che abbiamo già studiato: il controllore cache vede arrivare indirizzi fisici esattamente come prima (arrivano dalla MMU invece che direttamente dalla CPU, ma questo è irrilevante). Inoltre, la presenza della cache non turba il funzionamento della MMU: la presenza della cache è trasparente anche per la MMU. Questa è la soluzione adottata nei processi Intel/AMD64.

I bit A e D, infine, sono legati all'implementazione dell'area di swap, a cui accenniamo solo dal punto di vista teorico (non la realizzeremo nel sistema didattico). La MMU setta ad 1 il bit A di una entrata quando la usa, cioè durante un accesso ad un indirizzo all'interno della corrispondente pagina. Se l'accesso era in scrittura, la MMU setta anche il bit D. Consideriamo, per esempio, un sistema che gestisca il caricamento (*swap-in*) e scaricamento (*swap-out*) dei processi da e verso un dispositivo di swap (per esempio, un hard disk). In questo

⁵Linux e FreeBSD fanno al contrario.

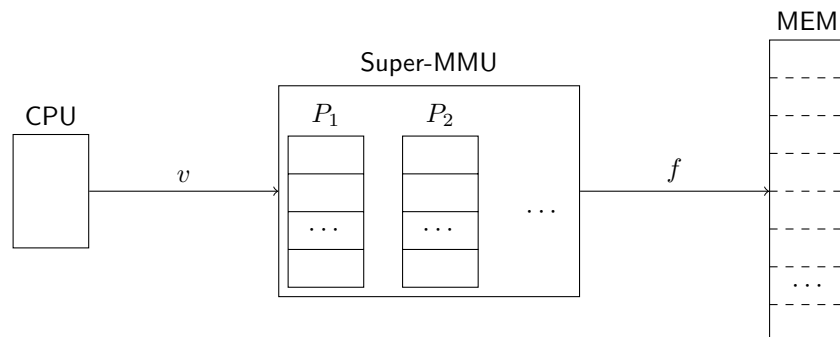


Figura 7: Traduzione degli indirizzi con una Super-MMU. Quando è in esecuzione il processo P_i , la Super-MMU traduce gli indirizzi usando la corrispondente tabella.

caso lo stato iniziale dei processi si trova sullo swap. Ogni volta che il sistema carica le pagine di un processo dallo swap in memoria dovrebbe porre $D=0$ in tutte le entrate della tabella di corrispondenza. Al momento di eseguire uno swap-out del processo, il sistema può evitare di riscrivere nello swap le pagine che non sono state modificate mentre erano in RAM. Per scoprire quali siano queste pagine, può esaminare le entrate delle tabelle (di livello 1) notando in quali di esse il bit D è diventato 1: queste puntano a pagine che sono state modificate e vanno salvate; per tutte le altre il salvataggio si può evitare, in quanto la copia già presente nello swap è ancora valida. Il bit A può essere usato per capire quali pagine/tabelle sono più usate o sono state usate più recentemente, ed è di ausilio soprattutto all'implementazione della *paginazione su domanda*, in cui non vengono caricati in memoria tutte le pagine di un processo ma solo le pagine a cui il processo effettivamente accede, sfruttando il bit P e intercettando l'eccezione di page fault.

3 La Super-MMU

Precedentemente, per fare in modo che ogni processo fosse isolato dagli altri, avevamo una coppia di valori `contesto[I_LINF]` e `contesto[I_LSUP]` per ogni processo, con una sola coppia attiva ad ogni istante (quella appartenente al processo in esecuzione). Analogamente, ora dovremo avere una intera tabella di corrispondenza distinta per ogni processo, con una sola tabella attiva ad ogni istante (quella appartenente al processo in esecuzione). Per introdurre i dettagli un po' alla volta, in modo da concentrarci prima sugli aspetti più importanti, introduciamo prima una Super-MMU che contiene essa stessa tutte le tabelle di corrispondenza, di tutti i processi, al proprio interno (Figura 7).

È disponibile un [semplice esempio](#) che illustra quando detto finora, facendo uso della Super-MMU. Uno degli scopi dell'esempio è di far vedere come la memoria virtuale sia completamente *trasparente* al programmatore utente.

Tabelle multilivello

G. Lettieri

20 Aprile 2023

1 Trie-MMU: tabella su più livelli

Calcoliamo la dimensione delle tabelle di corrispondenza usate dalla Super-MMU. La memoria virtuale è almeno di 2^{48} byte¹ e ogni pagina è grande 2^{12} byte, quindi la memoria virtuale contiene

$$\frac{2^{48}}{2^{12}} = 2^{36} = 64 \text{ Gi pagine.}$$

La tabella di corrispondenza di ogni processo deve avere una entrata per ognuna di queste pagine. Ogni entrata deve contenere almeno i bit P, R/W, U/S, PCD, PWT, A, D e il numero di frame che fornisce i bit da 12 a 51 dell'indirizzo fisico, per un totale di 47 bit, arrotondati in 6 byte. Se poi vogliamo che la dimensione di ogni entrata sia una potenza di 2 dovremo usare almeno 8 byte. In conclusione, la tabella di corrispondenza dovrà essere di

$$64 \text{ Gi} \times 8 \text{ B} = 512 \text{ GiB.}$$

Difficilmente, quindi, possiamo pensare di avere un dispositivo di memoria che possa contenere anche una sola di queste tabelle, figuriamoci una per ogni processo.

Per affrontare il problema notiamo che la stragrande maggioranza dei programmi ha bisogno soltanto di una piccola frazione dei 2^{48} byte disponibili di memoria virtuale. Vorremmo avere una struttura dati che contenga le sole entrate effettivamente utilizzate.

Introduciamo quindi Trie-MMU, una MMU del tutto identica alla Super-MMU, tranne che per il formato della tabella di corrispondenza. Come la Super-MMU, Trie-MMU possiede una memoria interna in cui salvare le tabelle di corrispondenza e un registro, **cr3**, che serve ad individuare la tabella di corrispondenza attiva ad ogni istante. La speranza è di poter usare una memoria interna molto più piccola rispetto a quella richiesta dalla Super-MMU.

¹Come detto precedentemente, omettiamo di considerare il caso di memoria virtuale grande 2^{57} byte.

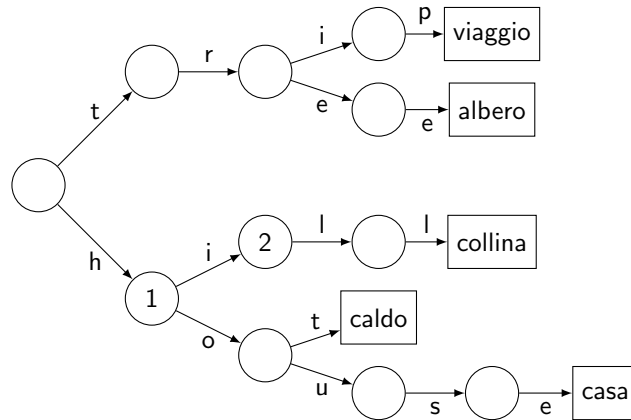


Figura 1: Un esempio di trie con chiavi e valori di tipo stringa.

1.1 La struttura dati *trie*

La struttura dati utilizzata da Trie-MMU è un *bitwise trie*, che è una variante di trie. I trie sono strutture dati ad albero che permettono di mappare chiavi di tipo stringa in valori, in modo che i caratteri successivi della chiave guidino la ricerca all'interno dell'albero. Consideriamo, per esempio, il trie mostrato in Figura 1. L'albero memorizza le associazioni $\text{trip} \mapsto \text{viaggio}$, $\text{tree} \mapsto \text{albero}$, $\text{hill} \mapsto \text{collina}$, $\text{hot} \mapsto \text{caldo}$ e $\text{house} \mapsto \text{casa}$. Si noti come gli archi dell'albero siano contrassegnati con i caratteri delle chiavi e il valore associato ad ogni chiave si trovi nella foglia che si raggiunge partendo dalla radice e seguendo il percorso indicato dalla chiave².

Un modo di implementare un trie è di avere, in ogni nodo dell'albero, un array di 128 entrate, ciascuna delle quali contenga il puntatore al prossimo nodo da visitare in base al codice ASCII del prossimo carattere della chiave.

Ogni nodo si trova sul percorso di tutte le chiavi che iniziano con lo stesso prefisso. Per esempio, il nodo marcato con “2” si trova nel percorso di tutte le chiavi che iniziano con “hi”. Un puntatore nullo nell'array di un nodo indica che il trie non contiene chiavi con il corrispondente prefisso. Per esempio, una ricerca della chiave “history” nel trie di Figura 1 seguirebbe il ramo “h” dalla radice per arrivare al nodo “1”, quindi il ramo “i” per arrivare al nodo “2”. Qui troverebbe un puntatore nullo associato al carattere “s” e la ricerca si concluderebbe con un fallimento. L'inserimento di una nuova associazione chiave/valore nel trie comporta una visita dell'albero come in una ricerca, ma creando eventuali nodi mancanti fino alla foglia che deve contenere il valore.

Nel nostro caso la chiave è il numero di pagina e il valore che vi vogliamo associare è il corrispondente numero di frame. Per questo scopo possiamo usare

²In un trie generico il valore associato alla chiave può trovarsi anche in alcuni dei nodi intermedi (non foglia). Per esempio, il valore “ciao” associato alla chiave “hi” si troverebbe direttamente nel nodo 2. Questo non accade nel bitwise trie che interessa noi.

un *bitwise* trie, che funziona esattamente come un trie ma, al posto dei caratteri, usa gruppi di bit della chiave. In particolare, il numero di pagina è composto da 36 bit che possiamo raggruppare in 4 gruppi di 9 bit. Ogni nodo del bitwise trie conterrà dunque una tabella di $2^9 = 512$ entrate con puntatori, eventualmente nulli, al nodo successivo. Le foglie stesse possono essere tabelle indicizzate dall'ultimo gruppo di 9 bit della chiave. In questo caso le entrate delle tabelle foglie conterranno il numero di frame associato al numero di pagina. Si arriva dunque alla struttura dati illustrata in Figura 2. Ciascun nodo dell'albero di Figura 2, foglie incluse, è una tabella di 512 entrate, ciascuna grande 8 byte. Ogni tabella è grande dunque 4096 byte. L'albero ha al massimo 4 livelli, che per convenzione vengono numerati da 4 a 1 (il livello delle foglie), in modo da poter parlare di tabelle di livello 4, di livello 3 e così via. È molto comodo rappresentare i numeri di pagina in base 8, in quanto ciascuna cifra in base 8 corrisponde a 3 bit del numero di pagina, e dunque ciascun gruppo di 9 bit può essere rappresentato da 3 cifre in base 8. Per esempio, supponiamo che la Trie-MMU si trovi a dover tradurre l'indirizzo virtuale $v = (000\ 777\ 000\ 777\ 1234)_8$. Il numero di pagina è $(000\ 777\ 000\ 777)_8$. I primi 9 bit del numero di pagina sono $(000)_8$. La Trie-MMU usa quindi l'entrata di indice 0 (vale a dire la prima entrata) della tabella di livello 4 per trovare la prossima tabella da consultare. La Trie-MMU passerà dunque alla tabella di livello 3 in alto in Figura 2. Qui usa i successivi 9 bit, $(777)_8$ per sapere quale entrata di questa tabella deve consultare per raggiungere la prossima tabella, di livello 2. Dunque la Trie-MMU userà l'ultima entrata della tabella e passerà alla seconda tabella di livello 2 dall'alto in Figura 2. Qui userà i bit $(000)_8$ del terzo gruppo e proseguirà verso la terza tabella di livello 1 dall'alto. Infine, troverà la traduzione che stava cercando nell'entrata $(777)_8$ di questa tabella.

Il formato delle entrate delle tabelle di livello 1 è mostrato in Fig. 3 e rispecchia quello dell'architettura Intel/AMD a 64 bit. Chiameremo queste entrate *descrittori di pagina virtuale* o anche *descrittori di livello 1*, essendo contenuti nelle tabelle di livello 1. Si noti che i descrittori contengono tutte le informazioni che abbiamo già introdotto parlando della Super-MMU. I bit P, R/W, U/S, PWT e PCD sono scritti dalla routine di sistema che attiva il processo (nel nostro caso, `activate_p()`) e soltanto letti dalla Trie-MMU. I bit A e D, invece, sono letti e scritti sia dal software (di sistema) che dalla Trie-MMU.

Ogni volta che il sistema carica le pagine di un processo in memoria (alla prima attivazione, oppure dopo uno *swap-in* in seguito ad uno *swap-out*), dovrebbe porre $D=0$ in tutte le entrate della tabella di corrispondenza. Al momento di eseguire uno *swap-out* del processo, il sistema può evitare di salvare tutte le pagine del processo nel dispositivo di swap ri-esaminando le entrate e notando in quali di esse il bit D è diventato 1: queste puntano a pagine che sono state modificate e vanno risalvate; per tutte le altre il salvataggio si può evitare, in quanto la copia già presente nello swap è ancora valida. Il bit A può essere usato per capire quali pagine/tabelle sono più usate o sono state usate più recentemente, ed è di ausilio soprattutto all'implementazione della paginazione su domanda.

Tutte le tabelle di livello 2, 3 e 4 hanno lo stesso formato. Ciascuna di esse

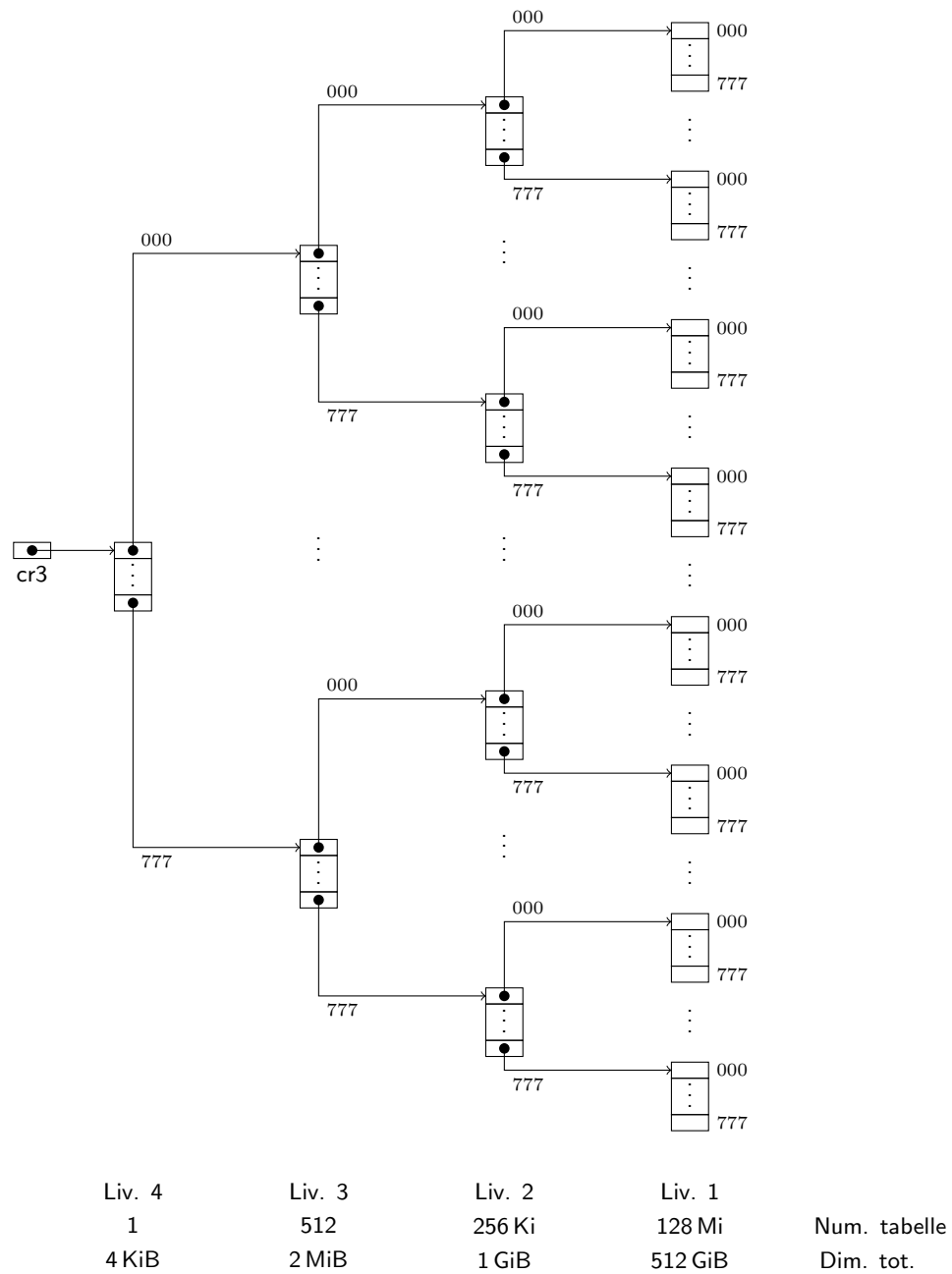


Figura 2: Tabella di corrispondenza su 4 livelli, implementata con un bitwise trie.

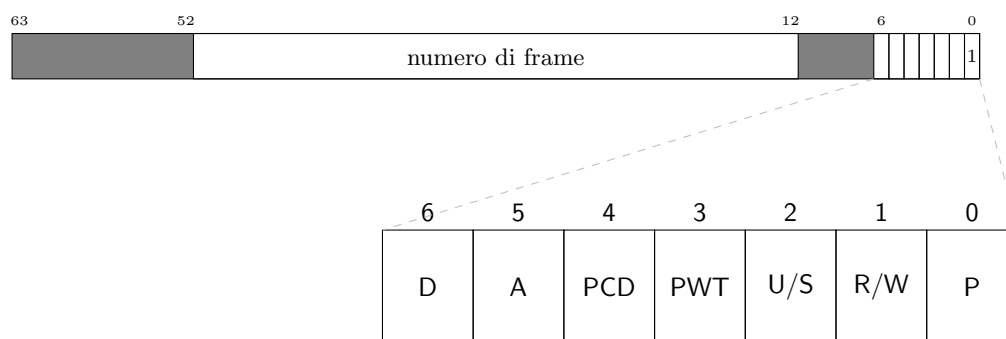


Figura 3: Descrittore di pagina virtuale (tabelle di livello 1).

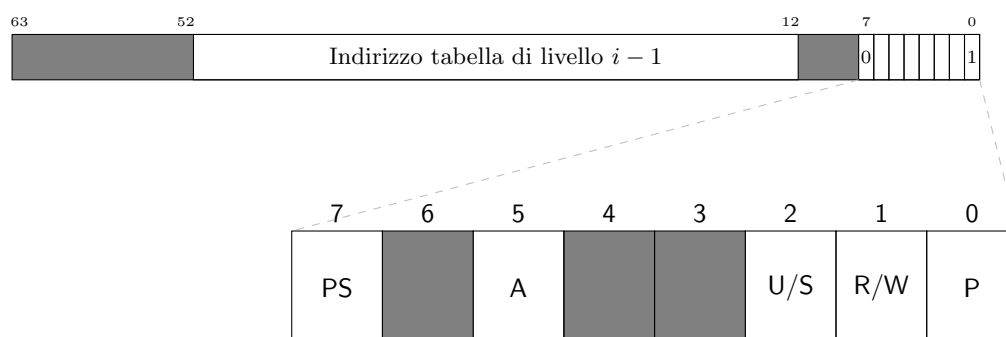


Figura 4: Descrittore di livello i , con $i = 2, 3, 4$.

contiene 512 entrate con il formato illustrato in Fig. 4. Il formato è simile a quello dei descrittori di livello 1 mostrati in Fig. 3: ci sono ancora i bit P, R/W e U/S e A, nelle stesse posizioni dei bit omonimi di Fig. 3; il campo “Indirizzo tabella di livello $i - 1$ ” occupa la stessa posizione del campo “numero di frame” di Fig. 3; per il momento non ci preoccupiamo del nuovo bit PS e assumiamo che valga 0. Si noti che all’indirizzo della tabella mancano i bit da 0 a 11: la Trie-MMU assume che questi bit siano 0, cioè che tutte le tabelle partano da indirizzi che sono multipli di 4 KiB (allineamento naturale). Questo vale anche per la tabella di livello 4: i 12 bit meno significativi di **cr3** devono essere tutti a 0.

Chiameremo le entrate delle tabelle di livello 2 *descrittori di livello 2* o *descrittori delle tabelle di livello 1*. Si noti il decremento del livello: i descrittori di livello 2 sono contenuti in tabelle di livello 2 e descrivono tabelle di livello 1. Allo stesso modo chiameremo le entrate delle tabelle di livello 3 *descrittori di livello 3* o *descrittori delle tabelle di livello 2* e, infine, chiameremo le entrate della tabella di livello 4 *descrittori di livello 4* o *descrittori delle tabelle di livello 3*.

Anche se simili, i descrittori di livello 2, 3 e 4 non vanno confusi con i descrittori di livello 1: i primi descrivono tabelle, mentre quelli di livello 1 descrivono pagine. I descrittori di tabella servono a trovare le tabelle di livello inferiore, ma solo le tabelle foglia contengono la traduzione cercata.

In Fig. 5 abbiamo riassunto il processo di traduzione operato dalla Trie-MMU, mostrando più in dettaglio le operazioni svolte da Trie-MMU, assumendo che tutti i bit P valgano 1. Al primo passo Trie-MMU deve leggere il corretto descrittore di livello 4, che si trova nella sua memoria interna. Per farlo deve eseguire una operazione di lettura in memoria, ovviamente specificando l’indirizzo. La tabella di livello 4 è un vettore di descrittori, ciascuno grande 8 byte, e la Trie-MMU vuole leggere il descrittore il cui indice è contenuto nei bit 39–47 di V (indice di livello 4), sia i_4 . L’indirizzo a cui vuole leggere è dunque **cr3** + $i_4 \times 8$. Si noti che, per quanto detto sull’allineamento naturale delle tabelle, questa operazione non comporta una vera somma, ma solo una concatenazione di bit. Anche la moltiplicazione per 8 consiste, ovviamente, nella concatenazione con tre bit costanti pari a 0. Letto il descrittore di livello 4, Trie-MMU ne estrae il campo indirizzo (bit 12–51), lo concatena con i bit 30–38 di V e con altri tre bit a 0, ottenendo così l’indirizzo del descrittore di livello 3. E così anche per i descrittori di livello 2 e 1. Arrivata al livello 1, la Trie-MMU estrae il campo F (bit 12–51), lo concatena con l’offset di V e ottiene così la traduzione.

Durante la traduzione la Trie-MMU esegue anche altri compiti, analoghi ai compiti aggiuntivi che svolgeva la Super-MMU, ma applicati alla struttura multi-livello:

- controlla tutti i bit R/W: una operazione di scrittura è permessa solo se tutti e 4 i bit lungo il percorso la permettono;
- controlla tutti i bit U/S: una operazione (di lettura o scrittura) è permessa solo se tutti e 4 i bit lungo il percorso la permettono;

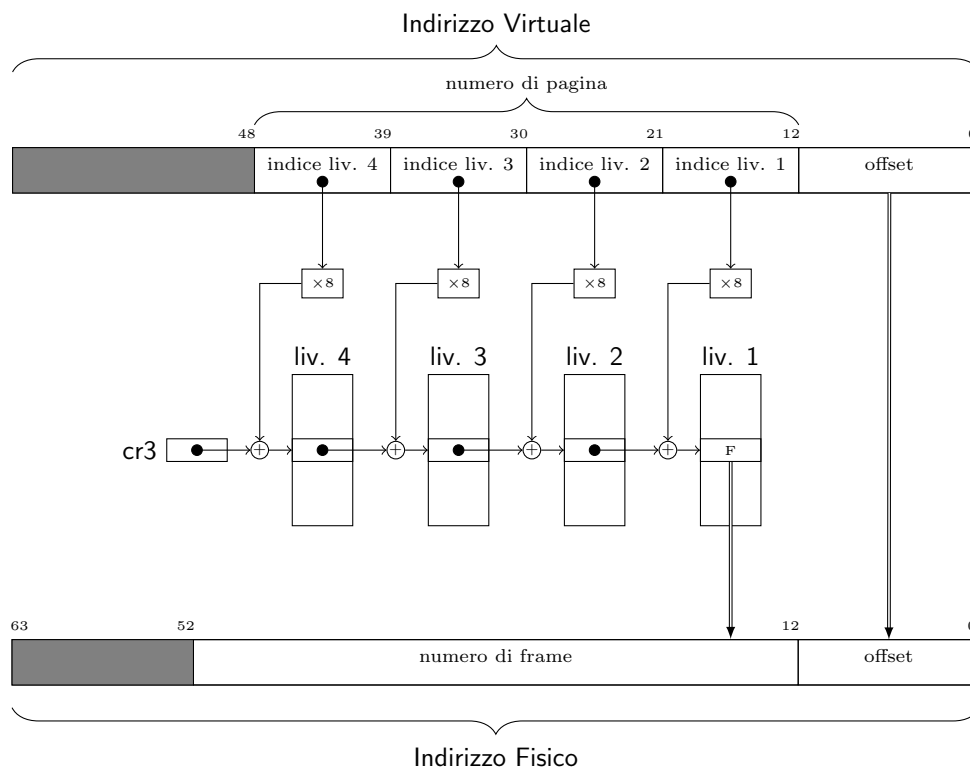


Figura 5: Traduzione da indirizzo virtuale a fisico (pagine di 4 KiB).

- passa al controllore cache le informazioni contenute nei bit PWD e PCD nel descrittore di livello 1;
- pone a 1 tutti e 4 i bit A incontrati, se non lo erano già;
- in caso di scrittura, pone a 1 il bit D nella tabella di livello 1 (i descrittori di livello maggiore di 1 non hanno il bit D).

Se uno qualunque dei bit P incontrati durante la traduzione vale 0, la Trie-MMU smette di tradurre e solleva una eccezione di page fault. La routine di sistema che gestisce il fault terminerà il processo con un errore.

Ciascuna delle tabelle di corrispondenza della Super-MMU deve essere sostituita da uno di questi trie (in altre parole, ci serve un trie per ogni processo). Nella parte bassa di Figura 2 abbiamo riportato il numero massimo di tabelle per ogni livello del trie e quanto spazio occupa ogni livello. Si noti che le tabelle di livello 1 occupano complessivamente 512 GiB, esattamente come la tabella della Super-MMU che stiamo cercando di sostituire. In effetti, se rimettessimo insieme tutte le tabelle di livello 1, in ordine, riatterremmo la tabella di corrispondenza della Super-MMU. Siccome a questo spazio dobbiamo aggiungere quello richiesto dai livelli superiori del trie, ci rendiamo conto che il trie completo occupa *più* spazio della tabella di corrispondenza originaria. Anche dal punto di vista del tempo richiesto per eseguire la traduzione ci stiamo perdendo: la tabella unica della Super-MMU richiede un unico accesso in memoria, mentre il trie ne richiede 4. Qual è dunque il vantaggio di passare al trie? Un vantaggio è che possiamo evitare di allocare le parti di trie relative a indirizzi virtuali che il processo non usa. Per esempio, se un processo non usa nessun indirizzo il cui numero di pagina inizi con $(777)_8$, il trie di questo processo non ha bisogno di tutto il sottoalbero inferiore di Figura 2. L'omissione di questo sottoalbero si ottiene semplicemente ponendo a 0 il bit P dell'entrata $(777)_8$ della tabella di livello 4. Dal momento che la maggior parte dei processi userà soltanto una piccola parte dei possibili indirizzi virtuali, ci rendiamo conto che in questo modo il trie avrà quasi sempre una dimensione ragionevole.

1.2 Regioni e sottoregioni

Un altro modo per pensare alle operazioni svolte dalla Trie-MMU è di ragionare in termini di regioni naturali (che, ricordiamo, sono intervalli di indirizzi con dimensione pari ad una potenza di 2 e allineate naturalmente). Possiamo identificare ciascuna tabella del trie specificando la sequenza di bit della chiave che porta dalla radice alla tabella in questione. Per esempio, la terza tabella di livello due dall'alto in Figura 2 è identificata dalla sequenza di 18 bit $(777000)_8$. La traduzione di tutti gli indirizzi virtuali che iniziano con questo prefisso deve passare da questa tabella. Questa tabella, dunque, è “responsabile” della traduzione dell'intera regione naturale, grande $2^{48-18} = 2^{30} = 1 \text{ GiB}$, il cui numero di regione è appunto $(777000)_8$. Aggiungendo ulteriori 9 bit possiamo identificare anche ogni singola *entrata* della tabella. Per esempio, i 27 bit $(777000777)_8$ identificano sia la terza tabella di livello 1 dal basso di Figura 2.

chiamiamola t , sia l'ultima entrata della seconda tabella di livello 2 dal basso, chiamiamola e . Di nuovo, la traduzione di tutti gli indirizzi virtuali che iniziano con $(777\,000\,777)_8$ deve passare dall'entrata e e poi da una delle entrate della tabella t . Tutti questi indirizzi virtuali appartengono alla stessa regione naturale grande $2^{48-27} = 2^{21} = 2\text{ MiB}$, il cui numero di regione è $(777\,000\,777)_8$. Possiamo dire che l'entrata e , o l'intera tabella t , sono responsabili della traduzione in questa regione.

In generale, diremo che ogni entrata di una tabella di livello i , con $1 \leq i \leq 4$, sarà responsabile della traduzione di una regione naturale di livello $i - 1$. Invece ogni tabella di livello i , nella sua interezza, sarà responsabile della traduzione di una regione naturale dello stesso livello i . Le regioni di livello 0 non sono altro che le pagine, grandi 2^{12} byte. In generale una regione di livello j , con $0 \leq j \leq 4$, è grande 2^{9j+12} byte. Quindi, ogni entrata di una tabella di livello 1 è responsabile della traduzione di una ben precisa pagina, mentre una intera tabella di livello 1, nel suo complesso, è responsabile della traduzione di una ben precisa regione di livello 1, grande $2^{9 \times 1 + 12} \text{ B} = 2\text{ MiB}$, la stessa regione di cui è responsabile l'entrata (in una tabella di livello 2) che punta alla tabella nel trie. E così via.

Paginazione: complementi

G. Lettieri

18 Aprile 2024

Eliminiamo ora tutte le semplificazioni che abbiamo introdotto e studiamo la MMU che si trova nei sistemi Intel/AMD a 64 bit.

La Trie-MMU aveva una memoria interna per memorizzare le tabelle dei vari livelli, ma per la vera MMU non funziona così. *Le tabelle devono essere memorizzate nella memoria fisica, insieme alle pagine dei processi e al codice e alle altre strutture dati del sistema.* Il registro **cr3** della MMU contiene semplicemente il numero di frame della tabella radice del TRIE corrente. La MMU si limita a realizzare in hardware il cosiddetto *table-walk*: ogni volta che riceve un indirizzo virtuale dalla CPU usa **cr3** e i campi del numero di pagina per consultare, in sequenza, le tabelle dei 4 (o 5) livelli del TRIE, fino a trovare il descrittore di pagina, eseguire la traduzione e infine completare l'accesso richiesto dalla CPU.

Questo rende la MMU realizzabile in pratica, ma ci costringe ad affrontare nuovi problemi: dove troviamo lo spazio per le tabelle in memoria fisica? Inoltre, visto che il table-walk richiede tanti accessi aggiuntivi in memoria per ogni accesso iniziato dalla CPU, come possiamo rendere efficiente questo meccanismo?

1 Memoria fisica in memoria virtuale

Preoccupiamoci per prima cosa di dove mettere le tabelle dei TRIE. Concettualmente farebbero tutte parte di M1: sono strutture dati del sistema, inaccessibili agli utenti. Tuttavia, per motivi puramente pratici, conviene allocarle in M2. Infatti, come le pagine degli utenti, anche le tabelle sono grandi 4 KiB e devono essere allineate naturalmente. Ogni tabella, dunque, occupa esattamente un frame di M2 e quindi possiamo usare una stessa lista di frame liberi sia per l'allocazione delle tabelle che delle pagine. In questo modo evitiamo anche di dover stabilire *a-priori* quanto spazio dedicare alle une o alle altre.

Questo comporta, però, che il modulo sistema deve poter accedere liberamente a tutti i frame di M2, in modo da poter creare, modificare e consultare liberamente le tabelle in qualunque frame. Il sistema deve anche, ovviamente, accedere a tutto M1, quindi dobbiamo permettergli di accedere a tutta la memoria fisica, indipendentemente da quale sia il TRIE attivo. Allo stesso tempo dobbiamo continuare a negare questo accesso ai processi utente, se non per le

parti che contengono le loro pagine. La cosa più semplice sarebbe di avere una MMU che si disattiva ogni volta che la CPU passa a livello sistema, ma la MMU che abbiamo non si comporta così. Possiamo però creare una traduzione che non abbia alcun effetto, che di fatto è equivalente a disattivare la MMU. Creiamo quindi delle traduzioni “identità” che lascino inalterati (li traducano in sé stessi) tutti gli indirizzi che vanno da 0 fino all'ultimo indirizzo della memoria fisica. Poi, inseriamo queste traduzioni nello spazio di indirizzamento di ogni processo. Questo permette alle routine di sistema di usare indirizzi fisici proprio come se la MMU fosse disattivata, indipendentemente da quale processo si trova in esecuzione. È come se nella parte alta dello spazio di indirizzamento di ogni processo avessimo creato una finestra che permette di vedere la memoria fisica così come è. Ovviamente questa finestra è accessibile solo da livello sistema, in quanto per tutte le traduzioni nella metà alta dello spazio di indirizzamento abbiamo posto U/S=sistema.

Questa finestra deve essere creata prima di attivare la memoria virtuale. All'avvio del sistema, il processore parte con la memoria virtuale disattivata ed esegue la routine di inizializzazione. Questa può allocare e inizializzare tutte le tabelle necessarie alla definizione della finestra e poi attivare la memoria virtuale. A questo punto la routine di inizializzazione può continuare ad utilizzare indirizzi fisici come stava facendo prima dell'attivazione. Quando si passa a livello utente queste traduzioni diventano inaccessibili e ridiventano accessibili ogni volta che si ritorna a livello sistema.

2 Il TLB

Per ogni accesso in memoria la MMU deve prima consultare fino a 4 tabelle per poter eseguire la traduzione¹. Per ogni tabella consultata, se il corrispondente bit A è a zero, la MMU deve anche eseguire un ulteriore accesso (in scrittura) per settarlo a 1. Se consideriamo che il nostro programma deve accedere continuamente in memoria, sia per prelevare le sue stesse istruzioni, sia per prelevare o scrivere gli operandi che si trovano in memoria, ci rendiamo subito conto che l'impatto della MMU sul tempo di esecuzione di un programma può essere molto grande. È vero che subito dopo la MMU c'è la cache, e quindi possiamo sperare che molti di questi accessi non debbano realmente arrivare fino alla memoria, ma resta il fatto che, anche nel migliore dei casi, quello che prima era un unico accesso in cache si è ora trasformato in una sequenza di 5 accessi in cache.

Per affrontare questo problema si introduce una nuova cache, chiamata TLB (Translation Lookaside Buffer), che è specifica per la MMU. Lo scopo di questa cache è di ricordare le *traduzioni* utilizzate più recentemente, dove per traduzioni intendiamo quelle contenute nei descrittori di livello 1. Non ci interessano invece i descrittori di livello più alto, il cui unico scopo è di permettere di raggiungere, dato l'indirizzo virtuale da tradurre, il descrittore di livello 1. Una volta raggiunto questo descrittore la MMU può ricordare (nel TLB) quale sia

¹Fino a 5 nei modelli con 57 bit significativi nell'indirizzo virtuale.

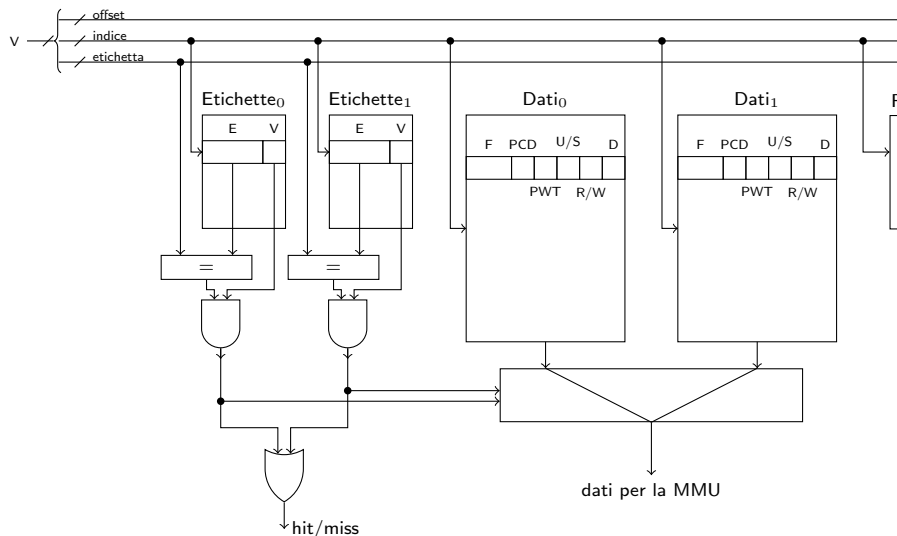


Figura 1: Un esempio di TLB a 2 vie (circuito di lettura).

la traduzione da fare per l'indirizzo virtuale in questione: non ha bisogno di ricordare tutto il percorso.

Quando la MMU deve tradurre un indirizzo controlla prima se il TLB contiene già il descrittore che sta cercando; altrimenti si comporta come abbiamo visto fin'ora, seguendo tutta la catena di tabelle dal livello 4 fino al livello 1 (table-walk); alla fine, oltre ad eseguire la traduzione e completare l'accesso in memoria per conto del processore, memorizzerà nel TLB il descrittore appena usato, possibilmente rimpiazzandone un altro. Tipicamente il TLB è una memoria associativa ad insiemi e il descrittore da rimpiazzare sarà scelto in base ad un algoritmo di pseudo-LRU. In Figura 1 è mostrato un esempio di TLB a 2 vie. I campi dati di ogni via memorizzano i campi *F* dei descrittori di livello 1. La parte *offset* dell'indirizzo V in ingresso non è utilizzata per accedere alla memoria cache, ma andrà direttamente a far parte dell'indirizzo fisico. Un TLB moderno potrebbe avere 8 vie e contenere 1024 descrittori in totale (quindi $1024/8 = 128$ indici).

Il TLB, come tutte le cache, è trasparente al software, persino al software di sistema: l'architettura non prevede istruzioni che permettano al software di esaminarne il contenuto. Le uniche istruzioni che l'architettura mette a disposizione sono quelle che permettono di *invalidarlo* in tutto o in parte. Queste operazioni si rendono necessarie quando il software modifica qualcosa nelle tabelle, rendendo dunque non più valide le informazioni che potrebbero trovarsi nel TLB. In particolare, le istruzioni disponibili per l'invalidazione sono due:

- `movq %rax, %cr3`, che già conosciamo; questa istruzione cambia potenzialmente tutta la tabella di livello 4 usata fino al momento prima,

quindi tutte le informazioni contenute nel TLB sono da considerarsi non più valide e l'intero TLB viene svuotato;

- *invlpg operando in memoria*: questa istruzione dice al TLB di invalidare la traduzione relativa all'indirizzo dell'operando passato come argomento.

Ogni volta che si cambia processo verrà eseguita una `movq %rax, %cr3` per caricare il puntatore alla tabella di livello 4 del processo entrante. Questa istruzione avrà l'effetto di svuotare il TLB, che è quello che vogliamo, in quanto le traduzioni in esso presenti facevano riferimento alla memoria virtuale del processo uscente.

Il TLB deve permettere alla MMU di svolgere tutte le sue funzioni senza consultare l'albero di traduzione. Ricordiamo che la MMU, oltre ad eseguire la traduzione, deve anche controllare i permessi sui bit U/S e R/W, aggiornare i bit A e D e passare al controllore cache i bit PWT e PCD. Questi ultimi due bit vengono memorizzati insieme al numero di frame e, in caso di hit, passati alla MMU che li girerà al controllore cache.

Consideriamo i bit U/S: la MMU ne incontra fino a quattro durante la traduzione, ma non c'è bisogno che il TLB li ricordi tutti e quattro, in quanto la regola dice che l'accesso è vietato da livello utente se anche uno solo di essi è zero. È dunque sufficiente che la MMU calcoli l'AND dei bit incontrati nel percorso e carichi soltanto questo nel TLB. Lo stesso discorso vale per i bit R/W.

Un discorso diverso va fatto per i bit A e D, in quanto questi bit sono *scritti* dalla MMU durante la traduzione, e non vogliamo certo che la MMU sia costretta a percorrere l'albero per aggiornare questi bit anche quando ha trovato la traduzione nel TLB (cosa che renderebbe il TLB del tutto inutile). Per il bit A la soluzione adottata è semplice: la MMU li aggiorna solo durante un table-walk, e dunque solo quando *non* trova la traduzione nel TLB. Quindi, fino a che una traduzione si trova nel TLB, i corrispondenti bit A nel percorso di traduzione non verranno ulteriormente modificati dalla MMU. Questo è un problema solo se il software, per qualche motivo, azzerava qualche bit A, perché in quel caso tail bit resterebbero a zero anche in presenza di ulteriori accessi. In questo caso deve essere il software stesso che si deve preoccupare di invalidare, in tutto o in parte, il TLB dopo aver azzerato i bit A, costringendo dunque la MMU a ripercorrere l'albero e ri-aggiornare i bit A al prossimo accesso.

Per il bit D il discorso è più complesso, perché può succedere che vi sia un primo accesso in sola lettura all'interno di una pagina, seguito da un accesso in scrittura. Nel primo accesso la MMU non porta, ovviamente, D a 1, e la traduzione viene caricata nel TLB. Al secondo accesso la MMU trova la traduzione nel TLB e, se non prendessimo provvedimenti, non accedrebbe all'albero e lascerebbe il bit D sempre a zero, pur essendovi stata una scrittura all'interno della pagina. L'informazione offerta dal bit D sarebbe quindi inaffidabile. In questo caso è complicato trovare un rimedio puramente software, in quanto il software non sa cosa contenga il TLB. La soluzione adottata è hardware: il TLB ricorda anche il valore del bit D al momento del caricamento della traduzione e causa una *miss* per gli accessi in scrittura con D uguale a 0. Nell'esempio precedente accade dunque questo: nel primo accesso (sola lettura) il TLB carica la

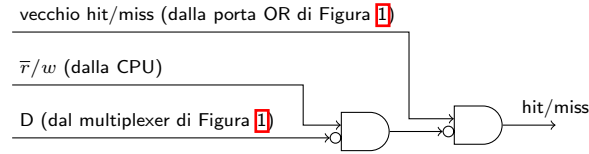


Figura 2: Soluzione hardware al problema dell'inconsistenza del bit D.

traduzione e il valore corrente di D, che è 0. Nel successivo accesso in scrittura il TLB si accorge che, anche se la traduzione è presente, il bit D è ancora a zero. Genera dunque un miss, forzando la MMU a percorrere l'albero, aggiornando di conseguenza il bit D. Si noti che a quel punto la MMU ricaricherà la traduzione nel TLB, questa volta con il bit D a 1: i successivi accessi in scrittura non causeranno dunque ulteriori miss. Questa soluzione può essere realizzata aggiungendo il circuito di Figura 2 a valle del circuito di Figura 1²

3 Pagine di grandi dimensioni

Avere TRIE con 4 (o 5) livelli richiede molto spazio in memoria fisica e allunga i tempi del table-walk. Entrambi i costi possono essere ridotti usando pagine più grandi di 4 KiB. Si tratta però di un compromesso, in quanto usare pagine più grandi può aumentare lo spreco di memoria all'interno delle pagine stesse (frammentazione interna).

L'architettura Intel/AMD a 64 bit ci permette di avere pagine più grandi di 4 KiB usando il bit PS nei descrittori di livello 3 e 2. Anche questo bit è scritto dalle routine di sistema e soltanto letto dalla MMU. La Fig. 3 mostra il formato del descrittore di livello 2 nel caso in cui il bit PS valga 1. Si vede che il descrittore contiene ora un campo F che va dai bit 21 a 51, invece del puntatore alla tabella di livello 1. La Fig. 4 mostra la traduzione da indirizzo virtuale a fisico eseguita in questo caso: quando la MMU arriva alla tabella di livello 2 e trova il bit PS a 1, usa come offset tutti i bit da 0 a 20 e li concatena al campo F trovato nel descrittore. In questo modo abbiamo eliminato una tabella di livello 1, risparmiando il relativo spazio. Il meccanismo è compatibile e può convivere con le pagine di 4 KiB: la MMU si comporta come sempre nei

²I processori 80386 e 80486 contenevano in realtà un paio di registri, TR6 e TR7, che servivano a testare il TLB e, di fatto, potevano essere usati per leggerne il contenuto. Per esempio, scrivendo un indirizzo di pagina in TR6 e settando il bit 0 si poteva poi leggere in TR7 la sua traduzione presa dal TLB (se presente) e nella parte bassa di TR6 tutti i bit associati, incluso D. Questi registri furono spostati tra i registri MSR (Model Specific Register) nel Pentium e poi rimossi nei processori successivi. La presenza di questi registri, in ogni caso, non basta da sola ad evitare la soluzione hardware di Figura 2, perché se una traduzione venisse rimpiazzata nel TLB prima che il software l'abbia letta tramite TR6 e TR7, l'informazione sul bit D sarebbe comunque persa. Per risolvere completamente il problema il TLB dovrebbe anche comportarsi come una cache write-back, ricopiando nella tabella opportuna il contenuto del bit D prima di rimpiazzare una entrata, ma questo richiederebbe di realizzare il table-walk anche nel TLB, perché la tabella andrebbe ritrovata a partire dall'unica informazione di cui il TLB dispone, che è il numero di pagina.

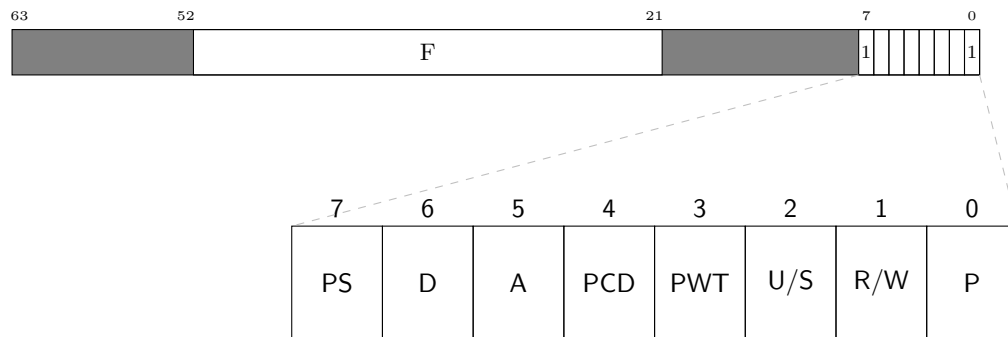


Figura 3: Descrittore di pagina virtuale da 2 MiB (tabelle di livello 2).

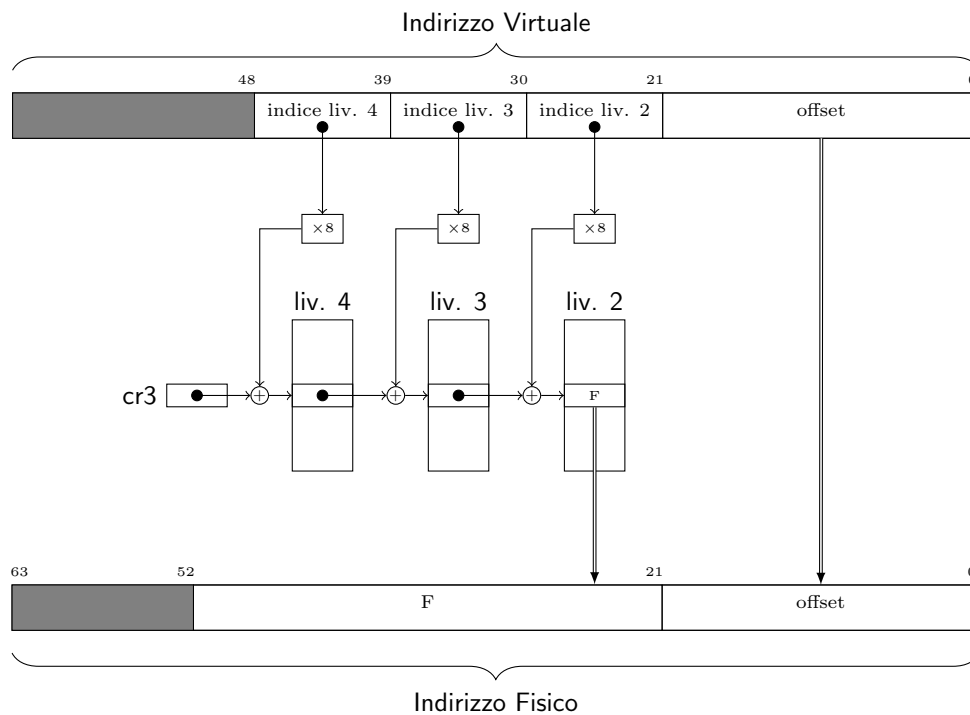


Figura 4: Traduzione da indirizzo virtuale a fisico (pagine di 2 MiB).

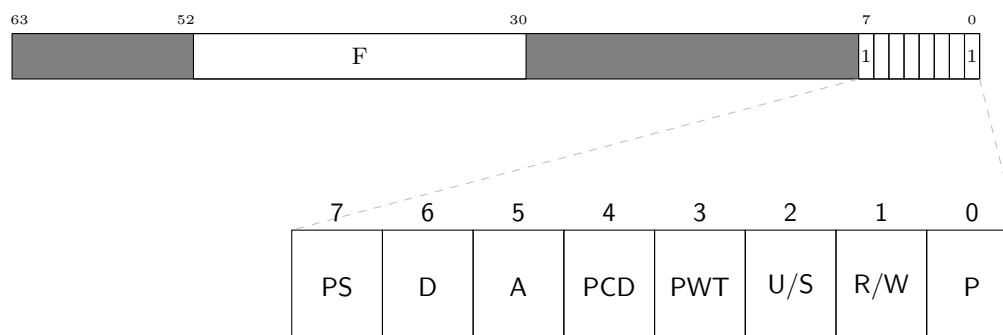


Figura 5: Descrittore di pagina virtuale da 1 GiB (tabelle di livello 3).

livelli 4 e 3 e scopre di dover eseguire il nuovo tipo di traduzione solo quando arriva al livello 2 e trova il bit PS pari a 1; se lo avesse trovato a 0 avrebbe proseguito fino al livello 1, come sempre. Ogni descrittore di livello 2 può avere il bit PS a 1 o a 0 indipendentemente dagli altri, quindi nello stesso spazio di indirizzamento possiamo usare sia pagine di 2 MiB, sia pagine di 4 KiB, a seconda della convenienza.

I processori più recenti permettono di avere PS=1 anche nei descrittori di livello 3. La Fig. 5 mostra il formato del descrittore di livello 3 in questo caso, in cui si vede che il campo F sostituisce anche qui il campo che punta alla tabella di livello 2. La Fig. 6 mostra la traduzione eseguita dalla MMU in questo caso. Si vede come l'offset è ora su 30 bit, e dunque le pagine sono ora grandi 1 GiB. Questo tipo di traduzione è molto utile per creare la finestra sulla memoria fisica (si veda la sezione 1).

3.1 Interazione con il TLB

Il TLB per le pagine di 4 KiB non è in grado di memorizzare direttamente la traduzione di una pagina più grande, in quanto il numero di bit di offset è diverso. Se vogliamo continuare a usare un unico TLB, la traduzione di una pagina di 2 MiB (per esempio) deve essere scomposta nelle equivalenti 512 traduzioni di pagine da 4 KiB, occupando dunque 512 posizioni del TLB. Anche se alcuni processori più vecchi adottavano questa soluzione, i processori moderni hanno invece un diverso TLB per ogni dimensione di pagina supportata. Si noti che la MMU deve cercare la traduzione in *ciascuno* di questi TLB, perché ogni indirizzo potrebbe essere tradotto usando pagine di qualunque dimensione, e non è possibile saperlo in anticipo. Per fortuna la ricerca può essere svolta in parallelo, e dunque non influisce negativamente sulle prestazioni. La presenza di questi ulteriori TLB è anzi un motivo in più per usare pagine di grandi dimensioni, quando possibile, in modo da sfruttare questi TLB aggiuntivi e alleggerire la pressione sul TLB principale.

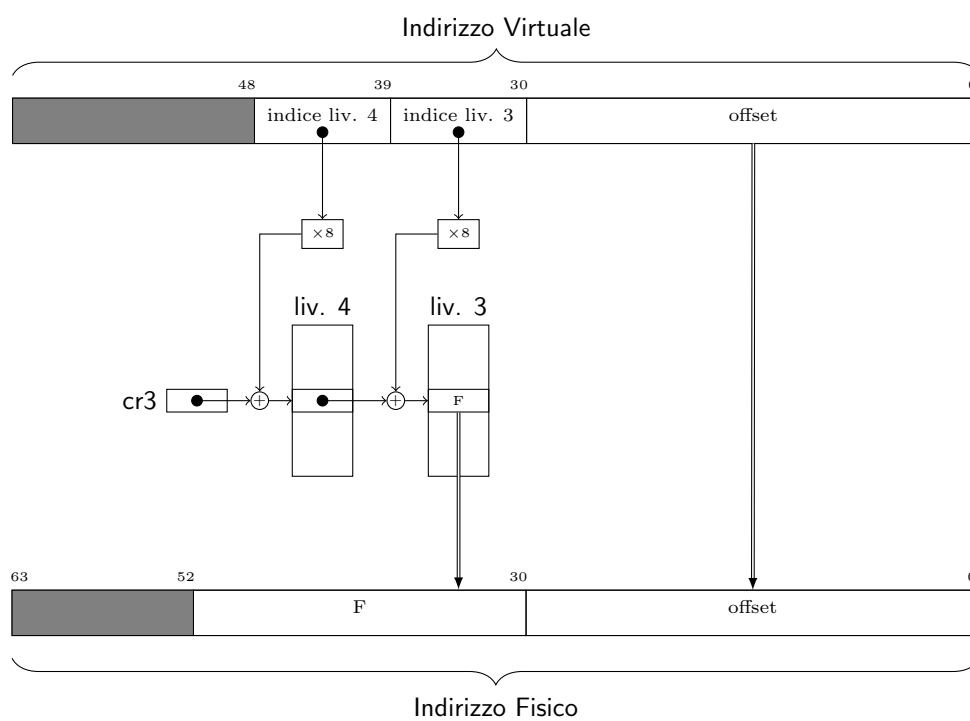


Figura 6: Traduzione da indirizzo virtuale a fisico (pagine di 1 GiB).

Funzioni di supporto per la paginazione

G. Lettieri

11 Aprile 2024

La libreria `libce` definisce i tipi numerici `paddr` `vaddr`, che rappresentano rispettivamente indirizzi fisici e virtuali. I due tipi hanno solo lo scopo di ricordare al programmatore quando si suppone che un indirizzo debba essere virtuale o fisico, ma sono entrambi equivalenti ad un intero senza segno su 64 bit.

La libreria contiene anche alcune strutture dati e funzioni di uso generale, a cui si può accedere includendo il file `vm.h`. Nel seguito descriviamo il contenuto di questo file.

1 Calcoli con gli indirizzi

Queste funzioni eseguono semplici calcoli sugli indirizzi virtuali.

1.1 La funzione `norm()`

La funzione `vaddr norm(vaddr a)` serve a *normalizzare* un indirizzo virtuale, cioè a rendere i 16 bit più significativi tutti uguali al bit numero 47. Può essere anche usata per controllare se un dato indirizzo (per esempio ricevuto da una sorgente non fidata, come il livello utente) è normalizzato o meno:

```
if (norm(v) == v) {  
    // v è normalizzato  
} else {  
    // v non è normalizzato: errore  
}
```

1.2 La funzione `dim_region()`

La funzione `natq dim_region(int liv)` restituisce la dimensione in byte di una regione di livello `liv`. Si ricordi che abbiamo chiamato “regione di livello *i*” l’intervallo di indirizzi coperti da una singola entrata di un tabella di livello *i* + 1. Quindi, per esempio, una regione di livello 0 è grande 4096 byte (pagina di livello 1), mentre una regione di livello 1 è grande 2 MiB (pagina di livello 2). La funzione può essere anche utilizzata per ottenere le maschere che permettono

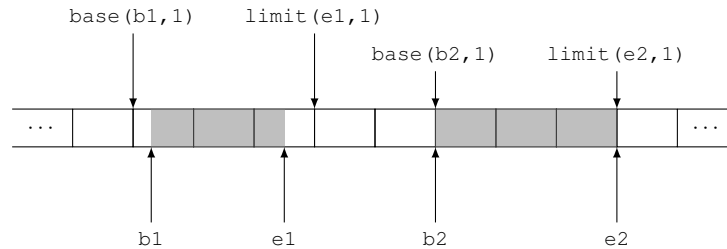


Figura 1: Esempio di calcolo di `base()` e `limit()` per due intervalli di indirizzi, `[b1, e1)` e `[b2, e2)`.

di estrarre da un indirizzo virtuale il numero di pagina e l'offset. Per esempio, se `v` è un indirizzo virtuale e vogliamo sapere in che pagina di livello 2 cade, e a quale offset:

```
natq mask = dim_region(1) - 1;
natq base = v & ~mask; // indirizzo base della pagina
natq offset = v & mask; // offset nella pagina
```

Questo perché `dim_region(liv)` ha la forma di 2^n , che in binario è un 1 seguito da n zeri, e dunque `dim_region(liv) - 1` produce una maschera di n bit ad 1.

1.3 Le funzioni `base()` e `limit()`

La base della regione di livello `liv` in cui cade un indirizzo `v` si può ottenere anche dalla funzione `vaddr base(vaddr v, int liv)`. Invece, la funzione `vaddr limit(vaddr e, int liv)` serve a calcolare la base della prima regione di livello `liv` che si trova a destra di un intervallo `[b, e)` senza toccarlo (si veda la Figura 1).

2 Manipolare singole entrate

Queste definizioni e funzioni aiutano a leggere/modificare singole entrate delle tabelle.

2.1 Il tipo `tab_entry`

Il tipo `tab_entry` rappresenta una entrata di una tabella, di qualunque livello. Il file contiene la definizione di un po' di costanti, una per ogni bit del byte di accesso delle entrate, che possono essere usati come maschere per estrarre, settare o resettare i vari bit. Per esempio, se `e` è un riferimento ad una entrata di una tabella, possiamo

- esaminare il bit `P` con `"if (e & BIT_P) { /*qualcosa */ }"`;

- settare il bit U/S con “`e |= BIT_US`”;
- resettare il bit R/W con “`e &= ~BIT_RW`”

e così via.

2.2 Le funzioni **extr/set_IND_FISICO()**

La funzione `paddr extr_IND_FISICO(tab_entry e)` può essere usata per estrarre l'indirizzo fisico contenuto nell'entrata `e`. Tale indirizzo rappresenta l'indirizzo (fisico) della tabella di livello inferiore o, nel caso di tabelle foglia, la base del frame in cui è mappata una pagina virtuale.

La funzione `set_IND_FISICO(tab_entry& e, paddr p)` serve invece a settare il campo “numero di frame” dell'entrata `e` con il numero di frame dell'indirizzo fisico `p`, senza modificare il byte di accesso.

3 Lavorare su singole tabelle

Queste funzioni aiutano a leggere/modificare una tabella del TRIE alla volta.

3.1 Le funzioni **i_tab()** e **get/set_entry()**

La funzione `int i_tab(vaddr v, int liv)` estrae dall'indirizzo virtuale `v` l'indice che la MMU usa per consultare le tabelle di livello `liv`. Per esempio, se `liv` è 2, la funzione restituisce l'indice contenuto nei bit 29–21 di `v` (si veda avanti, sezione [5.3.1](#)).

La funzione `tab_entry& get_entry(paddr t, int i)`, dato l'indirizzo fisico `t` di una tabella e un indice `i`, restituisce un riferimento all'entrata `i`-esima della tabella stessa (il riferimento potrà essere effettivamente usato solo se è accessibile la finestra sulla memoria fisica).

Per esempio, se vogliamo settare il bit R/W nell'entrata relativa ad un indirizzo virtuale `v` in una tabella di livello 2 di indirizzo fisico `tab`, possiamo scrivere

```
tab_entry& e = get_entry(tab, i_tab(v, 2));
e |= BIT_RW;
```

Come ulteriore esempio, supponiamo che `tab4`, `tab3`, `tab2` e `tab1` siano gli indirizzi fisici di quattro tabelle inizialmente vuote e che vogliamo costruire il percorso di traduzione che mappi l'indirizzo virtuale `v` nell'indirizzo fisico `p`, in modo che sia consentito l'accesso in scrittura ma non quello da livello utente. Possiamo scrivere:

```
// aggancio tab4->tab3
tab_entry& e4 = get_entry(tab4, i_tab(v, 4));
set_IND_FISICO(e4, tab3);
e4 |= BIT_P | BIT_RW;
```

```

// aggancio tab3->tab2
tab_entry& e3 = get_entry(tab3, i_tab(v, 3));
set_IND_FISICO(e3, tab2);
e3 |= BIT_P | BIT_RW;
// aggancio tab2->tab1
tab_entry& e2 = get_entry(tab2, i_tab(v, 2));
set_IND_FISICO(e2, tab1);
e2 |= BIT_P | BIT_RW;
// installo la traduzione v -> p
tab_entry& e1 = get_entry(tab1, i_tab(v, 1));
set_IND_FISICO(e1, p);
e1 |= BIT_P | BIT_RW;

```

Esiste anche la funzione

```
void set_entry(paddr tab, natl j, tab_entry se)
```

che può essere usata per sovrascrivere completamente l'entrata j della tabella tab con il nuovo valore se . Rispetto a modificare una entrata tramite il riferimento ottenuto da `get_entry(tab, j)`, ha il vantaggio di gestire automaticamente l'eventuale contatore di entrate valide della tabella tab (si veda più avanti, sezione [5.3.1](#)).

3.2 Le funzioni `copy_des()` e `set_des()`

Queste sono due funzioni di utilità che permettono di copiare una o più entrate da una tabella ad un'altra (`copy_des()`) o di sovrascrivere una o più entrate di una tabella con uno stesso valore (`set_des()`). Rispetto a scrivere un semplice ciclo, hanno il vantaggio di gestire automaticamente l'eventuale contatore di entrate valide nella tabella di destinazione (si veda più avanti, sezione [5.3.1](#)).

4 Accesso all'hardware

Queste funzioni permettono di accedere ai registri della MMU o di interagire con il TLB.

4.1 Le funzioni `readCR3()`/`loadCR3()` e `readCR2()`

Queste funzioni sono scritte in assembler e permettono di leggere i registri **cr3** e **cr2**, e scrivere nel registro **cr3**. In particolare, `loadCR3()` va usata per attivare un nuovo albero di traduzione, passandole l'indirizzo fisico della tabella radice. Si ricordi che ha l'effetto collaterale di invalidare tutto il TLB.

Il registro **cr2** contiene l'ultimo indirizzo virtuale che ha causato una eccezione di page fault e non è scrivibile da software.

4.2 Le funzioni per invalidare il TLB

La funzione `invalida_entrata_TLB(vaddr v)` serve a invalidare la traduzione associata all'indirizzo virtuale `v`, nel caso il TLB ne stesse conservando una copia. È scritta in assembler e usa l'istruzione `invlpg`. La funzione va usata ogni qual volta si cambia qualcosa nel percorso di traduzione di `v`. Non solo, dunque, se si cambia la traduzione, ma anche se si cambia qualche bit di uno qualunque dei byte di accesso che si incontrano nel percorso di traduzione di `v`, perché il TLB memorizza anche quelli, o comunque assume che siano sempre nello stato in cui li aveva visti la MMU al momento del caricamento della traduzione.

La funzione `invalida_TLB()` serve ad invalidare tutto il contenuto del TLB. È equivalente a `loadCR3(read(CR3))`. Ha senso chiamarla se sono stati fatti molti cambiamenti (per esempio, azzeramento di tutti i bit A dopo averli esaminati) e dunque diventa conveniente rispetto a chiamare tante volte `invalida_entrata_TLB()`.

5 Lavorare con interi TRIE

Le funzioni più utili messe a disposizione dalla libreria permettono di lavorare con interi TRIE, usando internamente le funzioni definite qui sopra.

5.1 L'iteratore `tab_iter`

Si tratta di un iteratore che permette di visitare tutte le entrate dell'albero di traduzione coinvolte, a tutti i livelli, nella traduzione di tutti gli indirizzi di un dato intervallo. La visita è del tipo *depth first* e può essere eseguita sia in ordine anticipato che posticipato. Tutte le entrate sono visitate una sola volta e le entrate foglia (che contengono le traduzioni) sono visitate rispettando l'ordine crescente degli indirizzi virtuali.

L'iteratore va costruito specificando l'indirizzo fisico della tabella radice dell'albero, la base dell'intervallo e la sua lunghezza (1 per default). Ad ogni istante, a meno che la visita non sia terminata, l'iteratore si trova su una qualche entrata di una qualche tabella dell'albero. Appena costruito si troverà sull'entrata della tabella radice relativa all'indirizzo base dell'intervallo da visitare. L'iteratore può essere spostato sulla prossima entrata (secondo l'ordine *depth first*) con il metodo `next()`. L'operatore di conversione a **bool** restituisce **false** quando la visita è terminata. Le funzioni membro `get_e()`, `get_tab()` e `get_l()` e permettono di ottenere, rispettivamente, un riferimento all'entrata su cui si trova l'iteratore, l'indirizzo fisico della tabella che contiene questa entrata e il livello (4, 3, 2 o 1) di questa tabella. La funzione `get_v()`, invece, restituisce il più piccolo indirizzo virtuale la cui traduzione passa da questa entrata.

Consideriamo prima il caso particolare in cui l'intervallo consiste di un unico indirizzo e rifacciamoci all'esempio alla fine della Sezione [3.1](#). Possiamo stampare tutto il percorso di traduzione di `v` nel seguente modo:

```

for (tab_iter it(tab4, v); it; it.next()) {
    printf("tab %x, liv %d, entry %x\n",
        it.get_tab(),
        it.get_l(),
        it.get_e());
}

```

Il ciclo **for** costruisce un iteratore per l'albero di radice `tab4` e per l'intervallo di indirizzi virtuali $[v, v + 1)$ (non avendo passato la lunghezza dell'intervallo come terzo argomento viene assunto il default di 1). Nella prima iterazione `it` si trova sull'entrata di `tab4` che abbiamo chiamato `e4` nella sezione 3.1. La `printf()` stamperà dunque l'indirizzo `tab4`, il livello 4 e il contenuto di `e4`, vale a dire l'indirizzo `tab3` e il byte di accesso. Nella seconda iterazione `it` si sposterà su `e3` e la `printf()` stamperà l'indirizzo `tab3`, il livello 3 e il contenuto di `e3`, cioè l'indirizzo `tab2` e il byte di accesso. Lo stesso accadrà per il livello 2 e il livello 1. A quel punto la visita è terminata e l'espressione "`it`" (che invoca l'operatore di conversione a **bool**) restituirà **false**, terminando il ciclo.

Supponiamo di modificare il codice qui sopra cambiando il ciclo `for` in

```

for (tab_iter it(tab4, v, 2*DIM_PAGINA); it; it.next()) {

```

Ora vogliamo visitare le traduzioni delle due pagine `v` e `v+DIM_PAGINA` (la pagina successiva a `v`, supponendo che `v` non sia l'ultima pagina dello spazio di indirizzamento e non sia adiacente al "buco" nello spazio). Supponendo che l'albero sia sempre quello creato in Sezione 3.1, l'iteratore seguirebbe lo stesso percorso di prima e, dopo aver visitato `e1`, si sposterebbe sull'entrata di `tab1` successiva, oppure, se `e1` fosse l'ultima entrata di `tab1` (quella di indice 511), sull'entrata di `tab2` successiva a `e2` (o ancora più su, se anche `e2` fosse l'ultima entrata di `tab2`). La `printf()` mostrerebbe la tabella e il livello su cui l'iteratore si è fermato e il contenuto dell'entrata corrente, che nel nostro caso sarebbe tutto nullo. Al prossimo passo la visita sarebbe terminata, perché la pagina `v+DIM_PAGINA` non ha una traduzione e non ci sono altre pagine nell'intervallo. Se invece anche `v+DIM_PAGINA` avesse una traduzione, l'iteratore scenderebbe lungo il suo percorso, fermandosi ad ogni livello fino al livello 1, e solo allora terminerebbe la visita.

La visita in ordine anticipato è utile quando vogliamo *costruire* l'albero di traduzione per un certo intervallo di indirizzi. Ogni volta che l'iteratore si ferma possiamo esaminare il bit `P` dell'entrata corrente e, se vale 0 e non siamo ancora arrivati al livello 1, allocare e agganciare una nuova tabella di livello inferiore. Per far questo sfruttiamo il fatto che `get_e()` ci restituisce un *riferimento* all'entrata su cui si trova l'iteratore, permettendoci dunque di modificarla. Al prossimo passo l'iteratore si sposterà sulle entrate rilevanti di questa nuova tabella e noi potremo continuare ad allocare ed agganciare le tabelle di livello inferiore oppure, arrivati al livello 1, installare le traduzioni. Qui possiamo usare la funzione membro `get_v()` per chiedere all'iteratore qual è l'indirizzo

virtuale il cui percorso di traduzione porta all'entrata su cui ci troviamo, in modo da poter scegliere correttamente la traduzione da associarvi. Questo è il meccanismo usato dalla funzione `map()` del modulo `sistema`.

L'iteratore può essere usato anche per eseguire una visita in ordine posticipato, nel seguente modo

```
tab_iter it(tab4, v);
for (it.post(); it; it.next_post()) {
    printf("tab %x, liv %d, entry %x\n",
        it.get_tab(),
        it.get_l(),
        it.get_e());
}
```

In questo caso il codice mostrerà le entrate del percorso partendo dal livello 1 e salendo fino al 4. La visita in ordine posticipato è utile quando vogliamo *distruggere* un albero di traduzione, in quanto ci permette di eliminare i livelli inferiori dell'albero prima di esaminare i livelli superiori. Questo è il meccanismo usato dalla funzione `unmap()` del modulo `sistema`.

Quando vogliamo esaminare il percorso di traduzione di un singolo indirizzo conviene usare il seguente codice

```
tab_iter it(tab4, v);
while (it.down())
    ;
```

La funzione membro `down()` prova soltanto a scendere nell'albero, seguendo il percorso di traduzione di `v`, fermandosi alla prima foglia (sia perché ha trovato un bit `P` a zero, sia perché è arrivata alla traduzione). All'uscita dal **while** l'iteratore si trova ancora sull'entrata foglia, che possiamo così esaminare e/o modificare. Questo non è sempre vero con gli altri tipi di visita.

5.2 La funzione `trasforma()`

La funzione `paddr trasforma(paddr root, vaddr v)` permette di tradurre l'indirizzo virtuale `v` nel corrispondente indirizzo fisico in base al TRIE con radice `root`. Internamente la funzione usa un `tab_iter` per visitare il TRIE in software nello stesso modo in cui lo farebbe la MMU in hardware. Se `v` non ha traduzione, restituisce 0.

Per tradurre usando il TRIE corrente è sufficiente usare `readCR3()`:

```
paddr p = trasforma(readCR3(), v);
```

5.3 Le funzioni `map` e `unmap`

In molti casi il modo più semplice di manipolare i TRIE per creare o eliminare traduzioni è di usare queste funzioni.

La funzione `map()` riceve l'indirizzo fisico `tab` della tabella radice di un trie, gli estremi di un intervallo `[begin, end)` di indirizzi virtuali (allineati alla pagina) e un parametro template `getpaddr` che si deve comportare come una funzione da `vaddr` a `paddr`. La funzione `map()` creerà, nell'albero di radice `tab`, le traduzioni $v \mapsto \text{getpaddr}(v)$ per tutti gli indirizzi di pagina v nell'intervallo `[begin, end)`. La funzione riceve anche un parametro `flags` con cui si può specificare il valore desiderato per i flag U/S, R/W, PWT e PCD. Per esempio, per creare delle traduzioni identità nell'intervallo `[1000, 80 0000)` (esadecimale) in modo che siano accessibili in scrittura da livello sistema, si può scrivere:

```
paddr identity_map(vaddr v)
{
    return v;
}

void some_function()
{
    ...
    map(tab, 0x1000, 0x800000, BIT_RW, &identity_map);
    ...
}
```

La funzione creerà il mapping

$$1000 \mapsto \text{identity_map}(1000) = 1000,$$

poi il mapping

$$2000 \mapsto \text{identity_map}(2000) = 2000$$

e così via fino a

$$7f\,f000 \mapsto \text{identity_map}(7f\,f000) = 7f\,f000.$$

Se invece vogliamo mappare gli stessi indirizzi su dei nuovi frame di M2, basta sostituire la funzione passata come `getpaddr()`:

```
paddr my_alloc_frame(vaddr v)
{
    return alloca_frame();
}

void some_function()
{
    ...
    map(tab, 0x1000, 0x800000, BIT_RW, &my_alloc_frame);
    ...
}
```


In questo caso `map()` allocherà un nuovo frame per ogni indirizzo di pagina nell'intervallo, quindi mapperà la pagina su quel frame.

Al posto dei puntatori a funzione si possono usare espressioni *lambda*, che sono spesso più comode. Un'altra possibilità è di usare oggetti istanza di classi/strutture che ridefiniscono `operator()`. Questo è utile quando, per creare correttamente le traduzioni, non ci basta conoscere l'indirizzo virtuale e abbiamo bisogno di portarci dietro altre informazioni. Un esempio, nel modulo `sistema`, è dato dalla funzione `carica_modulo()`, che deve creare un mapping per ogni segmento di un file ELF. Qui vediamo un esempio più semplice: supponiamo di dover creare un mapping tra lo stesso intervallo di prima e degli indirizzi fisici arbitrari, scritti in un array `paddr a[]`. Possiamo operare così:

```
class my_addrs {
    paddr *pa;
    int i;
public:
    my_addrs(paddr *pa_) : pa(pa_), i(0) {}

    paddr operator()(vaddr v) {
        return pa[i++];
    }
};

void some_function()
{
    paddr a[] = { ... };
    my_addrs m(a);

    map(tab, 0x1000, 0x800000, BIT_RW, m);
}
```

Opzionalmente, `map()` può creare le traduzioni usando pagine di livello maggiore di 1. È sufficiente passare il livello desiderato come ulteriore parametro. Per esempio, la funzione `crea_finestra_FM()` usa pagine di livello 2 e passa questo valore come ultimo argomento di `map()`.

La funzione `unmap()` esegue l'operazione inversa di `map()`: distrugge tutti le traduzioni in un dato intervallo di indirizzi virtuali. La funzione si preoccupa di deallocare (tramite `rilascia_tab()`) anche tutte le tabelle che diventano vuote dopo aver eliminato le traduzioni dell'intervallo. La funzione riceve un parametro template `putpaddr` che l'utente può usare per decidere cosa fare di ogni indirizzo fisico che prima era mappato da qualche indirizzo virtuale nell'intervallo. Per esempio, per distruggere il mapping creato tramite `identity_map()` non è necessario fare niente e `putpaddr` può essere una NOP:

```
void do_nothing(vaddr v, paddr p, int lvl)
{
```

```

}

void some_function()
{
    ...
    unmap(tab, 0x1000, 0x800000, &do_nothing);
    ...
}

```

Invece, per disfare i mapping creati tramite `my_alloc_frame()` è necessario chiamare `rilascia_frame()` su tutti i frame non più mappati:

```

void my_rel_frame(vaddr v, paddr p, int lvl)
{
    rilascia_frame(p);
}

void some_function()
{
    ...
    unmap(tab, 0x1000, 0x800000, &my_rel_frame);
    ...
}

```

Si noti che la funzione `putpaddr` riceve anche un parametro `vaddr v` e un parametro `int lvl`, che contengono l'indirizzo virtuale e il livello della pagina che era precedentemente mappata sull'indirizzo fisico `p`, nel caso queste informazioni siano necessarie per capire cosa fare di `p`.

5.3.1 Funzioni usate da `map` e `unmap`

Le funzioni `map` e `unmap` usano alcune funzioni per allocare e deallocare le tabelle. La `libce` fornisce una versione semplificata di queste funzioni, in cui le tabelle vengono allocate sullo heap e non vengono mai deallocate. Il modulo `sistema`, invece, fornisce una versione più sofisticata che mantiene per ogni tabella un contatore delle entrate valide (cioè le entrate con il bit di presenza `P` settato) e permette di deallocare le tabelle quando questo contatore vale zero.

Implementazione della memoria virtuale

G. Lettieri

29 Aprile 2024

In questa ultima parte studiamo l'implementazione della paginazione nel nucleo didattico.

1 La memoria virtuale nel nucleo

Realizzeremo un caso ibrido, in cui i processi hanno sia zone di memoria condivise tra tutti, sia zone di memoria private per ciascuno. Tutti i processi utente avranno caricato nella propria memoria virtuale un unico programma: quello contenuto nel modulo utente. Ogni processo, però, partirà eseguendo una funzione del programma che può anche essere diversa per ciascuno.

La memoria virtuale di ogni processo sarà organizzata come in Figura 1. La Figura mostra lo spazio di indirizzamento virtuale di due processi, P_1 e P_2 , insieme al possibile contenuto della memoria fisica. Le annotazioni che si trovano ai lati sinistro di P_1 valgono anche per P_2 e per qualunque altro processo, e lo stesso per le annotazioni che si trovano a destra di P_2 . La memoria virtuale di ogni processo è suddivisa *a priori* nello stesso modo, come mostrato sulla destra della memoria virtuale di P_2 : la parte che va dall'indirizzo 0000 0000 0000 0000 all'indirizzo 0000 7fff ffff ffff ($2^{47} - 1$) è dedicata al sistema (bit U/S pari a 0 in tutte le traduzioni), mentre la parte che va da ffff 8000 0000 0000 a ffff ffff ffff ffff è dedicata all'utente (bit U/S pari a 1 in tutte le traduzioni). Gli indirizzi virtuali della prima pagina (da 0 a fff) sono lasciati non mappati, per intercettare dereferenziazioni di `nullptr` indipendentemente dal livello di privilegio del processore.

Ogni parte (utente o sistema) dello spazio di indirizzamento di ogni processo è suddivisa in ulteriori sezioni. Sulla sinistra della memoria virtuale di P_1 abbiamo mostrato alcune costanti (definite in `sistema.cpp`) che contengono gli indirizzi di inizio e fine delle varie sezioni. Per “indirizzo di fine” intendiamo il primo indirizzo che non fa parte della sezione: gli indirizzi di una sezione vanno da quello di inizio, incluso, a quello di fine, escluso. Il nome delle costanti è composto da tre parti: 1) la stringa “ini” per l'indirizzo di inizio o “fin” per l'indirizzo di fine; 2) la stringa “sis” per le sezioni *sistema*, “mio” per la sezione *modulo I/O* e la stringa “utn” per le sezioni *utente*; 3) il carattere “c” per le sezioni *condivise* o “p” per quelle private. Quindi, per

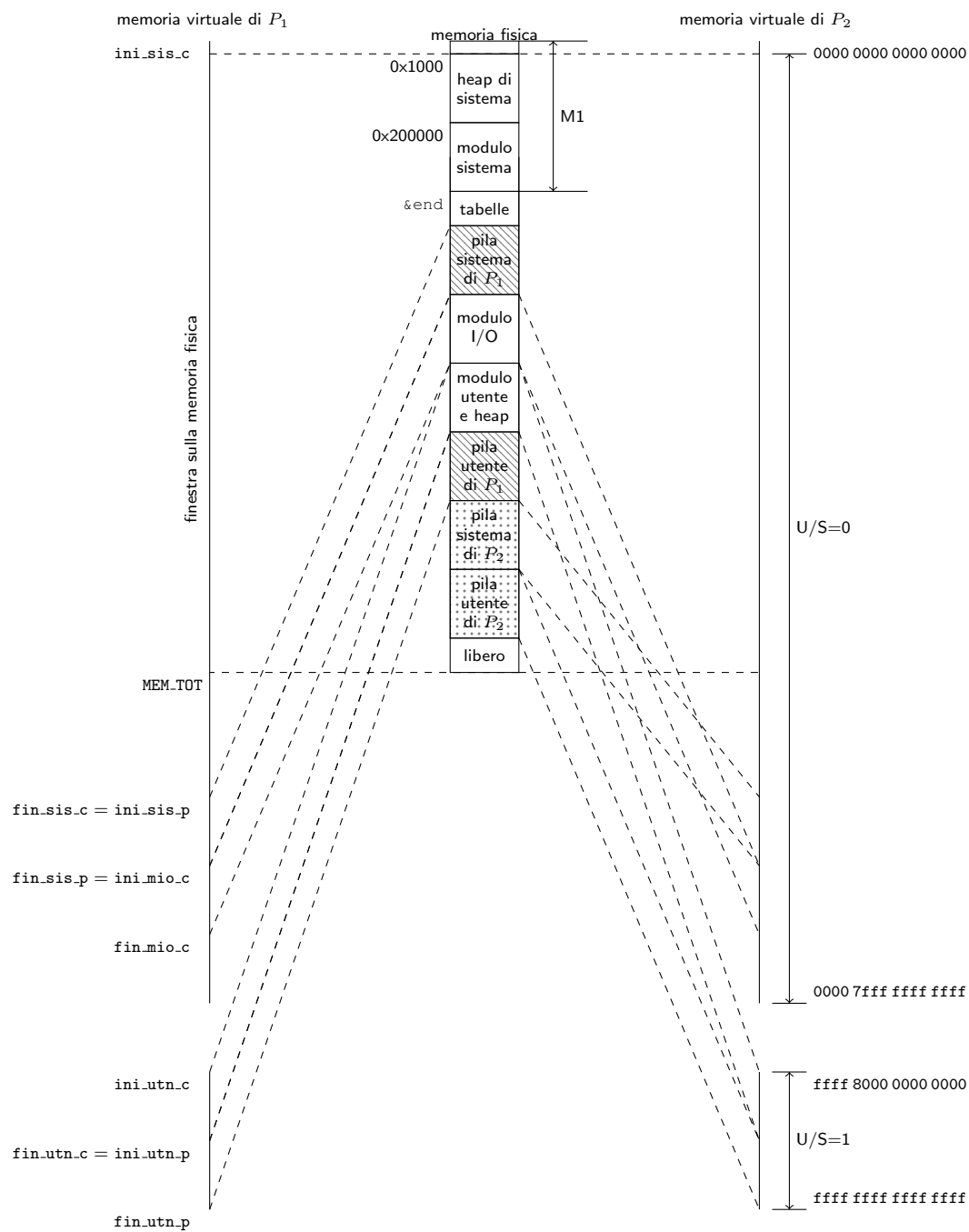


Figura 1: Esempio di memoria virtuale con due processi (figura non in scala).

esempio, `fin_sis_c` è l'indirizzo di fine della sezione `sistema/condivisa`. Le sezioni condivise contengono le stesse traduzioni in tutti i processi.

Le sezioni della memoria virtuale di ogni processo sono elencate di seguito.

sistema/condivisa: contiene la finestra sulla memoria fisica. La dimensione della sezione è decisa *a priori*, mentre la dimensione della finestra dipende dalla memoria fisica installata e può essere più piccola (come illustrato in Figura, la finestra arriva solo fino a `MEM_TOT`). La sezione contiene anche delle traduzioni identità (non mostrate in Figura) per quegli indirizzi che corrispondono a registri di periferiche (come per esempio l'APIC). Il resto della sezione è lasciato inutilizzato.

sistema/privata: contiene la pila sistema del processo. Si noti che questa pila è utilizzata sia dalle funzioni del modulo `sistema`, sia da quelle del modulo `I/O`, in quanto questo gira a livello sistema e ogni processo ha una sola pila di livello sistema.

IO/condivisa: contiene il modulo `I/O`, vale a dire le sezioni `.text`, `.data` estratte dal file `build/io` e mappate al loro indirizzo di collegamento. Contiene inoltre lo *heap I/O*, vale a dire la zona di memoria su cui lavorano gli operatori **new** e **delete** quando usati dal modulo `I/O`.

utente/condivisa: contiene il modulo utente, vale a dire le sezioni `.text` e `.data` estratte dal file `build/utente` e caricate al loro indirizzo di collegamento. Analogamente al modulo `I/O`, contiene inoltre lo *heap utente*, che è usato dagli operatori **new** e **delete** quando sono invocati dal modulo utente.

utente/privata: contiene la pila utente del processo.

La memoria fisica è suddivisa, come al solito, in una parte M1 e una parte M2 (per motivi di spazio, solo la parte M1 è indicata esplicitamente in Figura; la parte M2 è quella rimanente). La suddivisione della parte M2 della memoria fisica in Figura 1 è puramente esemplificativa: i processi saranno in generale più di due; inoltre, se osserviamo il sistema ad un qualunque istante, potremo trovare le varie parti in un ordine qualsiasi e in frame non contigui della memoria fisica.

La parte M1 della memoria fisica è organizzata nel modo seguente.

- I primi due MiB contengono lo *heap di sistema*. Si tratta di una zona di memoria in cui le primitive del modulo `sistema` allocano i `des_proc`, le strutture `richiesta` (usate dalla primitiva `delay`), e in generale qualunque struttura dati che debbe essere allocata dinamicamente tramite gli operatori **new** e **delete** (parte degli indirizzi sono occupati da altre cose, come la memoria video in modalità testo, che si trova all'indirizzo `b8000`). Quando il modulo `sistema` parte, il boot loader ha già usato parte dello heap per allocare alcune strutture dati (in particolare, le tabelle che contengono le traduzioni per la finestra di memoria e il segmento TSS).

- La parte dedicata al modulo sistema contiene più precisamente le sezioni **.text** e **.data** estratte dal file `build/sistema` e caricate al loro indirizzo di collegamento. L’inizio della parte M2 verrà ottenuta, a tempo di inizializzazione, dal simbolo `end` definito nel modulo sistema.

Le linee tratteggiate tra le memorie virtuali di P_1 e P_2 e la memoria fisica vogliono rappresentare la corrispondenza tra le sezioni della memoria virtuale di ciascun processo e le parti di memoria fisica in cui sono tradotte. Le parti a sfondo bianco delle memoria fisica corrispondono a sezioni condivise tra tutti i processi. Si noti, per esempio, come le sezioni “modulo I/O” e “modulo utente e heap” corrispondano alla stessa parte della memoria fisica sia per P_1 che per P_2 . Le sezioni private, invece, usano traduzioni diverse in ciascun processo e corrispondono a parti diverse della memoria fisica: si veda per esempio la sezione utente/privata (tra gli indirizzi `ini_utn_p` e `fin_utn_p`) che corrisponde a “pila utente di P_1 ” per il processo P_1 e “pila utente di P_2 ” per il processo P_2 ; un discorso analogo vale per la parte sistema/privata. Si noti che alcune parti della memoria fisica sono accessibili esclusivamente tramite la finestra: in particolare, tutta la parte M1 e i frame di M2 che o contengono tabelle o sono vuote (la parte indicata con “libero” in Figura 1). Alcune parti di M2 sono accessibili sia dalla finestra, sia da altre sezioni. Ciò è dovuto semplicemente al fatto che la finestra permette di accedere a tutta la memoria fisica, indipendentemente dal contenuto, e dunque contiene traduzioni anche per tutti i frame che contengono le pagine della memoria virtuale di ogni processo.

2 Implementazione

In questa sezione esaminiamo le strutture dati e le funzioni che si trovano nel modulo sistema e sono relative all’implementazione della memoria virtuale.

La suddivisione dello spazio di indirizzamento dei processi nelle sue varie parti è stabilita da alcune costanti definite in `costanti.h` (si vedano i commenti in “suddivisione della memoria virtuale” all’interno del file). Per semplicità le varie parti occupano ciascuna un numero intero di regioni di livello massimo (quindi 512 GiB per i trie a 4 livelli), e dunque le possiamo distinguere in base alle entrate utilizzate nella tabella radice. Per esempio, la parte sistema/condivisa usa la prima entrata della radice (indice 0), mentre la parte sistema/privata usata la seconda (indice 1).

Anche le dimensioni delle varie parti *fisiche* della memoria dei processi sono decise *a priori* con delle costanti: la dimensione delle pile, sistema e utente, e la dimensione degli heap dei tre moduli (sistema, I/O e utente). Lo stesso vale per la RAM totale del sistema, che è decisa dalla costante `MEM_TOT`. Per aumentare o diminuire la RAM usata dal sistema (M1+M2) occorre modificare questa costante e ricompilare. Si noti che lo script `run` configura la macchina virtuale con la quantità di RAM stabilita da questa stessa costante.

```

1  struct des_frame {
2      union {
3          natw nvalide;
4          natl prossimo_libero;
5      };
6  };

```

Figura 2: Il descrittore di frame.

2.1 Gestione della memoria fisica

Il sistema deve sapere quali frame della memoria sono liberi o occupati, in modo da poterli allocare per le pagine e le tabelle dei processi. Dal momento che tutti i frame sono equivalenti, è sufficiente mantenere una lista dei frame liberi da cui si estrae e si inserisce sempre in testa.

In generale, il sistema ha bisogno di associare anche altre informazioni ad ogni frame. Per questo scopo il modulo sistema alloca un *descrittore di frame* per ogni frame della memoria fisica. Il descrittore ha il tipo mostrato in Figura 2 e contiene le seguenti informazioni:

- `prossimo_libero`: (significativo solo se il frame è libero) il numero del prossimo frame nella lista dei frame liberi;
- `nvalide`: (significativo solo se il frame contiene una tabella) quante entrate con `P` diverso da zero sono contenute nella tabella.

Si noti che, dal momento che le due informazioni non sono mai utili contemporaneamente, sono dichiarate all'interno di una **union**, che permette di risparmiare spazio.

I descrittori di frame sono raccolti in un array, `vdf[]`, indicizzato dal numero di frame. La funzione `init_frame()`, chiamata in fase di inizializzazione, si preoccupa di inizializzare questo array, inserendo tutti i frame di M2 nella lista dei frame liberi. Da questo punto in poi è possibile allocare e deallocare frame usando le funzioni:

- `paddr alloca_frame()`, che prova ad estrarre un frame dalla lista dei liberi e ne restituisce l'indirizzo (fisico) di base (0 se la lista è vuota);
- **void** `rilascia_frame(paddr p)`, che inserisce il frame di indirizzo fisico `p` nella lista dei liberi.

Le funzioni di utilità `inc_ref()`, `dec_ref()` e `get_ref()` servono a manipolare il contatore `nvalide` di una tabella di cui conosciamo l'indirizzo fisico. Il modulo sistema ridefinisce la funzione `alloca_tab()` della libce in modo che allochi la tabella dalla lista dei frame liberi di M2, invece che dallo heap di sistema.

2.2 Creazione delle parti condivise

Tutte le traduzioni relative alle parti condivise dei processi (sistema, I/O e utente) possono essere create una volta per tutte all'avvio del sistema. I processi possono condividere tutto il sottoalbero che implementa queste traduzioni, a partire dal livello inferiore a quello della radice (il livello 3 nei processori con trie a 4 livelli). Per esempio, possiamo creare la traduzione della finestra sulla memoria una sola volta. Tutte le entrate di indice 0 delle tabelle radice di tutti i processi punteranno poi alla stessa tabella di livello inferiore, e dunque tutto il sottoalbero sarà condiviso tra tutti i processi. Questo ci permette di risparmiare molto spazio, e anche del tempo ogni volta che creiamo un nuovo processo.

Le traduzioni della parte sistema/condivisa sono create dal boot loader, tramite la funzione `crea_finestra_FM()`, prima di abilitare la paginazione. Il modulo sistema ritrova il TRIE allocato dal boot loader tramite l'indirizzo contenuto in `cr3`: Il boot loader mappa in se stessi tutti gli indirizzi da `DIM_PAGINA` a `MEM_TOT`, in modo da creare la finestra sulla memoria fisica, lasciando non mappata la prima pagina in modo da poter intercettare le dereferenziazioni di `nullptr`. Le traduzioni della finestra sono implementate con pagine di livello 2 dove possibile, e sono impostate con bit R/W a 1 (scrittura permessa) e bit U/S a zero (accesso consentito solo da livello sistema). Questa traduzione permette anche di accedere alle sezioni `.text`, `.data` e `.bss` del modulo sistema, che sono collegate e caricate a partire dall'indirizzo `0x200000` (2 MiB, inizio del terzo megabyte di RAM). Si noti che la finestra comprende anche la memoria video in modalità testo, che si trova nella pagina di indirizzo base `0xb8000`. Nella traduzione degli indirizzi della memoria video viene anche settato il bit PWT, in modo che le scritture raggiungano effettivamente la memoria video e non si fermino in cache per un tempo indefinito. Non è invece necessario disabilitare del tutto la cache, in quanto nel nostro caso la memoria video può essere modificata solo dal software e, dunque, possiamo sfruttare la cache per le operazioni di lettura. Sempre nella stessa funzione sono anche create le traduzioni identità che permettono di accedere all'APIC, i cui registri, come sappiamo, sono mappati nello spazio di memoria. In questo caso dobbiamo completamente disabilitare la cache, settando il bit PCD. Il boot loader, per permettere l'accesso anche ad eventuali altre periferiche con registri mappati nello spazio di memoria, crea traduzioni identità on cache disabilitata per tutti gli indirizzi da `MEM_TOT` fino a 4 GiB (escluso).

Le traduzioni delle parti IO/condivisa e utente/condivisa sono create invocando la funzione `carica_modulo()`. Il boot loader di QEMU ha copiato i file `io` e `utente` in memoria (nel secondo megabyte di RAM). Gli indirizzi di partenza delle due copie sono scritti in una struttura dati che il boot loader passa alla routine di inizializzazione del modulo sistema¹. La vera e propria creazione delle traduzioni per i due moduli si trova nella funzione `carica_modulo()`,

¹In realtà c'è un passaggio intermedio tramite il boot loader della `libce`, in quanto il boot loader di QEMU porta il processore soltanto nella modalità a 32 bit. Il boot loader di `libce` passa alla modalità a 64 bit e cede il controllo al modulo sistema, passandogli gli stessi parametri ricevuti dal boot loader di QEMU.

invocata una volta per il modulo I/O e un'altra per il modulo utente. Questa funzione interpreta il file ELF copiato in memoria e ne consulta la tabella di programma, per sapere quali parti del file devono essere mappate nello spazio di indirizzamento. Per ciascuna di esse alloca opportunamente dei frame dalla parte M2, vi copia il corrispondente contenuto del file (o provvede ad azzerare i byte del frame, nel caso del **.bss**) e crea le traduzioni. Inoltre, alloca ulteriori frame destinati a contenere lo heap del modulo e li mappa in coda all'ultimo indirizzo virtuale menzionato nel file ELF (normalmente, dunque, subito dopo le sezioni **.data** e **.bss**). A questo punto lo spazio occupato dai due file copiati dal boot loader di QEMU può essere riutilizzato ed entra a far parte dello heap del modulo sistema.

Tutte queste traduzioni vengono create nello stesso albero di traduzione che aveva inizialmente creato il boot loader e la cui radice si trova ancora in **cr3**.

2.3 Creazione dei processi

La creazione dei processi si trova nella funzione `crea_processo()`. Oltre ad allocare e inizializzare un nuovo `des_proc` nei modi che già conosciamo, la funzione si deve preoccupare di creare tutta la memoria virtuale del processo. Per far questo alloca una nuova tabella radice e vi copia tutte le entrate relative alle parti condivise, prendendole dalla tabella radice che in questo momento è puntata da **cr3**. Nel caso dei primi processi, creati in fase di inizializzazione, questa è la tabella radice di cui abbiamo parlato nella sezione precedente. Tutti gli altri processi sono creati da altri processi, e in questo caso il registro **cr3** punta alla tabella radice del processo genitore. In ogni caso, l'effetto è che le entrate relative alle parti condivise punteranno tutte agli stessi sottoalberi, come avevamo anticipato.

La funzione `crea_processo()` deve poi creare le parti private. Queste comprendono la pila sistema (parte sistema/privata) e, per i processi di livello utente, anche la pila utente (parte utente/privata). Queste sono create allocando un numero prestabilito di frame (in base alle costanti definite in `costanti.h`) e poi mappandoli contigualmente partendo dagli indirizzi inferiori delle rispettive parti dello spazio di indirizzamento.

La funzione, ricordiamo, deve anche *inizializzare* la pila sistema, inserendovi le cinque parole quaduple che verranno poi lette dalla **iretq** la prima volta che il processo andrà in esecuzione. Qui bisogna fare particolare attenzione: la funzione non può scrivere nella pila sistema del processo appena creato usando gli indirizzi della parte sistema/privata, in quanto al momento è ancora attivo lo spazio di indirizzamento del processo *genitore* e quegli indirizzi verrebbero tradotti dalla MMU nei frame della pila sistema di questo processo, e non del processo creato. Per poter scrivere nella pila sistema del processo creato, la funzione sfrutta la finestra sulla memoria fisica, accedendo direttamente ai frame della pila tramite il loro indirizzo fisico.

2.4 Le funzioni di supporto

Per gestire correttamente l'allocazione e la deallocazione delle tabelle occorre fare attenzione al contatore delle entrate libere, che va aggiornato ogni volta che si cambia un bit *P* all'interno di una tabella. Il modo più semplice è di usare le funzioni definite nella libce e riportate qui sotto. Queste funzioni usano internamente le funzioni `inc_ref()` e `dec_ref()` per tenere il contatore sempre aggiornato.

- `paddr alloca_tab()`
(ridefinita nel modulo sistema) alloca un frame destinato a contenere una tabella; rispetto ad una semplice `alloca_frame()` provvede anche ad azzerare sia il frame, sia il contatore `nvalide` nel suo descrittore;
- `void rilascia_tab()`
(ridefinita nel modulo sistema) rilascia un frame che conteneva una tabella; rispetto ad una semplice `rilascia_frame()`, controlla anche che il contatore `nvalide` non sia diverso da zero, in modo da intercettare errori di programmazione in cui si tenta di rilasciare tabelle che ancora contengono entrate valide;
- `void set_entry(paddr tab, natl j, tab_entry se)`
fa il contrario della `get_entry()`, settando l'entrata *j*-esima della tabella di indirizzo fisico `tab` con il nuovo valore `se`; si preoccupa di aggiornare opportunamente il contatore `nvalide` se il valore del bit *P* cambia per effetto dell'assegnamento;
- `void copy_des(paddr src, paddr dst, natl i, natl n)`
copia *n* entrate a partire dalla *i*-esima della tabella di indirizzo fisico `src` nelle corrispondenti entrate della tabella di indirizzo fisico `dst`; si preoccupa di aggiornare opportunamente il contatore `nvalide` della tabella `dst`; questa funzione è utile, per esempio, quando si crea un nuovo processo e si devono copiare le entrate delle parti condivise dalla tabella radice del processo genitore;
- `void set_des(paddr dst, natl i, natl n, tab_entry e)`
setta *n* entrate a partire dalla *i*-esima della tabella di indirizzo fisico `dst`, impostandole tutte uguali a `e` e aggiornando opportunamente il contatore `nvalide`; questa funzione è utile soprattutto quando `e` è zero perché si sta distruggendo un albero di traduzione (per esempio, quando un processo termina);

Il bus PCI

G. Lettieri

30 Aprile 2024

1 Il bus di espansione

Come molti altri computer dell'epoca e come i suoi discendenti di oggi, anche il PC AT era *espandibile*. In questo contesto l'aggettivo “espandibile” significa che il sistema contiene un *bus di espansione* con degli *slot* in cui possono essere inserite delle *schede di espansione* che possono aggiungere funzionalità al sistema: schede video, schede audio, schede di rete, etc. Così come per altri computer dell'epoca (come per esempio l'Apple II) queste schede di espansione potevano essere create e vendute da aziende indipendenti, senza una preventiva approvazione dell'IBM. Questa mancanza di centralizzazione ha creato una serie di scomodità per gli utenti delle schede di espansione. Per focalizzare le idee, concentriamoci sul problema seguente: le schede hanno registri e/o memoria interna a cui devono essere assegnati indirizzi nello spazio di I/O e/o di memoria affinché il software vi possa accedere. Nel PC AT, però, sono le schede stesse a riconoscere gli indirizzi dei propri registri, senza nessuna regola stabilita *a priori*. Questo può andar bene se qualcuno coordina la produzione delle schede, ma è un problema se le schede sono prodotte da aziende indipendenti e concorrenti. È chiaro che se l'azienda produttrice della scheda A e l'azienda produttrice della scheda B scelgono indirizzi sovrapposti per i loro registri, la scheda A e la scheda B non possono essere usate contemporaneamente nello stesso PC. Inoltre, a causa di queste possibili sovrapposizioni, il software non ha un modo affidabile per scoprire se una certa scheda è installata o meno: tutto ciò che può fare e provare a scrivere e leggere qualcosa dagli indirizzi dei registri di quella scheda per vedere se ottiene ciò che si aspetta, ma così facendo rischia di inviare comandi ad un'altra scheda che per caso usa quegli stessi indirizzi.

2 Lo standard PCI

Non solo le schede di espansione erano prodotte da aziende indipendenti: dal momento che il PC IBM era costruito con parti standard, aziende concorrenti dell'IBM crearono infiniti cloni “compatibili” del PC stesso, in grado di far girare lo stesso software e installare le stesse schede di espansione, malgrado i tentativi dell'IBM di arginare il fenomeno per vie legali. Il bus del PC AT venne chiamato ISA, per *Industry Standard Architecture*. Era uno standard di fatto

che sostanzialmente descriveva il bus del PC AT, con tutti i suoi limiti. Per risolvere i problemi del bus ISA occorreva definire un nuovo standard e sperare che tutti vi aderissero. Ne furono proposti diversi, ma l'unico che ebbe successo fu lo standard *Peripheral Component Interconnect* (PCI) proposto dall'Intel nel 1992. Lo standard PCI stabilisce quali sono i collegamenti del bus e il protocollo che le schede devono usare per comunicare. Inoltre, definisce tre spazi di indirizzamento: lo spazio di memoria, lo spazio di I/O e uno *spazio di configurazione*. I primi due spazi sono del tutto analoghi a quelli che già conosciamo e sono quelli che il software deve utilizzare, nel funzionamento a regime, per dialogare con le periferiche connesse al bus. Resta, ovviamente, il vincolo che i registri delle periferiche occupino indirizzi non sovrapposti, ma questo viene garantito nel seguente modo:

- le periferiche che rispettano lo standard non possono scegliere autonomamente gli indirizzi dei propri registri, ma devono contenere dei comparatori programmabili in modo che questi indirizzi siano impostati dal software;
- il software può impostare questi comparatori all'avvio del sistema, tramite il nuovo spazio di configurazione.

Lo spazio di configurazione è fatto in modo tale da poter accedere a dei registri di configurazione che ogni periferica deve avere per rispettare lo standard, e di poterlo fare in modo univoco senza conflitti. Accedendo a questi registri di configurazione il software di avvio (il PCI BIOS, nell'idea dello standard) può scoprire in modo affidabile quali periferiche sono connesse al bus (ricavandone anche il codice del venditore e il tipo), di quanti indirizzi ciascuna di esse ha bisogno, e programmare di conseguenza i comparatori in modo che non ci siano sovrapposizioni.

Una peculiarità dello standard PCI è che non è legato ad un particolare processore e prevede un dispositivo detto *ponte ospite-PCI* che faccia da tramite tra il bus dove risiede il processore (detto *bus locale*) e il bus PCI (si veda la Figura 1). Un'altra peculiarità dello standard è di prevedere non un unico bus, ma un *albero* di bus la cui radice è il bus PCI a cui è collegato il ponte ospite-PCI; gli altri bus sono collegati all'albero tramite ponti PCI-PCI, fino ad un massimo di 256 bus. Ad ogni bus si possono collegare fino a 32 dispositivi, e ciascun dispositivo può contenere da una a 8 funzioni (per esempio, una singola scheda di espansione conta come un dispositivo, ma al suo interno potrebbe contenere sia la funzione video che la funzione audio). Alcune funzioni possono fare da ponte verso altri tipi di bus: avremo dunque ponti PCI-ISA, ponti PCI-ATA, ponti PCI-USB, e così via.

2.1 Operazioni di lettura e scrittura

Sul bus possono transitare principalmente operazioni di lettura o scrittura in uno dei tre spazi (I/O, memoria o configurazione). Ciascuna operazione, detta *transazione*, è iniziata da un dispositivo detto *iniziatore* che cerca di operare su un altro dispositivo detto *obiettivo*. Lo standard permette a qualsiasi dispositivo

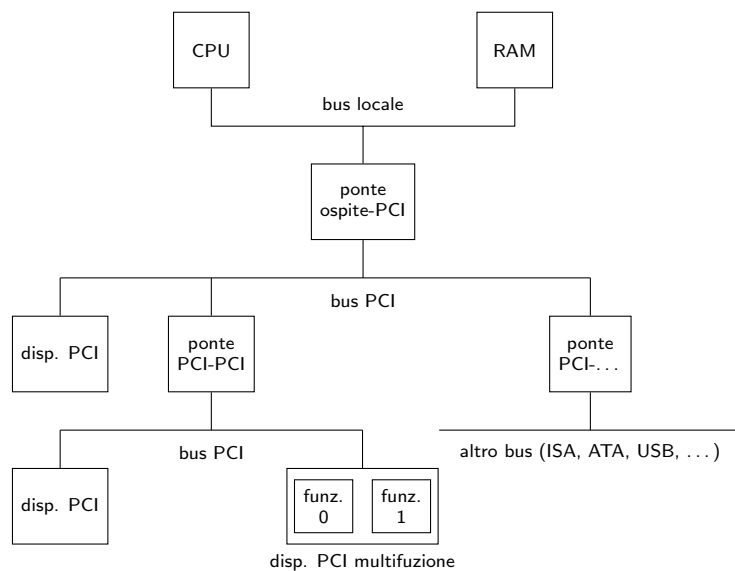


Figura 1: Esempio di architettura con bus PCI. Al bus PCI principale (il più vicino alla CPU) sono collegati 4 dispositivi (i tre ponti e il dispositivo sulla sinistra). Al secondo bus PCI sono collegati tre dispositivi (il ponte e i due dispositivi in basso). Il dispositivo in basso a destra contiene due funzioni; tutti gli altri ne hanno soltanto una. I dispositivi possono essere collegati direttamente al bus, oppure trovarsi su schede di espansione PCI inserite in appositi slot.

di essere, in transazioni diverse, iniziatore o obiettivo. Questo permette, tra le altre cose, di realizzare il meccanismo di accesso diretto alla memoria di cui parleremo in seguito. Per il momento possiamo pensare che l'unico iniziatore sia il ponte ospite-PCI, che inizia delle transazioni sul bus PCI per tradurre le operazioni analoghe avviate dal processore sul bus locale.

Le transazioni si svolgono in più fasi: una fase di indirizzamento e una o più fasi di scambio dati. Durante la fase di indirizzamento l'iniziatore specifica il tipo di operazione (lettura o scrittura e spazio di indirizzamento) e l'indirizzo del primo byte da leggere o scrivere. Durante questa fase tutti i dispositivi che hanno registri in quello spazio confrontano l'indirizzo specificato dall'iniziatore con quello contenuto nei propri comparatori. Al più un dispositivo troverà una corrispondenza, e quello diventa l'obiettivo della transazione. Nelle successive fasi di scambio dati avviene il trasferimento dei dati dall'iniziatore all'obiettivo (scrittura) o dall'obiettivo all'iniziatore (lettura).

I principali collegamenti del bus sono i seguenti:

FRAME# (uscita per l'iniziatore, ingresso per l'obiettivo) delimita ("incornicia") l'inizio e il termine di ogni transazione;

DEVSEL# (ingresso per l'iniziatore, uscita per l'obiettivo) attivato dall'obiettivo che riconosce uno dei propri indirizzi durante la fase di indirizzamento

C/BE#[3:0] (uscita per l'iniziatore, ingresso per l'obiettivo) codificano il tipo di operazione nella fase di indirizzamento (C[3:0]), e fungono byte-enable nelle fasi di trasferimento dati (BE#[3:0]);

AD[31:0] (ingresso/uscita per tutti) codificano l'indirizzo nella fase di indirizzamento e trasportano i dati nelle fasi di trasferimento dati;

TRDY# (*target ready*, ingresso per l'iniziatore, uscita per l'obiettivo) handshake nelle fasi di scambio dati;

IRDY# (*initiator ready*, uscita per l'iniziatore, ingresso per l'obiettivo) handshake nelle fasi di scambio dati;

STOP# (ingresso per l'iniziatore, uscita per l'obiettivo) serve a terminare prematuramente una transazione;

CLK (ingresso per tutti) segnale di clock; tutti i dispositivi campionano i loro segnali in ingresso sul fronte in salita di questo segnale.

I segnali il cui nome termina in "**#**" sono attivi bassi, mentre i nomi seguiti da quadre rappresentano un gruppo di più segnali. Come si vede, il bus è a 32 bit sia per gli indirizzi che per i dati. Venne definita una estensione a 64 bit, che non fu molto usata e poi venne superata dal PCI Express. Noi ci limiteremo alla versione a 32 bit.

Il protocollo è il seguente: l'iniziatore pilota C[3:0] e AD[31:0] con il tipo di operazione e l'indirizzo iniziale del trasferimento, quindi attiva FRAME# per segnalare l'inizio di una transazione. Se uno dei dispositivi riconosce il

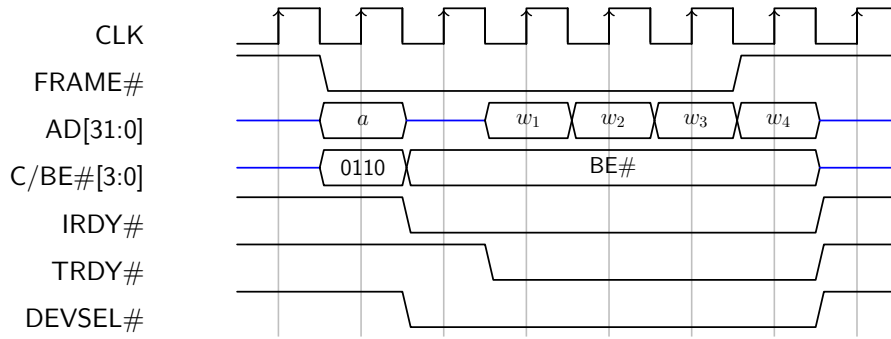


Figura 2: Esempio di transazione di lettura nello spazio di memoria, con 4 fasi di trasferimento dati. La transazione inizia quando FRAME# diventa attivo, indicando la validità dell'indirizzo *a* sulle linee AD e del comando 0110 sulle linee C (0110 specifica una transazione di lettura nello spazio di memoria). In questo esempio un dispositivo riconosce il comando e l'indirizzo, quindi attiva DEVSEL# al clock successivo. Si noti che a questo punto il pilotaggio delle linee AD deve passare dall'iniziatore all'obiettivo; lo standard impone che l'obiettivo mantenga TRDY# disattivato per almeno un clock, per dare il tempo all'iniziatore di scollegare le proprie uscite AD. Ora l'iniziatore pilota le linee BE# (in una transazione di lettura sono tipicamente sempre tutte attive) e l'obiettivo pilota le linee AD. Nell'esempio IRDY# e TRDY# sono sempre attivi durante le fasi di trasferimento dati, quindi le 4 parole $w_1, \dots, w_4$ sono trasferite alla velocità massima. Durante il quarto trasferimento l'iniziatore disattiva FRAME#, segnalando la fine della transazione.

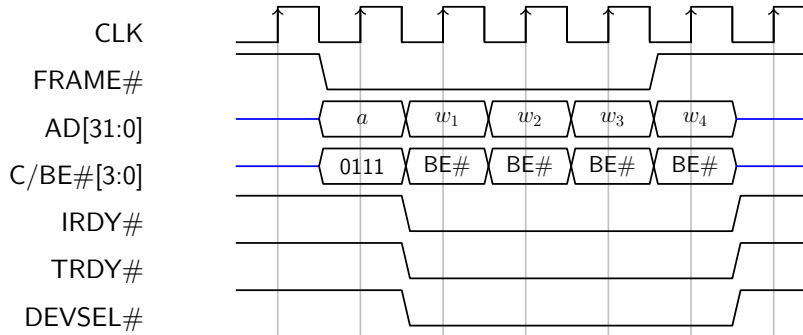


Figura 3: Esempio di transazione di scrittura con 4 fasi di trasferimento dati. Rispetto alle operazioni di lettura, l'obiettivo può attivare TRDY# già dal clock successivo a quello in cui l'iniziatore attiva FRAME#, in quanto il pilotaggio delle linee AD[31:0] non deve passare da un dispositivo all'altro. Sempre rispetto ad una transazione di lettura, questa volta è probabile che le linee BE#[3:0] non siano sempre tutte attive, quindi l'esempio le mostra in cambiamento in ogni fase. Il resto del protocollo è del tutto simile.

tipo di operazione e l'indirizzo attiva DEVSEL#. Se nessun dispositivo attiva DEVSEL# entro un certo numero di clock, l'iniziatore termina l'operazione con un errore, altrimenti si prosegue con le fasi dati. Nelle operazioni di lettura (si veda l'esempio in Figura 2) l'obiettivo pilota le linee AD[31:0] e segnala la loro validità attivando la linea TRDY#, quindi attende che l'iniziatore segnali di aver ricevuto i dati attivando IRDY#; viceversa, l'iniziatore attende che TRDY# sia attivo, quindi campiona AD[31:0] (sullo stesso fronte del clock in TRDY# attivo) e, quando è pronto per il prossimo trasferimento, attiva IRDY#. Nelle transazioni di scrittura (esempio in Fig. 3) vale l'opposto: l'iniziatore pilota le linee AD[31:0] e BE#[3:0] e attiva IRDY#, quindi attende TRDY#, mentre l'obiettivo attiva TRDY# quando è pronto a ricevere e attende IRDY# per sapere quando AD[31:0] e BE#[3:0] sono validi. Ogni fase dati trasferisce al più 4 byte allineati naturalmente. Ciascuna fase dati si conclude quando sia IRDY# che TRDY# sono attivi sullo stesso fronte di salita del clock.

Il riutilizzo delle stesse linee per scopi diversi riduce i costi a scapito della velocità di trasferimento, ma questa è in gran parte recuperata dalla possibilità di eseguire più fasi dati con una singola fase di indirizzamento. In tutte le transazioni (sia lettura che scrittura) è l'iniziatore a decidere il numero di fasi trasferimento: se FRAME# è attivo quando IRDY# è attivo, l'iniziatore sta chiedendo una ulteriore fase dati. Viceversa, l'obiettivo riconosce l'ultima fase di trasferimento quando campiona IRDY# attivo con FRAME# disattivo. L'obiettivo può attivare STOP# per terminare forzatamente la transazione. Un motivo per farlo può essere il seguente: per poter eseguire più fasi dati, l'obiettivo deve poter memorizzare internamente l'indirizzo di partenza e poi incrementarlo autonomamente in ogni fase dati. Un dispositivo che non è in grado

di farlo attiverà `STOP#` la prima volta che attiva `TRDY#`. Questo costringe l'iniziatore a iniziare una nuova transazione per il prossimo trasferimento.

Anche se, dal punto di vista del bus PCI, l'iniziatore delle transazioni è il ponte ospite-PCI, questo non inizia mai transazioni di sua spontanea volontà, ma solo per tradurre analoghe operazioni iniziate dalla CPU (o dal controllore cache) sul bus locale. Nella nostra macchina il ponte risponde a tutte le operazioni nello spazio di I/O e ad alcune operazioni nello spazio di memoria. Il ponte ha alcuni registri nello spazio di I/O del processore e risponde direttamente alle operazioni che li indirizzano (si veda la Sezione 2.2.2), ma per il resto traduce tutte le operazioni nelle analoghe transazioni sul bus PCI, dialogando con l'obiettivo per conto della CPU o del controllore cache. La traduzione è trasparente: le operazioni nello spazio di I/O del processore si traducono in transazioni nello spazio di I/O PCI allo stesso indirizzo, e analogamente per lo spazio di memoria. Dal punto di vista della CPU (e dunque del software) è come se i dispositivi obiettivo di queste transazioni si trovassero direttamente sul bus locale. Affinchè questo avvenga, però, i dispositivi devono essere prima configurati.

2.2 Lo spazio di configurazione

Lo standard PCI prevede uno spazio di configurazione di 64 parole (da 4 byte) per ogni funzione (ogni dispositivo ha almeno una funzione). In questo spazio ciascuna funzione rende accessibili i propri registri di configurazione, seguendo lo schema illustrato in Figura 4, dove abbiamo evidenziato solo i registri che ci interessano. Non tutti i registri menzionati in Figura 4 devono essere sempre presenti, ma, con la possibile eccezione del ponte ospite-PCI, tutte le funzioni devono avere i registri che nella Figura hanno lo sfondo grigio¹.

I registri obbligatori hanno il seguente significato:

Vendor ID (lettura) un codice che identifica il produttore della funzione, assegnato ad ogni produttore da una autorità centrale²;

Device ID (lettura) un codice che identifica la funzione, assegnato dal produttore;

Command (lettura/scrittura) permette di abilitare o disabilitare varie capacità della funzione (vedere sotto);

Status (lettura) descrive alcune capacità della funzione, più lo stato di alcuni eventi, per lo più legati al verificarsi di errori;

Class Code (lettura) un codice che identifica in modo generico il tipo di funzione;

¹La revisione 2.2 dello standard ha aggiunto altri registri obbligatori, che non consideriamo per semplicità.

²Il PCI SIG (PCI Special Interest Group), <https://pcisig.com/>.

+3	+2	+1	+0	offset
Device ID		Vendor ID		0x00
Status		Command		0x04
Class Code			Revision ID	0x08
...	Header Type	...	Cache Line Size	0x0c
Base Address Register 0				0x10
Base Address Register 1				0x14
Base Address Register 2				0x18
Base Address Register 3				0x1c
Base Address Register 4				0x20
Base Address Register 5				0x24
...				0x28
...		...		0x2c
...				0x30
...			...	0x34
...				0x38
...	...	Intr. Pin	Intr. Line	0x3c

Figura 4: Lo spazio di configurazione di ogni funzione PCI.

Revision ID (lettura) numero di revisione della funzione, assegnato dal produttore;

Header Type (lettura) tipo di header; deve essere 0 per tutte le funzioni che non siano ponti PCI-PCI o PCI-CardBus.

Del registro Command ci interessano solo i bit 0, 1 e 2. I bit 0 e 1 abilitano la funzione a rispondere alle transazioni nello spazio di I/O e di memoria, rispettivamente. Questi bit sono per default a zero (transazioni disabilitate) e il software di inizializzazione deve metterli a 1 dopo aver assegnato alla funzione gli indirizzi necessari nei corrispondenti spazi. Come vedremo in seguito, il bit 2 abilita la funzione a comportarsi da *bus master*, qualora ne abbia la capacità. Se una funzione non ha registri nello spazio di memoria o di I/O (o non ha la capacità di comportarsi da bus master), il corrispondente bit vale 0 e non è scrivibile.

2.2.1 Transazioni nello spazio di configurazione

Nel seguito ci limiteremo a considerare il caso di un singolo bus PCI, quello direttamente collegato al ponte ospite-PCI, che è l'unico sicuramente presente.

Tramite le transazioni nello spazio di configurazione si può accedere ai registri di configurazione di ciascuna funzione, e a nient'altro. Inoltre, solo il ponte ospite-PCI può essere iniziatore di queste transazioni.

Ogni dispositivo PCI diverso dal ponte ospite-PCI, e ogni slot di espansione, ha un ingresso chiamato IDSEL. Il ponte ospite-PCI, invece, possiede un diverso collegamento verso ciascuno di questi ingressi. Per accedere ad un particolare registro di una particolare funzione, il ponte deve selezionare il dispositivo che contiene la funzione desiderata attivandone il segnale IDSEL, e poi passargli il numero della funzione (da 0 a 7), l'offset della parola che contiene il registro (colonna destra in Figura 4) e i byte enable relativi al registro. A parte l'uso di IDSEL, le transazioni nello spazio di configurazione sono per il resto simili a quelle negli altri spazi: nella fase di indirizzamento, oltre ad attivare FRAME#, il ponte attiva l>IDSEL appropriato e usa AD[7:0] per passare l'offset (con AD[1] e AD[0] sempre a zero) e AD[10:8] per passare il numero di funzione. Se l>IDSEL attivato è effettivamente collegato ad un dispositivo (potrebbe essere collegato ad uno slot di espansione vuoto), questo deve rispondere attivando DEVSEL# e poi procedere come nelle altre transazioni. Si noti che, dal momento che lo standard impone che DEVSEL# debba essere attivato entro un limite di tempo prestabilito, il ponte dispone di un modo affidabile per riconoscere gli slot vuoti: sono quelli in cui DEVSEL# non viene attivato in tempo.

2.2.2 Accesso allo spazio di configurazione via software

Anche nel caso di transazioni nello spazio di configurazione l'iniziatore è il ponte ospite-PCI dal punto di vista del bus PCI, ma le transazioni sono in realtà iniziate dal software in esecuzione sulla CPU. Questa volta, però, la CPU non dispone di istruzioni che permettano di interagire direttamente con lo spazio di

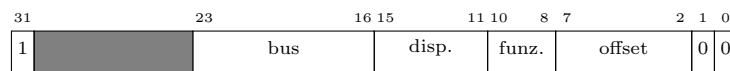


Figura 5: Configuration Address Port (CAP)

configurazione. Invece, il ponte mette a disposizione due registri nello spazio di I/O della CPU, e tramite questi registri il software può accedere indirettamente allo spazio di configurazione PCI. I due registri, entrambi a 32 bit, sono i seguenti:

Configuration Address Port (CAP) (indirizzo 0xCF8) permette di selezionare una funzione e l'offset della parola a cui si vuole accedere;

Configuration Data Port (CDP) (indirizzo 0xCFC) permette di accedere alla parola selezionata tramite CAP.

Per individuare una particolare funzione si usano tre numeri: il numero di bus, il numero del dispositivo nel bus e il numero della funzione nel dispositivo. Nel nostro caso, il numero di bus è sempre 0. Il numero di dispositivo dipende unicamente dal piedino IDSEL a cui il dispositivo è collegato, mentre il numero di funzione è deciso dal costruttore del dispositivo, con il vincolo che la funzione 0 deve essere sempre presente. In ogni caso, i tre numeri sono stabiliti *a priori* in modo univoco, ed è questo che permette allo spazio di configurazione di essere accessibile già dall'avvio del sistema.

I tre numeri e l'offset della parola a cui si vuole accedere vanno scritti in CAP nel modo illustrato in Figura 5. Una volta che CAP è stato impostato, il ponte ospite-PCI tradurrà le letture e le scritture in CDP in corrispondenti transazioni nello spazio di configurazione. In particolare, il ponte attiverà l'uscita IDSEL che corrisponde al numero di dispositivo selezionato in CAP, copierà il numero di funzione e l'offset da CAP sulle linee AD[10:0], e userà come BE#[3:0] i primi 4 byte enable del processore.

Se l'operazione in CDP è di lettura, il ponte fa anche un'altra cosa: se nessun dispositivo risponde alla corrispondente transazione di lettura nello spazio di configurazione (nessuno attiva DEVSEL# in tempo), il ponte restituisce un valore di tutti 1 al processore. La lettura di questo valore permette quindi al software di rilevare in maniera affidabile l'assenza di un dispositivo nella posizione selezionata. In particolare, dal momento che nessun produttore ha ID 0xFFFF, le funzioni PCI installate sul sistema possono essere scoperte all'avvio provando a leggere il Vendor ID da tutti i possibili numeri di dispositivo (da 0 a 31) con funzione 0 (che, ricordiamo, ogni dispositivo deve avere): la lettura di un valore diverso da 0xFFFF indica la presenza di un dispositivo, che poi può essere ulteriormente ispezionato e configurato.

2.2.3 I Base Address Register (BAR)

Lo scopo principale della fase di configurazione dei dispositivi è quello di assegnare indirizzi univoci negli spazi di I/O e/o di memoria, in modo che da quel

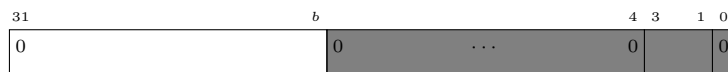


Figura 6: Base Address Register (BAR), caso di blocco nello spazio di memoria. I bit in grigio non sono scrivibili.

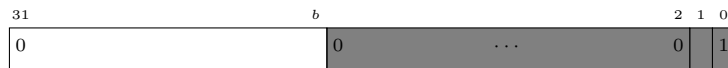


Figura 7: Base Address Register (BAR), caso di blocco nello spazio di I/O. I bit in grigio non sono scrivibili.

momento in poi il software possa accedere ai registri direttamente. Si tenga presente che in questo caso stiamo parlando dei registri specifici della funzione (per esempio, il registro RBR di una interfaccia di ingresso), diversi dai registri che si trovano nello spazio di configurazione. Il costruttore della scheda deve definire uno o più *blocchi* di indirizzi, ciascuno con una dimensione che sia una potenza di due, all'interno dei quali rendere disponibili i registri specifici della funzione. Il costruttore deve anche decidere, per ciascun blocco, se deve trovarsi nello spazio (PCI) di I/O o di memoria, e quale è l'organizzazione interna di ciascun blocco (a che offset si trovano i vari registri all'interno del blocco). Quello che non può decidere, però, è l'indirizzo di partenza dei blocchi. Invece, per ogni blocco, deve prevedere un Base Address Register (BAR) nello spazio di configurazione (offset 0x10–0x24 in Figura 4). Sarà il software di inizializzazione a scegliere gli indirizzi di partenza in modo che non vi siano conflitti con i blocchi delle altre funzioni.

I BAR sono organizzati come in Figura 6 e 7. Si noti che alcuni bit non sono scrivibili e servono a dare al software di inizializzazione indicazioni sul tipo di blocco e sulla sua dimensione. In particolare, il bit 0 indica se il blocco è pensato per lo spazio di I/O (bit 0 uguale a 1) o di memoria (bit 0 uguale a 0). La dimensione del blocco è 2^b , dove b è il numero di bit che non sono scrivibili. Il software di inizializzazione può scoprire quanto vale b provando a scrivere tutti 1 nel BAR e rileggendo il risultato, per vedere quali bit sono stati effettivamente scritti. Si noti che $b \geq 4$ per i blocchi di memoria, che quindi hanno una dimensione minima di 16 byte, mentre $b \geq 2$ per i blocchi di I/O, corrispondenti ad una dimensione minima di 4 byte³.

I blocchi possono essere assegnati solo a regioni allineate naturalmente alla loro dimensione: il software di inizializzazione sceglierà la regione e ne scriverà il numero nei bit scrivibili del BAR. In questo modo il BAR (una volta azzerati i 4 o 2 bit meno significativi) conterrà l'indirizzo di partenza del blocco.

Dopo aver assegnato una regione a un blocco, il software di inizializzazione setterà il bit 0 o 1 del registro di Comando (a seconda che il blocco sia di I/O

³Nei blocchi di memoria i bit 1–3 contengono ulteriori indicazioni sul tipo di memoria, che tralasciamo per semplicità; nei blocchi di I/O il bit 1 è riservato e vale sempre 0.

o di memoria). Da quel momento in poi la funzione risponderà alle transazioni i cui indirizzi cadono in quella regione.

Nella macchina QEMU l'inizializzazione è fatta dal PCI BIOS, quindi viene completata prima che il nostro software parta. In teoria è sempre possibile rifarla, ma noi ci limiteremo ad accettare le impostazioni del BIOS. Tutto ciò che ci serve è sapere quali indirizzi il BIOS ha assegnato ai vari blocchi, e questo possiamo farlo leggendo i BAR.

2.3 Le interruzioni

I dispositivi PCI possono generare richieste di interruzione, ma molti dei dettagli non sono fissati dallo standard, in quanto dipendono dal particolare sistema in cui si trova il bus.

Ogni dispositivo ha fino a quattro piedini di uscita per le richieste di interruzione: INTA#, INTB#, INTC# e INTD#. Ogni funzione di un dispositivo può essere collegata ad al più uno di questi piedini (anche nessuno), e più funzioni dello stesso dispositivo possono essere collegate allo stesso piedino. Il piedino utilizzato da una funzione deve essere leggibile dal registro di configurazione chiamato "Intr. Pin" in Figura 4 (un byte, sola lettura). Un valore 0 in questo registro indica che la funzione non genera richieste di interruzione, 1 indica che usa il pin INTA#, 2 indica INTB#, etc.

Chi scrive il software che deve accedere alla funzione, però, non vuole sapere questo; piuttosto, ha bisogno di sapere a quale piedino del controllore delle interruzioni ciascuna funzione è collegata. Lo standard non lo dice, in quanto si ferma ai piedini INTA#, INTB#, etc., lasciando piena libertà al progettista su come collegare questi piedini al resto. Il progettista, però, deve garantire che il BIOS scriva questa informazione nel registro "Intr. line" (un byte, lettura/scrittura) in Figura 4. Nel nostro caso il BIOS scrive in "Intr. line" proprio il numero del piedino dell'APIC a cui la funzione è collegata, quindi possiamo limitarci a leggere questo registro.

2.4 Il bus PCI nel PC

Nel seguito faremo riferimento all'architettura di Fig. 8, con un unico bus PCI e due ponti, uno verso il bus ISA (dove si trovano le vecchie periferiche del PC AT, come la tastiera e il timer) e uno verso il bus ATA, dove si trovano gli hard disk. Questa architettura era comune intorno agli anni 2000. Il ponte ospite-PCI veniva chiamato *North Bridge* e spesso conteneva altre funzioni, come quella di controllore della memoria (la memoria era dunque collegata al North Bridge invece che al bus locale, ma noi continueremo a supporre che sia collegata direttamente al bus locale, come in Figura 8); i ponti PCI-ISA e PCI-ATA erano inglobati in un altro dispositivo che svolgeva queste e altre funzioni, chiamato *South Bridge*. I PC moderni adottano lo standard PCI Express, che ha superato il vecchio standard PCI mantenendo la compatibilità a livello software. Noi ci concentreremo solo sullo standard PCI, che è molto più semplice del PCI Express

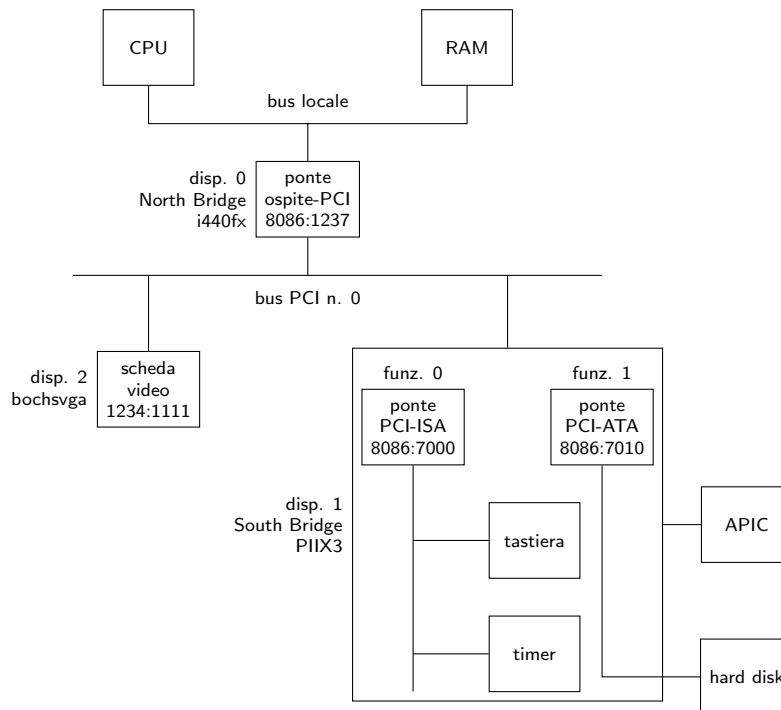


Figura 8: Architettura dei bus delle periferiche nella macchina QEMU. I numeri all'interno delle funzioni PCI rappresentano VendorID:DeviceID. Il North Bridge (i440fx) risponde a tutte le operazioni di lettura/scrittura sul bus locale che o indirizzano lo spazio di I/O, oppure sono relative a indirizzi di memoria superiori a quelli della RAM installata, o appartengono a zone del primo MiB tradizionalmente riservate alle schede di espansione (per es. la memoria video). Le transazioni intercettate nello spazio di memoria del bus locale sono tradotte in analoghe transazioni nello spazio di memoria PCI sul bus 0. L'i440fx risponde direttamente alle transazioni nello spazio di I/O del bus locale che indirizzano i registri CAP e CDP, e traduce in transazioni nello spazio di I/O del bus PCI tutte le altre. Per realizzare le transazioni nello spazio di configurazione PCI, l'i440fx ha 16 collegamenti IDSEL realizzati riutilizzando le linee AD[26:11] (il numero di dispositivi PCI installabili sul bus 0 è dunque limitato a 16 invece dei 32 permessi dallo standard). Il South Bridge (PIIX3) è un dispositivo PCI multi-funzione contenente al suo interno un ponte PCI-ISA, un ponte PCI-ATA e un ponte PCI-PCI (non mostrato); svolge anche altre funzioni non legate allo standard PCI, come permettere l'accesso all'APIC e emulare il vecchio controllore della tastiera e l'interfaccia timer.

ed quello emulato dalla nostra macchina virtuale QEMU. Inoltre, conoscere lo standard PCI facilita l'eventuale studio dello standard PCI Express.

I/O nel nucleo

G. Lettieri

3 Maggio 2024

Come abbiamo visto nell'esempio che motivava l'introduzione della protezione, le operazioni di I/O offrono un'ottima opportunità per sfruttare appieno l'ambiente multiprogrammato. Ricordiamo brevemente gli aspetti essenziali dell'esempio. Tipicamente, un processo che inizia una operazione di I/O non può proseguire finché l'operazione non è terminata, e d'altro canto l'operazione stessa è normalmente lenta e non ha bisogno della CPU per essere portata avanti, o ne ha bisogno in minima parte. L'idea è quindi di *bloccare* i processi che iniziano una operazione di I/O e sbloccarli (riportarli in coda pronti) quando l'operazione è terminata. In questo modo altri processi potranno andare in esecuzione mentre è in corso l'operazione di I/O e la CPU sarà sfruttata più efficientemente. Si noti che il sistema deve sapere che l'operazione è completata mentre è in esecuzione un processo che, in generale, è completamente scorrelato da essa. Il completamento dell'operazione dovrà essere segnalato da una interruzione, in modo che il sistema possa riacquistare il controllo della CPU e svolgere le operazioni necessarie, tra cui sbloccare il processo che aveva richiesto l'operazione.

Quanto appena descritto è un esempio di *sincronizzazione*: vogliamo che il processo che ha iniziato l'I/O possa proseguire solo dopo che l'operazione di I/O è stata completata.

In ambiente multiprocesso il sistema deve anche preoccuparsi di coordinare l'accesso alle periferiche. Tipicamente, mentre una periferica è impegnata in una operazione di I/O, non può essere usata per altre operazioni. Se un processo vuole usare una periferica mentre questa è occupata, deve aspettare che la precedente operazione termini e la periferica ritorni libera. A differenza del precedente, questo non è un problema di sincronizzazione, ma di *mutua esclusione*: se due processi, P_1 e P_2 , vogliono entrambi usare la stessa periferica, è per noi indifferente se la usa prima P_1 e poi P_2 o viceversa: l'unica cosa che ci interessa è che non la usino contemporaneamente. Si noti infine che la mutua esclusione possiamo garantirla separatamente per ogni periferica: se P_1 e P_2 devono usare due periferiche distinte non hanno bisogno di coordinarsi.

Ricordiamo ora che i processi di cui stiamo parlando sono processi *utente* e, come al solito, sono da considerarsi non fidati. Non possiamo dunque aspettarci che gli utenti si coordinino tra di loro per usare le periferiche uno alla volta, o che sospendano volontariamente i propri processi per far andare avanti quelli

degli altri. Per imporre la mutua esclusione e la sincronizzazione di cui sopra procediamo come al solito:

- impediamo agli utenti di parlare direttamente con le periferiche e
- forniamo delle primitive per svolgere le operazioni di I/O sotto il controllo del sistema.

Per realizzare il primo punto facciamo in modo che il campo IOPL del registro **rflags** dei processi utente specifichi il valore “sistema”. In questo modo, mentre il processore si trova a livello utente, genera una eccezione ogni volta che si prova ad eseguire una istruzione di **in** o **out**.¹ Notare che, indipendentemente da questo problema, siamo sostanzialmente obbligati a settare il campo IOPL a “sistema”, in quanto questo è anche il modo in cui vietiamo agli utenti l'utilizzo delle istruzioni **sti** e **cli**, che abilitano e disabilitano le interruzioni esterne mascherabili. Per vietare l'accesso alle periferiche che hanno i registri mappati in memoria ricorriamo invece alla MMU, in uno dei due modi possibili: o non inserendo traduzioni che portino ai registri delle periferiche, o proteggendo le traduzioni con i bit “S/U” nei descrittori.

Occupiamoci ora della struttura generale delle primitive di I/O che il sistema deve fornire. Una tipica primitiva di lettura avrà una interfaccia simile a questa:

```
extern "C" void read_n(natl id, char* buf,
                      natq quanti);
```

La primitiva riceve un parametro `id`, che serve ad identificare la periferica da cui il processo vuole leggere, e un indirizzo `buf` che punta ad un buffer in cui l'utente vuole ricevere i dati. Faremo l'ipotesi che i dati siano sempre una sequenza di byte. Il parametro `quanti` specifica il numero di byte che l'utente vuole leggere. È l'utente che deve preoccuparsi di dichiarare un buffer grande a sufficienza per contenere i byte richiesti. Vedremo che il sistema può imporre altre limitazioni su questo buffer.

Analogamente, la tipica primitiva di scrittura avrà questa interfaccia verso gli utenti:

```
extern "C" void write_n(natl id, const char* buf,
                       natq quanti);
```

I parametri hanno lo stesso significato del caso precedente, ma ora il buffer puntato da `buf` deve contenere i dati che l'utente vuole scrivere (la primitiva si limita a leggerli, da cui il **const**).

1 Realizzazione con primitiva e driver

Concentriamoci su una generica operazione di lettura, con interfaccia utente `read_n()` come dichiarata qui sopra. Assumiamo che nel sistema siano installate diverse periferiche simili, identificate dunque dal parametro `id`. Assumiamo

¹Per vietare **in** e **out** sono necessari anche altri accorgimenti, che tralasciamo per semplicità; maggiori dettagli si trovano nel codice.

inoltre che queste periferiche siano in grado di trasferire un byte alla volta, inviando una richiesta di interruzione ogni volta che è disponibile un nuovo byte. L'interfaccia di ogni periferica avrà un registro di controllo per abilitare e disabilitare le interruzioni (CTL) e un registro di ingresso da cui leggere il nuovo byte (RBR). La lettura da RBR funziona da risposta alla richiesta di interruzione: l'interfaccia non genera una nuova richiesta di interruzione fino a quando non ha avuto una risposta a quella precedente.

L'operazione sarà svolta in parte dalla primitiva e in parte da un *driver*, che andrà in esecuzione ad ogni richiesta di interruzione da parte dell'interfaccia:

- la primitiva ha lo scopo di avviare l'operazione di I/O e bloccare il processo, garantendo anche la mutua esclusione;
- il driver ha il compito di trasferire effettivamente i byte e sbloccare il processo quando l'operazione si è conclusa.

Per gestire la mutua esclusione e la sincronizzazione vogliamo usare i semafori, ma questo comporta che la primitiva non può essere atomica, e dunque non deve salvare e caricare lo stato del processo all'entrata e all'uscita dal sistema. In questo caso ha senso immaginare che sia il processo stesso ad eseguire la primitiva, sospendendosi e risvegliandosi al suo interno in corrispondenza delle invocazioni delle primitive semaforiche.

Consideriamo ora un processo P_1 che invoca la primitiva `read_n`. Questa, come per tutte le primitive, è in realtà solo una piccola funzione scritta in Assembler nel file `utente.s`. La funzione invoca la primitiva vera e propria tramite una istruzione **int**, che permette l'innalzamento del livello di privilegio:

```

1  .global read_n
2  read_n:
3      int $IO_TIPO_RN
4      ret

```

Nell'entrata corrispondente al tipo `IO_TIPO_RN` della tabella IDT dovrà essere installato un gate con che punti a `a_read_n`. Questa sarà una funzione scritta in assembler nel file `sistema.s`:

```

1  .extern c_read_n
2  a_read_n:
3      call c_read_n
4      iretq

```

Si noti che, come dicevamo, la `a_read_n` non salva lo stato, lasciando che la primitiva venga eseguita nel contesto del processo P_1 che l'ha invocata.

Passiamo ora alla parte C++ della primitiva. Dato l'identificatore `id`, la primitiva ha bisogno di ottenere alcune informazioni sulla corrispondente periferica (l'indirizzo dei suoi registri, ma non solo). Per memorizzare tutte queste informazioni prevediamo un descrittore di operazione di I/O definito come segue:

```

1  struct des_io {
2      ioaddr iRBR, iCTL;
3      char* buf;
4      natq quanti;
5      natl mutex;
6      natl sync;
7  };

```

È sufficiente avere un array di tali descrittori e usare `id` come indice al suo interno. Ogni descrittore contiene tre tipi di informazioni: gli indirizzi dei registri (riga 2), le informazioni (ricevute dall'utente) su dove i byte vanno trasferiti (righe 3-4) e due identificatori di semafori (righe 5-6). Per realizzare la sincronizzazione e la mutua esclusione, come abbiamo anticipato, la `c_read_n` utilizzerà i semafori, che servono proprio a questo. Il semaforo `mutex` è inizializzato a 1 all'avvio del sistema e serve a garantire la mutua esclusione tra i processi che vogliono usare la periferica `id`. Il semaforo `sync` è inizializzato a 0 all'avvio del sistema e serve a realizzare la sincronizzazione tra il processo che ha richiesto l'operazione e la conclusione dell'operazione stessa.

La `c_read_n` è strutturata nel modo seguente:

```

1  extern "C" void c_read_n(natl id, natb *buf, natl quanti)
2  {
3      // controllo sui parametri
4      des_io *d = &array_des_io[id];
5
6      sem_wait(d->mutex);
7      d->buf = buf;
8      d->quanti = quanti;
9      outputb(1, d->iCTL);
10     sem_wait(d->sync);
11     sem_signal(d->mutex);
12 }

```

Alla riga 4 ottiene un puntatore al descrittore della interfaccia, per comodità di scrittura. Le righe 6 e 11 garantiscono la mutua esclusione: solo un processo alla volta può eseguire le righe 7-10.

Le righe 7 e 8 trasferiscono le informazioni sul buffer dell'utente all'interno del descrittore. Da qui le leggerà il driver quando andrà in esecuzione.

La riga 9 abilita le interruzioni (supponiamo che sia sufficiente scrivere 1 nel registro CTL). Da questo momento l'interfaccia può inviare richieste di interruzione ogni volta che ha un nuovo byte da trasferire. Si noti che per il momento le interruzioni esterne sono mascherate, in quanto ci troviamo nel modulo sistema.

La riga 10 blocca P_1 sul semaforo di sincronizzazione. A questo punto il sistema può passare ad eseguire altri processi. Mentre sono in esecuzione questi altri processi le interruzioni sono abilitate. Si noti che P_1 è bloccato all'interno della `sem_wait` alla riga 10 e dunque ancora dentro la zona di mutua esclusione.

Fino a quando P_1 non verrà sbloccato e non eseguirà la `sem_signal(d->mutex)` alla riga **11** l'interfaccia non potrà essere usata da altri processi, come volevamo. Se un altro processo (andato in esecuzione in seguito al blocco di P_1) invocherà la `read_n` sulla stessa interfaccia, arriverà alla riga **6** e si bloccherà.

Passiamo ora ad esaminare il driver. Si noti che abbiamo già visto un esempio di driver quando abbiamo parlato della primitiva `delay()` e le considerazioni generali valgono ancora: le ripetiamo per comodità. Il driver andrà in esecuzione per effetto di una richiesta di interruzione da parte dell'interfaccia (richiesta che arriverà alla CPU tramite l'APIC). Il driver gira a livello sistema e, per poter tornare a livello utente, deve terminare anch'esso con una **`iretq`**. Anche il driver, dunque, avrà una parte scritta in assembler:

```

1  .extern c_driver
2  a_driver_i:
3      call salva_stato
4      movq $i, %rdi
5      call c_driver
6      call apic_send_EOI
7      call carica_stato
8      iretq

```

Per fare in modo che `a_driver_i` vada in esecuzione ogni volta che l'interfaccia genera una interruzione, occorre conoscere il tipo dell'interruzione generata e preparare il corrispondente gate della IDT in modo che punti a `a_driver_i`.

Chiamiamo P_2 il processo in esecuzione all'arrivo dell'interruzione.

Il driver salva e ripristina lo stato di P_2 (linee **3** e **7**) perché può dover cambiare il processo in esecuzione. Infatti, quanto l'ultimo byte è stato trasferito, il driver deve risvegliare P_1 (che ora è bloccato nella `sem_signal(d->sync)` dentro `c_read_n`). Se P_1 ha una priorità maggiore di P_2 , è P_1 che deve andare in esecuzione, mentre P_2 deve tornare in coda pronti. La `carica_stato` alla riga **7** quindi, carica lo stato di P_2 o di P_1 , a seconda dei casi.

Alle righe 4-5 si chiama il driver vero e proprio, `c_driver`, scritto in C++. Dal momento che stiamo assumendo di trattare interfacce simili, `c_driver` può essere una funzione generica e ricevere un parametro che gli indichi l'interfaccia da gestire (linea **4**). Si noti che il parametro che `a_driver_i` passa a `c_driver` è una costante. Questo perché ogni interfaccia genera una richiesta di interruzione di tipo diverso, quindi è sufficiente associare ad ogni tipo di interruzione una diversa copia di `a_driver_i`, ciascuna con una costante diversa.

Alla riga **6** inviamo l'End Of Interrupt al controllore APIC, in modo che lasci passare le interruzioni a priorità minore o uguale di quella appena gestita dal driver.

Si ricordi che il driver usa le risorse di del processo che ha interrotto, che in questo caso è P_2 . In particolare usa la sua pila sistema e anche la sua memoria virtuale, dal momento che non stiamo cambiando il valore di **`cr3`**.

Vediamo ora il codice di `c_driver`:

```

1  extern "C" void c_driver(natl id)
2  {
3      des_io *d = &array_des_io[id];
4
5      d->quanti--;
6      if (d->quanti == 0) {
7          outputb(0, d->iCTL)
8          des_sem *s = &array_dess[d->sync];
9          s->counter++;
10
11         if (s->counter <= 0) {
12             des_proc* lavoro = rimozione_lista(s->pointer);
13             inspronti(); // preemption
14             inserimento_lista(pronti, lavoro);
15             schedulatore(); // preemption
16         }
17     }
18     char c = inputb(d->iRBR);
19     *d->buf = c;
20     d->buf++;
21 }

```

Alla riga [3](#) il driver ottiene un puntatore al descrittore della interfaccia, per comodità di scrittura.

Lo scopo principale del driver è leggere il nuovo byte dall'interfaccia e copiarlo nel buffer dell'utente, cosa che viene fatta alle righe [18-19](#). Alla riga [20](#) il puntatore al buffer viene incrementato, in modo che il prossimo byte venga copiato nella locazione successiva.

Alla riga [5](#) il driver decrementa `d->quanti`, che così contiene sempre il numero di byte ancora da leggere. Se `d->quanti` è arrivato a zero il byte letto alla riga [18](#) è l'ultimo byte richiesto dall'utente. Si deve quindi disabilitare l'interfaccia a generare interruzioni (riga [7](#)) e risvegliare il processo che aveva iniziato l'operazione (righe [8-16](#)).

Ci sono alcune cose da notare:

1. la disabilitazione delle interruzioni (riga [7](#)) è eseguita *prima* della lettura del byte (riga [18](#));
2. invece di chiamare `sem_signal()` ripetiamo la parte centrale del suo codice (righe [8-16](#));
3. la scrittura in `d->buf` (riga [19](#)) è eseguita mentre è attiva la memoria virtuale di P_2 , anche se il buffer era stato allocato da P_1 .

Il punto 1 è una conseguenza del fatto che la lettura del byte fa da risposta alla richiesta di interruzione da parte dell'interfaccia, che a quel punto può generarne un'altra se ha un nuovo byte disponibile. Quindi, se leggiamo l'ultimo

byte mentre le interruzioni sono abilitate, è possibile che l'interfaccia generi una nuova interruzione, rimandando in esecuzione il driver, anche se nessun processo ha iniziato una operazione di lettura. Il driver leggerebbe questo byte e lo copierebbe dove punta `d->buf`, andando a sovrascrivere parti casuali della memoria.

Il punto 2 è una conseguenza del fatto che `sem_signal()` salva e ripristina lo stato, ma il driver non è un processo e non ha un suo descrittore di processo. In particolare, se `c_driver` chiamasse `sem_signal()` salverebbe lo stato del driver nel descrittore processo attivo, che è P_2 . Per di più sovrascriverebbe lo stato salvato alla riga 3 di `a_driver_i` (avremmo violato la regola di non avere due `salva_stato` senza una `carica_stato` in mezzo).

Dal momento che il driver fa parte del modulo sistema, potremmo pensare di chiamare direttamente `c_sem_signal()`, evitando il salvataggio e caricamento dello stato. Purtroppo, però, il controllo sulla validità del semaforo (`sem_valido()`) fallirebbe, in quanto `liv_chiamante()` assume che `c_sem_signal()` sia stata invocata per effetto di una istruzione `int`. Nel nostro caso, invece, `liv_chiamante()` accedrebbe alla pila sistema di P_2 e scambierebbe il livello di P_2 , che molto probabilmente è utente, per il livello del chiamante della `c_sem_signal()`; dal momento che il semaforo `d->sync` era stato allocato da livello sistema, segnalerebbe un errore. Non ci resta dunque che ricopiare il codice della `c_sem_signal()` escludendo i controlli dei parametri (o definire una ulteriore funzione che contenga solo tale codice). Si noti che il driver, in questo modo, manipola le code dei processi, e dunque deve essere eseguito con le interruzioni disabilitate. In altre parole, il driver deve essere atomico. Questo comporta che anche le richieste di interruzione a precedenza maggiore dovranno aspettare che il driver termini, prima di poter essere gestite.

Il punto 3 è un problema se P_1 ha allocato il buffer nella sua sezione utente/privata, dal momento che gli indirizzi virtuali delle sezioni private hanno significati diversi per ogni processo. In questo caso `c_driver` scriverebbe nella sezione utente/privata di P_2 , non di P_1 , causando un doppio problema: P_1 non riceverebbe i suoi dati e P_2 si vedrebbe sovrascrivere parti casuali della sua memoria. Dobbiamo quindi richiedere che i buffer per l'I/O vengano sempre allocati nella parte utente/condivisa. Dal punto di vista del linguaggio C++, nel nostro caso, questo comporta che i buffer devono essere dichiarati globali o allocati nello heap, e mai dichiarati come variabili locali. Questo perché le variabili locali vengono allocate in pila e abbiamo deciso che le pile si trovano in parti private. Nei sistemi che bloccano i processi che causano page fault, il buffer deve anche essere residente (non rimpiazzabile), in quanto il driver, non essendo un processo, non può essere bloccato.

Esaminiamo infine come devono essere impostati i campi del gate che porta a `a_read_n` e di quello che porta a `a_driver_i`. Per il primo avremo:

- il campo DPL (Descriptor Privilege Level) che indica che il gate può essere utilizzato da livello utente;
- il campo L (Level) che indica che, dopo il salto, il processore si deve trovare a livello sistema;

- il campo I/T che può indifferentemente indicare “Interrupt” o “Trap”, dal momento che la primitiva non manipola direttamente le code e i descrittori dei processi, ma noi assumeremo “Interrupt” per uniformità con il resto del codice del modulo sistema.

Per il gate della `a_driver_i` avremo:

- il campo DPL (Descriptor Privilege Level) che indica che il gate può essere utilizzato solo da livello sistema;
- il campo L (Level) che indica che, dopo il salto, il processore si deve trovare a livello sistema;
- il campo I/T che indica “Interrupt”, per quanto detto prima.

L'impostazione del campo DPL deriva da questa constatazione: tutti i gate della IDT possono essere usati indifferentemente da tutti e tre i meccanismi di interruzione (interruzioni esterne, eccezioni e istruzione `int`). Impostando DPL a livello sistema impediamo all'utente di invocare il driver in modo spurio, tramite una `int`.

1.1 Controllo sui parametri

Alla riga [3](#) della `c_read_n()` abbiamo lasciato un commento in cui si dice che è necessario controllare i parametri. Questo è sempre vero per tutte le primitive, in quanto i parametri arrivano dal livello utente che, per definizione, non è fidato. In particolare, prima di usare `id` alla riga [4](#), dobbiamo assicurarci che `id` sia un valido identificatore di periferica. Se abbiamo raggruppato tutti i descrittori all'inizio dell'array è sufficiente ricordare l'indice dell'ultimo e confrontarlo con `id`.

Si noti che, invece, non c'è bisogno di controllare che `id` sia valido alla riga [3](#) di `c_driver_i`, in quanto in quel caso `id` è stato passato da `a_driver_i` che, facendo parte del modulo sistema, è fidata.

In `c_read_n()` è molto importante controllare i parametri `buf` e `quanti`, per evitare il cosiddetto problema del *Cavallo di Troia*. L'utente, invece di passare l'indirizzo di un buffer che ha correttamente allocato, potrebbe passare (usando dei cast o scrivendo direttamente in assembler) l'indirizzo di qualunque cosa, anche di parti della memoria che appartengono al sistema o ad altri processi. I controlli di protezione eseguiti dalla CPU e dalla MMU sono inefficaci in questo caso: il passaggio dell'indirizzo “cattivo” alla primitiva comporta solo la scrittura di un numero in un registro e la CPU non controlla alcunché, in quanto non può conoscere lo scopo di questa operazione. L'apparentemente innocuo Cavallo di Troia attraversa dunque le mura del sistema. Quando, successivamente, il sistema tenta di usare l'indirizzo per leggere o scrivere (nel nostro caso ciò accade alla riga [19](#) di `c_driver_i`), ecco che il cavallo si apre, in quanto la primitiva gira a livello sistema e dunque anche la MMU non esegue alcun controllo. Se non prendiamo provvedimenti, l'utente riesce a costringere il

sistema a scrivere su (o leggere da) una zona di memoria a cui lui, normalmente, non avrebbe accesso.

Dobbiamo anche controllare che il buffer che l'utente ci passa non contenga indirizzi che non sono mappati, o non sono normalizzati, in quanto questi causerebbero una eccezione nel driver alla riga [19](#), ma abbiamo detto che il driver deve essere atomico e, dunque, non deve generare eccezioni.

Il modulo sistema mette a disposizione seguente funzione che può essere usata in questi casi:

```
bool c_access(vaddr begin, natq dim,  
             bool writeable, bool shared);
```

La funzione controlla che l'intervallo `[begin, begin+dim)` non attraversi il massimo indirizzo (non ci sia un *wrap around*) e non contenga indirizzi che fanno parte del "buco" (e dunque non sono normalizzati). Inoltre, percorre l'albero di traduzione e controlla che tutti gli indirizzi dell'intervallo siano mappati, accessibili da livello utente e, se `writeable` è **true**, anche scrivibili. Inoltre, se `shared` è **true**, controlla anche che tutto l'intervallo sia contenuto nella zona utente/condivisa.

La funzione `c_read_n()`, dunque, può eseguire il seguente codice

```
if (!c_access(begin, quanti, true, true)) {  
    flog(LOG_WARN, "buf non valido");  
    abort_p();  
}
```

Si noti che passiamo **true** come parametro `writeable`, perché il driver dovrà *scrivere* nel buffer (in una `c_write_n()` avremmo passato **false**). Inoltre, dal momento che ci troviamo in una primitiva non atomica (che dunque non salva e carica lo stato), non è sufficiente chiamare `c_abort_p()`; per abortire il processo, ma si deve chiamare `abort_p()`, che salva lo stato, chiama `c_abort_p()` e poi carica lo stato.

Modulo I/O

G. Lettieri

10 Maggio 2024

La gestione delle interruzioni con il meccanismo del driver è poco flessibile ed efficiente, per due motivi:

- il driver deve essere eseguito con le interruzioni disabilitate, in quanto manipola direttamente le code dei processi;
- il driver non si può bloccare, in quanto non è un processo.

La disabilitazione delle interruzioni può causare problemi, in quanto costringe anche le interruzioni ad alta priorità ad aspettare che il driver termini la propria esecuzione. Il fatto che il driver non si possa bloccare limita fortemente le cose che può fare, soprattutto in un sistema più complicato in cui si debba gestire anche la rete o i file system.

Il secondo problema si può risolvere “trasformando” il driver in un processo. Più precisamente, facciamo in modo che l’interruzione non mandi in esecuzione l’intero driver, ma solo un piccolo *handler* che ha il solo scopo di mandare in esecuzione un processo, il quale si preoccuperà di svolgere le istruzioni che prima erano svolte dal driver.

Il problema delle interruzioni disabilitate può essere ridotto facendo girare questo processo (come tutti gli altri processi) a interruzioni abilitate, identificando tutti i punti in cui accede a strutture dati condivise (nel nostro caso, le code dei processi) e disabilitando le interruzioni solo in quei punti, usando le istruzioni `cli` e `sti`. Questa soluzione, per quanto utilizzata in sistemi reali, comporta però grandi complicazioni nella scrittura del codice. Possiamo invece adottare una soluzione molto più semplice: se il processo non fa parte del nucleo e appartiene invece ad un modulo distinto, che non è collegato con il modulo sistema, non ha modo di accedere alle code dei processi se non invocando delle primitive di sistema. Poiché le primitive di sistema girano a interruzioni disabilitate, ecco che l’accesso alle code dei processi è protetto.

Introduciamo allora un nuovo modulo, detto modulo *I/O*, distinto dal modulo sistema e dal modulo utente. Lo scopo di questo modulo è di realizzare le primitive per l’accesso alle periferiche, gestendo le richieste di interruzione tramite processi, detti processi “esterni” (nel senso che sono esterni al modulo sistema, anche se svolgono funzioni di sistema). Nella nostra implementazione, i file che contengono il codice di questo nuovo modulo si trovano nella cartella

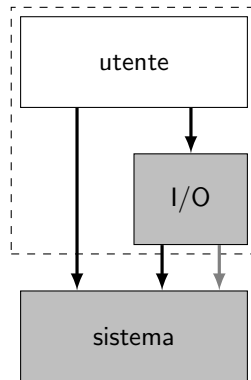


Figura 1: Relazioni tra i moduli. I moduli in grigio sono eseguiti con la CPU a livello sistema. I moduli dentro il riquadro tratteggiato sono eseguiti con le interruzioni mascherabili abilitate. Le frecce rappresentano possibili invocazioni di primitive: le nere sono accessibili anche da livello utente e le grigie solo da livello sistema.

`io` e sono `io.cpp` e `io.s`. La loro compilazione e collegamento produrrà il file `build/io`, che poi verrà caricato in memoria durante l'avvio del sistema e mappato nello spazio di indirizzamento di ogni processo, nella parte che abbiamo chiamato *IO/condivisa*. Il modulo `io` conterrà tutto ciò che è legato alle primitive di I/O, incluse le primitive di I/O invocabili dall'utente e le loro strutture dati.

La Figura 1 mostra le relazioni tra i moduli che compongono il sistema. Si noti che ora l'utente ha accesso sia a primitive che sono realizzate direttamente nel modulo sistema (`sem_ini()`, `sem_wait()`, `sem_signal()`, `delay()`, `activate_p()`, `terminate_p()`), sia a nuove primitive che sono realizzate nel modulo I/O. L'accesso a queste nuove primitive avviene sempre tramite la tabella IDT, con l'esecuzione di una istruzione **int**. Le relative entrate della tabella, però, punteranno a funzioni che sono definite nel modulo I/O.

Idealmente vorremmo che il codice contenuto in questo nuovo modulo girasse ad un livello intermedio tra utente e sistema in quanto deve avere più diritti degli utenti (deve poter interagire direttamente con le interfacce di I/O), ma non deve accedere direttamente alle strutture dati del sistema (IDT, GDT, code dei processi, tabelle della paginazione, etc.) Purtroppo il processore dispone di soli due livelli e scegliamo, dunque, di far girare anche questo modulo a livello sistema (in Figura, entrambi i moduli sono su sfondo grigio). Il fatto di compilare separatamente il modulo I/O e il modulo sistema protegge comunque le strutture dati del sistema da molti errori, ma sicuramente non da tutti (per esempio, un accesso errato in memoria da parte del codice I/O può comunque modificare la memoria riservata al modulo sistema).

A differenza del codice del modulo sistema, il codice del modulo I/O girerà a interruzioni abilitate, come il codice del modulo utente. Questo vale sia per

```

1      # file: sistema.s
2      handler_i:
3          call salva_stato
4          call inspronti
5          movq a_p+$i*8, %rax
6          movq %rax, esecuzione
7          call carica_stato
8          iretq

```

Figura 2: Schema generale di un handler.

il codice dei processi esterni, sia per il codice delle nuove primitive realizzate nel modulo I/O. Eventuali problemi di mutua esclusione dovranno essere risolti utilizzando i semafori forniti dal modulo sistema. Infatti, nella Figura si vede che anche il modulo I/O fa uso di primitive fornite dal modulo sistema; in particolare, usa le primitive semaforiche.

Il modulo I/O ha però accesso anche a primitive ad esso dedicate e non accessibili da livello utente. Per evitare che gli utenti possano invocare queste primitive è sufficiente che il bit DPL dei corrispondenti gate sia impostato a Sistema.

Una delle primitive riservate al modulo I/O è la primitiva `activate_pe()`, che serve ad attivare un processo esterno. Questa primitiva ha gli stessi parametri della normale `activate_p()`, con un ulteriore parametro che è il numero del piedino dell'APIC da cui arriveranno le richieste a cui il processo deve rispondere. Il modulo sistema avrà una tabella `a_p` con una entrata per ogni piedino dell'APIC. La `activate_pe()`, dopo aver creato il processo, inserirà il corrispondente `des_proc` nella opportuna entrata di questa tabella, invece di inserirlo in coda pronti.

Per ogni possibile interruzione, il modulo sistema deve predisporre un handler che si preoccupa di mettere in esecuzione il corrispondente processo esterno, prendendo il `des_proc` da questa tabella. Quindi, se i è uno dei piedini dell'APIC, dovrà esistere un `handler_i` che metta in esecuzione il `des_proc` preso da `a_p[i]`. Questi handler non sono inizialmente associati a nessuna entrata della IDT. Il motivo è che l'entrata della IDT, ovvero il tipo dell'interruzione, determina anche la priorità che l'APIC assegna alla richiesta di interruzione. Vogliamo fare in modo che questa priorità sia consistente con la precedenza del corrispondente processo esterno, come assegnata dalla `activate_pe()`. La soluzione adottata prevede che tale precedenza debba avere la forma `MIN_EXT_PRIO + prio`, dove `MIN_EXT_PRIO` è una costante definita dal sistema (in `costanti.h`) e `prio` è un numero naturale minore di 256. La `activate_pe()` programmerà l'APIC in modo che il piedino i invii il tipo `prio` e all'entrata `prio` della IDT installerà un gate che punti a `handler_i`.

L'handler associato alla richiesta di interruzione proveniente dal piedino i -esimo avrà la forma mostrata in Figura 2. Alla riga 3 salva lo stato del

```

1 // file: io.cpp
2 extern "C" estern(natl i)
3 {
4     des_io *d = &array_des_io[i];
5
6     for (;;) {
7         ...
8         wfi();
9     }
10 }

```

Figura 3: Schema generale di un processo esterno.

```

1 # file: io.s
2 .global wfi
3 wfi:
4     int $TIPO_WFI
5     ret

```

Figura 4: Funzione di interfaccia per la primitiva `wfi()`.

processo che stava girando quando la richiesta è stata accettata e alla riga 4 lo inserisce forzatamente in testa alla coda pronti (se era in esecuzione, era sicuramente quello a priorità maggiore tra quelli in coda pronti). Nelle righe 5-6 esegue l'equivalente del codice `esecuzione=a_p[i]`. La successiva coppia “`call carica_stato; iretq`” (righe 7 e 8) cederà il controllo al processo esterno.

I processi esterni seguiranno tutti lo schema generale mostrato in Figura 3. In Figura si assume che nel sistema ci siano più periferiche simili, ciascuna gestita da un diverso processo esterno, il cui codice può essere però scritto una volta per tutte. Tramite il parametro `i` il codice accede al descrittore di una specifica interfaccia (linea 4). Si noti che tale parametro era stato passato alla `activate_pe()` al momento della creazione del processo, in modo del tutto analogo a quanto avviene con la `activate_p()`. Dopo la loro creazione i processi esterni non terminano più e, dopo aver risposto ad una richiesta di interruzione, si limitano a sospendersi in attesa della prossima. Il loro corpo è composto dunque da un ciclo infinito (linee 6-9) che, dopo ogni iterazione, termina con una invocazione della primitiva di sistema `wfi()` (*wait for interrupt*), come si vede alla linea 8. Anche questa primitiva è riservata al modulo I/O.

La Figura 4 mostra il codice della funzione di interfaccia `wfi()`, usata dal modulo I/O per invocare la primitiva di sistema vera e propria, mostrata in Figura 5. La primitiva salva lo stato del processo esterno (linea 3), invia l'EOI al controllore APIC (linea 4) e mette in esecuzione un altro processo (linee 5-7), di fatto sospendendo il processo esterno, che a questo punto potrà andare nuo-

```

1      # file: sistema.s
2      a_wfi:
3          call salva_stato
4          call apic_send_EOI
5          call schedulatore
6          call carica_stato
7          iretq

```

Figura 5: La primitiva di sistema `wfi()`.

vamente in esecuzione solo quando il corrispondente handler verrà nuovamente invocato, in risposta ad una nuova richiesta di interruzione da parte della stessa interfaccia.

Si noti che possiamo affermare con certezza che, dopo la prima volta che il processo esterno è andato in esecuzione, la coppia “**call carica_stato**; **iretq**” al termine dell’handler associato (linee 6-7 di Figura 2) caricherà lo stato salvato alla linea 3 della `a_wfi`. Infatti, se l’handler è in esecuzione vuol dire che l’interfaccia ha inviato una richiesta che l’APIC ha fatto passare, e dunque l’APIC deve aver ricevuto l’EOI per la richiesta precedente, e dunque l’ultima cosa che il processo esterno aveva fatto in precedenza era proprio chiamare la `wfi()`.

Non possiamo invece sapere quale processo verrà messo in esecuzione al termine della `a_wfi`. Questo perché il processo esterno gira a interruzioni abilitate e dunque, mentre è stato in esecuzione, vari processi potrebbero essere finiti in coda pronti (per esempio, processi sospesi su una `delay()`, o processi che attendevano la conclusione di operazioni di I/O su altre periferiche a maggiore priorità, etc.).

1 Esempio di primitiva di lettura

Torniamo a considerare una generica operazione di lettura, con interfaccia utente

```

extern "C" void read_n(natl id, char* buf, natq quanti);

```

L’operazione sarà svolta in parte dalla primitiva e in parte da un processo esterno messo in esecuzione da un handler ad ogni richiesta di interruzione da parte dell’interfaccia.

Consideriamo ora un processo P_1 che invoca la primitiva `read_n()`. Questa, come per tutte le primitive, è in realtà solo una piccola funzione scritta in Assembler nel file `utente.s`. La funzione invoca la primitiva vera e propria tramite una istruzione **int**, che permette l’innalzamento del livello di privilegio:

```

1      # file: utente.s
2      .global read_n
3      read_n:

```

```

4      int $IO_TIPO_RN
5      ret

```

All'entrata `IO_TIPO_RN` della tabella IDT dovrà essere installato un gate con che punti a `a_read_n`. Questa sarà una funzione scritta in assembler nel file `io.s`:

```

1      # file: io.s
2      .extern c_read_n
3      a_read_n:
4          call c_read_n
5          iretq

```

Notiamo che la funzione `a_read_n` *non* chiama le funzioni `salva_stato` e `carica_stato`, esattamente come l'analogia nel caso del driver. Il motivo è lo stesso: la primitiva `read_n()` non è atomica ed è eseguita dal processo che la invoca. In questo caso, però, non c'è modo di chiamare `salva_stato` o `carica_stato` per sbaglio, in quanto si tratta di funzioni definite nel modulo sistema, che non è collegato con il modulo I/O.

Passiamo ora alla parte C++ della primitiva. Anche in questo caso la primitiva ha bisogno di ricordare alcune informazioni relative alla periferica, quindi prevediamo un descrittore di operazioni di I/O, identico a quello visto precedentemente:

```

1      // file: io.cpp
2      struct des_io {
3          natw iRBR, iCTL;
4          char* buf;
5          natl quanti;
6          natl mutex;
7          natl sync;
8      };

```

Prevederemo come al solito un array di tali descrittori e useremo `id` come indice al suo interno. In questo caso, però, l'array sarà definito in `io.cpp`.

Anche la `c_read_n` è identica a quella vista precedentemente, che riportiamo qui per comodità:

```

1      // file: io.cpp
2      extern "C"
3      void c_read_n(natl id, natb *buf, natl quanti)
4      {
5          des_io *d = &array_des_io[id];
6
7          sem_wait(d->mutex);
8          d->buf = buf;
9          d->quanti = quanti;
10         outputb(1, d->iCTL);

```

```

11     sem_wait(d->sync);
12     sem_signal(d->mutex);
13 }

```

C'è però una differenza rispetto al caso precedente, anche se non si vede: questa primitiva è ora definita nel modulo I/O (file `io.cpp`) e gira ad interruzioni abilitate (il suo gate è di tipo *trap*). Dobbiamo quindi preoccuparci di proteggere le risorse usate dalla primitiva rispetto ai possibili accessi da parte di altri processi che potrebbero interromperla e cercare di accedere alle stesse risorse. In questo caso le risorse sono il descrittore `des_io` e la periferica stessa (i suoi registri). In particolare, la primitiva potrebbe essere interrotta:

1. da altri processi che tentano di invocare `read_n()` sulla stessa interfaccia;
2. dal processo esterno associato all'interfaccia.

(Interruzioni da altri processi diversi da questi non potrebbero accedere alle risorse utilizzate dalla `read_n()` e quindi non ci interessano). Il primo problema è risolto tramite la mutua esclusione garantita dal semaforo `d->mutex`. Si noti come la mutua esclusione è rilasciata dopo aver atteso, sul semaforo `d->sync`, che il trasferimento sia interamente completato. Chiunque volesse utilizzare la periferica mentre è già un corso un altro trasferimento dovrà dunque mettersi in fila alla riga 7, aspettando che il trasferimento sia finito.

Il secondo problema è risolto sfruttando il fatto che il processo esterno potrà andare in esecuzione solo dopo che abbiamo abilitato l'interfaccia a generare richieste di interruzione (riga 10), quindi è sufficiente completare tutti gli accessi al `des_io` prima di quel momento (linee 8-9).

Vediamo ora il codice del processo esterno:

```

1  // file: io.cpp
2  extern "C" void estern(natl id)
3  {
4      des_io *d = &array_des_io[id];
5
6      for (;;) {
7          d->quanti--;
8          if (d->quanti == 0)
9              outputb(0, d->iCTL);
10         char c = inputb(d->iRBR);
11         *d->buf = c;
12         d->buf++;
13         if (d->quanti == 0)
14             sem_signal(d->sync);
15         wfi();
16     }
17 }

```


Lo scopo principale del processo esterno, come quello del driver, è di leggere il nuovo byte dall'interfaccia e copiarlo nel buffer dell'utente. Il codice del processo esterno è dunque molto simile a quello del driver, ma ci sono delle differenze dovute al fatto che ora ci troviamo in un vero processo. In particolare, il test su `d->quanti == 0` deve essere ripetuto due volte:

1. alla riga [8](#) in quanto dobbiamo eventualmente disabilitare le interruzioni (riga [9](#)) *prima* di leggere il byte dall'interfaccia (riga [10](#));
2. alla riga [13](#) in quanto dobbiamo risvegliare il processo che aveva richiesto il trasferimento (riga [14](#)) *dopo* aver effettivamente trasferito l'ultimo byte (righe [10-11](#)).

Per il punto 1 vale esattamente lo stesso discorso già fatto nel caso del driver: se lasciassimo le interruzioni abilitate e leggessimo l'ultimo byte, l'interfaccia potrebbe generare una nuova richiesta, portando al trasferimento di un ulteriore byte che non era stato richiesto.

Il punto 2 è dovuto al fatto che, a differenza del driver, il processo esterno non è atomico. In particolare, la `sem_signal()` (linea [14](#)) potrebbe mettere in esecuzione il processo risvegliato, se questo ha priorità maggiore del processo esterno. Il processo risvegliato (che era quello che aveva richiesto il trasferimento) potrebbe così cominciare ad usare i dati, prima che l'ultimo byte sia stato effettivamente trasferito.

La Figura [6](#) mostra un esempio di evoluzione del sistema supponendo che un processo P_1 invochi una `read_n()` chiedendo di trasferire 2 byte dall'interfaccia i . Si suppone che sia pronto anche un processo P_2 , a priorità più bassa, e che non ci siano altre richieste di interruzioni oltre quelle dell'interfaccia i durante tutto il trasferimento.

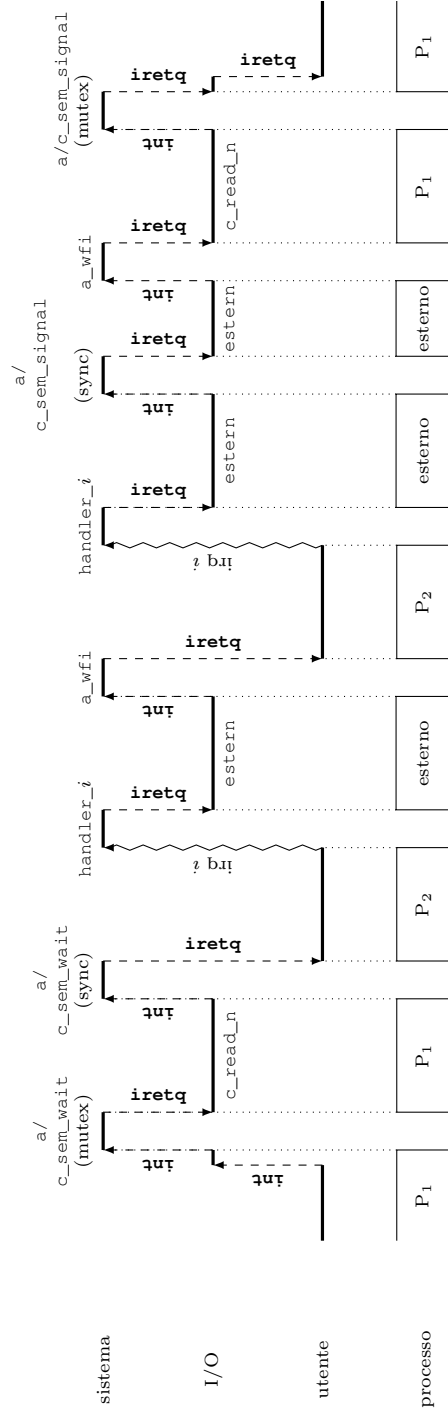


Figura 6: Esempio di esecuzione temporale in seguito all'invocazione di `read_n()` da parte di un processo `P1` (trasferimento di due byte). Le tre righe in alto mostrano il modulo che contiene il codice attualmente in esecuzione. La riga in basso mostra quale processo sta eseguendo il codice. In particolare, quando siamo nel modulo sistema tutti i processi sono fermi e il codice è eseguito atomicamente, mentre il codice del modulo I/O è sempre eseguito da un processo (quello che ha invocato la primitiva oppure un processo esterno).

DMA e PCI Bus Mastering

G. Lettieri

20 Maggio 2024

1 Direct Memory Access

Abbiamo visto, fino ad ora, due modalità di trasferimento dati da un dispositivo alla memoria, o viceversa:

- a “controllo di programma”;
- tramite interruzioni.

Nella prima modalità il software controlla periodicamente che il dispositivo sia pronto leggendo un qualche registro di stato, quindi opera il trasferimento con una o più operazioni di lettura da registri del dispositivo, seguite da scritture in memoria, o viceversa. Nella modalità con interruzioni il trasferimento avviene nello stesso modo, ma è iniziato solo quando il dispositivo segnala la propria disponibilità tramite una richiesta di interruzione. La modalità a controllo di programma è più veloce di quella a interruzioni, ma, come sappiamo, è complicata da programmare se si vuole che il processore possa fare anche altro nei momenti in cui il dispositivo non è pronto; entrambe le modalità prevedono comunque un coinvolgimento del processore, che deve eseguire le istruzioni di lettura e scrittura, e comportano che i dati vengano scambiati sul bus due volte: una volta tra RAM e CPU e una volta tra CPU e dispositivo.

La modalità DMA (Direct Memory Access), che è la terza e ultima modalità di trasferimento dati, prevede invece che sia direttamente il dispositivo ad eseguire le necessarie operazioni di lettura o scrittura sulla RAM, una volta istruito dal software. In generale, il software vorrà eseguire un trasferimento dati dal dispositivo verso un buffer in RAM (operazione di ingresso o lettura) o da un buffer in RAM verso un dispositivo (operazione di uscita o scrittura). Supponiamo che il buffer si trovi all'indirizzo b e sia grande n byte, occupando dunque gli indirizzi $[b, b + n)$. L'indirizzo b e il numero n devono essere comunicati al dispositivo, insieme a tutte le altre eventuali informazioni necessarie a definire il trasferimento richiesto. Da quel momento in poi il dispositivo si preoccuperà di eseguire autonomamente le operazioni in RAM, al proprio ritmo. Ovviamente il dispositivo deve essere dotato di un sommatore che gli permetta di calcolare da solo gli indirizzi necessari a partire da b , e di un contatore che decrementi n ogni volta che è stato completato un trasferimento. Quando tutti i byte sono stati

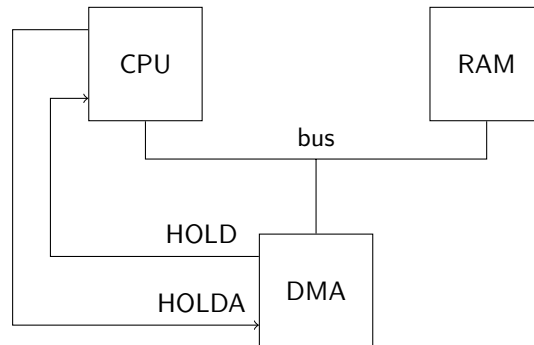


Figura 1: DMA, architettura di base.

trasferiti, il dispositivo segnerà il completamento dell'operazione settando opportunamente un suo registro di stato e, tipicamente, inviando una richiesta di interruzione. Assumiamo che il software non acceda al buffer per tutto il tempo che intercorre tra l'avvio dell'operazione (operata dal software stesso) e la sua conclusione (segnalata dalla richiesta di interruzione).

Dal punto di vista hardware possiamo considerare l'architettura schematizzata in Figura 1, dove abbiamo per il momento eliminato cache, MMU e bus PCI, ricreando l'architettura tipica dei primi PC IBM, o dei minicomputer PDP della DEC. Il dispositivo in grado di operare in DMA è collegato direttamente al bus dove si trovano anche la CPU e la RAM. Questo gli permette di dialogare con la RAM usando lo stesso protocollo già usato dalla CPU. Dal momento che il bus è condiviso, però, un solo dispositivo alla volta può pilotarlo: o la CPU, o il dispositivo DMA. L'accesso è arbitrato tramite i collegamenti HOLD/HOLDA fra dispositivo e CPU:

1. il dispositivo mantiene normalmente i suoi piedini di uscita in alta impedenza;
2. ogni volta che il dispositivo vuole eseguire un trasferimento sul bus, attiva HOLD;
3. in risposta, la CPU termina l'eventuale trasferimento in corso (che può essere anche nel mezzo di una istruzione), mette i suoi piedini di uscita in alta impedenza e attiva HOLDA;
4. il dispositivo attiva i suoi piedini di uscita ed esegue il trasferimento, quindi rimette le uscite in alta impedenza e disattiva HOLD;
5. la CPU disattiva HOLDA, attiva i suoi piedini e riprende il suo normale funzionamento.

In pratica, la CPU dà la precedenza al DMA nell'accesso al bus. La tecnica è chiamata "cycle stealing", in quanto il DMA ruba cicli di bus alla CPU.

Attenzione a non confondere il protocollo HOLD/HOLDA con l'inizio e la fine dell'intera operazione di trasferimento: il dispositivo DMA avvierà il protocollo HOLD/HOLDA ogni volta che sarà pronto a trasferire m byte, ma in generale sarà $m \leq n$ e il protocollo andrà ripetuto più volte fino al completamento dell'intera operazione di trasferimento richiesta dal software. Negli intervalli di tempo in cui HOLD è disattivo la CPU può continuare a usare il bus indisturbata.

Si noti però che, mentre HOLD è attivo, la CPU può solo eseguire una istruzione già prelevata che non preveda accessi in memoria. Durante il tempo di trasferimento, dunque, la CPU potrebbe essere rallentata nell'esecuzione del software. Il meccanismo del DMA resta comunque vantaggioso in almeno tre scenari:

1. se la CPU è più lenta della RAM;
2. se il trasferimento a controllo di programma non è abbastanza veloce per il dispositivo;
3. se il dispositivo deve trasferire i dati con più urgenza di quanto permesso dal meccanismo delle interruzioni.

Il primo scenario era comune, per esempio, negli home computer degli '80 del secolo scorso, e il DMA era usato per trasferire dati di una schermata dalla RAM alla scheda video mentre la CPU stava eseguendo il programma che preparava la prossima schermata. Il secondo e terzo scenario possono verificarsi anche oggi, per esempio con alcune schede di rete che possono ricevere o inviare decine di milioni di pacchetti al secondo a velocità di 200 Gbps (Gigabit per secondo).

Il meccanismo HOLD/HOLDA funziona se c'è un unico dispositivo in grado di operare in DMA. Ci limitiamo a questo caso perché, come vedremo, il bus PCI ci permette di aggirare il problema senza introdurre altri meccanismi.

1.1 Interazione con la cache

Ora prendiamo in considerazione l'esistenza della cache. Notiamo subito che, come mostrato in Figura 2, i segnali HOLD e HOLDA devono ora collegare dispositivo DMA e controllore cache, in quanto è quest'ultimo, e non la CPU, ad essere collegato direttamente al bus con la memoria. A parte questo, l'handshake per il possesso del bus resta lo stesso e valgono le stesse considerazioni fatte precedentemente.

L'introduzione della cache porta un grande vantaggio, ma anche alcune complicazioni. Il vantaggio è che è molto più probabile che la CPU riesca ad eseguire istruzioni mentre è in corso una operazione di DMA, perché può trovare istruzioni e operandi in cache. Il rallentamento della CPU durante i trasferimenti in DMA può ora considerarsi trascurabile.

Le complicazioni nascono dal fatto che le operazioni in DMA potrebbero coinvolgere parti di RAM che erano state precedentemente copiate in cache. Nel caso di cache con politica *write-back* la cache potrebbe anche contenere modifiche che non sono ancora state ricopiate in RAM. Esaminiamo i problemi,

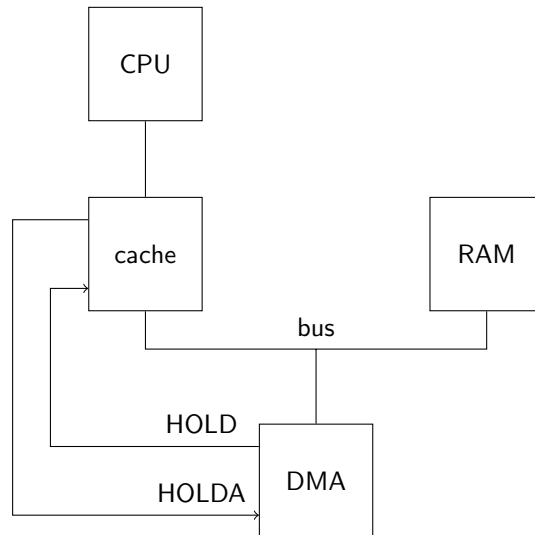


Figura 2: Interazione tra DMA e cache della CPU.

e le possibili soluzioni, partendo dal caso più semplice e passando poi a quelli più complessi. Ricordiamo che stiamo assumendo che il software, per tutta la durata del trasferimento, non acceda al buffer coinvolto nel trasferimento. Ci interessa soltanto, dunque, lo stato della cache al momento di inizio e di fine del trasferimento.

1.1.1 Cache con politica *write-through*

Questo è il caso più semplice, perché all'inizio del trasferimento tutte le cacheline eventualmente presenti in cache contengono lo stesso valore delle corrispondenti cacheline in RAM. Questo implica che non ci sono problemi nel caso di operazione di uscita in DMA (direzione da RAM a dispositivo). Nel caso di operazione di ingresso in DMA (direzione da dispositivo a RAM), invece, bisogna assicurarsi che tutte le cacheline coinvolte (anche parzialmente) nel trasferimento vengano o rimosse dalla cache, o aggiornate, altrimenti il contenuto della cache, a fine trasferimento, non sarebbe consistente con il contenuto della RAM. Il software, dunque, potrebbe leggere valori “vecchi” scambiandoli per nuovi. Il problema è che l'accesso diretto alla memoria rende non più vera l'assunzione su cui abbiamo basato il funzionamento della cache, cioè che la RAM non cambia valore se non per le scritture che arrivano dalla CPU.

Il problema può essere risolto automaticamente in hardware, oppure essere demandato al programmatore software. Nel caso di soluzione completamente hardware, dobbiamo fare in modo che il controllore cache osservi *tutte* le possibili sorgenti di scritture in RAM. Grazie al bus condiviso, possiamo sfruttare il fatto che il controllore cache può osservare cosa sta accadendo durante l'accesso

diretto alla RAM, ricevendo in ingresso le linee di controllo e di indirizzo. Questo meccanismo viene chiamato *snooping* (ficcanasare), in quanto il controllore cache “ficca il naso” in quello che sta facendo il DMA. Se le linee di controllo identificano una operazione di scrittura, il controllore può usare il contenuto delle linee di indirizzo per eseguire una normale ricerca all’interno della cache (del tutto analoga a quella eseguita quando è la CPU a chiedere un indirizzo). Nel caso di *hit*, il controllore può autonomamente provvedere a invalidare la corrispondente cacheline. Alla fine del trasferimento, se il software accede al buffer, i dati dovranno essere prelevati dalla RAM, che contiene i valori aggiornati. Se il controllore riceve in ingresso anche le linee dati può aggiornare la cacheline, invece di invalidarla. Questa operazione viene detta *snarfing* (ingurgitare) e, tipicamente, non è prevista nei processori Intel. Si tenga presente che lo snarfing è più complicato di quanto può apparire a prima vista, in quanto il dispositivo DMA potrebbe star trasferendo byte che coinvolgono solo parte di una o più cacheline, non intere cacheline.

Ci sono sistemi (per esempio quelli basati su processori ARM, comuni negli smartphone) in cui il problema non è risolto automaticamente in hardware e il programmatore deve risolverlo in software. Per farlo dispone di alcune istruzioni che gli permettono di interagire con il controllore cache, tipicamente per invalidare un intervallo di indirizzi. Il software deve eseguire queste istruzioni, specificando l’intervallo di indirizzi coinvolto nel trasferimento (nel nostro caso, gli indirizzi da b a $b + n$ escluso) subito dopo che il trasferimento sia terminato. Per le cache write-through questa operazione di invalidazione è necessaria solo per i trasferimenti in ingresso.

1.1.2 Cache con politica *write-back*

Consideriamo ora una cache con politica *write-back* e ipotizziamo che il dispositivo DMA voglia leggere o scrivere dei byte di una cacheline attualmente presente in cache (senza perdita di generalità, limitiamoci a considerare una sola cacheline alla volta). Se la cacheline non è “dirty”, la copia in cache contiene lo stesso valore che si trova nella corrispondente cacheline in RAM: ricadiamo negli stessi casi già visti per la cache di tipo write-through e possiamo adottare soluzioni analoghe. Se la cacheline è dirty, invece, ci scontriamo con dei problemi nuovi sia nelle operazioni di ingresso (da dispositivo a RAM) che nelle operazioni di uscita (da RAM a dispositivo):

- nelle operazioni di uscita, i dati più aggiornati si trovano in cache e se il DMA legge soltanto dalla RAM rischia di leggere valori vecchi;
- nelle operazioni di ingresso, il contenuto finale della RAM deve tenere conto sia dati contenuti in cache, sia di quelli che arrivano dal dispositivo DMA.

Si faccia attenzione al secondo punto, che è diverso dal problema che avevamo con la cache write-through: se la cache possiede una copia della cacheline in cui sta scrivendo il dispositivo DMA, e questa cacheline è marcata dirty, il

controllore cache non può, in generale, limitarsi a invalidare la propria copia. Infatti, ogni byte della RAM deve contenere l'ultimo valore scritto, sia se l'ultima scrittura proveniva dalla CPU, sia se proveniva dal DMA: se il dispositivo DMA non sovrascrive tutti i byte di una cacheline, gli altri byte devono continuare a memorizzare l'ultimo valore scritto dalla CPU, e questo valore si trova al momento soltanto in cache. Se il controllore cache invalidasse la propria copia senza eseguire prima un write-back, il contenuto più aggiornato di questi byte si perderebbe.

Le soluzioni hardware possibili sono diverse. Il problema in ingresso può risolversi con la tecnica di snooping adottata per le cache write-through, ma in questo caso il controllore *deve* implementare lo snarfing per le cacheline dirty. Il problema in uscita è più problematico. Infatti, è vero che il controllore cache può accorgersi tramite snooping che l'operazione di lettura dalla RAM iniziata dal dispositivo DMA coinvolge una cacheline dirty, ma cosa dovrebbe fare una volta scoperto ciò? In teoria dovrebbe essere il controllore a fornire i dati al dispositivo, invece della RAM, ma pilotare le linee dati con il valore aggiornato della cacheline comporta una corsa, in quanto anche la RAM sta vedendo la stessa operazione sul bus, e anche lei vorrà pilotare le linee dati. Occorre un coordinamento tra i tre attori coinvolti. Tipicamente il protocollo di accesso alla RAM viene modificato in modo che si svolga in più fasi temporalmente distinte:

1. c'è una prima fase, a cui la RAM non partecipa, in cui il dispositivo DMA comunica al controllore cache gli indirizzi a cui vuole accedere e il controllore cache risponde con il segnale hit/miss e l'eventuale stato del bit Dirty;
2. la prima fase è seguita da una o più fasi che dipendono dall'esito della prima.

La prima fase è in genere chiamata di snooping, ma in questo caso impropriamente, in quanto non si svolge "in segreto" come lo snooping di cui abbiamo parlato prima.

Consideriamo prima le operazioni di uscita. Se la fase di snooping è terminata con un miss o un hit non dirty, il dispositivo DMA può accedere normalmente alla RAM e il controllore cache non deve fare altro. Se la fase di snooping si è invece conclusa con un hit dirty si aprono varie possibilità, ma la più semplice è che il dispositivo DMA ceda il possesso del bus al controllore cache per permettergli di eseguire un write-back della cacheline in RAM; il dispositivo DMA può anche prelevare i dati che gli interessano mentre stanno transitando sul bus verso la RAM (in pratica eseguendo anche lui uno snooping con snarfing), altrimenti deve riacquisire il bus e rieseguire l'accesso quando il controllore cache ha terminato.

Se il controllore cache, come quelli Intel, non implementa lo snarfing, possiamo risolvere i problemi dei trasferimenti in ingresso in modo analogo a quello di uscita, prevedendo anche in questo caso una prima fase di snooping. Se la fase di snooping è terminata con miss o un hit non dirty, il dispositivo DMA può procedere con la normale scrittura in RAM e il controllore cache, in caso di

hit, deve invalidare la propria cacheline. Se la fase di snooping è terminata con un hit dirty, possiamo gestire il problema quasi esattamente come nel caso di ingresso: il dispositivo DMA cede il bus al controllore cache in modo da fargli eseguire il write-back in RAM, quindi riacquiesce il bus e riesegue la sua operazione di scrittura. A volte (come nel chipset emulato dalla nostra macchina QEMU) la soluzione è leggermente diversa: il controllore cache trasmette la cacheline dirty soltanto al dispositivo DMA (senza coinvolgere la RAM) e invalida la propria copia; il dispositivo DMA aggiorna la copia internamente con i dati che avrebbe dovuto scrivere in RAM, quindi scrive in RAM l'intera cacheline aggiornata.

Nel caso di soluzione interamente software, il programmatore può tipicamente ordinare al controllore cache di eseguire il write-back di un certo intervallo di indirizzi. Il software deve eseguire questa operazione su tutti gli indirizzi del buffer, e deve farlo necessariamente *prima* di avviare il trasferimento. Questo è ovviamente necessario per le operazioni di uscita (da RAM a dispositivo), in modo che la RAM sia correttamente aggiornata prima del trasferimento, ma è necessario anche per le operazioni di ingresso (da dispositivo a RAM), non solo per preservare il valore attuale degli eventuali byte non coinvolti nel trasferimento, ma anche per evitare che eventuali cacheline dirty non vengano ricopiate in RAM durante l'operazione o dopo che è terminata, col rischio di sovrascrivere quanto già scritto dal dispositivo. Durante il trasferimento il software deve anche fare attenzione a non toccare eventuali byte che si trovino nelle stesse cacheline del buffer, pur non appartenendovi. La cosa più semplice per assicurarsi che ciò non avvenga è di avere buffer con dimensione multipla della cacheline e allineati alla cacheline (nel qual caso si può anche adottare l'ottimizzazione descritta di seguito).

1.1.3 Trasferimento di intere cacheline

Il caso di trasferimento in ingresso con cacheline dirty può essere molto semplificato se il dispositivo DMA ha intenzione di sovrascrivere l'intera cacheline e il controllore cache ne viene informato. In questo caso, il controllore cache può limitarsi a invalidare la propria copia come se non fosse dirty, e la fase di snooping preliminare non è necessaria. Per questo scopo, il protocollo del bus prevede in genere una operazione di “Write and Invalidate” distinta dalla normale operazione di scrittura. Il dispositivo DMA può usare questa operazione ogni volta che è sicuro di star sovrascrivendo una intera cacheline. In generale, un intero trasferimento che coinvolge un buffer $[b, b + n)$ sarà composto, in base all'allineamento di b e al valore di n , da un possibile trasferimento di parte di una cacheline, seguito da zero o più trasferimenti di intere cacheline, e concluso da un eventuale trasferimento di parte di una cacheline. Il dispositivo DMA potrà usare normali scritture (con fase di snooping) soltanto per la prima e l'ultima operazione, se necessario, e usare Write and Invalidate per quelle intermedie.

Anche nel caso di soluzione puramente software il caso di sovrascrittura di intere cacheline può essere ottimizzato, se il processore dispone di una istruzione che permetta l'invalidazione *senza write back*. Il software deve comunque

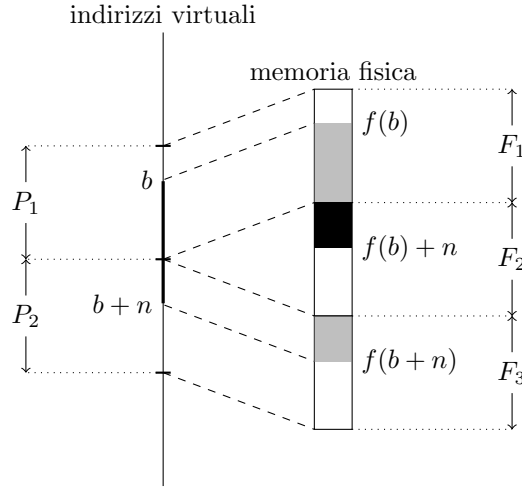


Figura 3: Un esempio con un buffer $[b, b + n]$ che attraversa due pagine (P_1 e P_2) mappate su due frame non contigui (F_1 e F_3).

usare questa operazione all'inizio del trasferimento, per evitare il write back di eventuali cacheline dirty durante o dopo l'operazione DMA.

1.2 Interazione con la memoria virtuale

Come sappiamo, tra la CPU e la cache troviamo la MMU, che traduce gli indirizzi virtuali (usati dal software) in indirizzi fisici (usati sul bus). Il dispositivo DMA, come collegato in Figura 2, non interagisce in alcun modo con la MMU e può utilizzare soltanto indirizzi fisici. Supponiamo che il software voglia eseguire un trasferimento in DMA da un buffer che si trova agli indirizzi *virtuali* $[b, b + n]$ verso un dispositivo (o viceversa). Saranno necessari i seguenti accorgimenti:

- al dispositivo andrà comunicato l'indirizzo *fisico* corrispondente a b , sia $f(b)$, e non l'indirizzo virtuale b ;
- se l'intervallo $[b, b + n]$ attraversa più pagine che non sono tradotte in frame contigui, il trasferimento deve essere spezzato in più trasferimenti in modo che ciascuno di essi coinvolga solo frame contigui;
- la traduzione degli indirizzi coinvolti in un trasferimento non deve cambiare mentre il trasferimento è in corso.

Tutti e tre i punti sono diretta conseguenza del fatto che il dispositivo non ha accesso alla traduzione da indirizzi virtuali a fisici. Consideriamo il primo punto: se comunicassimo b al dispositivo, questo lo userebbe come se fosse un indirizzo fisico, andando a leggere o scrivere in parti della memoria che non c'entrano niente con il buffer (tranne nel caso particolare in cui $f(b) = b$). Considera-

mo il secondo punto e supponiamo che $[b, b + n)$ attraversi due pagine, P_1 e P_2 , mappate su due frame non contigui, F_1 e F_3 (si veda la Figura 3). Se al dispositivo comunichiamo $f(b)$ ed n , questo leggerà (o scriverà) agli indirizzi fisici $[f(b), f(b) + n)$, invadendo dunque il frame F_2 , con effetti disastrosi. Il trasferimento, invece, deve essere spezzato in due parti: una da $f(b)$ fino alla fine di F_1 , e uno dall'inizio di F_3 fino a $f(b + n)$ escluso. Alcuni dispositivi possono essere programmati per eseguire autonomamente più trasferimenti in successione, specificando in anticipo l'indirizzo e il numero di byte di ciascun trasferimento. Anche nei dispositivi che non offrono questa funzione, comunque, è sempre possibile programmare i diversi trasferimenti uno dopo l'altro in software.

Consideriamo invece il terzo punto, immaginando di trovarci in un sistema multiprocesso che realizzi, per esempio, lo swap-in/swap-out dei processi per poter eseguire più processi di quanti ne possano entrare in RAM. Supponiamo che un processo P_1 avvii un trasferimento in DMA verso un suo buffer privato e, mentre il trasferimento è in corso, il sistema decida di eseguire lo swap-out di P_1 per caricare un altro processo P_2 al suo posto. Il dispositivo DMA è ignaro del cambiamento e continuerà leggere o scrivere agli indirizzi fisici precedentemente occupati dal buffer di P_1 e ora occupati da parti della memoria di P_2 , di nuovo con effetti disastrosi.

2 PCI Bus Mastering

La Figura 4 mostra un esempio di architettura con bus PCI. Come sappiamo, sul bus PCI ogni dispositivo (più precisamente, ogni funzione all'interno di ogni dispositivo) può comportarsi da "iniziatore" di una transazione. I dispositivi che sono in grado di essere iniziatori di transazioni sono detti *bus master*, e devono essere dotati dei collegamenti REQ/GNT verso l'arbitro, che ha il compito di coordinare l'accesso al bus tra i vari bus master. In Figura 4 abbiamo tre dispositivi collegati al bus PCI: il ponte, che è sempre bus master (in quanto deve iniziare le transazioni PCI per conto della CPU), un altro dispositivo bus master, e un terzo dispositivo che non è bus master. Solo il ponte e il secondo dispositivo hanno i collegamenti REQ/GNT con l'arbitro. Per ottimizzare i tempi, l'arbitraggio può svolgersi mentre è in corso una precedente transazione: il dispositivo che ottiene il GNT, prima di iniziare la propria transazione, deve attendere che il bus diventi libero. La condizione di bus libero si riconosce dal fatto che sia FRAME# che IRDY# sono disattivi.

Dal punto di vista software l'operazione si svolge come se il dispositivo bus master fosse collegato direttamente al bus principale; in particolare, il software dialoga soltanto con il dispositivo per comunicargli, per esempio, l'indirizzo (fisico) del buffer il numero di byte da trasferire e la direzione del trasferimento. Dal punto di vista hardware, però, tutto il trasferimento avviene per tramite del ponte. Ogni volta che il dispositivo vuole iniziare un trasferimento da o verso la RAM, inizia una transazione di memoria sul bus PCI, all'indirizzo opportuno. Il ponte è configurato in modo da comportarsi come obiettivo per tutte le

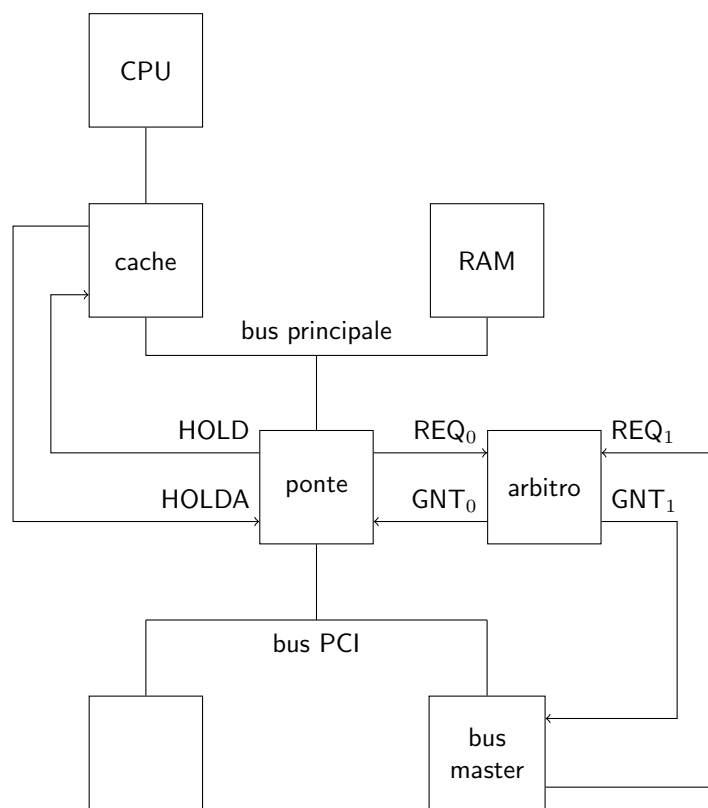


Figura 4: Bus Mastering PCI.

transazioni di memoria con indirizzi appartenenti alla RAM che si trova sul bus principale. Il ponte, quindi, risponde alla transazione iniziata dal bus master e, in caso di trasferimento da RAM a dispositivo, esegue le necessarie operazioni di lettura DMA sul bus principale; in caso di trasferimento dal dispositivo alla RAM, copia temporaneamente i dati in arrivo dal dispositivo in un buffer interno (*posted writes*) e, in parallelo, inizia le necessarie operazioni di scrittura in DMA verso la RAM. Sul bus principale il ponte si comporta esattamente come un normale dispositivo DMA e per lui valgono tutte le considerazioni che abbiamo svolto fino ad ora, in particolare per quanto riguarda il protocollo HOLD/HOLDA e le interazioni con la cache.

Si noti che il bus master può utilmente sfruttare la possibilità, offerta dallo standard PCI, di eseguire transazioni con più di una fase dati, in quanto conosce in anticipo il numero di byte da trasferire. Inoltre, anche il bus PCI prevede una transazione di tipo “Write and Invalidate” che può essere sfruttata dai dispositivi bus master per comunicare la loro intenzione di sovrascrivere intere cacheline. Lo standard PCI, però, richiede che il software comunichi ai dispositivi bus master la dimensione delle cacheline (si veda il campo “Cache Line Size” nello spazio di configurazione, Figura 4 della dispensa sul PCI); questo perché lo standard PCI non è legato ad un particolare modello di processore. Questo campo viene in genere scritto dal PCI BIOS durante la fase di inizializzazione del sistema.

2.1 Interazione con il meccanismo delle interruzioni

La presenza della bufferizzazione intermedia nel ponte crea un problema con i trasferimenti in ingresso (da dispositivo a RAM) e il meccanismo delle interruzioni. In particolare, quando il dispositivo ha trasmesso al ponte l’ultimo byte richiesto, invierà una richiesta di interruzione per segnalare il completamento del trasferimento. Ci troviamo a questo punto di fronte ad una corsa: da una parte la richiesta di interruzione, che passa dal controllore delle interruzioni e arriva alla CPU, dall’altra i byte destinati alla memoria, che passano dal ponte: è possibile che la richiesta di interruzione arrivi e sia accettata prima che il ponte sia riuscito a trasferire tutti i byte in RAM; il software potrebbe quindi accedere erroneamente al buffer quando ancora parte dei dati non sono stati aggiornati.

Anche questo problema può essere risolto in hardware o in software. Per risolverlo in software si può sfruttare il fatto che il ponte gestisce in modo strettamente FIFO tutti i trasferimenti di dati dal bus PCI verso il bus principale. La routine di interruzione, allora, prima di permettere l’accesso al buffer, potrebbe eseguire una lettura di un registro del dispositivo bus master. La risposta a questa lettura verrebbe accodata nel ponte *dopo* l’ultimo trasferimento di dati del bus master, e dunque l’istruzione di lettura verrebbe completata nella CPU necessariamente dopo che il ponte ha finito di trasferire i dati in RAM. Si noti che, dal momento che il dispositivo bus master aveva inviato una richiesta di interruzione, è probabilmente già previsto che la routine di interruzione debba leggere un qualche registro del dispositivo, per segnalare che la richiesta è stata servita: in tal caso, quest’unica lettura assolve entrambi i compiti.

Per risolvere il problema interamente in hardware, invece, si crea in genere un collegamento di handshake tra il controllore delle interruzioni e il ponte. Prima di inoltrare una qualunque richiesta di interruzione al processore, il controllore chiede l'OK al ponte; quest'ultimo non dà l'OK fino a quando non ha finito di trasferire tutti i dati ricevuti fino a quel momento. Questa è la soluzione adottata nel ponte che è emulato all'interno della nostra macchina QEMU.

Un'altra soluzione, più moderna, prevede che le richieste di interruzione non viaggino su linee separate, ma siano inoltrate come speciali transazioni sul bus PCI stesso, sotto forma di scritture a particolari indirizzi (Message Signaled Interrupts). In questo modo le richieste si accodano naturalmente ai dati e non ci sono problemi di corse.

La pipeline

G. Lettieri

13 Maggio 2025

In quest'ultima parte del corso ritorniamo a concentrarci sulla CPU. In particolare, esaminiamo alcuni aspetti avanzati dell'architettura che permettono di eseguire il flusso di istruzioni più velocemente, sfruttando le opportunità di svolgere più azioni contemporaneamente. Faremo sempre riferimento ai processori Intel, limitatamente a ciò che avviene all'interno di un singolo “core” (quindi, un unico flusso di istruzioni). Rispetto al resto del corso rimarremo però ad un livello più astratto e meno aderente ai dettagli della CPU reale, che in gran parte non sono noti (anche se molti sono stati dedotti a-posteriori tramite *reverse engineering*).

La prima e più antica tecnica che permette di velocizzare l'esecuzione di un flusso di istruzioni è quella della *pipeline* (catena di montaggio). In analogia con le catene di montaggio industriali, l'idea è di pensare alla CPU come una fabbrica che produce i risultati delle istruzioni. Al solito, conviene concentrarsi inizialmente sulle operazioni aritmetico/logiche, che hanno due operandi e producono un risultato. Per produrre questi risultati, ogni istruzione deve passare attraverso varie fasi di “lavorazione”: l'istruzione deve essere prelevata dalla memoria, decodificata, devono essere prelevati i suoi operandi, deve essere eseguita, e infine il risultato deve essere scritto nella destinazione.

Per esempio, pensiamo all'istruzione assembly

```
add %rax, 16(%rbx, %rcx, 8)
```

In memoria, l'istruzione è codificata con i byte 48 01 44 cb 10. Questi byte devono essere prelevati dalla memoria e portati in un registro interno del processore; quindi devono essere decodificati per capire che si tratta di una operazione di somma con due operandi: il primo si trova nel registro `rax` e il secondo si trova in memoria, ad un indirizzo che deve essere calcolato sommando 16 al contenuto di `rbx` e al contenuto di `rcx` moltiplicato per 8; una volta prelevati i due operandi, deve essere eseguita la somma e il risultato deve essere infine scritto in memoria allo stesso indirizzo calcolato in precedenza. Ora, possiamo immaginare che la rete logica che esegue la somma (la ALU) sia completamente distinta dalla rete logica che decodifica l'istruzione; quindi, per tutto il tempo in cui la CPU sta calcolando la somma, la rete di decodifica resta inutilizzata. L'idea è di usarla per iniziare a decodificare la prossima istruzione, mentre l'istruzione precedente non è stata ancora completata.

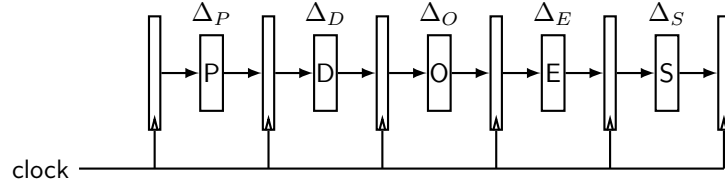


Figura 1: Esempio di pipeline

Più in generale, ci saranno altre reti logiche specializzate per compiti particolari, come accedere alla memoria o ai registri. Mentre una istruzione sta attraversando una certa fase del suo tragitto all'interno della CPU, sta occupando una sola particolare rete logica, mentre le altre sono al momento inutilizzate, e potrebbero essere utilizzate per svolgere il loro compito su altre istruzioni. Nel caso ideale sarà possibile identificare una serie di fasi attraverso cui tutte le istruzioni devono passare e tali che ogni fase usa una rete indipendente dalle altre. Nella metafora della catena di montaggio, ciascuna di queste reti è un operaio specializzato nel suo compito, gli operai sono in fila davanti al nastro, e il nastro trasporta la sequenza di istruzioni da elaborare. Per fissare le idee, supponiamo che le fasi siano quelle dell'esempio precedente: prelievo (P), decodifica (D), prelievo operandi (O), esecuzione (E) e scrittura risultato (S). Lo stato di una pipeline viene in genere illustrato nel modo seguente:

	t_0	t_1	t_2	t_3	t_4	t_5	t_6	t_7	t_8
i	P	D	O	E	S				
$i + 1$		P	D	O	E	S			
$i + 2$			P	D	O	E	S		
$i + 3$				P	D	O	E	S	
$i + 4$					P	D	O	E	S

Ogni riga della tabella mostra l'elaborazione di una singola istruzione nel tempo, mentre ogni colonna mostra lo stato della pipeline ad un certo periodo di clock. Nel periodo t_0 la pipeline contiene solo l'istruzione i , nello stadio P. Nel periodo successivo (t_1) l'istruzione i si sposta nello stadio D, liberando lo stadio P che viene occupato dall'istruzione successiva, $i + 1$. Nel periodo t_4 tutti gli stadi sono occupati, ciascuno da una istruzione diversa. Alla fine del periodo t_4 l'istruzione i viene completata, seguita dall'istruzione $i + 1$ alla fine del periodo t_5 , e così via.

Per realizzare la pipeline possiamo organizzare il processore come mostrato in Figura 1. Nella Figura, i blocchi P, D, O, E e S rappresentano le reti combinatorie che svolgono il corrispondente compito. Tra ogni coppia di reti inseriamo dei *registri di pipeline* che alla fine di ogni periodo di clock registrano il risultato dello stadio della pipeline che li precede e, all'inizio del clock successivo, lo rendono disponibile allo stadio della pipeline che li segue. Il periodo di clock Δ deve essere scelto in modo che sia maggiore del tempo di attraversamento di tutte le

reti combinatorie (più il tempo di setup dei registri, che assumiamo uguale per tutti i registri):

$$\Delta = \max\{ \Delta_P, \Delta_D, \Delta_O, \Delta_E, \Delta_S \} + t_{setup}.$$

Cosa guadagniamo rispetto ad un processore che faccia le stesse cose, ma senza pipeline? Se il processore senza pipeline esegue le varie fasi (P, D, O, E, S) in cicli di clock diversi, completa una istruzione ogni 5 periodi di clock, mentre il processore con pipeline (nel caso ideale) ne completa una ogni ciclo di clock, quindi il numero di istruzioni per unità di tempo aumenta di 5 volte, pari al numero di stadi della pipeline (detto anche *profondità della pipeline*). Se il processore senza pipeline esegue tutte le azioni in un unico periodo di clock, il suo periodo di clock deve essere almeno pari alla *somma* $\Delta_P + \Delta_D + \Delta_O + \Delta_E + \Delta_S$. In questo caso la pipeline ci permette di aumentare la frequenza di clock, ottenendo anche in questo caso un aumento del numero di istruzioni completate per unità di tempo, con un fattore praticamente uguale al numero di stadi della pipeline (purché tutti gli stadi abbiano un tempo di attraversamento all'incirca uguale).

1 Le alee

Ovviamente il caso in cui si riesce a completare una nuova istruzione ad ogni ciclo di clock è solo il caso ideale: in pratica ci sono varie condizioni che impediscono di tenere questo ritmo in modo costante. Un primo problema è dato dalle *alee*, situazioni che, se non correttamente gestite, possono portare ad una esecuzione errata del programma rispetto a come sarebbe stato eseguito su un processore senza pipeline. Le alee si distinguono in:

- alee strutturali;
- alee sui dati;
- alee sul controllo.

1.1 Alee strutturali

Le alee strutturali si verificano quando due o più istruzioni che si trovano in due stadi diversi della pipeline cercano di accedere contemporaneamente alla stessa risorsa. Per esempio, una istruzione i con un operando sorgente in memoria si trova nello stadio O, e un'altra istruzione j con operando destinazione in memoria si trova nello stadio S: entrambe cercherebbero di accedere contemporaneamente alla memoria. Le alee strutturali si possono risolvere introducendo degli “stalli” o “bolle” nella pipeline: nel caso di esempio, si ferma l'istruzione i nello stadio D fino a quando l'istruzione j non è uscita dallo stadio S. Si noti che mentre i è ferma nello stadio D, anche lo stadio precedente, P, deve fermarsi. In generale, data la natura strettamente sequenziale della pipeline, se una istruzione è ferma in un qualche stadio, anche tutti gli stadi precedenti si devono fermare. Nel periodo di clock in cui i sarebbe dovuta entrare nello

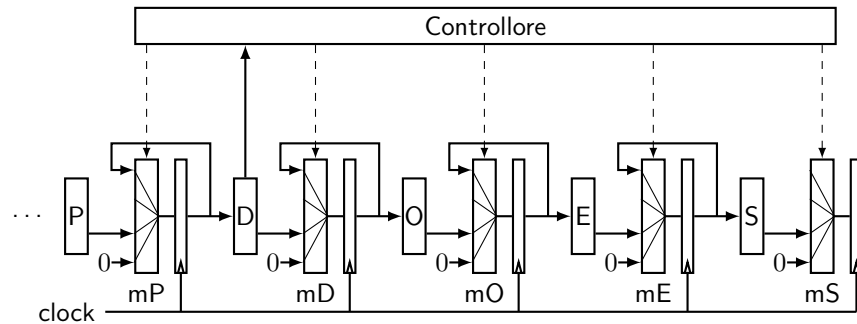


Figura 2: Circuito di controllo della pipeline

stadio O, lo stadio O sarà occupato da una cosiddetta “bolla”—essenzialmente una nop.

Operativamente, possiamo organizzare la pipeline come in Figura 2, dove abbiamo inserito un multiplexer di fronte ad ogni registro di pipeline, in modo che ogni registro possa ricevere o l’uscita dello stadio precedente, o la sua stessa uscita dal periodo precedente, o una costante che codifica la bolla (rappresentata con 0 in Figura). Aggiungiamo poi un controllore che tiene traccia di tutte le istruzioni che entrano nella pipeline (una volta decodificate) e, per ogni periodo, decide quale via attivare in ogni multiplexer (nella Figura abbiamo ommesso per il momento il circuito che deve precedere lo stadio P e decide da quale indirizzo prelevare la prossima istruzione). Per esempio, supponiamo che al clock t lo stadio E contenga una istruzione i che dovrà scrivere in memoria quando raggiunge lo stadio S, e che lo stadio D abbia decodificato una istruzione j che dovrà usare la memoria quando raggiunge lo stadio O. Il controllore riconosce l’alea strutturale e, per il periodo di clock $t + 1$, seleziona la via in alto del multiplexer mP (mantieni lo stato corrente), la via in basso del multiplexer mD (inserisci una bolla) e le vie normali dei restanti multiplexer. Durante il clock $t + 1$ l’istruzione j passa nello stadio S, dove esegue la sua scrittura in memoria indisturbata, mentre l’istruzione i è ferma nello stadio D. Per il clock $t + 2$ il controllore selezionerà tutte le vie normali, e l’istruzione i potrà riprendere il suo cammino.

Una volta introdotta nella pipeline, la bolla deve poi attraversare tutti gli stadi fino all’uscita: ogni bolla introdotta riduce quindi il numero di istruzioni completate per unità di tempo, e i progettisti delle pipeline cercano sempre di ridurle al minimo. Nel caso delle alea strutturali, una strategia che permette di ridurre le bolle/stalli è quella di aumentare le risorse hardware. Per esempio, nella pipeline di Figura 1 abbiamo un’alea strutturale con tutte le istruzioni che accedono in memoria, in quanto lo stadio P deve sempre accedere anch’esso alla memoria per prelevare altre istruzioni. Questa alea può essere risolta introducendo cache separate per le istruzioni e per i dati.

Un’altra strategia molto efficace consiste nel progettare il set di istruzioni direttamente in funzione della sua elaborazione tramite una pipeline. In partico-

lare, ogni istruzione dovrebbe fare poche cose semplici, affidando al compilatore il compito di codificare le operazioni più elaborate usando più istruzioni. I processori progettati in questo modo sono chiamati RISC, per Reduced Instruction Set Computer, e i loro ideatori li contrapposero ai processori CISC, per Complex Instruction Set Computer. I processori RISC sono nati nelle università negli anni '80 del secolo scorso; la loro adozione non è stata immediata, e anzi ne nacque una lunga polemica, ma alla fine l'architettura RISC è diventata quella prevalente al giorno d'oggi (anche nel caso dell'Intel, come vedremo).

Un set di istruzioni di tipo RISC prevede, tipicamente:

- istruzioni tutte della stessa grandezza (per es. 4 byte);
- pochi e semplici formati;
- istruzioni operative che lavorano esclusivamente sui registri, o al massimo su operandi immediati;
- istruzioni separate di load/store per accedere alla memoria, senza fare altre elaborazioni.

La tipica istruzione operativa RISC ha il formato “*op src₁, src₂, dst*”, dove *op* è il codice operativo e *src₁*, *src₂* e *dst* sono due registri sorgenti e una destinazione. Una istruzione di load può avere il formato “*ld offset(base), dst*” e una store “*st src, offset(base)*”, dove *src*, *dst* e *base* sono registri. Le istruzioni di load e store completano il loro accesso in memoria nello stadio E, e dunque non entrano mai in conflitto tra loro e con le altre istruzioni. Nello stadio O vengono solo letti registri e nello stadio S solo scritti registri, e le due operazioni possono essere svolte contemporaneamente nello stesso periodo di clock.

Il fatto che le istruzioni abbiano tutte la stessa grandezza semplifica lo stadio P, che può autonomamente calcolare l'indirizzo della prossima istruzione da prelevare, senza aspettare che questa venga decodificata (ovviamente se non si tratta di un salto, si vedano le alee sul controllo più avanti).

1.2 Alee sui dati

Le alee sui dati si presentano quando una istruzione potrebbe leggere un dato diverso rispetto a quello che avrebbe letto in un processore senza pipeline.

Consideriamo per esempio la sequenza di istruzioni (RISC)

```
add r1, r2, r3
sub r3, r4, r5
```

L'istruzione `sub` legge il registro `r3`, che in un processore senza pipeline dovrebbe contenere il risultato della `add` precedente. Nella pipeline, però, il registro `r3` verrà aggiornato alla fine dello stadio S, mentre l'istruzione `sub` ne ha bisogno all'inizio dello stadio O. Quando l'istruzione `sub` si trova nello stadio O, l'istruzione `add` si trova ancora nello stadio E, e devono passare ancora due cicli di clock prima che `r3` venga aggiornato. Se non si prendessero provvedimenti, la `sub` leggerebbe il *vecchio* contenuto di `r3`, non il valore calcolato dalla `add`.

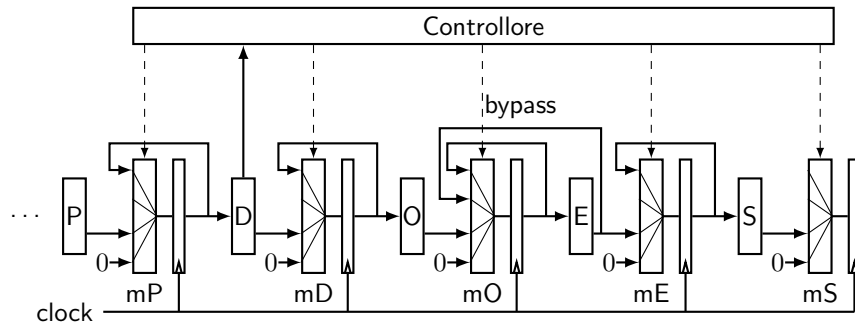


Figura 3: Circuito di bypass

Anche le alee sui dati si possono risolvere introducendo bolle. In questo caso il controllore dovrebbe tenere ferma la sub per due cicli nello stadio D, introducendo due bolle. In generale, tutte le alee possono risolversi introducendo bolle: al limite, se inseriamo 4 bolle dopo ogni istruzione, siamo ritornati al processore senza pipeline, che sicuramente funziona. Ovviamente, però, perderemmo tutti i vantaggi in termini di prestazioni.

Per risolvere le alee sui dati senza introdurre bolle possiamo aggiungere un circuito di *bypass* come in Figura 3. L'idea è che le istruzioni non hanno veramente bisogno di eseguire una lettura da un particolare registro, ma solo di ricevere il valore calcolato da una precedente istruzione. Nell'esempio precedente, la sub ha solo bisogno del risultato della add, e questo è già disponibile alla fine del ciclo di clock in cui la add si trova nello stadio E. Il controllore può a questo punto selezionare la via del bypass nel multiplexer mO, in modo che, al clock successivo, quando la sub si trova nello stadio E, riceva il risultato della precedente add al posto del valore letto dallo stadio O (il circuito in Figura è puramente indicativo: il circuito completo deve tenere conto del fatto che un operando arriva dal bypass e l'altro dallo stadio O).

1.3 Alee sul controllo

Le alee sul controllo si verificano quando il processo con pipeline potrebbe eseguire istruzioni diverse da quelle eseguite dal processore senza pipeline. Questo può succedere quando viene prelevata una istruzione di salto condizionale, o una istruzione di salto indiretto. In entrambi i casi l'indirizzo dell'istruzione successiva diventa noto solo quando l'istruzione di salto si trova più avanti nella pipeline, mentre lo stadio P ha bisogno di saperlo subito.

Anche qui il controllore della pipeline può risolvere il problema introducendo un numero sufficiente di bolle, ma ancora una volta si può provare a fare di meglio. In questo caso, il processore deve tentare di indovinare quale sarà la prossima istruzione, prima di eseguire completamente l'istruzione di salto, e continuare a prelevare istruzioni dall'indirizzo "predetto". Dopo alcuni cicli di clock, l'istruzione di salto sarà arrivata allo stadio in cui viene calcolata la

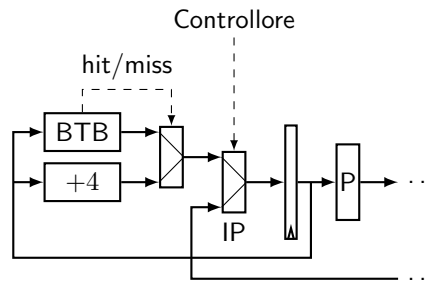


Figura 4: Branch Target Buffer

vera destinazione (per esempio nello stadio E). A quel punto il controllore può confrontare la vera destinazione con quella che era stata predetta: se aveva indovinato va tutto bene e le istruzioni prelevate nel frattempo possono proseguire attraverso la pipeline; se aveva sbagliato, per fortuna nessuna delle istruzioni prelevate ha ancora avuto effetti permanenti (tipo scritture in memoria o nei registri, che avvengono solo negli stadi E ed S), quindi possono essere tutte annullate (sostanzialmente trasformandole in bolle) e lo stadio P può ripartire a prelevare dall'indirizzo corretto. La bontà di questa strategia dipende da quanto efficacemente si riesce a prevedere le destinazioni dei salti, e molte risorse sono state investite nel tentativo di migliorare questo aspetto.

La strategia più semplice è di fare una previsione statica. Per esempio, se il salto è all'indietro, si può assumere che il programma contenga un ciclo e che questo ciclo venga effettivamente eseguito un po' di volte, quindi si può predire che il salto verrà fatto. Viceversa, se il salto è in avanti, si può assumere che il programmatore (o il compilatore) abbia organizzato il codice in modo che il caso più frequente sia quello in cui il salto non viene fatto, e dunque si può prevedere che non verrà fatto.

Strategie più sofisticate prevedono di ricordare in un nuovo dispositivo di cache, detto Branch Target Buffer (BTB), informazioni sui salti che sono stati osservati in passato, e basare la previsione su queste informazioni. La Figura 4 mostra una schematizzazione della parte di pipeline che pilota lo stadio di prelievo. Lo stadio P preleva l'istruzione all'indirizzo contenuto nell'Instruction Pointer (`rip` nei processori Intel). Il controllore decide se aggiornare l'Instruction Pointer con il valore predetto (via alta del secondo multiplexer) o il valore calcolato dalla pipeline (via bassa). Il valore predetto è ottenuto o dal valore precedente incrementato di 4 (ipotizzando istruzioni RISC) o, se il valore precedente è stato trovato nel BTB, dal valore predetto dal BTB stesso.

Una strategia molto semplice per le previsioni del BTB è di ricordare soltanto cosa l'istruzione aveva fatto l'ultima volta che era stata eseguita, e rifare la stessa cosa. Questa strategia non è molto efficace, perché in presenza di un ciclo sbaglia più del necessario: sbaglia sicuramente quando si esce dal ciclo, ma sbaglia di nuovo anche se il programma ripassa in seguito dallo stesso ciclo (perché ricorda che l'ultima volta il salto non era stato fatto).

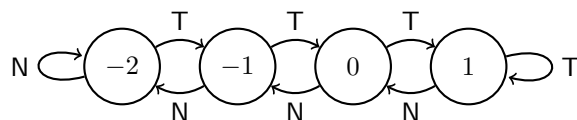


Figura 5: Predittore di salto (2 bit con saturazione).

Un metodo più efficace, che è anche quello comunemente usato, è di usare due bit organizzati come un contatore con saturazione. L'idea è illustrata in Figura 5: il contatore è inizialmente a zero e, ogni volta che il salto viene eseguito (rami T per *Taken*) il contatore viene aumentato (saturando a 1); ogni volta che il salto non viene eseguito (rami N per *Not taken*) il contatore viene decrementato. Il BTP predurrà che il salto non verrà fatto se il contatore è negativo, e fatto altrimenti. Nell'esempio precedente, il contatore arriverà presto a 1 mentre il ciclo viene eseguito e sbaglierà una volta all'uscita dal ciclo, ma questa volta riporterà il contatore a zero e, se lo stesso ciclo viene incontrato una seconda volta, predurrà correttamente che il salto verrà effettuato. I predittori moderni basano le loro previsioni sulla storia degli ultimi salti, memorizzata in degli shift-register in cui ogni bit rappresenta l'esito di un salto.

Notare che le istruzioni `ret` sono istruzioni di salto indiretto, e per sapere a quale indirizzo devono essere prelevate le prossime istruzioni occorre eseguire una operazione di lettura in memoria. Le istruzioni `ret` possono però essere trattate a parte, perché nei casi normali sono strettamente associate alle `call`. Molti processori (inclusi gli Intel) hanno un Return Address Stack (RAS), che è un buffer interno al processore, organizzato a pila. Ogni istruzione `call` inserisce l'indirizzo di ritorno in cima alla pila e ogni `ret` estrae dalla cima della pila. Notare che si tratta comunque di una previsione, sia perché i programmi sono liberi di usare le `ret` anche in modi non legati alle `call`, sia perché il RAS ha comunque una dimensione limitata e ricorda solo un piccolo numero degli ultimi indirizzi di ritorno.

2 La pipeline nei processori Intel

Una delle operazioni più complesse nell'elaborazione del set di istruzioni Intel è la decodifica: le istruzioni hanno lunghezza variabile e la codifica ha tanti casi particolari da gestire. Per esempio, una istruzione come “push registro” è codificata su un solo byte con valore esadecimale 50 più il codice del registro, secondo la corrispondenza `rax` = 0, `rcx` = 1, `rdx` = 2, `rbx` = 3, `rsp` = 4, `rbp` = 5, `rsi` = 6 e `rdi` = 8. Quindi, per esempio, “push `rbp`” è codificata con 55. Per usare i registri (o le parti di registro) aggiunti da AMD nel processore a 64 bit bisogna usare il cosiddetto prefisso REX, che ha un valore esadecimale da 40 a 4F¹. Per esempio, l'istruzione “push `r9`” si codifica su due byte, 41 51.

¹Nel processore a 32 bit questi byte codificavano le istruzioni “inc registro” e “dec registro” su un byte; nel processore a 64 bit queste istruzioni devono essere codificate su più byte.

Il prefisso REX va utilizzato anche se si vogliono usare operandi a 64 bit nelle istruzioni i cui operandi sono per default a 32 bit (praticamente tutte tranne le `push`, `pop` e poche altre). Quasi tutte le istruzioni che specificano due operandi hanno un codice operativo (su 1, 2 o 3 byte) e devono usare un ulteriore byte, detto ModR/M, che permette di specificare un registro (secondo la codifica di cui sopra, su 3 bit) e un altro operando che può essere a sua volta in un registro o in memoria. Il byte ModR/M permette di codificare gli indirizzamenti diretti, indiretti tramite registro, e gli indirizzamenti con base più offset; l'offset può essere codificato su 1, 2 o 4 byte e deve seguire il byte ModR/M. Se si utilizza un indirizzamento con registro indice (con o senza scala), è necessario un ulteriore byte, detto SIB (Scale Index Base). Se l'istruzione ha un operando immediato, anche questo può essere codificato su 1, 2 o 4 byte e si trova alla fine dell'istruzione. A tutto ciò aggiungiamo il fatto che l'istruzione può essere preceduta da uno o più byte di prefisso (come `rep`).

Per esempio, vediamo cosa significano i byte 48 01 44 cb 10, che codificano l'istruzione `add %rax, 16(%rbx, %rcx, 8)`. Il byte 48 è un prefisso REX, necessario perché l'istruzione usa operandi a 64 bit e la `add`, per default, ha operandi a 32 bit. Il byte 01 è il codice operativo della `add`, che specifica anche l'ordine degli operandi (la stessa `add` a operandi invertiti ha codice operativo 03 e tutto il resto uguale); segue il byte ModR/M che ha tre campi. Scritti in binario e partendo dal bit più significativo, i campi sono: Mod = 01, reg = 000 e r/m = 100. Il campo reg indica il registro `rax`, mentre la combinazione Mod = 01 e r/m = 100 indica la presenza di un byte SIB e di un offset codificato su un byte; il byte SIB è cb e l'offset è 10 (pari a 16); anche il byte SIB ha tre campi, che scritti in binario partendo dal bit più significativo sono: S = 11, I = 001 e B = 011, vale a dire S = 3, I = 1 e B = 3. I tre campi codificano una scala di 8 (2^3), il registro indice (I) `rcx` e il registro base (B) `rbx`.

Anche se i processori Intel hanno un set di istruzioni molto complesso e non pensato per la realizzazione di una pipeline, Intel riuscì ad introdurre una pipeline a 5 stadi nel processore 80486 del 1989. All'epoca il processore era ancora a 32 bit e mancavano molte delle estensioni aggiunte in seguito, come le istruzioni SSE. Lo schema con codice operativo, ModR/M e SIB era però già presente. Per gestirne la complessità, lo stadio di fetch della pipeline del 486 preleva sempre 16 byte allineati naturalmente, senza sapere dove iniziano e dove finiscono le istruzioni al suo interno. Cerca sempre di mantenere un buffer di due di questi blocchi da 16 byte consecutivi, perché una istruzione potrebbe trovarsi anche a cavallo di un confine di 16 byte. Lo stadio di decodifica è scomposto in due sottostadi, D1 e D2. D1 decodifica al più 3 byte dell'istruzione e fornisce allo stadio D2 informazioni su come procedere con il resto della decodifica.

Una ulteriore complicazione è data dal fatto che le istruzioni Intel richiedono tempi molto diversi tra loro: una `add` tra registri può essere completata in un clock dello stadio di esecuzione, ma una `add` come quella esaminata sopra, che deve leggere e poi anche riscrivere in memoria, richiede 3 clock nello stadio di esecuzione. Il circuito di controllo della pipeline del 486 deve quindi poter mantenere una istruzione nello stesso stadio anche per più di un clock, mettendo

in stallo le istruzioni precedenti. Questo accade anche nello stadio D2 della decodifica, se l'istruzione contiene offset o operandi immediati.

Pur con tutte queste limitazioni e complicazioni, il 486 comunque riuscì ad andare circa al doppio della velocità del suo predecessore (80386) anche a parità di frequenza di clock, e riuscì anche ad essere competitivo con i processori RISC dell'epoca.

Esecuzione fuori ordine

G. Lettieri

3 Giugno 2025

Il limite principale della pipeline è la sua natura strettamente sequenziale: se una istruzione si blocca in uno stadio, tutti gli stadi precedenti devono fermarsi. Tra i casi più problematici, si pensi ad una istruzione che deve accedere alla memoria nel caso in cui l'informazione cercata non si trovi in cache: l'istruzione dovrà restare bloccata nello stadio in cui ha eseguito l'accesso per tutto il tempo necessario a completare la lettura dalla memoria, e tutte le istruzioni successive (che si trovano in stadi precedenti della pipeline) dovranno aspettare. Con la tecnica dell'esecuzione fuori ordine, invece, il processore può esaminare le istruzioni successive e, se non “dipendono” da quella bloccata, farle andare avanti, in modo da poter svolgere lavoro utile mentre è in corso l'accesso in memoria.

Ovviamente dobbiamo capire cosa vuol dire che una istruzione dipende da un'altra. Vediamo prima un esempio: consideriamo un frammento di programma che calcoli una somma vettoriale:

```
for (int i = 0; i < 1000; i++)  
    a[i] = b[i] + c[i];
```

Il programma deve calcolare

```
a[0] = b[0] + c[0];  
a[1] = b[1] + c[1];  
...  
a[999] = b[999] + c[999];
```

ma non ha alcuna importanza che queste istruzioni vengano eseguite esattamente in quest'ordine: il risultato sarà lo stesso qualunque sia l'ordine. Supponiamo, per esempio, che `b[0]` non si trovi in cache: questo vuol dire che il calcolo del valore da scrivere in `a[0]` deve aspettare che `b[0]` venga prelevato dalla memoria, ma nel frattempo è del tutto lecito provare ad eseguire le istruzioni successive, perché il loro risultato *non dipende* da quanto vale `b[0]`.

L'idea può essere estesa ulteriormente: non solo non è necessario che le varie somme siano eseguite nell'ordine che aveva specificato il programmatore, ma non è neanche necessario che vengano eseguite sequenzialmente. Se avessimo 1000 ALU, potremmo anche eseguirle tutte contemporaneamente.

1 Dipendenze tra le istruzioni

Vediamo più precisamente cosa si intende per dipendenza tra istruzioni. Facciamo riferimento alle istruzioni di tipo RISC e ad un flusso di esecuzione, così come lo genererebbe un processore che esegua le istruzioni nell'ordine specificato dal programma. Consideriamo una istruzione i ed una istruzione j che la segue nell'ordine del programma (non necessariamente immediatamente dopo). Per il momento consideriamo solo le istruzioni operative “ op , $src1$, $src2$, dst ” e quelle di salto, tralasciando le istruzioni di accesso alla memoria. Diremo che j dipende da i :

- per i *dati* se j usa direttamente o indirettamente il risultato prodotto da i ;
- per i *nomi* se j scrive in uno qualunque dei registri a cui accede i (sia sorgenti che destinazione);
- per i *controllo* se j può essere eseguita o meno in base al risultato prodotto da i .

Vediamo un esempio di dipendenza (diretta) sui dati:

```
add x1, x2, x3
// altre istruzioni che non scrivono in x3
sub x4, x3, x5
```

Consideriamo come j l'istruzione `sub` e i l'istruzione `add`. L'istruzione `add` lascia il proprio risultato nel registro `x3`, e questo stesso risultato è letto dalla `sub`. Quindi la `sub` dipende (direttamente) per i dati dalla `add`: non possiamo eseguire la `sub` fino a quando la `add` non ha calcolato il suo risultato.

Vediamo invece quest'altro caso:

```
add x1, x2, x3
// altre istruzioni
sub x4, x5, x1
// altre istruzioni
mul x6, x7, x3
```

Consideriamo come i sempre l'istruzione `add` e come j la `sub`. L'istruzione `sub` scrive in uno dei registri dell'istruzione `add` (`x1`). Lo stesso vale se prendiamo come j l'istruzione `mul` (in questo caso il registro è `x3`). In entrambi i casi l'istruzione j scrive in uno dei registri della `add`, e dunque j dipende da i per i nomi. Se, per esempio, eseguiamo la `sub` prima della `add`, o mentre la `add` non è ancora terminata, la `add` potrebbe usare il valore scorretto di `x1`. Nel caso della `mul` il problema sarebbe diverso: se eseguiamo la `mul` prima della `add`, l'ultimo valore scritto in `x3` (da cui magari dipendono per i dati altre istruzioni successive) sarebbe sbagliato.

Possiamo riassumere le dipendenze sui dati e sui nomi nel seguente schema, in cui prendiamo in considerazione due istruzioni i e j , con j successiva ad i

nell'ordine del programma, tali che entrambe usino uno stesso registro come operando sorgente (quindi in lettura, abbreviato in “r”) o come destinatario (scrittura, “w”):

$j \backslash i$	r	w
r	–	dati
w	nomi ¹	nomi ²

Se entrambe le istruzioni leggono il registro non c'è dipendenza. Se i scrive e j legge lo stesso registro, j dipende da i per i dati. Se j scrive, allora dipende da i per i nomi. Il caso 1 (i legge e j scrive) viene detto anche *antidipendenza*, mentre il caso 2 (entrambe scrivono) è detto *dipendenza in uscita*. Nell'esempio precedente, l'istruzione `sub` ha una antidipendenza con l'istruzione `add`, mentre l'istruzione `mul` ha una dipendenza in uscita.

Infine, consideriamo le seguenti istruzioni:

```
jz x1, avanti
add x2, x3, x4
avanti:
sub x5, x6, x7
```

L'istruzione `add` può essere eseguita oppure no, in base al risultato dell'istruzione `jz`: diciamo che la `add` dipende per il controllo dalla `jz`.

A differenza delle dipendenze sui nomi e sui dati, che possono essere scoperte osservando un singolo flusso di esecuzione di un programma, le dipendenze sul controllo richiedono la conoscenza dell'intero programma. Nell'esempio precedente la `sub` viene eseguita in ogni caso, e dunque non dipende per il controllo dalla `jz`, ma ci basta modificare il programma nel seguente modo:

```
jz x1, avanti
add x2, x3, x4
jmp fine
avanti:
sub x5, x6, x7
fine:
```

ed ecco che anche la `sub` può essere eseguita oppure o no, in base al risultato della `jz`, e dunque dipende per il controllo da `jz`. Questa differenza è importante perché noi vorremmo che fosse il processore stesso a scoprire le dipendenze, ma il processore non conosce il programma: conosce solo le istruzioni che preleva, e dunque osserva solo un flusso di esecuzione (o processo, in senso astratto). In particolare, se la `jz` effettua il salto, il processore vedrà che la prossima istruzione è la `sub`, senza sapere quali altre istruzioni sono state saltate, e dunque senza poter distinguere i due casi qui sopra. Il processore è dunque costretto a fare una approssimazione molto cruda: appena preleva una istruzione di salto condizionale, deve assumere che da quel momento in poi *tutte* le istruzioni dipendano per il controllo da essa.

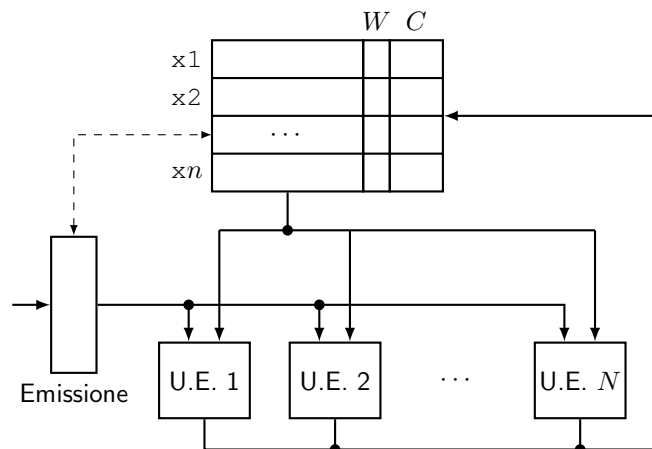


Figura 1: Architettura per l'esecuzione fuori ordine (caso base)

Quando abbiamo parlato di pipeline abbiamo introdotto le alee sui dati e sul controllo: attenzione a non confonderle con le omonime dipendenze. Le dipendenze sono una proprietà del programma da eseguire, mentre le alee sono delle situazioni indesiderate che si possono presentare o meno in base a come è strutturato il processore. Le dipendenze, se non correttamente gestite, possono portare a delle alee.

2 Stadio di emissione e dashboard

Introduciamo ora una architettura che ci permette di eseguire le istruzioni fuori ordine o in parallelo, tenendo traccia delle dipendenze. Per il momento continuiamo a preoccuparci solo delle istruzioni operative e di salto di tipo RISC, che hanno operandi esclusivamente di tipo registro (o al massimo di tipo immediato). Partendo dalla pipeline che abbiamo già visto, sostituiamo gli stadi di lettura operandi, esecuzione e scrittura operandi con l'architettura di Figura 1. Introduciamo:

- un nuovo stadio di Emissione, che riceve le istruzioni, ancora in ordine, dallo stadio di decodifica, e deve decidere se lasciarle proseguire oppure no;
- più di una unità di esecuzione, ciascuna in grado di eseguire una istruzione in parallelo con le altre unità;
- una *dashboard* che contiene informazioni su ciascun registro, oltre ai registri stessi.

Le istruzioni che si trovano negli stadi di esecuzione possono essere completate in qualsiasi ordine, scrivendo il loro risultato nel rispettivo registro di destinazione.

Le dipendenze tra queste istruzioni vanno tracciate e gestite in modo da evitare eventuali alee. Il compito di evitare le alee spetta allo stadio di emissione, ma ci sono molti modi in cui può farlo, più o meno efficientemente.

Partiamo da un caso base, che possiamo poi migliorare. Nel caso base, ogni volta che riceve una nuova istruzione, sia j , lo stadio di emissione deve verificare che j non abbia dipendenze con nessuna delle istruzioni che si trovano negli stadi di esecuzione; in caso contrario lo stadio va in stallo (bloccando anche gli stadi precedenti) fino a quando l'alea non si risolve, altrimenti *emette* l'istruzione e passa a valutare la successiva. L'emissione comporta in particolare che l'istruzione j viene assegnata ad uno stadio di esecuzione libero e inizia la sua esecuzione. Se ci fosse una istruzione i da cui j dipende e lo stadio di emissione non bloccasse j , si genererebbe un'alea, in quando j , una volta emessa, potrebbe terminare la propria esecuzione prima di i . Si noti che le uniche dipendenze che possono generare alee sono però solo queste, tra l'istruzione j appena arrivata e quelle che si trovano ancora negli stadi di esecuzione. Eventuali dipendenze tra j e istruzioni già completate non possono generare alee, in quanto j verrà completata sicuramente dopo di esse.

Per riconoscere la presenza di dipendenze, lo stadio di emissione consulta la dashboard, che contiene due informazioni per ogni registro: un flag W che vale 1 se una qualche istruzione in esecuzione ha quel registro come destinatario, e un contatore C che conta quante istruzioni in esecuzione hanno quel registro come sorgente. Quando una istruzione viene emessa, incrementa i campi C dei suoi registri sorgenti e setta a 1 il campo W del suo registro destinatario. Quando una istruzione viene completata, decrementa i campi C dei propri registri sorgenti e riporta a 0 il campo W del proprio registro destinatario.

Supponiamo che lo stadio di emissione riceva una istruzione j del tipo “*op, src1, src2, dst*”. Lo stadio consulta i campi W e C del registro *dst*: se uno o entrambi sono diversi da zero, l'istruzione j dipende per i nomi da una o più istruzioni attualmente in esecuzione (dipendenza in uscita se $W = 1$ o antidipendenza se $C \neq 0$), ed emetterla potrebbe causare una o più alee. Per evitarle, lo stadio di emissione entra in stallo fino a quando entrambi i campi W e C di *dst* non tornano a zero. Si noti che questo implica che, ad ogni istante, ciascun registro è scritto da al più una istruzione in esecuzione. Lo stadio consulta anche le entrate della dashboard relative ai registri *src1* e *src2* di j . Se il campo W di *src1* vale 1, una qualche istruzione i , ancora in esecuzione, ha come destinatario lo stesso registro da cui j vuole leggere, e dunque j dipende per i dati da i . L'istruzione j non può essere emessa in questo momento, perché altrimenti avremmo una alea sui dati. Anche in questo caso lo stadio di emissione entra in stallo fino a quando W non passa a zero. Lo stesso vale per il registro *src2*. Riassumendo, il caso base prevede che lo stadio di emissione emetta le istruzioni operative solo quando il registro destinatario non è attualmente usato ($W = 0$ e $C = 0$) ed entrambi i registri sorgenti non sono oggetto di scrittura ($W = 0$).

Se lo stadio di emissione riceve una istruzione di salto condizionale, controlla che il registro sorgente non abbia W pari a 1 (dipendenza sui dati), altrimenti entra in stallo fino a quando W non passa a zero. Una volta emessa l'istruzione,

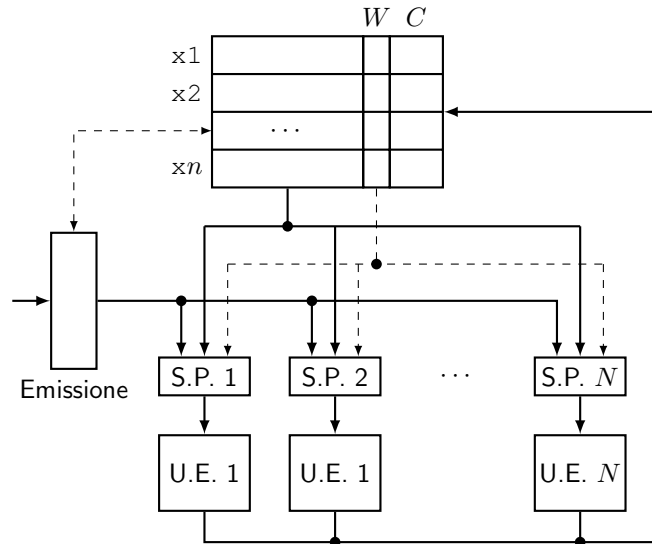


Figura 2: Ottimizzazione delle dipendenze sui dati

smette di ricevere nuove istruzioni fino a quando il salto non è stato completato (per evitare alee sul controllo).

3 Ottimizzazione delle dipendenze sui dati

Ogni stallo dello stadio di emissione riduce le prestazioni, quindi è importante cercare di eliminarli. Una prima ottimizzazione riguarda le dipendenze sui dati: l'osservazione fondamentale è che ogni istruzione può aspettare che siano pronti i propri dati indipendentemente dalle altre. Per esempio, consideriamo il seguente frammento di programma:

```
div x1, x2, x3
add x3, x4, x5
div x6, x7, x8
add x8, x9, x10
```

Lo stadio di emissione, in base alle regole stabilite fino ad ora, dovrebbe entrare in stallo alla prima add, per via della dipendenza sui dati con la prima div. Invece, introduciamo le cosiddette *stazioni di prenotazione* come in Figura 2. Lo scopo delle stazioni è di fornire un buffer in cui una istruzione può essere memorizzata in attesa che siano pronti i propri operandi (e nel frattempo tenere occupato uno stadio di esecuzione che possa eseguirla). Lo stadio di emissione copia le istruzioni in una stazione di prenotazione libera, dove l'istruzione monitora lo stato dei flag *W* dei proprio operandi sorgenti, e viene eseguita appena entrambi sono pronti. Nel frattempo lo stadio di emissione può procedere con le

istruzioni successive. Nell'esempio precedente, se ci sono 4 stadi di esecuzione, lo stadio di emissione può emettere tutte le istruzioni senza fermarsi: ciascuna di esse occuperà uno stadio di esecuzione; le due `div` potranno essere eseguite sostanzialmente in parallelo e, al loro termine, entrambe le `add` potranno essere eseguite anch'esse in parallelo.

È possibile anche avere stazioni di prenotazione in grado di memorizzare più di una istruzione, in genere organizzate in una coda FIFO davanti ad ogni stadio di esecuzione.

4 Ottimizzazione delle dipendenze sui nomi

Consideriamo una leggera variazione della sequenza di istruzioni precedente:

```
div x1, x2, x3
add x3, x4, x5
div x6, x7, x3
add x3, x9, x10
```

In questa nuova versione, la seconda `div` sta passando il risultato alla seconda `add` tramite il registro `x3` invece del registro `x8`. È chiaro che il risultato finale è lo stesso, ma questa nuova versione introduce delle dipendenze sui nomi: la seconda `div` ha ora una antipendenza con la prima `add` e una dipendenza in uscita con la prima `div`, e dunque può causare uno stallo nello stadio di emissione. Se il programmatore (o il compilatore) avesse scelto `x8`, o un qualunque altro registro non utilizzato, al posto di `x3`, il risultato della sequenza sarebbe stato lo stesso e lo stallo non ci sarebbe stato. In generale, quando indichiamo un registro come operando di una istruzione, quello che vogliamo realmente indicare è il risultato di una precedente istruzione: l'esatto registro usato non ha importanza. Se avessimo un numero sufficiente di registri, potremmo memorizzare il risultato di ogni istruzione in un registro diverso, e a quel punto non avremmo più dipendenze sui nomi. Questo è ovviamente irrealizzabile, ma possiamo sfruttare un'altra osservazione: non ci interessa eliminare *tutte* le dipendenze sui nomi, ma solo quelle che possono causare alee, e queste sono solo le dipendenze tra le istruzioni attualmente in esecuzione, che sono un numero ragionevole. Ci basta dunque avere un registro diverso per contenere il risultato di ogni istruzione in esecuzione.

La soluzione più semplice sarebbe quindi di prevedere un numero di registri sufficientemente grande, ma ci sono problemi pratici che ce lo impediscono: il numero di unità di esecuzione o di stazioni di prenotazione può essere aumentato facilmente al progredire della tecnologia, ma per aumentare il numero di registri visibili dal programmatore bisogna cambiare il set delle istruzioni. Anche quando ciò può essere fatto senza perdere la compatibilità con i programmi già esistenti, questi non potrebbero beneficiare dell'esistenza dei nuovi registri, a meno di non essere riscritti o ricompilati.

Possiamo però aumentare il numero di registri in modo totalmente trasparente per il software, introducendo uno stadio di *rinomina dei registri* tra lo

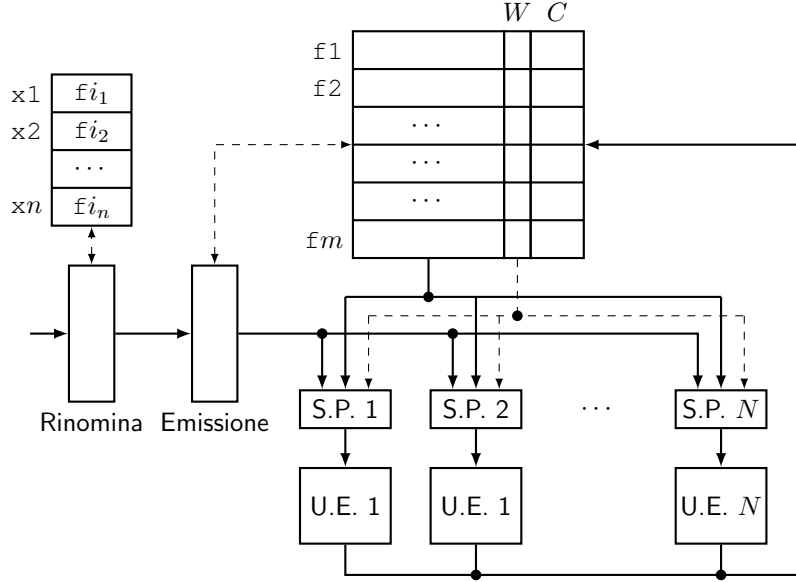


Figura 3: Ottimizzazione delle dipendenze sui nomi

stadio di decodifica e quello di emissione (Figura 3). Chiamiamo *registri logici* i registri usati dal programmatore/compiler, e introduciamo un numero (anche molto maggiore) di registri *fisici*. I registri logici hanno il solo scopo di specificare le relazioni tra le istruzioni (chi deve ricevere i dati da chi), ma il loro contenuto può trovarsi in uno qualsiasi dei registri fisici. Aggiungiamo una tabella che, ad ogni istante, associ ogni registro logico al registro fisico che contiene (o conterrà, se l'ultima istruzione che ha il registro logico come destinazione è ancora in esecuzione) l'ultimo valore scritto in quel registro fino a quell'istante. Chiamiamo $f1, f2, \dots$ i registri fisici. La dashboard conterrà le informazioni relative ai registri fisici, non più i logici. Quando lo stadio di rinomina riceve una nuova istruzione " $op, src1, src2, dst$ ", per prima cosa sostituisce $src1$ e $src2$ con i registri fisici corrispondenti in quel momento; quindi trova un registro fisico al momento inutilizzato ($W = 0$ e $C = 0$), sia fn , e crea una nuova associazione tra dst e fn (eventualmente la stessa già esistente). Per esempio, la sequenza precedente potrebbe essere trasformata nella seguente, supponendo che all'inizio tutti i registri fisici siano inutilizzati:

```
div f1, f2, f3
add f3, f4, f5
div f6, f7, f8
add f8, f9, f10
```

In particolare, all'arrivo della seconda *div* con destinazione $x3$, lo stadio di rinomina osserva che il corrispondente registro $f3$ è attualmente in uso (scritto dalla prima *div* e letto dalla prima *add*), quindi sceglie un nuovo registro, per

esempio `f8`, e associa `x3` a `f8`. All'arrivo della seconda `add`, che ha `x3` come primo operando sorgente, lo stadio di rinomina usa la nuova corrispondenza per permettere all'istruzione di leggere il risultato corretto, dal registro `f8`.

Lo stadio di emissione, a questo punto, non troverà mai dipendenze sui nomi, in quanto lo stadio di rinomina ha fatto in modo che le destinazioni di tutte le istruzioni abbiano sempre $W = 0$ e $C = 0$. Abbiamo dunque eliminato tutti gli stalli di questo tipo.

5 Istruzioni che accedono alla memoria

Consideriamo ora le istruzioni che accedono alla memoria: “`ld offset(base), dst`” che esegue una lettura, e “`st src, offset(base)`” che esegue una scrittura. Consideriamo due istruzioni i e j , in cui j viene dopo i ed è una `st`, mentre i è una `ld` o una `st`. Anche in questo caso possiamo avere dipendenze sui nomi e sui dati, se i e j accedono ad uno o più byte in comune. Rispetto al caso delle istruzioni operative e di salto, ci sono delle difficoltà in più:

- non possiamo pensare di avere una dashboard che mantenga W e C per ogni byte della memoria;
- per sapere se ci sono byte in comune, occorre prima calcolare gli indirizzi (sommando *offset* al contenuto del registro *base*).

Possiamo rendere il problema più gestibile se rinunciamo a parte del parallelismo o dei possibili riordinamenti. Per esempio, se prevediamo un'unica unità di esecuzione per le scritture in memoria, tutte le `st` saranno eseguite nell'ordine in cui vengono emesse, evitando *a priori* le alee dovute alle dipendenze sui nomi in uscita. Per risolvere le altre dipendenze (dati e antidipendenze) occorre procedere in due fasi. L'unità di esecuzione delle scritture avrà un cosiddetto *store buffer*, in cui le istruzioni `st` vengono inserite in ordine ed estratte in ordine una alla volta per essere eseguite. Ogni entrata dello store buffer contiene un campo “indirizzo” e un campo “dato”; il campo “indirizzo” contiene l'indirizzo completo, una volta calcolato, e il campo “dato” il valore da scrivere, letto dal registro sorgente dell'istruzione. Quando entrambi i campi sono stati riempiti l'istruzione è pronta per essere eseguita, ma la scrittura verrà effettuata solo quando tutte le scritture che la precedono saranno completate. L'istruzione verrà rimossa dallo store buffer solo quando la sua scrittura sarà stata completata. Lo store buffer, quindi, contiene informazioni su tutte le scritture “in corso” (emesse, ma non ancora completate), che sono le uniche che possono causare alee.

Le istruzioni `ld` devono andare in uno o più *load buffer* (possiamo averne più di uno, in quanto l'ordine delle letture non è importante). Ogni entrata dei load buffer memorizza l'indirizzo da cui l'istruzione vuole leggere. Per poter proseguire, una istruzione `ld` deve attendere che vengano calcolati tutti i campi “indirizzo” delle istruzioni `st` che si trovano già nello store buffer. A quel punto, bisogna confrontare l'indirizzo della `ld` con tutti quegli indirizzi e, in caso di

sovrapposizione, attendere che le corrispondenti `st` vengano completate. Questo evita le alee dovute alle dipendenze sui nomi. Come ottimizzazione, una `ld` che trovi i dati che sta cercando nello store buffer, può ottenerli direttamente da lì (*store forwarding*).

Infine, restano le alee causate dalle antidipendenze. Per risolvere anche queste è sufficiente che le istruzioni `st`, appena il loro indirizzo è stato calcolato, aspettino a loro volta che vengano calcolati tutti gli indirizzi di tutte le `ld` che erano già in qualche load buffer quando la `st` è stata emessa. Poi è necessario confrontare l'indirizzo della `st` con tutti gli indirizzi delle `ld` e, in caso di sovrapposizione, attendere che le rispettive `ld` vengano prima completate.

6 Realizzazione nei processori Intel

I meccanismi precedenti, così complessi già per le semplici istruzioni RISC, sembrerebbero impossibili da applicare al processore x86, ma con il Pentium Pro del 1995 l'Intel ha trovato il modo di introdurre queste soluzioni avanzate anche nei propri processori. Il trucco, usato in tutti i processori Intel da allora in poi, è di decodificare internamente le istruzioni x86 in microistruzioni simili alle istruzioni RISC, e poi eseguire le microistruzioni usando una architettura simile a quella descritta qui sopra. Il formato delle istruzioni RISC usato dall'Intel non è noto ufficialmente, ma se assumiamo come esempio le istruzioni che abbiamo usato fin'ora, possiamo pensare che una istruzione complessa come `"add %rax, offset(%rbx, %rcx, 8)"` venga tradotta nella sequenza

```
shl 3, rcx, x1
add x1, rbx, x2
ld  offset(x2), x3
add rax, x3, x4
st  x4, offset(x2)
```

Nella sequenza vediamo che, oltre ai registri "architetturali" `rax`, `rcx`, etc., si usano anche altri registri "microarchitetturali", non accessibili al programmatore, per contenere risultati intermedi (nell'esempio li abbiamo chiamati `x1`, `x2`, etc.). Lo stadio di decodifica può anche riusare sempre gli stessi registri microarchitetturali per i risultati intermedi, semplificando così il suo compito di traduzione. Il successivo stadio di rinomina dei registri provvederà a risolvere tutte le dipendenze sui nomi così introdotte.

Esecuzione speculativa

G. Lettieri

10 Giugno 2025

Nell'architettura costruita fin'ora, l'emissione di una istruzione di salto causa lo stallo dello pipeline nello stadio di emissione, fino a quando l'istruzione non è stata completata e la destinazione del salto non è nota. Questo ci permette di evitare le alee dovute alle dipendenze sul controllo, al costo di uno stallo per ogni istruzione di salto. Il BTB (Branch Target Predictor) è ancora presente, e permette agli stadi di prelievo e decodifica di continuare a lavorare anche mentre è in esecuzione una istruzione di salto, eventualmente annullando il lavoro fatto nel caso in cui la previsione si rivelasse sbagliata.

Osserviamo che gli stalli possono durare anche molti cicli di clock: si pensi al caso in cui l'istruzione di salto dipende per i dati da una precedente `ld` il cui operando sorgente non è in cache. Ha dunque senso cercare di ottimizzare anche questo tipo di dipendenze. L'idea è di non mettere in stallo lo stadio di emissione mentre è in corso l'esecuzione di una istruzione di salto, ma di continuare ad emettere le istruzioni che nel frattempo vengono prelevate e decodificate dall'indirizzo predetto dal BTB. Ovviamente, deve essere possibile annullare l'effetto di queste istruzioni quando il BTB sbaglia previsione. La tecnica che permette tutto ciò è detta *esecuzione speculativa*.

1 Stadio di ritiro

L'idea fondamentale è di introdurre un nuovo stadio di *ritiro* dopo lo stadio di esecuzione. Gli effetti delle istruzioni eseguite sono resi permanenti solo nello stadio di ritiro, e fino ad allora possono sempre essere annullati. Inoltre, mentre le istruzioni possono essere eseguite fuori ordine nello stadio di esecuzione, sono sempre ritirate in ordine. L'ordine è quello in cui le istruzioni sono state emesse, che è l'ordine originario nel flusso di esecuzione. In questo modo, quando viene ritirata una istruzione di salto, lo stadio di ritiro può verificare se la predizione del BTB era corretta e, in caso contrario, annullare tutte le istruzioni successive e comunicare allo stadio di prelievo l'indirizzo corretto da cui iniziare nuovamente a prelevare.

Lo stadio di ritiro usa un Reorder Buffer (ROB), organizzato come una coda FIFO. Lo stadio di emissione inserisce le istruzioni emesse in coda al ROB, oltre a copiarle in una stazione di prenotazione. Ogni entrata del ROB ha un flag che dice quando l'istruzione è stata completata. Man mano che le istruzioni

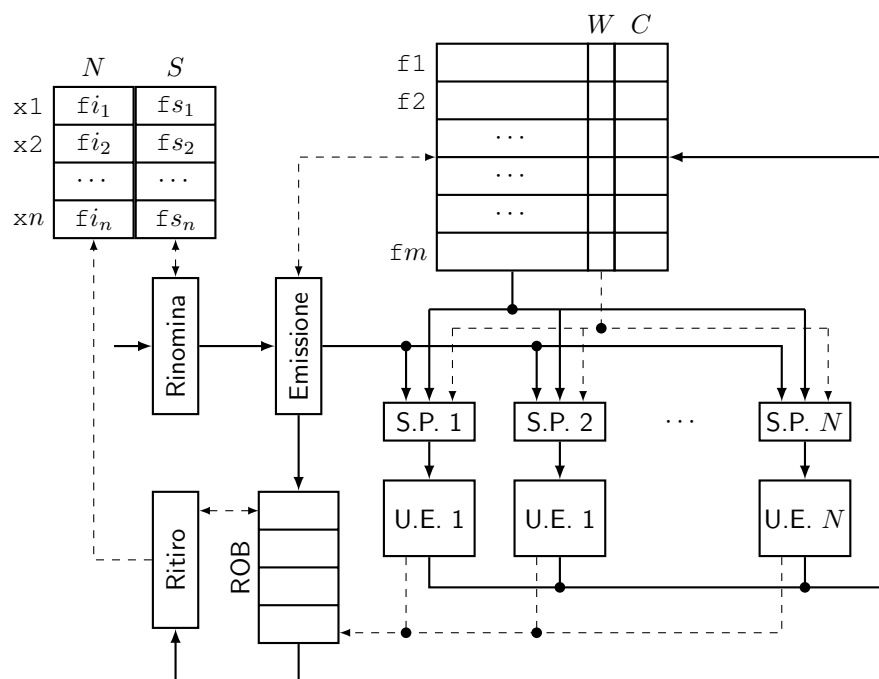


Figura 1: Esecuzione speculativa

vengono completate, in qualsiasi ordine, aggiornano il loro flag nel ROB. Lo stadio di ritiro monitora la testa del ROB e, quando vede che l'istruzione in testa è stata completata, la ritira. Se si tratta di una istruzione operativa, rende permanente il suo effetto (per esempio, scrittura in un registro). Se si tratta di una istruzione di salto predetta correttamente, non fa altro (le istruzioni successive sono state prelevate correttamente). Infine, come abbiamo detto, se si tratta di una istruzione di salto predetta in modo errato, svuota il ROB e fa ripartire lo stadio di prelievo dall'indirizzo corretto.

Vediamo ora come si possono rendere permanenti o annullare gli effetti delle istruzioni operative e di accesso alla memoria.

1.1 Istruzioni operative

Per quanto riguarda le istruzioni operative, l'unico effetto che modifica lo stato è la scrittura nel registro di destinazione. Qui possiamo sfruttare la distinzione, già introdotta, tra registri logici e registri fisici. Per ogni registro logico x ricordiamo *due* registri fisici, invece di uno solo:

- un registro fisico *speculativo* $S(x)$, che contiene, o conterrà, il risultato dell'ultima istruzione emessa e non ancora ritirata che ha x come destinazione;
- un registro fisico *non speculativo* $N(x)$, che contiene il risultato dell'ultima istruzione ritirata che aveva x come destinazione.

Supponiamo che lo stadio di rinomina dei registri riceva l'istruzione “*op src₁, src₂, dst*”. Per prima cosa sostituisce src_1 e src_2 con i loro registri speculativi $S(src_1)$ e $S(src_2)$; quindi sceglie un nuovo registro fisico f non utilizzato e lo fa diventare il nuovo registro speculativo di dst ($S(dst)$ diventa f), e infine sostituisce dst con f . L'informazione di quale fosse dst viene mantenuta in campi aggiuntivi dell'istruzione, e resta disponibile nel ROB. Se l'istruzione viene ritirata, lo stadio di ritiro rende permanente l'effetto dell'istruzione facendo diventare f il registro non speculativo di dst ($N(dst)$ diventa f). Se invece l'istruzione viene annullata, lo stadio di ritiro aggiorna la tabella dello stadio di rinomina, in modo che il registro speculativo di dst ridiventi il vecchio registro non speculativo ($S(dst)$ diventa $N(dst)$).

1.2 Istruzioni che accedono alla memoria

Per le operazioni che scrivono in memoria (*st*), sfruttiamo la presenza dello store buffer, che avevamo dovuto introdurre per evitare le alee causate dalle dipendenze sui nomi. Ci basta imporre che le scritture non possano essere effettivamente eseguite, e restino dunque nello store buffer, fino a quando l'istruzione non è stata ritirata. Lo stadio di ritiro, in caso di istruzione di salto predetta male, oltre a svuotare il ROB svuoterà anche lo store buffer.

Le istruzioni *ld* non presentano (apparentemente) problemi, e possiamo lasciarle accedere alla memoria anche prima che vengano ritirate. Anzi, uno

dei vantaggi dell'esecuzione speculativa è proprio quello di poter anticipare le istruzioni di lettura, in modo che eventuali cacheline mancanti possano essere prelevate in anticipo rispetto a quando il programma ne avrà effettivamente bisogno.

1.3 Gestione delle eccezioni

Supponiamo che una istruzione generi una eccezione mentre si trova nello stadio di esecuzione. Per esempio, una istruzione di accesso in memoria causa un page fault, o una istruzione di divisione scopre che il divisore è zero. L'eccezione non può essere gestita immediatamente, perché il processore non sa se l'istruzione andava davvero eseguita. Per rimediare a questo problema è sufficiente scrivere che l'istruzione ha causato una eccezione nella corrispondente entrata del ROB. Sarà lo stadio di ritiro, quando e se l'istruzione verrà ritirata, a far partire la gestione dell'eccezione. La gestione dell'eccezione è simile alla gestione di un salto predetto male: tutte le istruzioni successive vengono annullate e il prelievo ripartirà dall'indirizzo della routine che gestisce l'eccezione.

Negli Intel, come sappiamo, la gestione di una eccezione è una operazione piuttosto complicata, che deve svolgere diversi compiti (accesso alla IDT, eventuale cambio pila, salvataggio in pila di 5 parole quaduple, etc.). È probabile che, in caso di fault, lo stadio di ritiro immetta nello stadio di esecuzione una sequenza di microistruzioni che svolgono tutte le operazioni necessarie; l'ultima di queste microistruzioni sarà un salto all'indirizzo della routine di gestione dell'eccezione, letta dalla IDT.

2 Vulnerabilità della speculazione

Agli inizi di Gennaio 2018, sono state rivelate una serie di vulnerabilità dei processori, note come Meltdown e Spectre, che hanno mostrato come la tecnica della speculazione nasconda importanti insidie che possono compromettere il meccanismo di protezione del processore. Da allora ne sono state scoperte molte altre, non solo nei processori Intel. Alcuni di queste vulnerabilità sono causate da difetti di progettazione che possono essere corretti, mentre altre (come quelle del genere Spectre) sembrano essere causate da un difetto intrinseco nell'idea stessa della speculazione.

La domanda che ci dobbiamo porre è questa: è vero che *tutti* gli effetti di una istruzione possono essere cancellati, nel caso di speculazione errata? La risposta è quasi sempre no. Consideriamo, per esempio, una istruzione `ld`: se l'istruzione ha causato una miss in cache, la cache caricherà la corrispondente cacheline e questa non verrà poi rimossa, nel caso in cui l'istruzione venisse in seguito annullata. I progettisti di processori, fino al 2018, sembrano aver lavorato assumendo che ciò non avesse importanza, ma gli autori di Meltdown e Spectre hanno dimostrato il contrario. È molto semplice, da programma, capire se una data cacheline è in cache oppure no, perché i tempi di accesso nei due casi sono molto diversi: pochi cicli di clock contro centinaia. Questa differenza può

essere misurata, in quanto i processori possiedono dei timer con la precisione di qualche nanosecondo, e comunque la misura può essere resa robusta ripetendo l'esperimento tante volte.

Prendiamo in considerazione Meltdown, che è una vulnerabilità del primo tipo (difetto di progettazione). L'idea è di provare ad accedere alla memoria del kernel da livello utente. L'accesso causerà un page fault, che però verrà gestito solo quando l'istruzione arriverà in testa al ROB. Nel frattempo, il processore completerà comunque l'accesso (questo è il difetto di progettazione) e riuscirà ad eseguire diverse altre istruzioni. Queste istruzioni verranno annullate nel momento in cui lo stadio di ritiro riconoscerà il fault, ma i loro effetti in cache resteranno, e il programma potrà poi osservarli. Attenzione: il processo che esegue il programma dovrebbe essere poi terminato a causa del fault, ma in vari sistemi operativi (compresi Linux e Windows) i processi utente possono chiedere al kernel di associare dei propri gestori alle eccezioni, e in questo modo possono continuare ad eseguire anche dopo un fault. L'attaccante, quindi, può osservare lo stato della cache prodotto dalle istruzioni eseguite speculativamente dopo l'accesso vietato. Quali istruzioni potrebbe mettere l'attaccante dopo l'accesso? Osserviamo il seguente frammento:

```
xorl %rax, %rax
mov indirizzo_proibito, %al
shl $6, %rax
mov buf(%rax), %rbx
```

L'attaccante legge un byte dall'indirizzo proibito e poi accede ad un indirizzo che dipende dal byte letto, in particolare all'indirizzo $\text{buf} + b \times 64^1$, dove b è il valore letto. Sostanzialmente, accede ad una cacheline diversa per ogni possibile valore del byte segreto. Prima di eseguire questo codice, l'attaccante avrà dichiarato un buffer `buf` di 256×64 byte, e si sarà assicurato di rimuoverlo dalla cache (ci sono istruzioni per farlo). Dopo l'attacco, proverà ad accedere ad ogni cacheline coperta da `buf`, misurando il tempo richiesto da ogni accesso. Se tutto va bene, 255 accessi richiederanno centinaia di cicli di clock (cache miss) e solo uno richiederà pochi cicli di clock (cache hit): questa deve essere la cacheline a cui il processore aveva acceduto durante l'esecuzione speculativa. Supponiamo che l'accesso in questione sia relativo alla i -esima cacheline: vuol dire che il byte letto all'indirizzo proibito valeva i . L'attaccante è riuscito a leggere un byte dalla memoria del kernel; ora può incrementare `indirizzo_proibito` e leggere il successivo, e un byte alla volta riuscirà a leggere tutta lo spazio di indirizzamento riservato al kernel. Cosa troverà mappato in questo spazio? Fino al 2018, Linux aveva una finestra FM, proprio come il nostro nucleo didattico, e quindi in quello spazio di indirizzamento era mappata tutta la memoria fisica, in cui erano contenuti i frame di tutti i processi. Sfruttando Meltdown, un processo di un utente avrebbe potuto leggere la memoria dei processi degli altri utenti.

¹In realtà è necessario moltiplicare b per 4096 in modo da accedere a pagine diverse. Questo serve a disattivare un meccanismo che non abbiamo visto (il prefetch), che altrimenti confonderebbe le misure.

In risposta a Meltdown, gli sviluppatori di Linux fecero in modo di rimuovere la finestra FM ogni volta che il processore gira a livello utente, per poi ripristinarla ogni volta che il kernel riprende il controllo. Questo, ovviamente, ha un costo in termini di TLB miss.